

Maestría en Ingeniería de Software
y Sistemas Informáticos

Desarrollo Seguro de Software y Auditoría de la Ciberseguridad

Desarrollo Seguro de Software y Auditoría de la
Ciberseguridad

Tema 1. El problema de la seguridad en el software

Índice

Esquema

Ideas clave

- 1.1. Introducción y objetivos
- 1.2. El problema de la seguridad en el software
- 1.3. Vulnerabilidades y su clasificación
- 1.4. Propiedades del software seguro
- 1.5. Principios del diseño de seguridad del software
- 1.6. Tipos de S-SDLC
- 1.7. Estándares de Seguridad del Software
- 1.8. Referencias bibliográficas

A fondo

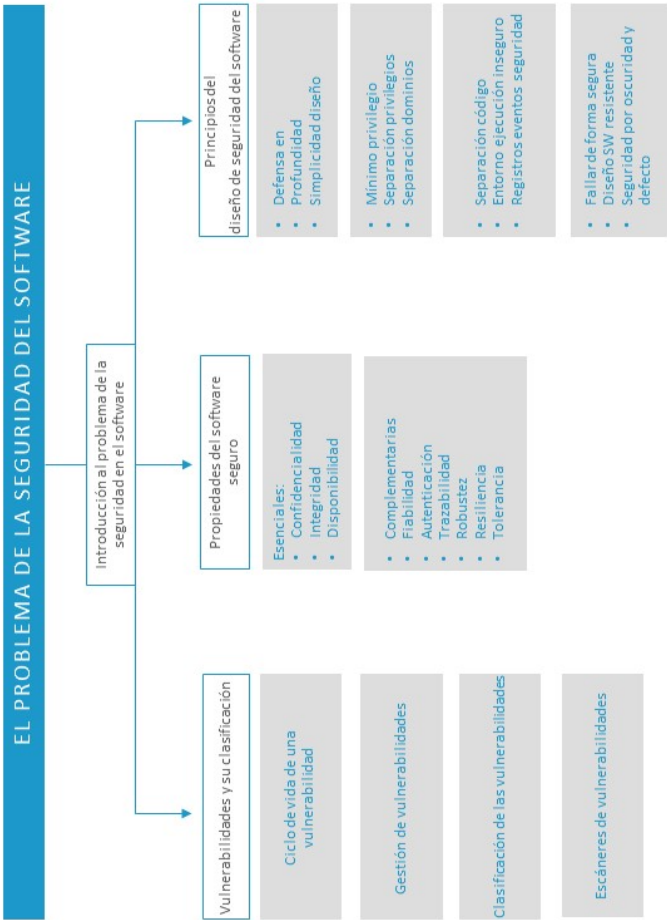
The application of a new secure software development life cycle (S-SDLC) with agile methodologies

Effective Filter for Common Injection Attacks in Online Web Applications

BSIMM7 software security framework

Security Review & Audit - Design, Architecture And Code Review

Test



1.1. Introducción y objetivos

Actualmente, las **tecnologías de seguridad de red** pueden ayudar a aliviar y mitigar los ciberataques, pero no resuelven el problema de seguridad real del software. Estas defensas no están destinadas, normalmente, a defender la **capa de aplicación**, que, según los últimos informes de amenazas, es donde más están aumentando los ataques.

Además, mediante **ingeniería social**, un atacante puede comprometer una máquina desde el interior; a través de esta podrá atacar a las demás empezando por las que dispongan de **aplicaciones vulnerables**. Se hace necesario, por lo tanto, disponer de un **software seguro** que funcione en un entorno agresivo y malicioso.

Los **objetivos** del presente tema son los siguientes:

- ▶ Introducir al alumno en los principales conceptos que abarca la **seguridad del software** en cuanto a los beneficios que produce y su importancia en la seguridad global de un sistema.
- ▶ Conocer los estándares de **gestión de vulnerabilidades**.
- ▶ Estudiar las **propiedades de un software seguro**.
- ▶ Estudiar los **principios de diseño seguro** para tener en cuenta en la arquitectura y el diseño de cualquier aplicación.
- ▶ Introducir una serie de **estándares de seguridad** aplicables a los procesos de **desarrollo seguro de software**.

1.2. El problema de la seguridad en el software

Hoy en día, los **ataques cibernéticos** son cada vez más **frecuentes, organizados y costosos** en el daño que infligen a las administraciones públicas, empresas privadas, redes de transporte, redes de suministro y otras infraestructuras críticas, desde la energía a las finanzas hasta el punto de poder llegar a ser una amenaza a la prosperidad, la seguridad y la estabilidad de un país.

La sociedad está cada vez más vinculada al **ciberespacio**. Un elemento importante del mismo lo constituyen el software o las aplicaciones que proporcionan los servicios, utilidades y funcionalidades. Sin embargo, estas aplicaciones presentan defectos, debilidades de diseño o configuraciones inseguras que originan vulnerabilidades que pueden ser explotadas por atacantes de diversa índole, desde aficionados hasta organizaciones de **cibercrimen** o, incluso, estados en acciones de **ciberguerra**, utilizándolas como plataformas de ataque al comprometer los sistemas y las redes de la organización.

Nadie quiere un **software defectuoso**, especialmente los desarrolladores, cuyo código incorrecto es el problema. En un informe de Klocwork (2004) se indica que las principales causas y aspectos que influyen en la aparición de vulnerabilidades son las siguientes:

- ▶ Tamaño excesivo y **complejidad** de las aplicaciones.
- ▶ **Mezcla de código** proveniente de varios orígenes, como compras a otra compañía, reutilización de otros existentes, etc., lo que puede producir comportamientos e interacciones no esperados de los componentes del software.
- ▶ Integración de los componentes del **software defectuoso**, lo que establece, por ejemplo, relaciones de confianza inadecuadas entre ellos.

- ▶ **Debilidades y fallos** en la especificación de requisitos y diseño no basados en valoraciones de riesgo y amenazas.
- ▶ No realizar **pruebas de seguridad** basadas en riesgo.
- ▶ Uso de **entornos de ejecución** con componentes que contienen vulnerabilidades o código malicioso embebido, como pueden ser capas de middleware, sistema operativo u otros componentes COTS.
- ▶ **Falta de herramientas** y un entorno de pruebas adecuado que simule correctamente el entorno real de ejecución.
- ▶ **Cambios de requisitos** del proyecto durante la etapa de codificación.
- ▶ Mezcla de **equipos de desarrolladores**, entre los que podemos tener equipos propios de desarrollos, asistencias técnicas, entidades subcontratadas, etc.
- ▶ **Falta de conocimiento** de prácticas de programación segura de los desarrolladores en el uso de lenguajes de programación.
- ▶ No controlar la **cadena de suministro del software**, lo que puede dar lugar a la introducción de código malicioso en origen.
- ▶ No realizar un seguimiento, por parte de los desarrolladores, de **guías normalizadas de estilo en la codificación**.
- ▶ Fechas límite de entrega de proyectos **inamovibles**.
- ▶ Cambio en la **codificación** en base al requerimiento de nuevas funcionalidades.
- ▶ **Tolerancia** a los defectos.
- ▶ **No tener actualizadas** las aplicaciones en producción con los parches correspondientes, configuraciones erróneas, etc.

Las aplicaciones son amenazadas y atacadas no solo en su fase de operación, sino que, también, en todas las fases de su ciclo de vida (Goertzel, K. M., 2009): desarrollo, distribución e instalación, operación, mantenimiento o sostenimiento.

En base a lo expuesto, se considera necesario que las diferentes organizaciones públicas o privadas dispongan de un **software fiable y resistente** a los ataques; es decir, de confianza, con un número de **vulnerabilidades explotables** que sea el mínimo posible. En respuesta a esto, nace la **seguridad del software**, que en el documento de referencia de SAFECode se define como: «La confianza que el software, hardware y servicios están libres de vulnerabilidades intencionadas o no intencionadas y que funcionan conforme a lo especificado y deseado» (SafeCode, 2010).

En este sentido, se puede definir la **seguridad del software** como:

El conjunto de principios de diseño y buenas prácticas a implantar en el SDLC para detectar, prevenir y corregir los defectos de seguridad en el desarrollo y adquisición de aplicaciones; de forma que se obtenga un software de confianza y robusto frente a ataques maliciosos, que realice solo las funciones para las que fue diseñado, que esté libre de vulnerabilidades, ya sean intencionalmente diseñadas o accidentalmente insertadas durante su ciclo de vida, y se asegure su integridad, disponibilidad y confidencialidad.

Para conseguir lo anterior y minimizar al **máximo** los ataques en la capa de aplicación y, por lo tanto, en número de **vulnerabilidades explotables**, es necesario incluir la seguridad, desde un principio, en el **ciclo de vida de desarrollo del software (SDLC)**, incluyendo buenas prácticas de seguridad en todas y cada una de

las fases de este, como, por ejemplo, requisitos de seguridad, casos de abuso, análisis de riesgo, análisis de código, pruebas de penetración dinámicas, etc.

Un beneficio importante que se obtendría de incluir un **proceso sistemático** que aborde la seguridad desde las primeras etapas del SDLC sería la **reducción de los costes de corrección de errores y vulnerabilidades**, pues estos son más altos conforme más tarde son detectados. Acorde a lo publicado por NIST (National Institute of Standards and Technology), el coste que tiene la corrección de código o vulnerabilidades después de la publicación de una versión es hasta treinta veces mayor que su detección y corrección en etapas tempranas del desarrollo.

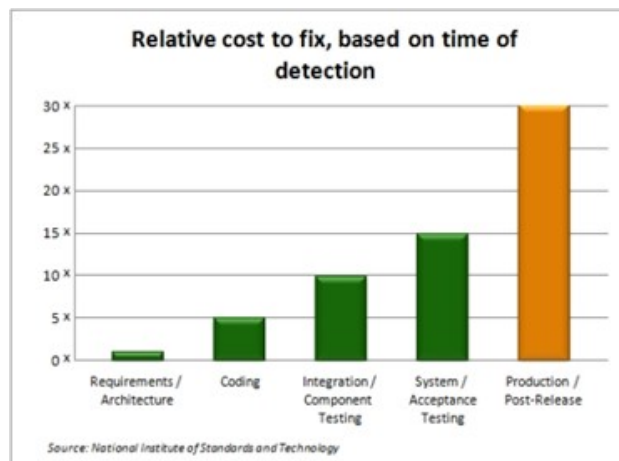


Figura 2. Coste relativo de corrección de vulnerabilidades en función de la etapa de desarrollo. Fuente: ImagineX consulting, 2018.

La empresa Microsoft, una de las primeras empresas en aplicar un **modelo S-SDLC**, presentó los siguientes datos al año de tener implantado el citado modelo (Caro, 2016):

- El **75 %** de las vulnerabilidades están relacionadas con las aplicaciones.

- ▶ Se genera alrededor de un **40-50 %** de reducción en el número de vulnerabilidades tras la implantación de un S-SDLC en el primer año (un 75-80 % sobre vulnerabilidades críticas).
- ▶ Una reducción de un **50 %** en el número de vulnerabilidades implica una reducción de un **75 %** en los costes de gestión de la configuración y respuesta a incidentes.

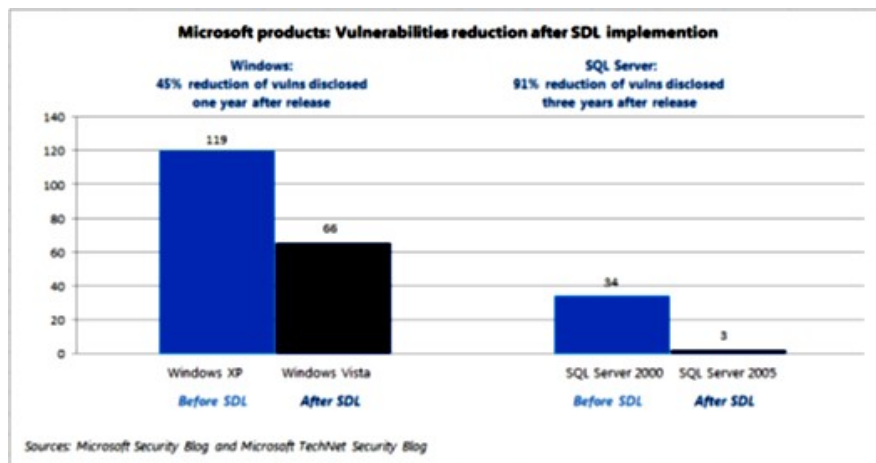


Figura 3. Datos de la empresa Microsoft. Fuente: Caro, 2016.

1.3. Vulnerabilidades y su clasificación

Se define **vulnerabilidad** como un fallo de programación, configuración o diseño que permite a los atacantes, de alguna manera, alterar el comportamiento normal de un programa, violar la política de seguridad del software y realizar algo malicioso, como alterar información sensible, interrumpir o destruir una aplicación o tomar su control.

Se puede decir que es un subconjunto del fenómeno más grande que constituyen los *bugs* (errores de programación) y *flaws* (errores de diseño) del software.

Sus fuentes se deben a:

- ▶ **Fallos de implementación.** Fallos provenientes de la codificación de los diseños del software realizados, por ejemplo: desbordamientos de búfer, formato, condiciones de carrera, *path traversal*, *cross-site scripting*, inyección SQL, etc.
- ▶ **Fallos de diseño.** Los sistemas hardware o software contienen, frecuentemente, fallos de diseño o debilidades (*flaws*) que pueden ser utilizados para realizar un ataque. Por ejemplo, TELNET no fue diseñado para su uso en entornos hostiles, para eso se implementó SSH.
- ▶ **Fallos de configuración.** La instalación de software, por defecto, implica, generalmente, la instalación de servicios que no se usan, pero que pueden presentar debilidades que comprometan la máquina.

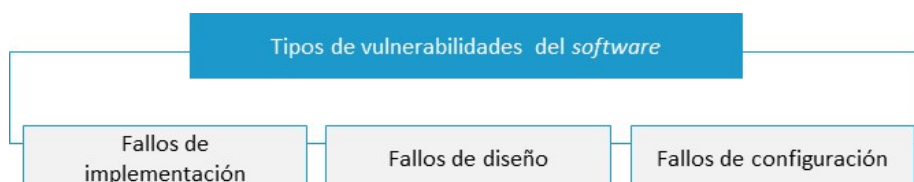


Figura 4. Tipos de vulnerabilidades del software. Fuente: Elaboración propia.

Casi todos los fallos que se producen en un software provienen de **fallos de implementación y diseño**, pero solamente algunos resultan ser vulnerabilidades de seguridad. Un fallo debe tener algún impacto o característica relevante para ser considerado un error de seguridad; es decir, tiene que permitir a los atacantes la posibilidad de lanzar un **exploits** que les permita hacerse con el control de un sistema.

Exploits: es una instancia particular de un ataque a un sistema informático que aprovecha una vulnerabilidad específica o un conjunto de ellas.

Una vulnerabilidad se define (INCIBE, 2011), básicamente, por cinco factores o parámetros que deben identificarla.

- ▶ **Producto:** productos a los que afecta, versión o conjunto de ellas.
- ▶ **Dónde:** componente o módulo del programa donde se localiza la vulnerabilidad.
- ▶ **Causa y consecuencia:** fallo técnico concreto que cometió el programador a la hora de crear la aplicación que es el origen de la vulnerabilidad.
- ▶ **Impacto:** define la gravedad de la vulnerabilidad, indica lo que puede conseguir un atacante que la explotase.
- ▶ **Vector:** técnica del atacante para aprovechar la vulnerabilidad, se le conoce como «vector de ataque».

Gestión de vulnerabilidades

Dada la gran cantidad de **vulnerabilidades** descubiertas, se hace necesario disponer de estándares que permitan referenciarlas unívocamente para poder conocer su gravedad de forma objetiva y obtener el conocimiento necesario para mitigarlas. Existen varios estándares, catálogos o bases de datos que, a continuación, pasamos a comentar:

- ▶ **Common Vulnerabilities and Exposures (CVE)**. Es un diccionario o catálogo público de vulnerabilidades, administrado por MITRE, que normaliza su descripción y las organiza desde diferentes tipos de vista (vulnerabilidades web, de diseño, implementación, etc.).

MITRE: organización sin ánimo de lucro de carácter público que trabaja en las áreas de ingeniería de sistemas, tecnologías de la información, concepto de operación y modernización de empresas.

Cada **identificador CVE** incluye:

- ▶ Identificador con el siguiente formato:

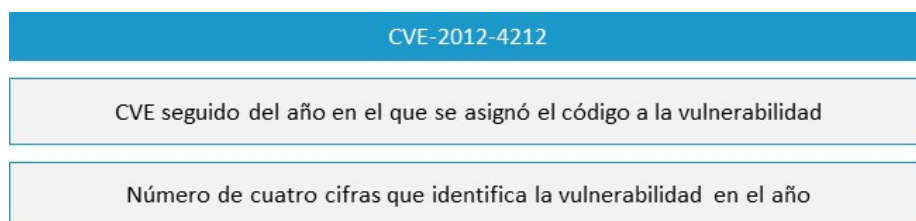


Figura 7. Identificador CVE. Fuente: Elaboración propia.

- ▶ Breve descripción de la vulnerabilidad.
- ▶ Referencias.

- **Common Vulnerability Scoring System (CVSS)**. Es, básicamente, un sistema que escalona la severidad de una vulnerabilidad de manera estricta a través de fórmulas, proporcionando un estándar para comunicar las características y el impacto de una vulnerabilidad identificada con su código CVE. Su modelo cuantitativo asegura una medición exacta y repetible, a la vez que permite ver características subyacentes que se usaron para generar su puntuación.

También permite organizar la **priorización** de las actividades de remediación o parcheo de las vulnerabilidades. En la figura se muestra el proceso de cálculo de una severidad:



Figura 8. Cálculo puntuación CVSS. Fuente: Elaboración propia.

El cálculo se realiza en base a tres tipos de métricas: **base**, **temporales** y **ambientales**, las dos últimas son opcionales. En cuanto a las **métricas base**, se tienen dos subconjuntos:

- **Explotabilidad**: vectores de acceso, complejidad de acceso y autenticación.
- **Impacto**: confidencialidad, integridad y disponibilidad.

El Instituto Nacional de Estándares y Tecnología de los Estados Unidos posee una página web donde se puede realizar el cálculo de la severidad de una vulnerabilidad conforme al estándar CBSS. Accede a ella a través del siguiente enlace: <https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator>

- ▶ **Common Weakness Enumeration (CWE)**. Estándar internacional y de libre uso que ofrece un conjunto unificado de debilidades o defectos del software medibles, que permite un análisis, descripción, selección y uso de herramientas de auditoría de seguridad de software y servicios que pueden encontrar debilidades en el código fuente y sistemas, así como una mejor comprensión y gestión de los puntos débiles de un software relacionados con la arquitectura y el diseño. Sus principales objetivos son:
 - Proporcionar un **lenguaje común** para describir los defectos y debilidades de seguridad del software en su arquitectura, diseño y codificación.
 - Proporcionar un **estándar de comparación** de herramientas de auditoría de la seguridad del software.
 - Proporcionar una línea base para la **identificación de vulnerabilidades**, su mitigación y los esfuerzos de prevención.

Incluye los siguientes tipos de debilidades del software:
desbordamientos del *buffer*, formato de cadenas, estructura y
problemas de validación, errores de ruta, interfaz de usuario,
autenticación, gestión de recursos, manipulación de datos, verificación
de datos, inyección de código, etc.

Cada identificador CWE incluye los siguientes campos de información:

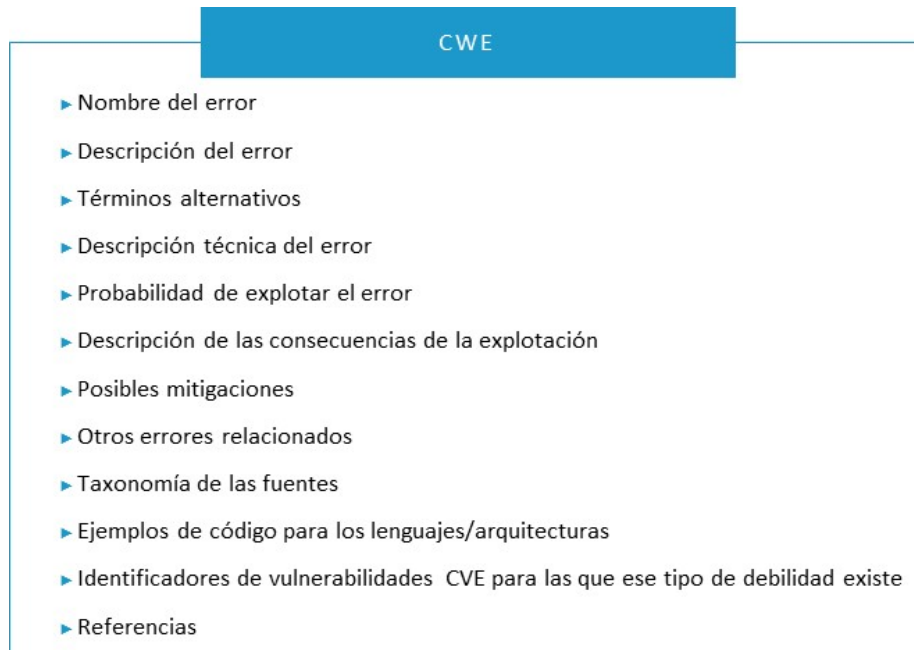


Figura 9. Campos de información de cada entrada CWE. Fuente: Elaboración propia.

1.4. Propiedades del software seguro

Básicamente se tienen dos conjuntos de **propiedades** que definen a un **software seguro** del que no lo es. Las primeras son las **esenciales**, comunes a la seguridad de cualquier sistema, cuya ausencia afecta gravemente a la seguridad de una aplicación; y, un segundo conjunto, las **complementarias** a las anteriores, que no influyen en su seguridad pero que ayudan a mejorarla en gran medida.

Las principales propiedades esenciales que distinguen al **software de confianza** del que no lo es, son:

- ▶ **Integridad.** Capacidad que garantiza que el código, los activos manejados, las configuraciones y el comportamiento no puedan ser, o no hayan sido, modificados o alterados por personas, entidades o procesos **no autorizados**, tanto durante la **fase de desarrollo** como en la **fase de operación**. Entre las modificaciones que se pueden realizar, tenemos: sobreescritura, inclusión de puertas traseras, borrado, corrupción de datos, etc.
- ▶ **Disponibilidad.** Capacidad que garantiza que el software es **operativo y accesible** por personas, entidades o procesos autorizados de forma que se pueda acceder a la información y a los recursos o servicios que la manejan conforme a las especificaciones de estos.
- ▶ **Confidencialidad.** Capacidad de preservar que cualquiera de sus características, activos manejados, están **ocultas** a usuarios no autorizados, de forma que se garantice que solo las personas, entidades o procesos autorizados pueden acceder a la información.

Un ejemplo de ataque podría ser un **desbordamiento de buffer** (*buffer overflow*) algo que consigue el control total de la máquina y permite violar las tres propiedades anteriores al poder robar información del sistema, cuentas de usuario, corromper

ficheros del sistema e incluso apagar la máquina y borrar los ficheros necesarios para que no vuelva a arrancar.

Estas tres primeras serían las **propiedades esenciales** mínimas fundamentales que debería de disponer todo software, a las que habría que añadir las siguientes **propiedades complementarias**:

- ▶ **Fiabilidad.** Capacidad del software de **funcionar de la forma esperada** en todas las situaciones a la que estará expuesto en su entorno de funcionamiento; es decir, que la posibilidad de que un agente malicioso pueda alterar la ejecución o resultado de una manera favorable para el atacante está significativamente reducida o eliminada.
- ▶ **Autenticación.** Capacidad que permite a un software **garantizar que una persona, entidad o proceso es quien dice ser**, o bien que garantiza la fuente de la que proceden los datos.
- ▶ **Trazabilidad.** Capacidad que garantiza la posibilidad de **imputar las acciones relacionadas** en un software que la ha originado.
- ▶ **Robustez.** Capacidad de **resistencia a los ataques** realizados por los agentes maliciosos (*malware, hackers, etc.*).
- ▶ **Resiliencia.** Capacidad del software de aislar, **contener y limitar** los daños ocasionados por fallos causados por la explotación de una vulnerabilidad de este y recuperarse reanudando su operación en (o por encima de) cierto nivel mínimo predefinido de servicio aceptable en tiempo oportuno.
- ▶ **Tolerancia.** Capacidad del software para **tolerar los errores y fallos** que resultan de ataques con éxito y seguir funcionando como si los ataques no se hubieran producido.

Las propiedades que distinguen al **software de confianza** se ilustran en la siguiente figura:

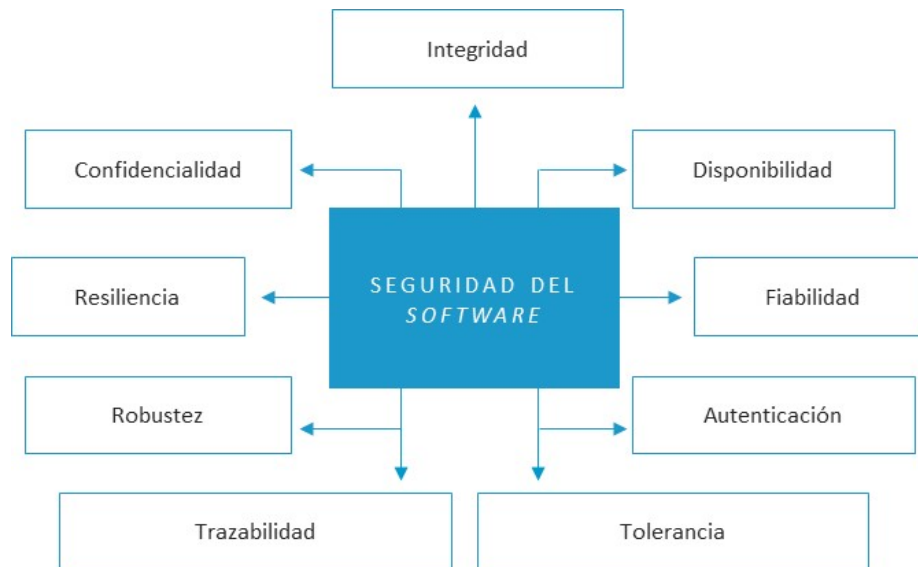


Figura 11. Propiedades seguridad del software. Fuente: Elaboración propia.

Existe una serie de factores que influyen en la probabilidad de que un software sea **consistente** con las propiedades anteriormente mostradas (Mead y Woody, 2016). Estos incluyen:

- ▶ **Principios de diseño y buenas prácticas de desarrollo.** Las prácticas utilizadas para desarrollar el software y los principios de diseño que lo rigen.
- ▶ **Herramientas de desarrollo.** El lenguaje de programación, bibliotecas y herramientas de desarrollo utilizadas para diseñar, implementar y probar el software y la forma en la que fueron utilizados por los desarrolladores.
- ▶ **Componentes adquiridos.** Tanto los componentes de software comercial como libre en cuanto a cómo fueron evaluados, seleccionados e integrados.

- ▶ **Configuraciones desplegadas.** Cómo el software se configuró durante la instalación en su entorno de producción.
- ▶ **Ambiente de operación.** La naturaleza y configuración de las protecciones proporcionadas por el entorno de ejecución o producción.
- ▶ **Conocimiento profesional.** El nivel de concienciación y conocimiento de seguridad que poseen los analistas, diseñadores, desarrolladores, probadores y mantenedores del software, o su falta de este.

1.5. Principios del diseño de seguridad del software

La adopción de estos principios de diseño constituye una base fundamental de las **técnicas de programación segura**, tanto para la protección de aplicaciones web, normalmente más expuestas a los **ciberataques** al estar desplegadas masivamente en Internet, como otras más tradicionales del tipo **cliente-servidor**.

Los principales **principios de diseño** para tener en cuenta en el desarrollo de toda aplicación son:

PRINCIPIO	OBJETIVO
Defensa en profundidad	Introducir múltiples capas de seguridad para reducir la probabilidad de compromiso del sistema.
Simplicidad del diseño	Reducir la complejidad del diseño para minimizar el número de vulnerabilidades explotables por el atacante y debilidades en el sistema.
Mínimo privilegio	Lo que no está expresamente permitido está prohibido.
Separación de dominios	Minimizar la probabilidad de que actores maliciosos obtengan fácilmente acceso a las ubicaciones de memoria u objetos de datos en el sistema.
Separación código, ejecutables y datos, configuración y programa	Reducir la probabilidad de que un ciberatacante que haya accedido a los datos del programa fácilmente pueda localizar y acceder a los archivos ejecutables y datos de configuración del programa.
Entorno de producción o ejecución inseguro	Evitar vulnerabilidad aplicando una serie de principios de validación de las entradas.
Registro de eventos de seguridad	Generar eventos (<i>logs</i>) de seguridad, para garantizar las acciones realizadas por un ciberatacante se observan y registran.
Fallar de forma segura	Reducir la probabilidad de que un fallo en el software pueda saltarse los mecanismos de seguridad de la aplicación, dejándolo en un modo de fallo inseguro vulnerable a los ataques.
Diseño de <i>software</i> resistente	Reducir al mínimo la cantidad de tiempo que un componente de un software defectuoso o fallido sigue siendo incapaz de protegerse de los ataques.
La seguridad por oscuridad: error	Concienciarse de que la seguridad por oscuridad es un mecanismo de defensa que puede proporcionar a un agente malicioso información para comprometer la seguridad de una aplicación.
Seguridad por defecto	Reducir la superficie de ataque de una aplicación o sistema.

Tabla 1. Objetivos y principios de seguridad. Fuente: elaboración propia.

Defensa en profundidad

Uno de los principios más importantes de una **estrategia defensiva** efectiva es la **defensa en profundidad**, que se define en la guía CCN-STIC-400 (CCN, 2013) como:

«Estrategia de protección consistente en introducir múltiples capas de seguridad, que permitan reducir la probabilidad de compromiso en caso de que una de las capas falle y en el peor de los casos minimizar el impacto». (CCN, 2013).

La arquitectura del **software y hardware de base**, que constituirá el entorno de ejecución donde la aplicación vaya a ser instalada, debería contar con una variedad de servicios de **seguridad y protecciones** que reduzcan y dificulten la probabilidad de que una acción maliciosa alcance el software, ya que:

- ▶ Se minimiza la exposición de las propias vulnerabilidades al mundo exterior.
- ▶ Se reduce la visibilidad externa de los componentes confiables principales.
- ▶ Se aíslan los componentes no confiables, de forma que su ejecución se vea limitada y sus malos comportamientos no afecten o amenacen la operación confiable de los demás.

El **aislamiento** significa que el software o componente no confiable dispone de recursos específicos para su ejecución, como memoria, espacio en disco duro, interfaz de red virtual, etc., en un entorno aislado. Para su implementación, se suelen utilizar máquinas virtuales que, además, pueden proporcionar otras características que ayudan a mejorar la **fiabilidad, confiabilidad y resistencia**, como el balanceo de carga, soporte para la restauración de imágenes, etc.

Objetivo: introducir múltiples capas de seguridad para reducir la probabilidad de compromiso del sistema.

Este principio propone un **enfoque defensivo** que implanta protecciones o mecanismos de seguridad en todos los niveles del sistema o capas del modelo Open Systems Interconnection (OSI). Las **medidas de seguridad** a implementar en cada capa podrán variar en función del entorno de operación del sistema. Sin embargo, el principio base, o general, permanece inalterable; por ejemplo, para las capas siguientes tendríamos:

CAPA	NOMBRE	AMENAZAS	CONTROLES
7	Aplicación	▶ Debilidades de diseño. ▶ Errores de programación. ▶ Malware. ▶ Intrusiones.	▶ Cortafuegos de aplicaciones. ▶ Proxy reversos. ▶ IDS Host, antivirus. ▶ S-SDLC. ▶ Análisis estático de código. ▶ Análisis de riesgo arquitectónico. ▶ Bastionado. ▶ Validación de entradas. ▶ Política de contraseñas.
6	Presentación	▶ Fuga de información. ▶ Divulgación de contenido.	▶ Cifrado datos sensibles. ▶ RBAC. ▶ Sistemas de prevención de pérdidas de datos (DLP).
5	Sesión	▶ Bypass de autenticación. ▶ Identificadores débiles de sesión. ▶ Spoofing. ▶ MITM. ▶ Robo de credenciales. ▶ Divulgación de datos. ▶ Ataques de fuerza bruta.	▶ Fuertes controles de autenticación. ▶ Control de acceso. ▶ Sesión única y aleatoria. ▶ Generación de ID con funciones criptográficas. ▶ Cifrado de la transmisión y el almacenamiento. ▶ Niveles de bloqueo de cuentas.
4	Transporte	▶ Pérdida de paquetes. ▶ <i>Fingerprinting</i>. ▶ Enumeración. ▶ Apertura de puertos.	▶ Cortafuegos Stateful inspection. ▶ TLS. ▶ Monitorización de puertos. ▶ Control flujo.

CAPA	NOMBRE	AMENAZAS	CONTROLES
3	Red	▶ Spoofing de las rutas y direcciones IP. ▶ Suplantación de identidad.	▶ Cortafuegos de filtrados de paquetes. ▶ ACLs, 802.1X, NAC, Kerberos, SSH. ▶ Bastionado de equipos de red para evitar ataques ARP, DHCP, etc. ▶ Monitoreo tráfico de red (<i>broadcast</i>). ▶ IDS/IPS ▶ IPsec. ▶ SIEM.
2	Enlace	▶ <i>Spoofing</i> de dirección MAC ▶ <i>Bypass</i> de VLAN. ▶ Errores <i>Spanning Tree</i> ▶ Ataques inalámbricos	▶ Filtrado de MAC. ▶ <i>Firewalls</i>. ▶ Segmentación VLAN. ▶ <i>Wireless</i> con autenticación y cifrado fuertes.
1	Físico	▶ Accidentes naturales (inundación, pérdidas energía eléctrica, etc.). ▶ Robo e intrusiones físicas. ▶ <i>Keyloggers</i> y otros interceptores de datos.	▶ Sistemas de detección de intrusiones físicas, sistemas de control de acceso. ▶ CCTV. ▶ Redundancias en los sistemas de alimentación eléctrica (generadores externos). ▶ Sistemas de alimentación ininterrumpida. ▶ Blindajes electromagnéticos. ▶ Sistemas de protección contra incendios.
0	Normativas	▶ Ataques a la confidencialidad, ingeniería social.	▶ Políticas de Seguridad de la Información. ▶ Organización de la seguridad. ▶ Procedimientos y estándares. ▶ Formación y concienciación en seguridad.

Tabla 2. Principio de defensa en profundidad. Fuente: elaboración propia.

Otro aspecto importante es la verificación de la cadena de suministros mediante la comprobación del *hash*, código de firma, aplicados al código ejecutable mediante la **validación de la integridad** de esta en el momento de la entrega, la instalación o en el tiempo de ejecución, para determinar:

- ▶ Si el código se **originó** a partir de una fuente de confianza.
- ▶ Si la **integridad** del código de se ha visto comprometida.

Simplicidad del diseño

La realización de un diseño tan sencillo como sea posible, y la redacción de unas especificaciones de este fácilmente **comprensibles y simples**, es una forma de obtener un **nivel de seguridad mayor**. Esto disminuye la probabilidad de que se incluyan debilidades de diseño y errores de programación que resulten en vulnerabilidades que comprometan la **seguridad** de la aplicación. Algunas de las opciones específicas de diseño del software que lo simplifican son (Mead y Woody, 2016):

- ▶ **Limitar** el número de estados posibles en el software.
- ▶ Favorecer los **procesos deterministas** sobre los no deterministas.
- ▶ Se debe evitar la **funcionalidad innecesaria** o los mecanismos de seguridad innecesarios.
- ▶ Utilizar **una sola tarea** en lugar de realizar múltiples tareas, siempre que sea práctico.
- ▶ El uso de **técnicas de sondeo** en lugar de interrupciones.
- ▶ Diseñar los componentes del software con el **conjunto mínimo** de características y capacidades que se requieran para realizar sus tareas en el sistema.

- ▶ La descomposición en subsistemas o componentes de un programa debería adaptarse a su **descomposición funcional**, para permitir una asignación uno a uno de los segmentos de programa a sus fines previstos.
- ▶ Desacoplar los componentes y procesos para minimizar las **interdependencias** entre ellos impedirá que un fallo o anomalía en un componente o proceso afecte a los estados de otros.
- ▶ No implementar características o **funciones innecesarias**.
- ▶ **Facilidad de uso**. Por ejemplo, Single Sign On (SSO) es un buen ejemplo que ilustra la simplificación de la autenticación de usuario.

Como podemos ver, el implementar **arquitecturas complejas** cuando se puede resolver el diseño de forma simple puede **perjudicar** la seguridad del sistema.

Objetivo: reducir la complejidad del diseño para minimizar el número de vulnerabilidades explotables por el atacante y debilidades en el sistema.

Mínimo privilegio

De acuerdo con el Software Assurance (2012), se define el **mínimo privilegio** como:

«Mínimo privilegio es un principio según el cual se concede a cada entidad (usuario, proceso o dispositivo) el conjunto más restrictivo de privilegios necesarios para el desempeño de sus tareas autorizadas. La aplicación de este principio limita el daño que puede resultar de un accidente, error o el uso no autorizado de un sistema. También reduce el número de interacciones potenciales entre los procesos privilegiados o programas, por lo que se minimiza la probabilidad de ocurrencia de usos maliciosos de privilegios, no deseados o inapropiados» (Software Assurance Pocket Guide Series, 2012).

Una de las principales razones por las que es necesario que una entidad se ejecute con los **mínimos privilegios posibles** es debido a que, si un ciberatacante consigue comprometer una máquina, o es capaz de inyectar código malicioso en un proceso del sistema, este se debería ejecutar con los **mismos privilegios** que tuviera el usuario en la máquina o el proceso.

Este principio requiere que el diseñador realice una lista de las **entidades de software** con los recursos que utiliza y las tareas que debe realizar en el sistema, especificando para cada una los **privilegios reales** estrictamente necesarios. Es normal cometer el error de asignar un usuario general con un conjunto de privilegios que le permitirá realizar todas las tareas, incluidas las no necesarias.

La **programación modular** ayuda a implementar menos privilegios, además de hacer que el código sea más legible, reutilizable y fácil de mantener. Cada unidad de código (clase, método, etc.) tiene un único propósito y las operaciones que puede realizar están limitadas solo a las que están diseñadas. Ejemplos de errores comunes son:

- ▶ Aplicación con derechos de administrador.
- ▶ Instalación de aplicaciones y servicios con el usuario de administrador.
- ▶ Usuarios con derechos de administrador.
- ▶ Servicios y procesos con privilegios por tiempo indefinido.

Objetivo: lo que no está expresamente permitido está prohibido.

Separación de privilegios

Es un principio relacionado con el anterior **mínimo privilegio** que implica la asignación a las diferentes entidades de un rol de las siguientes propiedades:

- ▶ Asignación de un **subconjunto** de funciones o tareas de todas las que ofrece el sistema.
- ▶ Acceso a los **datos necesarios** que debe gestionar para llevar a cabo su función en base a una serie de roles definidos.

Se **evita** así que todas las entidades sean capaces de acceder a la totalidad o llevar a cabo todas las funciones del sistema con privilegios de «superusuario» y, por lo tanto, que ninguna entidad tenga todos los privilegios necesarios para modificar, sobrescribir, borrar o destruir todos los componentes y recursos de la aplicación. Como ejemplo tenemos el servidor web, en donde el **usuario final** solo requiere de la capacidad de leer el contenido publicado e introducir datos en los formularios HTML. El **administrador**, por el contrario, tiene que ser capaz de leer, escribir y eliminar contenido y modificar el código de los formularios HTML.

Objetivo: asignación a las diferentes entidades de un rol que implique el acceso a un subconjunto de funciones o tareas y a los datos necesarios.

Separación de dominios

Es un principio que en unión con los dos anteriores, mínimo privilegio y separación de privilegios, permite **minimizar la probabilidad** de que actores maliciosos obtengan fácilmente acceso a las ubicaciones de memoria u objetos de datos del sistema. Para que el diseño cumpla con este principio, deben utilizar **técnicas de compartimentación** de los usuarios, procesos y datos de forma que las entidades:

- ▶ Solo deben ser capaces de realizar las tareas que son absolutamente necesarias.

- ▶ Llevarlas a cabo solamente con los datos que tengan permiso de acceso.
- ▶ Utilizar el espacio de memoria y disco que tengan asignado para la ejecución de esas funciones.

Objetivo: minimizar la probabilidad de que actores maliciosos obtengan fácilmente acceso a las ubicaciones de memoria u objetos de datos del sistema.

El **aislar las entidades de confianza** en su propia área de ejecución (con recursos dedicados a la misma) de otras menos confiables (procesadores de texto, software de descarga, etc.), permite reducir al mínimo su exposición a otras entidades e interfaces externas susceptibles de ser atacadas por agentes maliciosos.

Las **técnicas** que tenemos para llevar a cabo lo anterior son:

- ▶ Sistema operativo confiable.
- ▶ Máquinas virtuales. Además, pueden proporcionar otras características que ayudan a mejorar la fiabilidad, confiabilidad y resistencia, como el balanceo de carga, soporte para la restauración de imágenes, etc.
- ▶ Funciones *sandboxing* de lenguajes de programación como Java, Perl, .NET (Code Access Security), etc.
- ▶ Sistemas Unix: *chroot jails*.
- ▶ Trusted Processor Modules (TPM).

Separación de código, ejecutables y datos de configuración y de programa

Este principio pretende **reducir** la probabilidad de que un ciberatacante que haya accedido a los datos del programa fácilmente pueda **localizar y acceder** a los archivos ejecutables y **datos de configuración** de este, lo que le daría la posibilidad

de manipular el funcionamiento del sistema a su interés e incluso el escalado de privilegios.

La mayoría de las técnicas de separación de los datos de programa, configuración y archivos ejecutables se realizan en la **plataforma de ejecución** (procesador más sistema operativo). A continuación, vemos las principales:

- ▶ Utilizar plataformas con arquitectura Harvard.
- ▶ Establecer **permisos de escritura y lectura** de los datos de programa y sus metadatos al programa que los creó, a menos que exista una necesidad explícita de otros programas o entidades para poder leerlos.
- ▶ Los datos de configuración del programa solo deben poder ser leídos y modificados por el **administrador**.
- ▶ Los datos utilizados por un *script* en un servidor web deben ser colocados **fuera del árbol de documentos** de este.
- ▶ Prohibir a los programas y *scripts* escribir archivos en **directorios escribibles**, como el de UNIX/TMP.
- ▶ Almacenar los archivos de datos, configuración y programas ejecutables en los **directorios separados** del sistema de archivos.
- ▶ Los programas y *scripts* que están configurados para ejecutarse como servidor web de usuario *nobody* (debe suprimirse) deben ser modificados para funcionar bajo un **nombre de usuario específico**.
- ▶ Cifrar todos los archivos ejecutables e implementar un **módulo de decodificación** que se ejecute como parte del inicio del programa para descryptarlos al iniciar su funcionamiento.
- ▶ Incluir **técnicas de cifrado de archivos y firma digital o almacenamiento** en un servidor de datos externos con conexión cifrada (por ejemplo, mediante Secure

Sockets Layer —SSL— o Transport Layer Security —TLS—) para aislar los datos de configuración del software de la manipulación y eliminación, en caso de que las técnicas de control de acceso al sistema no sean lo suficientemente fuertes. Ello requerirá que el software incluya la lógica de cifrado para descifrar y validar la firma del archivo de configuración al inicio del programa.

- Implantar **clonado de sistemas** como una medida de recuperación desde un servidor remoto (que guardaría las imágenes y los datos de configuración) mediante una red de comunicaciones fuera de banda específica y cifrada.

Objetivo: reducir la probabilidad de que un ciberatacante que haya accedido a los datos del programa fácilmente pueda localizar y acceder a los archivos ejecutables y datos de configuración del programa.

Entorno de producción o ejecución inseguro

Al asumir que todos los componentes del entorno de producción y sistemas externos son inseguros o no confiables, se pretende reducir la exposición de los componentes del software a agentes potencialmente maliciosos que hayan podido penetrar en el **perímetro de defensa** (los límites de los dispositivos de protección perimetral) de la organización, y comprometer una máquina desde la que puedan expandirse e iniciar otros ataques a otras pertenecientes a la red (*pivoting*).

El software debe ser diseñado con una **mínima dependencia de los datos externos**. Se debe tener control completo sobre cualquier fuente de entrada de datos, tanto los proporcionados por la plataforma de ejecución como los de los sistemas externos, y se deberán **validar todos los datos** provenientes de las diferentes fuentes del entorno antes de utilizarlos, pues implican una posibilidad de ataque.

Hay que realizar una descomposición del sistema en sus componentes principales y **realizar una lista de las diferentes fuentes de datos externas**, entre las que podemos tener las siguientes:

- ▶ **Llamadas a sistema operativo.** Se deben evitar realizándolas a través de middleware o API's.
- ▶ Llamadas a otros programas en la **capa de aplicación.**
- ▶ Llamadas a una **capa de middleware** intermedia.
- ▶ **API's a los recursos del sistema.** Las aplicaciones que utilizan las API no deberían ser utilizadas por usuarios humanos.
- ▶ Referencias a objetos del sistema.
- ▶ En **aplicaciones cliente-servidor**, el flujo de datos entre los mismos.
- ▶ En **aplicaciones web**, el flujo de datos ente el cliente y el servidor.

Gran parte de los ataques a los **sistemas TIC** actualmente se deben a fallos o carencias en la **validación** de los datos de entrada, el confiar en la fuente que las originó hace a la aplicación **vulnerable a ataques** originados en el cliente al modificar los datos en el origen o durante su transporte. Los atacantes pueden **manipular** diferentes tipos de datos de entrada con el propósito de encontrar vías de compromiso de la aplicación.

Todos los tipos de entrada deben ser **validados y verificados** mediante pruebas específicas. Según Goertzel, K. M. y Winograd, T. (2008), entre los diferentes tipos de entradas a las aplicaciones, tenemos:

- ▶ Parámetros de línea de comandos.
- ▶ Variables de entorno.

- ▶ Localizadores de recursos universales (direcciones URL) e identificadores (URI).
- ▶ Referencias a nombres de archivo.
- ▶ Subida de contenido de archivos.
- ▶ Importaciones de archivos planos.
- ▶ Cabeceras Hyper Text Transfer Protocol (HTTP).
- ▶ Parámetros HTTP GET.
- ▶ Campos de formulario (especialmente los ocultos).
- ▶ Las listas de selección, listas desplegables.
- ▶ *Cookies*.
- ▶ Comunicaciones mediante Java *applets*.

Los tipos de aplicaciones que más probabilidad presentan de sufrir este tipo de ataques son las del tipo **cliente-servidor**, **portales web** y **agentes proxy**. Para evitar este tipo de vulnerabilidades, se deben aplicar una serie de **principios de validación de las entradas**, entre los que tenemos (Goertzel, K. M., Winograd, T., 2008):

- ▶ Centralizar la lógica de validación de las entradas.
- ▶ Asegurar que la validación de la entrada no puede ser evitada.
- ▶ Confiar en **listas blancas** validadas y filtrar las **listas negras**.
- ▶ Validar todas las entradas de usuario, incluidas las realizadas a través de *proxies* y agentes que actúen en nombre de los usuarios.

- ▶ Rechazar todos los contenidos ejecutables en las entradas provenientes de fuentes no autorizadas.
- ▶ Verificar que los programas que solicitan las llamadas tienen derecho (por la política) a emitirlos.
- ▶ Definir reacciones significativas a errores de validación de entrada.
- ▶ Validar los datos de salida de la aplicación antes de mostrarlos al usuario o redirigirlos a otro sistema.

Objetivo: evitar vulnerabilidades aplicando una serie de principios de validación de las entradas.

Registro de eventos de seguridad

Tradicionalmente, los **sistemas de auditoría** se dedicaban solo a grabar las acciones realizadas por los usuarios de estos. Sin embargo, hoy en día es necesario dotar a las aplicaciones de la capacidad de generar eventos (*logs*) de seguridad para garantizar que todas las acciones realizadas por un agente malicioso se observan y registran, contribuyendo a la capacidad de reconocer patrones y aislar o bloquear la fuente del ataque de modo que se evite su éxito; en definitiva, dotarle de cierta capacidad de detección de **intrusiones**.

Las principales diferencias que distinguen los sistemas con registros de auditoría de seguridad de registro de los registros de eventos estándar son:

- ▶ El tipo de información capturada en el registro de auditoría: eventos de seguridad.
- ▶ Capacidad de gestión de los incidentes relacionados con los eventos de seguridad.

- ▶ Posibilidad de que los eventos de seguridad registrados en la aplicación o sistema puedan ser utilizados en procesos reactivos después de la ocurrencia de un incidente.
- ▶ El nivel de protección de integridad.

Objetivo: generar eventos (*logs*) de seguridad para garantizar que las acciones realizadas por un ciberatacante se observan y registran.

Fallar de forma segura

El propósito de este principio es el de reducir la probabilidad de que un fallo en el software pueda saltarse los mecanismos de seguridad de la aplicación, dejándolo en un estado de **fallo inseguro**, vulnerable a los ataques. Es imposible implementar una aplicación perfecta que nunca falle, la solución consiste en saber en todo momento cuál es su estado y tener implementado un mecanismo, en caso de que falle.

Su objetivo es mantener la **resiliencia** (confidencialidad, integridad y disponibilidad) del software por defecto, para que sea seguro. Algunas de las características específicas del diseño que minimizan la probabilidad de que el software pase a un **estado inseguro** son:

- ▶ Implementar temporizadores tipo *watchdog*.
- ▶ Implementar una lógica de control de excepciones.
- ▶ El software siempre debe comenzar y terminar su ejecución en un estado seguro.
- ▶ Evitar problemas de sincronización y secuenciación.

Objetivo: reducir la probabilidad de que un fallo en el software pueda saltarse los mecanismos de seguridad de la aplicación, dejándolo en un modo de fallo inseguro, vulnerable a los ataques.

Diseño de software resistente

Dos propiedades importantes del software seguro es la **robustez y la resiliencia**, el siguiente principio pretende aumentar la resistencia de las aplicaciones reduciendo al mínimo la cantidad de tiempo que un componente de software defectuoso o fallido sigue siendo incapaz de protegerse de los ataques. Algunas de las características de diseño que aumentarán la **resistencia** del software incluyen (Goertzel, K. M. y Winograd. T., 2008):

- ▶ Capacidad de autocontrol y de limitar el uso de los recursos.
- ▶ Uso de técnicas de monitorización.
- ▶ Detección de estados anómalos.
- ▶ Proporcionar una realimentación que permita que todos los supuestos y modelos en los que el programa tome decisiones sean validados antes de ejecutarlas.
- ▶ Aprovechar cualquier redundancia y funciones de recuperación.
- ▶ Uso de plataformas virtuales.
- ▶ Uso de técnicas de recuperación.
- ▶ Determinar la cantidad de información a proporcionar en los mensajes de error.

Objetivo: reducir al mínimo la cantidad de tiempo que el componente de un software defectuoso o fallido sigue siendo incapaz de protegerse de los ataques.

La seguridad por oscuridad: error

Una de las asunciones que se debe realizar a la hora de diseñar una aplicación es que el atacante obtendrá, con el tiempo, acceso a todos los diseños y todo el código fuente de esta. La **seguridad por oscuridad** es un mecanismo de defensa que consiste en ocultar información sobre la aplicación de forma que sea difícil de obtener, pero que en poder de un atacante puede proporcionarle un medio para comprometerla.

El **principio de diseño abierto** (*open design*) establece que la implementación de salvaguardas de seguridad debe ser independiente del diseño, de modo que la revisión de este no comprometa la protección que ofrecen las salvaguardas. Esto es particularmente aplicable en la **criptografía**, donde los mecanismos de protección están desacoplados de las claves que se utilizan para las operaciones criptográficas, y los algoritmos utilizados para el cifrado y descifrado están abiertos y disponibles para que cualquier persona pueda revisarlos.

No es aconsejable **depender** de la seguridad por oscuridad, solamente indican que es válido usarla como una pequeña parte de una **estrategia general** de defensa en profundidad, para confundir y dificultar las actividades de ataque de un intruso. A continuación, se muestran algunos ejemplos de seguridad por oscuridad que pueden constituir un error:

- ▶ Guardar información en archivos binarios.
- ▶ Contraseñas en código fuente.
- ▶ Capetas ocultas en un servidor web.
- ▶ Falsa sensación de seguridad.
- ▶ Criptografía privada.

- Cambio del nombre del usuario «administrador».

Objetivo: concienciarse de que la seguridad por oscuridad es un mecanismo de defensa que puede proporcionar a un agente malicioso información para comprometer la seguridad de una aplicación.

Seguridad por defecto

Los sistemas deben diseñarse y configurarse de forma que garanticen la seguridad por defecto (ENS, 2010):

- El sistema proporcionará la **mínima funcionalidad** requerida para que la organización solo alcance sus objetivos y no alcance ninguna otra funcionalidad adicional.
- Las funciones de operación, administración y registro de actividad serán las **mínimas necesarias**, y se asegurará que solo sean accesibles por las personas, o desde emplazamientos o equipos autorizados, de manera que se puedan exigir, en su caso, restricciones de horario y puntos de acceso facultados.
- En un sistema de explotación se eliminarán o desactivarán, mediante el control de la configuración, las funciones que **no sean de interés** e, incluso, aquellas que sean inadecuadas al fin que se persigue.
- El uso ordinario del sistema ha de ser **sencillo y seguro**, de forma que una utilización insegura requiera de un acto consciente por parte del usuario.

Una cadena es tan fuerte como sus eslabones más débiles. Este principio de seguridad establece que la resistencia de su software contra los ataques dependerá, en gran medida, de la protección de sus componentes más débiles, se trate del código, un servicio o una interfaz. Una ruptura en el eslabón más débil resultará en una brecha de seguridad.

El principal **objetivo** del principio de seguridad, por defecto, es el de **minimizar la superficie de ataque** de cualquier aplicación o sistema TIC, deshabilitando aquellos servicios y elementos no necesarios y activando solo los necesarios.

Usualmente, cuando se instala o pone en producción una aplicación o servicio se activan o instalan servicios que no se usan con normalidad y que pueden suponer un punto potencial de entrada para los atacantes. Se deben elegir los componentes que van a ser utilizados de forma explícita por los usuarios e instalarlos y configurarlos de forma segura. Hay que minimizar los puntos vulnerables de la aplicación o sistemas, pues amenazan su **seguridad global**, por muy «securizado» que se tenga el resto; en concreto, los siguientes:

- ▶ Entradas y salidas de red.
- ▶ Número de puertos abiertos.
- ▶ Puntos de entrada a la aplicación, autenticados o no autenticados, locales o remotos y privilegios administrativos necesarios.
- ▶ Número de servicios. Desactivar los instalados por defectos no usados.
- ▶ Número de servicios con privilegios elevados.
- ▶ Número de cuentas de usuario administrador.
- ▶ Estado de las listas de control de acceso a directorios y ficheros.
- ▶ Control de cuentas de usuario y claves de acceso por defecto a servicios.
- ▶ Contenido dinámico de páginas web.

Varias organizaciones, como el National Institute of Standards and Technology (NIST), la Agencia de Seguridad Nacional (NSA) y el Centro Criptológico Nacional

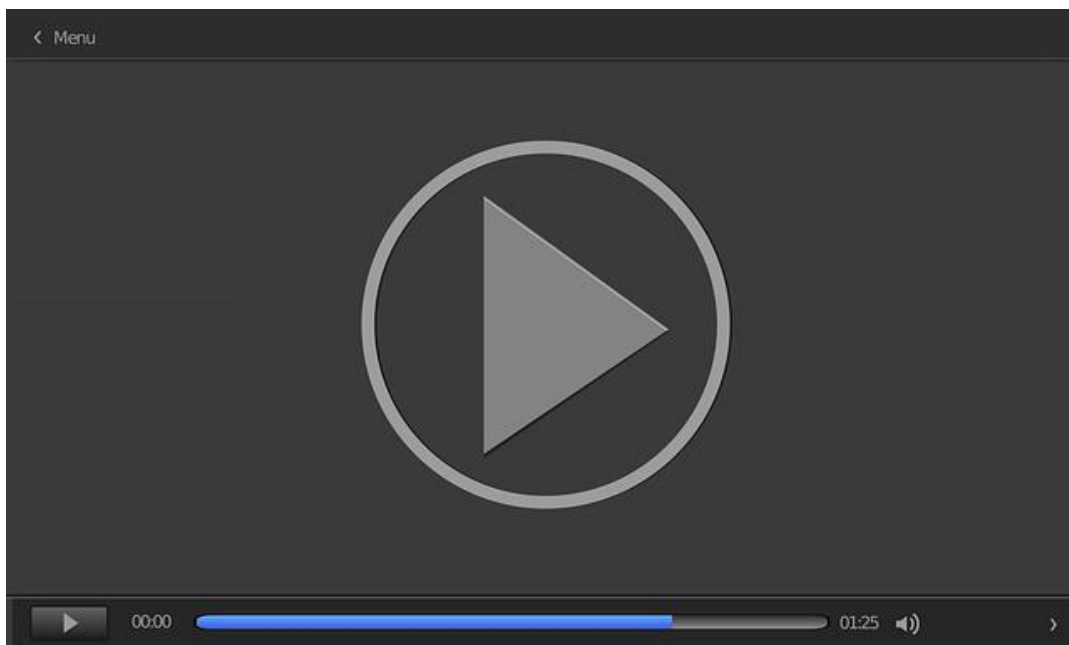
(CCN) han publicado **guías de seguridad de configuración y scripts** para productos COTS populares y de software libre que incluye la eliminación o restricción de servicios, usuarios, permisos y software innecesario.

El **bastionado de sistemas** es un proceso necesario en el marco de cualquier proyecto que contemple la aplicación de controles de seguridad sobre los sistemas TIC. Tiene como principio implementar todas las medidas de seguridad a **nivel técnico**, posibles para proteger un sistema sin que pierda la funcionalidad para la que fue destinado. Habrá casos en los que surjan conflictos que no se puedan superar, estos deben ser cuidadosamente documentados para proporcionar una justificación de la renuncia a los requisitos de configuración de seguridad que impidan el correcto funcionamiento del software.

Objetivo: reducir la superficie de ataque de una aplicación o sistema.

1.6. Tipos de S-SDLC

Para desarrollar software seguro y confiable es necesario una integración de buenas prácticas o actividades de seguridad en el ciclo de vida de desarrollo de software (SDLC). En este vídeo (*Tipos de ciclos de desarrollo de software seguro S-SDLC*) se presentan e introducen los principales conceptos sobre los ciclos de vida de desarrollo seguro del software (S-SDLC).

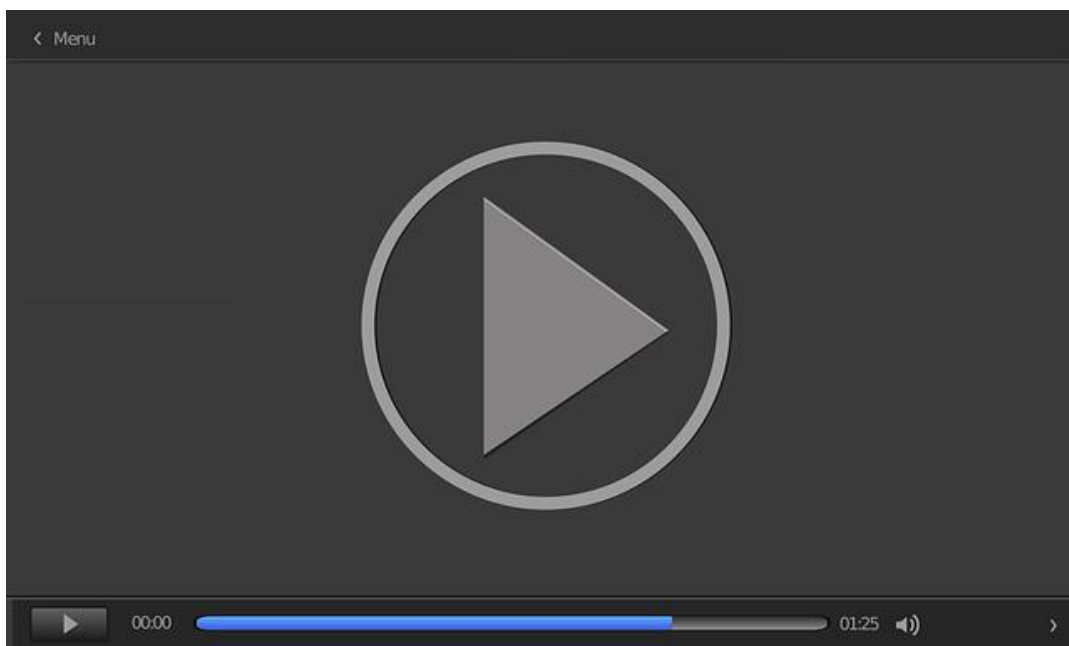


Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=78ad715e-5b7e-4a24-aae7-adf700cff4f7>

1.7. Estándares de Seguridad del Software

Las organizaciones deben implantar un estándar de aseguramiento de la calidad de software y un estándar de seguridad. El siguiente vídeo (*Estándares de aplicación a la Seguridad del Software*) presentará e introducirá las principales normas ISO relacionadas con la seguridad del software.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=822e5c90-2ac1-4f5c-857b-adf700cff575>

1.8. Referencias bibliográficas

Caro, A. (2016). *Modelos de desarrollo de software*. Catedra Viewext [Diapositivas].
<http://web.fdi.ucm.es/posgrado/conferencias/AndresCaroLindo-slides.pdf>

Catucci, F. (2018). *Operationalizing Web Application Security*. ImagineX consulting.
<https://www.qualys.com/docs/2018/qsc/las-vegas/qsc18-day2-23-operationalizing-web-application-security-imaginex.pdf>

Centro Criptológico Nacional. (2013). *Guía de Seguridad de la STIC (CCN-STIC-400)*. Manual de Seguridad de las Tecnologías de la Información y Comunicaciones.

ENS. (2010). Real Decreto 3/2010, de 8 de enero, por el que se regula el Esquema Nacional de Seguridad.

Goertzel, K. M., y Winograd, T. (2008). *Enhancing the Development Life Cycle to Produce Secure Software, Version 2.0*. United States Department of Defense Data and Analysis Center for Software.

Goertzel, K. M. (2009). *Introduction to Software Security*. Edición en español.

INCIBE. (2011). *Cuaderno de notas del Observatorio ¿Qué son las vulnerabilidades del software?*

Klocwork Inc. (2004). *Improving Software by Reducing Coding Defects Investing in software defect detection and prevention solutions to improve software reliability, reduce development costs, and optimize revenue opportunities*.

Mead, N., y Woody, C. (2016). *Cyber Security Engineering: A Practical Approach for Systems and Software Assurance*. Addison-Wesley Professional.

SAFECode. (2010). *Software Integrity Controls. Assurance-Based approach to minimizing risks in the software supply chain*.

Software Assurance Pocket Guide Series (2012). *Software Assurance in Acquisition and Contract Language*. Acquisition & Outsourcing, Volume I.

The application of a new secure software development life cycle (S-SDLC) with agile methodologies

De Vicente, J., Bermejo, J., Bermejo, J. R., y Sicilia, J. A. (2019). *The application of a new secure software development life cycle (S-SDLC) with agile methodologies*. Electronics, 8(11).

Este artículo propone un nuevo modelo S-SDLC aplicable a metodologías de desarrollo ágil de software. Además, contiene un estudio del estado del arte de modelos S-SDLC que complementa lo estudiado en la asignatura.

Effective Filter for Common Injection Attacks in Online Web Applications

Ibarra-Fiallo, S., Higuera, J. B., Intriago-Pazmiño, M., Higuera, J. R. B., Montalvo, J. A. S. y Cubo, J. *Effective Filter for Common Injection Attacks in Online Web Applications*. IEEE Access. <https://ieeexplore.ieee.org/document/9319139>

Los ataques de inyección contra aplicaciones web, por falta o una inadecuada validación de entradas, siguen siendo frecuentes, y organizaciones como OWASP los sitúan dentro del *top ten* de riesgos de seguridad para aplicaciones web. El objetivo principal de este trabajo es el desarrollo de una protección eficaz de las aplicaciones web contra los ataques de inyección más comunes mediante un filtro de validación de los campos de entrada a través de un conjunto de expresiones regulares y un proceso de sanitización.

BSIMM7 software security framework

Ishrath Sultana (s. f.). *BSIMM7 Software Security_Framework* [Vídeo]. YouTube.
<https://www.youtube.com/watch?v=5SHcoU6drqU>

En este vídeo se da una visión general de una importante iniciativa de seguridad del software llamada BSIMM y su forma de uso como una herramienta de medida para su organización, proveedores y su combinación con otros métodos de medición de seguridad.

Security Review & Audit - Design, Architecture And Code Review

Udacity. (2021, enero 15). *Security Review & Audit - Design, Architecture And Code Review* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=yykHTInfNVo>

En este vídeo se presentan los principales conceptos relativos a las revisiones de seguridad de las revisiones de diseño, arquitectura y código.

1. Indica cuál de las siguientes respuestas no es una causa de aparición de vulnerabilidades en el *software*:

- A. No realización de pruebas de seguridad basadas en riesgo.
- B. Cambios de requisitos del proyecto durante la etapa de especificación.
- C. Mezcla de código proveniente de varios orígenes.
- D. Tamaño excesivo y complejidad de las aplicaciones.

2. Indica la respuesta incorrecta respecto al ataque a las aplicaciones durante las diferentes fases de su ciclo de vida:

- A. Distribución e instalación. Ocurre cuando el instalador del *software* bastiona la plataforma en la que lo instala.
- B. Desarrollo. Un desarrollador puede alterar de forma intencionada o no el *software* bajo desarrollo.
- C. Operación. Cualquier *software* que se ejecuta en una plataforma conectada a la red tiene sus vulnerabilidades expuestas durante su funcionamiento. El nivel de exposición variará dependiendo de si la red es privada o pública, conectada o no a Internet, y si el entorno de ejecución del *software* ha sido configurado para minimizar sus vulnerabilidades.
- D. Mantenimiento o sostenimiento. No publicación de parches de las vulnerabilidades detectadas en el momento oportuno o incluso introducción de código malicioso por el personal de mantenimiento en las versiones actualizadas del código.

3. Señala la respuesta incorrecta. Las fuentes de las vulnerabilidades se deben a:
 - A. Fallos provenientes de la codificación de los diseños del *software* realizados.
 - B. Fallos provenientes de la cadena de distribución del *software*.
 - C. Los sistemas *hardware* o *software* contienen frecuentemente fallos de diseño que pueden ser utilizados para realizar un ataque.
 - D. La instalación de *software* por defecto implica, por lo general, la instalación de servicios que no se usan, pero que pueden presentar debilidades que comprometan la máquina.

4. Señala la respuesta incorrecta. Entre las técnicas y mecanismos para salvaguardar la integridad tenemos, por ejemplo:
 - A. Identificación del modo de transmisión y procesado de los datos por la aplicación.
 - B. Uso de arquitecturas de alta disponibilidad, con diferentes tipos de redundancias.
 - C. Uso de firma digital.
 - D. Estricta gestión de sesiones

5. Señala la respuesta incorrecta. Algunas de las opciones específicas de diseño del *software* que lo simplifican son:
 - A. Favorecer procesos deterministas sobre los no deterministas.
 - B. Limitar el número de estados posibles en el *software*.
 - C. El uso de técnicas de interrupciones en lugar de sondeo.
 - D. Desacoplar los componentes y procesos para minimizar las interdependencias entre ellos.

6. Para conseguir que el desarrollo de una aplicación posea las propiedades y principios de diseño del *software* seguro, se necesita que el personal de diseño y desarrollo lleve a cabo:

- A. Perspectiva administrador.
- B. Perspectiva defensor.
- C. Perspectiva usuario.
- D. Perspectiva atacante.

7. ¿Cómo se define la resiliencia?

- A. Capacidad de resistencia y tolerancia a los ataques realizados por los agentes maliciosos (*malware*, *hackers*, etc.).
- B. Capacidad que garantiza la posibilidad de imputar las acciones relacionadas en un *software* a la persona, entidad o proceso que la ha originado.
- C. Capacidad del *software* para aislar, contener y limitar los daños ocasionados por fallos causados por ataques de vulnerabilidades explotable del mismo, recuperarse lo más rápido posible de ellos y reanudar su operación en o por encima de cierto nivel mínimo predefinido de servicio aceptable en un tiempo oportuno.
- D. Capacidad del *software* para «tolerar» los errores y fallos que resultan de ataques con éxito y seguir funcionando como si los ataques no se hubieran producido.

8. ¿A qué principio de diseño le corresponde la siguiente afirmación? «Estrategia de protección consistente en introducir múltiples capas de seguridad, que permitan reducir la probabilidad de compromiso en caso de que una de las capas falle y en el peor de los casos minimizar el impacto».

- A. Seguridad por defecto.
- B. Separación de privilegios.
- C. Separación de dominios.
- D. Defensa en profundidad.

9. Señalar la respuesta incorrecta. Los elementos clave de un proceso de S-SDLC son:

- A. Gestión de configuración versiones.
- B Pruebas de seguridad.
- C. Hitos de control en las fases del SDLC.
- D. Despliegue y distribución.

10. Señala la respuesta incorrecta. El cálculo de código CVSS se realiza en base a tres tipos de métricas ambientales:

- A. Métricas estadísticas.
- B. Métricas base.
- C. Métricas temporales.
- D. Métricas ambientales.

Desarrollo Seguro de Software y Auditoría de la
Ciberseguridad

Tema 2. Seguridad en el ciclo de vida del software en las fases de análisis y diseño

Índice

Esquema

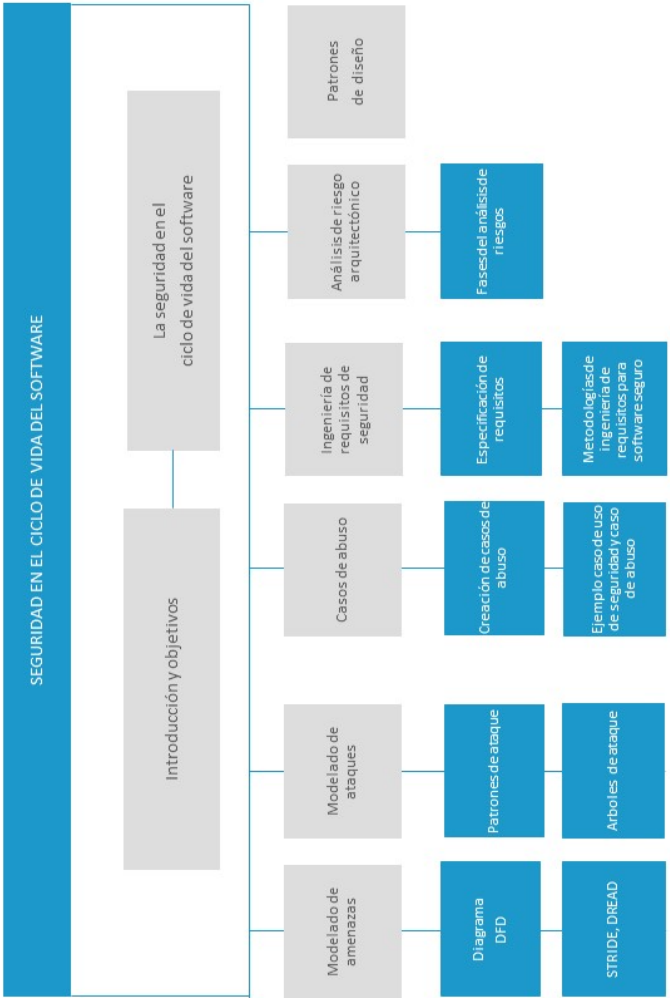
Ideas clave

- 2.1. Introducción y objetivos
- 2.2. La seguridad en ciclo de vida del software
- 2.3. Modelado de amenazas
- 2.4. Casos de abuso
- 2.5. Modelado de ataques
- 2.6. Ingeniería de requisitos de seguridad
- 2.7. Análisis de riesgo arquitectónico
- 2.8. Patrones de diseño
- 2.9. Referencias bibliográficas

A fondo

- Implementación simplificada del proceso SDL
- Secure Development Lifecycles (SDLC): Introduction and Process Models - Bart De Win
- Threat modeling tool demo
- Threat modeling
- Introduction, threat models

Test



2.1. Introducción y objetivos

Actualmente, el aumento de los ataques al software vulnerable ha dejado patente la insuficiencia de las protecciones a nivel de infraestructura. En este contexto, es conveniente minimizar al máximo los **ataques en la capa de aplicación** y, por lo tanto, el número de vulnerabilidades explotables.

El desarrollo de **software seguro y confiable** requiere de la adopción de un proceso sistemático o disciplina que aborde la seguridad en cada una de las fases de su ciclo de vida. Se debe integrar en él dos tipos de actividades de seguridad: la primera es el **seguimiento de unos principios de diseño seguro** (mínimo privilegio, por ejemplo); y la segunda, la inclusión de una serie de **buenas prácticas de seguridad** (especificación de requisitos de seguridad, casos de abuso, análisis de riesgo, análisis de código, pruebas de penetración dinámicas, etc.). A este **nuevo ciclo de vida** con prácticas de seguridad incluidas lo llamaremos **S-SDLC**.

Una de las principales ventajas de la adopción de un S-SDLC es el descubrimiento de **errores de codificación y debilidades de diseño** en las etapas tempranas de su desarrollo, lo que implica un importante ahorro de costes.

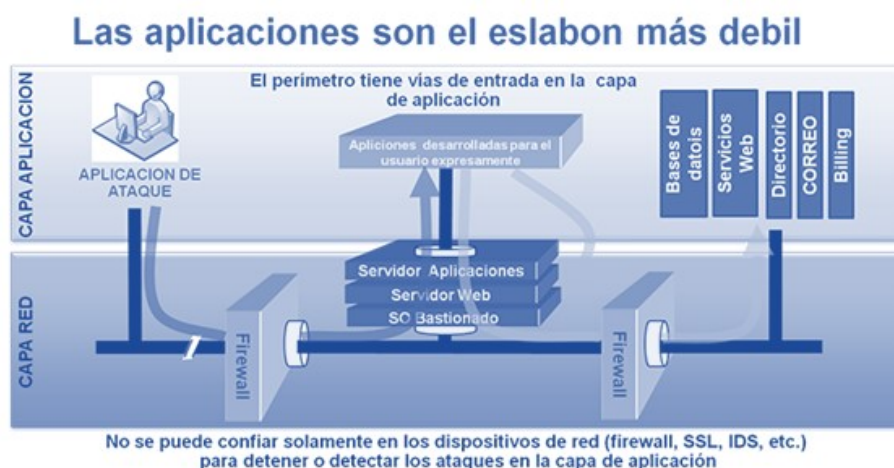


Figura 1. Ataques de la capa aplicación. Fuente: Hoglund, 2004.

Normalmente, el modelo más propicio para el desarrollo de software en las organizaciones suele ser una combinación de **dos o más modelos de los ciclos de vida** (cascada, espiral, etc.); sin embargo, lo importante no es el modelo seguido, si no la **incorporación de buenas prácticas de seguridad** en las diferentes fases de este, y unos hitos o puntos de control donde se verifiquen los entregables de las mismas.

A lo largo del tema se presentará un **conjunto mínimo de prácticas de seguridad** que deberían ser una parte integral del S-SDLC. Entre las razones principales para añadir prácticas de seguridad en el SDLC, tenemos:

- ▶ Mayor probabilidad de capturar adecuadamente los requisitos, tomar decisiones de diseño correctas y no cometer errores involuntarios de codificación.
- ▶ Dificultad de los agentes maliciosos para explotar vulnerabilidades y debilidades del software, pues el resultante de un ciclo de vida S-SDLC será más robusto en su entorno de ejecución y en su interacción con entidades externas.

Los **objetivos** del presente tema son los siguientes:

- ▶ Conocimiento y comprensión de las buenas prácticas de seguridad a incluir en un S-SDLC en las fases de requisitos y diseño.
- ▶ Profundizar en el estudio de una de las principales prácticas de seguridad, como es el análisis de riesgo arquitectónico.
- ▶ Introducir al alumno en prácticas de seguridad con el modelado de amenazas, los casos de abuso, el modelado de ataques, la ingeniería de requisitos de seguridad y los patrones de diseño.

2.2. La seguridad en ciclo de vida del software

La **seguridad del software** es algo más que la eliminación de las vulnerabilidades y la realización de pruebas de penetración. Un aspecto importante es la adopción por los gerentes de un enfoque sistémico que incorpore los **principios de diseño** y unas buenas prácticas de seguridad del **software touchpoint** en su ciclo de vida de desarrollo. Su objetivo es producir software **más seguro y confiable**, así como el poder verificar su seguridad. A este nuevo ciclo de vida lo denominaremos **ciclo de vida del software seguro S-SDLC**.

No existe una metodología de ingeniería de seguridad de software que haya probado unos mejores resultados que otras, todas se basan en la **inclusión de prácticas de seguridad**, fundamentalmente. Por lo tanto, no importa cuál es el modelo de ciclo de vida seguido, lo que sí es importante es que la seguridad sea considerada desde las **primeras etapas** del ciclo de vida del software.

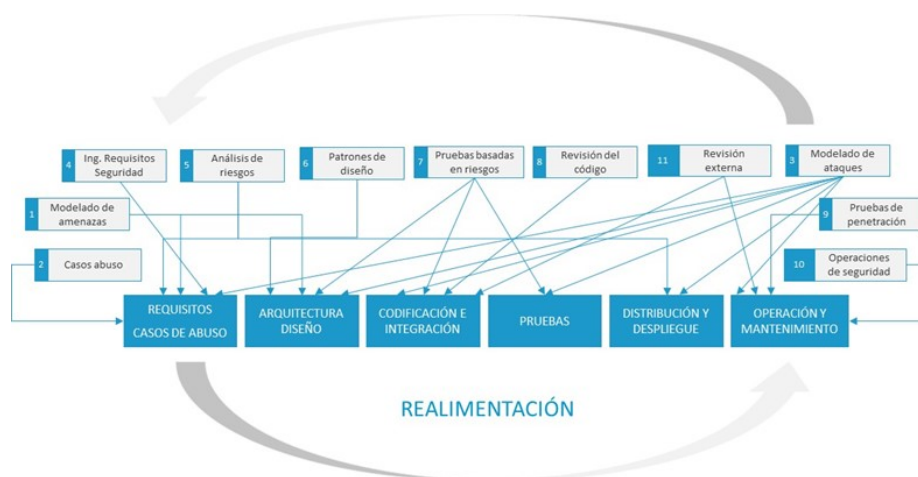


Figura 2. Mejores prácticas de seguridad del software en el SDLC. Fuente: elaboración propia.

La figura anterior propone un Ciclo de Vida de Desarrollo de Software SDLC en cascada, basado en el modelo de McGraw (2005). Para otro tipo, como los **iterativos**, sería similar, donde se especifican las actividades y prácticas de seguridad a efectuar en cada fase de este. A continuación, se enumeran esas actividades o mejores prácticas de seguridad del software:

- ▶ Modelado de Amenazas
- ▶ Casos de abuso.
- ▶ Modelado de ataques.
- ▶ Ingeniería requisitos de seguridad.
- ▶ Análisis de riesgo arquitectónico.
- ▶ Patrones de diseño.
- ▶ Pruebas de seguridad basadas en riesgo.
- ▶ Revisión de código.
- ▶ Pruebas de penetración.
- ▶ Operaciones de seguridad.
- ▶ Revisión externa.

En la siguiente tabla se representan las prácticas de seguridad y su relación en cada una de las fases donde son aplicables.

Fase SDLC \ Practica de Seguridad	Req.	Dise.	Codif.	Prueb.	Desp.	Oper.
Modelado de amenazas	X	X				
Casos de abuso	X					
Modelado de ataques	X	X	X	X	X	X
Ingeniería requisitos de seguridad	X					
Análisis de riesgo arquitectónico	X	X		X	X	X
Patrones de diseño		X				
Pruebas de seguridad basados en riesgo		X	X	X	X	X
Revisión de código			X			
Pruebas de penetración					X	X
Operaciones de seguridad						X
Revisión externa			X	X		X

Tabla 1. Relación entre las prácticas de seguridad y las fases del ciclo de vida. Fuente: elaboración propia.

En los siguientes apartados de este tema, se desarrollarán las **prácticas de seguridad** propuestas para el ciclo de vida.

2.3. Modelado de amenazas

Una **amenaza** para una aplicación de sistema software es cualquier actor, agente, circunstancia o evento que tiene el potencial de causarle **daño** a los datos o recursos a los que tiene, o permite, acceso (no se debe confundir con el concepto de vulnerabilidad).

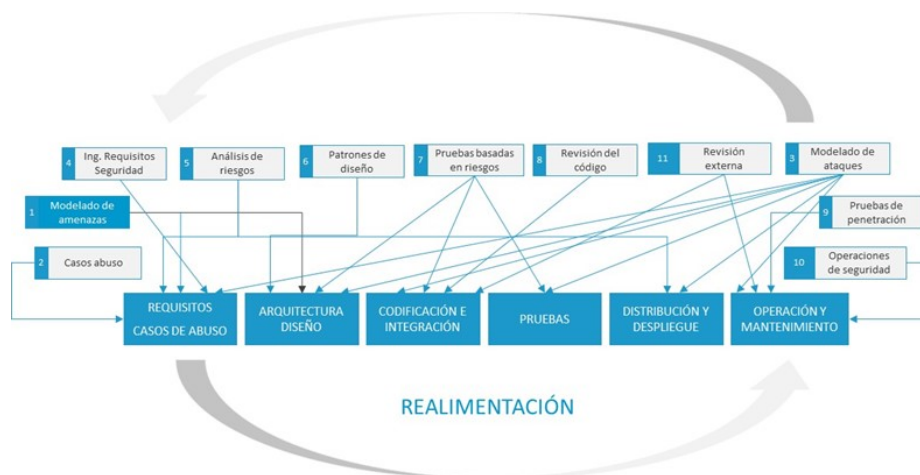


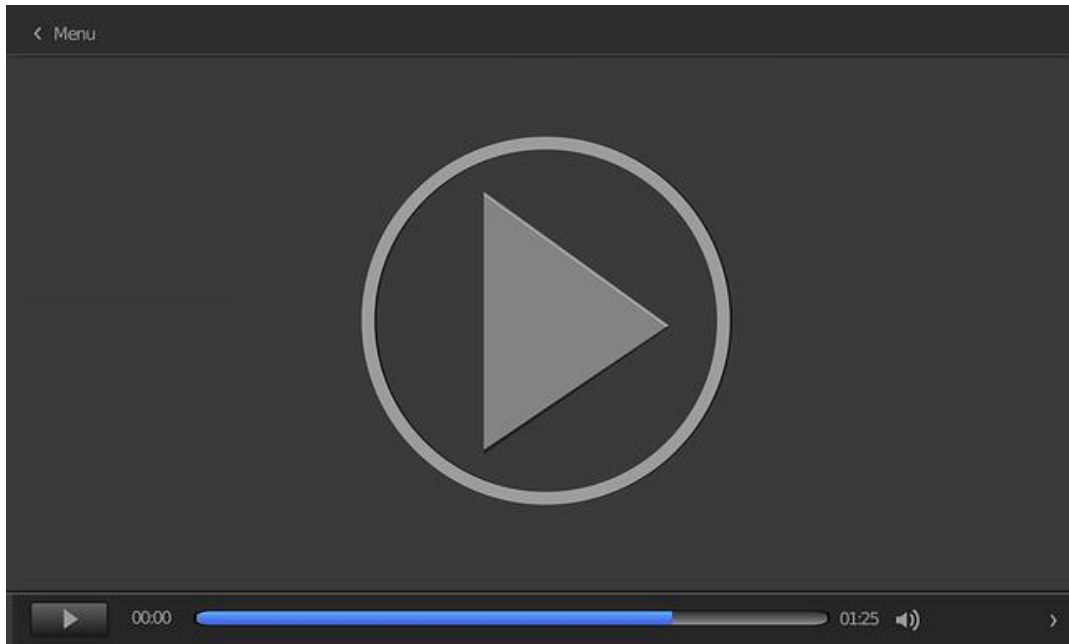
Figura 3. Modelado de amenazas. Fuente: elaboración propia.

Las amenazas se pueden clasificar según su **intencionalidad**:

- ▶ Involuntaria.
- ▶ Intencional, pero no malicioso.
- ▶ Maliciosa.

Esta importante práctica de seguridad se desarrolla en los siguientes vídeos:

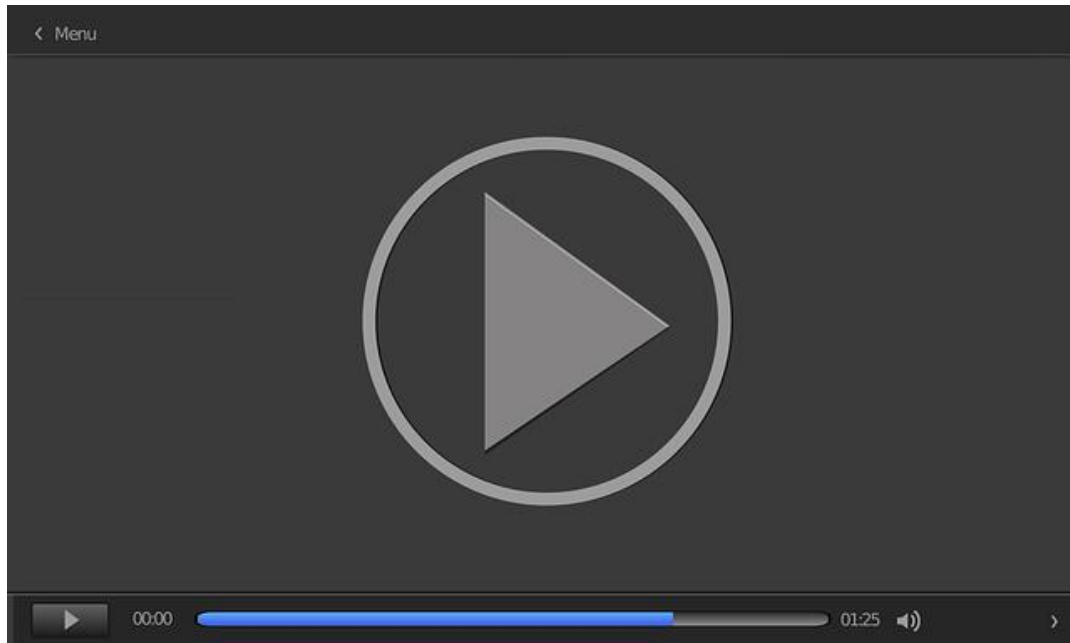
En este vídeo (*Modelado de Amenazas*) se presenta y estudia la buena práctica de seguridad Modelado de Amenazas, a desarrollar en las fases de análisis y diseño de un ciclo de vida de desarrollo de un software (SDLC).



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=ef5d6c46-b2cf-4f29-aedd-adf700cff411>

En este vídeo (*Microsoft Threat Modelling Tool*) se realiza una demostración práctica de la herramienta de modelado de amenazas para aplicaciones Microsoft Threat Modelling Tool.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=39afa698-2ea6-4dc9-9a5f-adf700cff387>

2.4. Casos de abuso

Los **diagramas de casos de uso** constituyen unas buenas prácticas para la obtención de los requisitos funcionales de una aplicación; sin embargo, para la obtención de requisitos de seguridad no lo son, sobre todo los **no funcionales o requisitos negativos** referidos a actividades que no debería realizar el sistema. Para solucionar lo anterior, se ha creado un tipo especializado de casos de uso que se utiliza para analizar y especificar las amenazas de seguridad. Xiaohong y sus colaboradores (2015) lo definen como:

«Un caso de abuso es la inversa de un caso de uso, es decir, una función que el sistema no debe permitir o una secuencia completa de acciones que resulta en una pérdida para la organización» (Xiaohong et al., 2015).

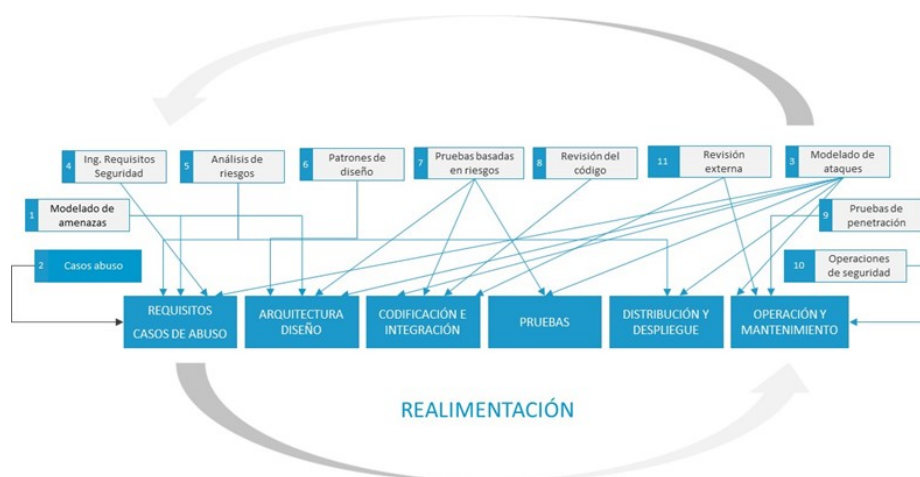


Figura 4. Casos de abuso. Fuente: elaboración propia.

Los **casos de abuso**, o casos de mal uso, son un instrumento que puede ayudar a pensar de la misma forma que lo hacen los atacantes. Pensando más allá de los aspectos normativos, funcionales y estudiando los eventos negativos o inesperados. Los **profesionales de seguridad de software** entienden mejor cómo crear software seguro, pues permiten obtener una mejor comprensión de las áreas de riesgo del sistema a través de:

- ▶ Identificación de los **objetivos de seguridad** que debe cumplir el software.
- ▶ Identificación de las **amenazas de seguridad** a ser mitigadas por el software.
- ▶ Identificación de los puntos en el software **susceptibles a ser atacados**.
- ▶ Obtener las restricciones o requisitos no funcionales de seguridad, requisitos negativos, necesarios para alcanzar los **objetivos de seguridad**.
- ▶ Obtener **requisitos de seguridad** funcionales, requisitos positivos, de los mecanismos que garanticen que el software cumple con los objetivos de seguridad.

Los casos de abuso constituyen un excelente medio de análisis de las amenazas que debe afrontar el software. Son apropiados para el análisis y especificación de restricciones, o requisitos negativos, ya que se basan en el uso indebido del sistema. Establecen la base para otros casos de uso de seguridad que proporcionan los medios para contrarrestar o mitigar las amenazas capturadas en los mismos y una manera altamente reutilizable de organizar, analizar y especificar sus requisitos de seguridad.

Los casos de abuso describen lo que el software **no debe hacer** en respuesta al uso incorrecto o malintencionado de este por un agente malicioso. Para cada caso de uso funcional, el desarrollador deberá explorar las formas en que esa función podría ser deliberadamente mal utilizada. También tiene que identificar todas las posibles relaciones entre casos de uso de seguridad y casos abuso, para especificar restricciones y requisitos de seguridad para evitar un mal uso o abuso de las funcionalidades del software.

En la siguiente figura se muestra la relación entre el caso de **uso de seguridad** y el de **abuso asociado**.

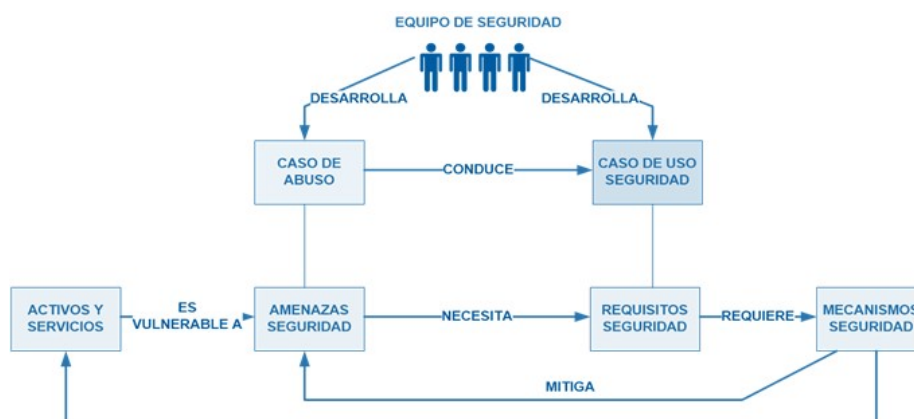


Figura 5. Relación entre el caso de uso de seguridad y el de abuso asociado. Fuente: adaptado de Donald, 2003.

En la siguiente tabla se resumen las diferencias entre los casos de uso de seguridad y los casos de abuso:

Característica	Casos de abuso	Casos uso seguridad
Uso	Analiza y especifica las amenazas a la seguridad y requisitos de seguridad no funcionales	Analiza y especifica los requisitos de seguridad funcionales
Criterio de éxito	Éxito del atacante	Éxito de la aplicación
Producido	Equipo de seguridad	Equipo de seguridad
Usado	Equipo de seguridad y requisitos	Equipo de seguridad y requisitos
Actor externo	Atacantes y usuarios	Usuarios
Conducido por	Vulnerabilidades de activos y amenazas	Casos de abuso

Tabla 2. Diferencias entre los casos de uso de seguridad y los casos de abuso. Fuente: adaptado de Donald, 2003.

Caso de uso de seguridad y caso de abuso

A continuación, presentamos un ejemplo de un diagrama de casos de uso de seguridad y sus casos de abuso asociados en su versión gráfica. La siguiente figura (Sindre y Opdahl, 2001) muestra un caso de abuso para un caso de comercio electrónico que presenta varias relaciones:

- **Relación caso abuso:** uso seguridad, «incluye» y «extiende».
- **Relación caso de uso:** abuso, «previene» y «detecta».

Los **casos uso** resultantes de especificar los requisitos de seguridad protegen al cajero automático y a sus usuarios de las amenazas potencialmente realizables por un cibercriminal.

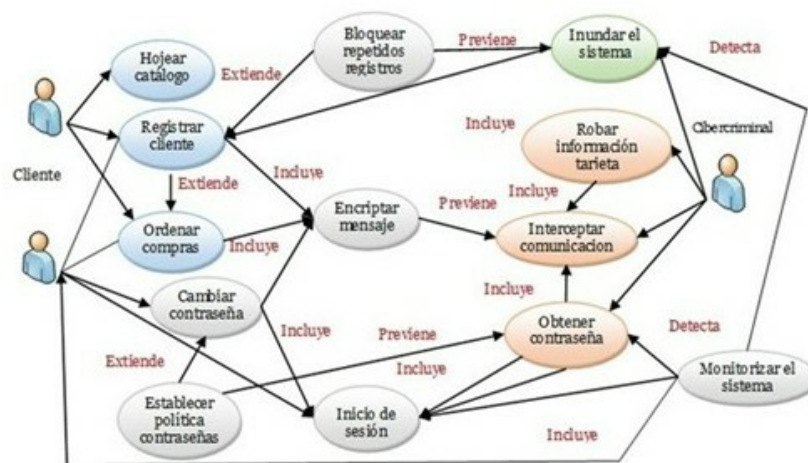


Figura 6. Ejemplo caso de uso de seguridad y caso de abuso. Fuente: adaptado de Sindre y Opdahl, 2001.

La siguiente tabla expone un ejemplo de especificación del caso de abuso **obtener contraseña** del diagrama de casos de uso y abuso de la figura anterior.

Campos	Descripción caso de abuso: «obtener contraseña»
Condiciones iniciales	- Pc1 El sistema dispone de un usuario «operador» con privilegios de administrador. - Pc2 El sistema permite al operador iniciar su sesión a través de la red.
Suposiciones	- As1 El operador utiliza la red para iniciar sesión en el sistema como operador (por todos los caminos.) - As2 El operador utiliza un acceso a Internet de su domicilio para iniciar sesión en el sistema como operador (por ap3.)
Peor caso amenaza (postcondición)	Wc1 El ciberatacante tiene privilegios de operador en el sistema de comercio electrónico por un tiempo ilimitado, es decir, nunca lo pierde
Garantía de captura (postcondición)	Cg1 El ciberatacante nunca obtiene privilegios de operador en el sistema de comercio electrónico
Normas relacionadas con el negocio	- Br1 El rol de operador del sistema de comercio electrónico dará privilegios completos en el equipo y los correspondientes equipos locales de la red. - Br2 solo el rol de operador del sistema de comercio electrónico dará los privilegios mencionados en Br1.
Potencial perfil del cibercriminal	Altamente cualificados, potencial administrador con intención criminal.
Acciones y amenaza	- Sh1 comercio electrónico - Posibles pérdidas si el cibercriminal consigue acceso como operador y sabotea el sistema - Pérdida de confianza si los problemas de seguridad se publican - Sh2 cliente - Pérdida de la privacidad si el cibercriminal utiliza el acceso como operador para obtener información sobre los hábitos de compra del cliente - Pérdida de potencial económico si el cibercriminal utiliza el acceso como operador para encontrar números de tarjetas de crédito
Alcance	Todo el negocio y el entorno empresarial
Nivel de abstracción	Meta del cibercriminal

Tabla 3. Especificación caso de abuso Obtener Contraseña. Fuente: elaboración propia.

2.5. Modelado de ataques

El principal **objetivo** de la seguridad del software es el **mantener sus propiedades de seguridad** frente a los ataques realizados por personal malicioso sobre sus componentes y reducir al mínimo posible sus vulnerabilidades explotables. Para conseguir que el desarrollo de una aplicación posea las propiedades y los principios de diseño del software seguro, se necesita que el personal de diseño y desarrollo generen dos perspectivas:

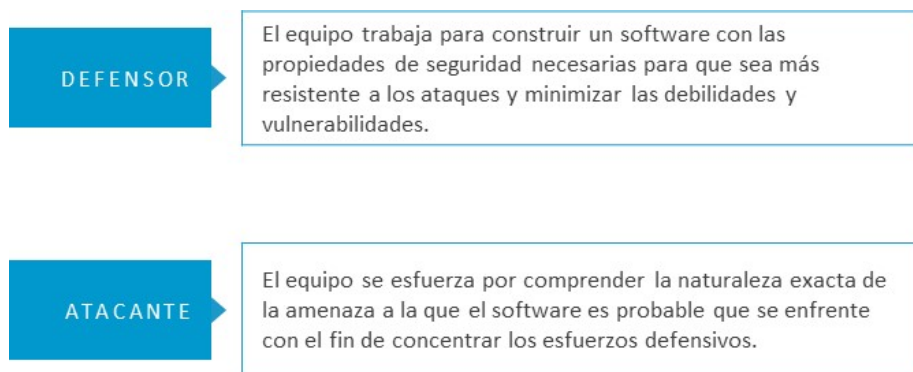


Figura 7. Perspectivas de modelado. Fuente: elaboración propia.

La **perspectiva del atacante** se suele modelar de las siguientes dos formas:

- ▶ Patrones de ataque.
- ▶ Árboles de ataque.

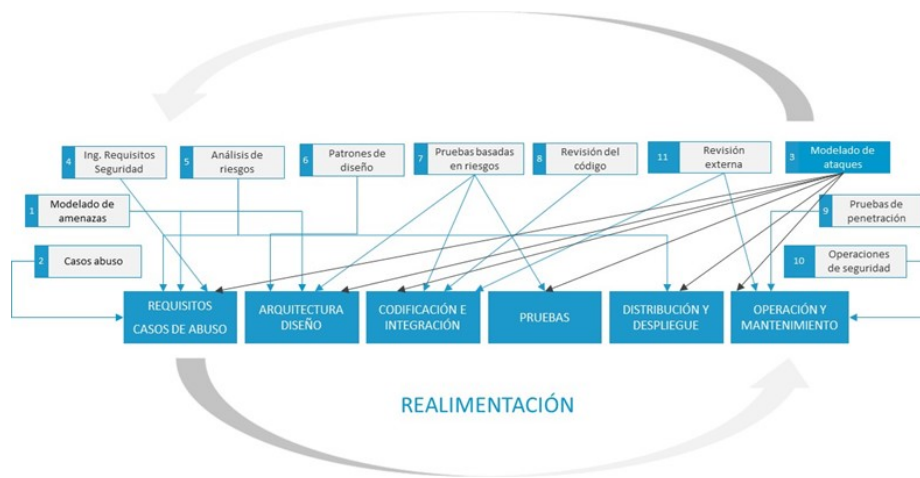


Figura 8. Modelado de ataques. Fuente: elaboración propia.

El uso combinado de patrones de ataque y de árboles de ataque captura la probabilidad de **cómo los ataques se pueden combinar y secuenciar**, esto proporciona una serie de datos que nos permiten ayudar a diseñar, en el software, una serie de respuestas encaminadas a mitigarlos.

Patrones de ataque

Un ataque aprovecha una **vulnerabilidad** de una aplicación mediante un *exploit* para obtener un beneficio del sistema como escalada de privilegios, robo y modificación de datos, modificaciones del funcionamiento, denegación de servicio, etc.

Un **patrón de ataque** se puede definir como:

Mecanismo o medio para capturar y representar la perspectiva y el conocimiento del ciberatacante con el suficiente detalle acerca de cómo los ataques llevan a cabo los métodos más frecuentes de explotación (*exploit*) y las técnicas usadas para comprometer el software.

En definitiva, constituyen una clasificación de los ataques y una **representación estructurada** del pensamiento del atacante para que el equipo de diseño y

desarrollo obtenga el conocimiento y los pasos necesarios a realizar para mitigar con mayor probabilidad las acciones de los ciberatacantes, reducir su impacto y seleccionar las políticas de seguridad más convenientes. Derivan del concepto de **patrones de diseño** aplicado en un contexto más destructivo que constructivo y están generados a partir de un análisis en profundidad de determinados ejemplos de *exploits* del mundo real.

Un catálogo de patrones de ataques, que proporciona un conjunto de definiciones comunes, una taxonomía de clasificación, un esquema de patrones de ataque y un conjunto de ellos, reales, lo constituye la iniciativa del MITRE **Common Attack Pattern Enumeration and Classification (CAPEC)**. Actualmente, incluye 519 patrones reales de ataque.

Accede al catálogo a través del aula virtual o desde la siguiente dirección web:

<http://capec.mitre.org>

Dependiendo del nivel de detalle que describe y su nivel de abstracción, los patrones de ataque constituyen un recurso que proporciona **valor** al conjunto del SDLC, dado que tiene diferentes utilidades en cada una de sus fases

En la siguiente tabla se muestra un resumen de los usos de los patrones de ataques en las diferentes fases del SDLC.

Fase	Utilidad
Especificación de requisitos	▶ Ayuda a identificar requisitos. ▶ Ayuda a identificar los riesgos a los que hará frente el software. ▶ Ayuda a definir el comportamiento del sistema para prevenir o reaccionar ante un tipo específico de ataque probable.
Diseño y arquitectura	▶ Proporciona ejemplos de ataques que aprovechan las vulnerabilidades de la arquitectura elegida. ▶ Proporcionan el contexto de las amenazas al que el software se va a enfrentar, de forma que permite diseñar arquitecturas seguras.
Codificación	▶ Específicas debilidades aprovechadas por los ataques, orientando qué técnicas o prácticas de seguridad de desarrollo evitan estas deficiencias.
Pruebas	▶ Pueden ser utilizados para orientar las pruebas de seguridad del software en un contexto práctico y realista identificando debilidades concretas para generar casos de prueba para cada componente.
Operación	▶ Permitirá la selección de políticas de seguridad y configuraciones acordes a las amenazas obtenidas de los patrones de ataque.

Tabla 4. Aplicación patrones de ataque a las diferentes fases del SDLC. Fuente: elaboración propia.

Un **patrón de ataque**, como mínimo, debe describir ampliamente el incidente, las habilidades y recursos necesarios para ejecutarlo con éxito, en qué contexto es aplicable y proporcionar información suficiente para permitir a los defensores evitar o mitigar eficazmente las acciones del atacante.

En la tabla siguiente se muestran los ítems de información de los que consta un patrón de ataque obtenido de CAPEC.

Ítem	Descripción
Nombre	► Identificador descriptivo del patrón de ataque.
Severidad	► En una escala aproximada típica (muy bajo, bajo, medio, alto, muy alto) de la gravedad del ataque.
Descripción	► Descripción detallada del ataque incluyendo la cadena de acciones tomadas por el atacante. Podría incluir árboles de ataque.
Prerrequisitos del ataque	► Describe las condiciones que deben existir, funcionalidades, características del software y comportamiento que debe exhibir para que un ataque de este tipo tenga éxito.
Probabilidad típica del <i>exploit</i>	► Es una escala aproximada (Muy Bajo, Medio Bajo, Alto, Muy Alto). Este campo se utiliza para capturar un valor global promedio típico para este tipo de ataque.
Métodos de ataque	► Describe el mecanismo de ataque utilizado por este patrón. Este campo puede ayudar a definir la superficie de ataque requerida por este ataque.
Ejemplos	► Contiene ejemplos explicativos o casos demostrativos de este ataque.
Conocimiento y habilidades requeridas del atacante	► Describe el nivel de habilidad o conocimiento específico requerido por un agente malicioso. Se codifica con una escala aproximada (bajo, medio, alto), así como un detalle de contexto.
Recursos requeridos	► Este campo describe los recursos (ciclos de CPU, direcciones IP, herramientas, etc.) requeridos por un atacante para ejecutar con eficacia este tipo de ataque.
Métodos de prueba	► Describe las técnicas normalmente utilizadas para investigar y reconocer un blanco potencial, determinar la vulnerabilidad y prepararse para este tipo de ataque.
Indicadores de un ataque	► Describe las actividades, eventos, condiciones o comportamientos que podrían servir como indicadores de que un ataque de este tipo es inminente, está en progreso o ha ocurrido.
Soluciones y mitigaciones	► Describe acciones o enfoques que pueden potencialmente prevenir o mitigar el riesgo de este tipo de ataque.

Ítem	Descripción
Motivación y consecuencias del ataque	► Comprende una selección de una lista enumerada de motivaciones definidas o consecuencias. Esta información es útil para la alineación de patrones de ataque a los modelos de amenaza.
Descripción del contexto	► Describe el contexto en el que este patrón es relevante y aporta más antecedentes para el ataque.
Vector de inyección	► Describe, lo más exactamente posible, el mecanismo y el formato de un ataque.
Payload	► Describe los datos de código, configuración u otros a ejecutar o activar, como parte de un ataque del vector de inyección.
Zona de activación	► Describe el área dentro del software de destino que es capaz de ejecutar o activar de otro modo la carga útil del vector inyección de un ataque de este tipo. Puede ser un intérprete de comandos, un código de máquina activa en un buffer, un navegador cliente, una llamada API del sistema, etc.
Impacto de activación del payload	► Descripción de los efectos que la activación del <i>payload</i> en la confidencialidad, integridad o disponibilidad del software solicitado.
Impacto CIA	► Describe el impacto de este patrón de ataque en las características de seguridad estándar confidencialidad, integridad y disponibilidad
Vulnerabilidades relacionadas	► Qué vulnerabilidades o debilidades puede el ataque explotar. Se referencia estándar de la industria CWE.
Requisitos de seguridad relevantes	► Identifica los requisitos de seguridad específicos que son relevantes para este tipo de ataques.
Principios de seguridad relacionados	► Identifica los principios de diseño de seguridad que son relevantes para identificar o mitigar este tipo de ataques.
Guías relacionadas	► Identifica las directrices de seguridad existentes que son relevantes para la identificación o mitigar este tipo de ataque.
Referencias	► Identifica las directrices de seguridad existentes que son relevantes para la identificación o mitigar este tipo de ataque.

Tabla 5. Alcance de información de un patrón de ataque. Fuente: elaboración propia

Árboles de ataque

Un árbol de ataque se puede definir como un método sistemático para caracterizar la seguridad de un sistema, basado en la combinación y dependencias de las vulnerabilidades de este, que un atacante puede aprovechar para comprometerlo.

En un árbol de ataque, el **objetivo del atacante** se coloca en la parte superior del árbol, documentándose las posibles alternativas de ataque en los diferentes recorridos del árbol por los nodos de nivel inferior que contienen objetivos

intermedios hasta llegar al nivel inferior que contiene los diferentes métodos o **técnicas de ataque**. Cada camino a través de un árbol de ataque representa un tipo de ataque único. Para cada alternativa se puede añadir recursivamente otras precursoras que alcancen diversos objetivos parciales que, en conjunto, logran el objetivo principal del atacante.

Mediante el examen de diferentes ramas del árbol de ataque, el analista puede identificar **todas las posibles técnicas o métodos** que podrían ser utilizados para comprometer la seguridad del sistema. Básicamente:

- ▶ Captura los **pasos realizados** en ataques con éxito.
- ▶ Captura **métodos generales y específicos** del sistema de ataque, propiedades del sistema y otras condiciones previas que lo hacen posible.
- ▶ Se enfoca en las **causas de las vulnerabilidades**, pero no identifica las contramedidas.
- ▶ Representan de forma **gráfica y textual** directrices de codificación segura y patrones de seguridad.

La representación de la **estructura de un árbol de ataque** se realiza conforme a los dos siguientes tipos:

- ▶ **Textual:** sigue una estructura de esquema numérico, el nodo raíz, o la meta, representada en el **primer nivel** sin sangría y cada objetivo de nivel inferior se enumeran con sangría de una unidad por nivel de descomposición:

Objetivo: Falsificar una Reserva de vuelos

1. Convencer al empleado de agregar una reserva falsa
 - 1.1 Chantaje empleado
 - 1.2 Amenazar empleado
2. Acceder y modificar la base de datos de vuelos
 - 2.1 Realizar una inyección SQL en la página web
 - 2.2 Iniciar una sesión en la base de datos
 - 2.2.1 Adivinar la contraseña
 - 2.2.2 Obtener la contraseña rastreando la red (sniff)
 - 2.2.3 Robar la contraseña del servidor
 - 2.2.3.1 Obtener una cuenta del servidor (AND)
 - 2.2.3.1.1 Desbordamiento de búfer
 - 2.2.3.1.2 Obtener acceso cuenta empleado
 - 2.2.3.2 Explotar condición de carrera acceso perfil protegido

Figura 9. Estructura textual de un árbol de ataque. Fuente: elaboración propia.

- **Grafica o semántica:** se construye, generalmente, con el nodo raíz, o la meta, en la **parte superior**, al descender por las ramas del árbol, se obtienen subobjetivos hasta que se llega al nivel inferior donde están los métodos de ataque. Un nodo se descompone como:

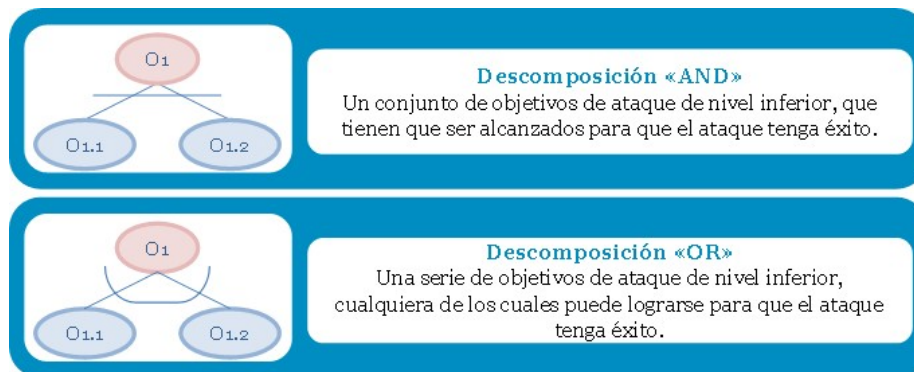


Figura 10. Aplicación patrones de ataque a las diferentes fases del SDLC. Fuente: elaboración propia.

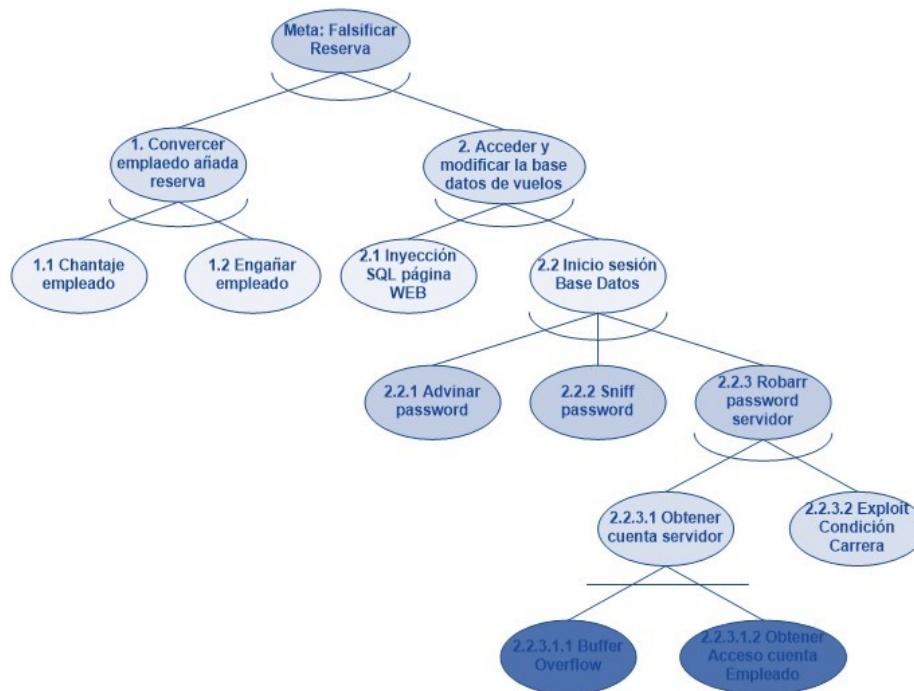


Figura 11. Vista conceptual de un árbol de ataque. Fuente: adaptado de Redwine, S. T. Jr., 2006.

2.6. Ingeniería de requisitos de seguridad

Gran parte de las vulnerabilidades y debilidades del software tienen su origen en unos requisitos inadecuados, inexactos e incompletos, principalmente debido a la falta o debilidad de la especificación de estos, que no determinan las funciones, restricciones y propiedades no funcionales del software que hacen que este sea previsible, confiable y resistente.

Los defectos en los requisitos cuestan de 10 a 200 veces más de corregir durante la **ejecución** que si se detectan durante su **especificación**. Además, es difícil y costoso mejorar significativamente la seguridad de una aplicación después de que esté en su entorno de producción.

La **fase de ingeniería** de requisitos cubre todas las actividades y tareas que deben realizarse antes de iniciar el diseño, su principal resultado es la especificación de los requisitos que definen los aspectos funcionales y no funcionales del software.

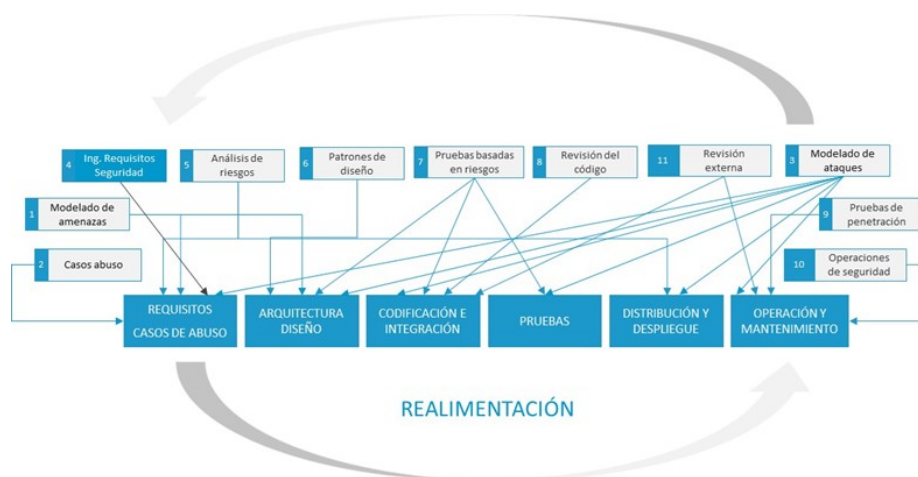


Figura 12. Ingeniería de requisitos. Fuente: elaboración propia.

Los requisitos de seguridad deben ser considerados simultáneamente junto con los demás requisitos, incluidos los relativos a funcionalidad, rendimiento y facilidad de uso. Ya que los requisitos de seguridad, a menudo, entran en conflicto con ellos, se hace necesario, por lo tanto, la integración satisfactoria de la ingeniería de requisitos de seguridad con la ingeniería de requisitos en su conjunto.

En la siguiente figura se muestra un alcance completo de los requisitos de seguridad de una aplicación:



Figura 13. Requisitos de seguridad. Fuente: elaboración propia.

- **Requisitos servicios de seguridad (funcionales o positivos).** Incluye la especificación de funciones que implementan una política de seguridad, como control de acceso, autenticación, autorización, cifrado y gestión de claves. Deben especificar:
 - **Propiedades que el software debe exhibir**, por ejemplo: el software debe tener un comportamiento correcto y predecible y disponer de capacidades de recuperación frente a ciberataques.

- **Nivel requerido de seguridad y salvaguardas de riesgo** de las funciones de seguridad.
- ▶ **Requisitos de software seguro (no funcionales o negativos).** Requisitos que afectan directamente a la probabilidad de que el software sea seguro. Estos abarcan, principalmente, los no funcionales, los que garantizan que el sistema seguirá siendo confiable, incluso cuando esa confianza se vea amenazada.
- ▶ **Operacionales.** Mas enfocados a los controles y normas que rigen los procesos de desarrollo, implementación y operación del software.

Es fundamental que los **requisitos de seguridad** del software sean:

- ▶ Completos.
- ▶ Precisos.
- ▶ Coherentes.
- ▶ Trazables.
- ▶ Verificables.

Teniendo en cuenta las prácticas de seguridad anteriormente presentadas, los pasos a realizar para la especificación de los requisitos de seguridad se explican a continuación:

- ▶ Dibujar **DFD** para el sistema que implementa los casos de uso.
- ▶ Identificar las **amenazas potenciales** para cada elemento del DFD mediante la metodología de modelado de amenazas.
- ▶ Identificar los **casos de uso** basados en las amenazas identificadas. Solo se identifican los nombres de los casos de abuso y el objetivo de cada caso de abuso.

- **Mapear las amenazas** obtenidas con los patrones de ataque de CAPEC, así como otras fuentes de patrones de ataque. Proporcionará información para la especificación de los caos de uso y valoración del riesgo de las amenazas
- **Usar los patrones de ataque recuperados** para especificar los casos de abuso identificados.
- Especificar los requisitos de seguridad funcionales en base a las **salvaguardas de seguridad** necesarias para mitigar las amenazas; y los no funcionales, en base a los patrones de ataque y caso de buso.

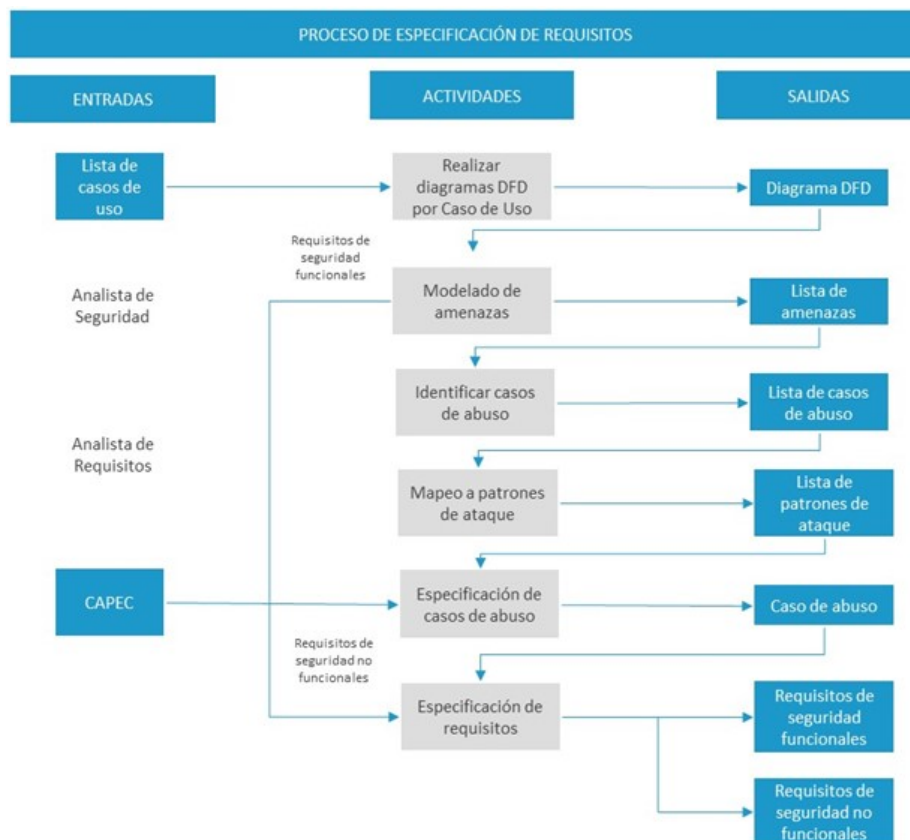


Figura 14. Proceso de especificación de requisitos de seguridad. Fuente: elaboración propia.

2.7. Análisis de riesgo arquitectónico

Según estudios y estadísticas disponibles, se sabe que, aproximadamente, un 50 % de los defectos de seguridad se deben a errores en la fase de diseño denominados **debilidades** o **flaws**, en el idioma inglés.

Identificando el riesgo, y después con su gestión, se pueden mitigar gran parte de los problemas de seguridad de un software.

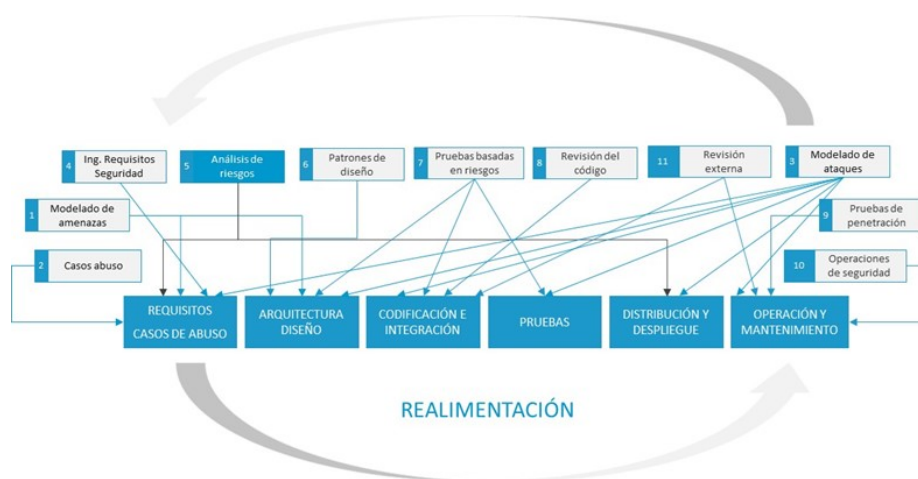


Figura 15. Análisis de riesgos. Fuente: elaboración propia.

En la figura anterior se puede observar que el análisis de riesgos está a caballo entre la **fase análisis de requisitos**, donde se obtienen los requisitos de seguridad, se modelan ataques y realizan casos de abuso, y la fase de la **fase de arquitectura y diseño**. Sigue en importancia a la revisión de código, si bien este hecho puede variar dependiendo de las características de la organización, de sus tipos de sistemas, cantidad de software, etc.

En la fase de análisis, al obtener los requisitos de seguridad se han llegado a bastantes conclusiones acerca de los activos del sistema y del impacto que los

posibles ataques pueden causar, pero es en la **fase de diseño de la arquitectura del sistema** donde se especifican todos los activos, porque se obtienen todos los componentes hardware y software del sistema, se configura la arquitectura y se deciden las salvaguardas concretas que van a protegerlo en función de los requisitos de seguridad y los casos de abuso.

Proceso análisis de riesgo arquitectónico

Cigital usa el **proceso de análisis de riesgo arquitectónico** que desarrolla un acercamiento a un análisis de riesgo que implica tres pasos básicos.

Modelo de Cigital, Building Security In:

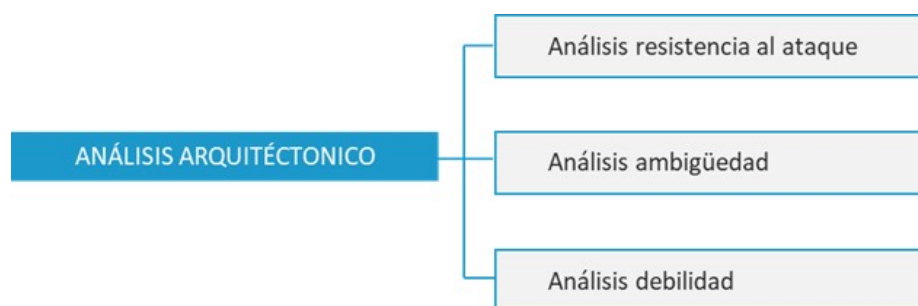


Figura 16. Fases análisis de riesgo arquitectónico. Fuente: elaboración propia.

- ▶ **Análisis de resistencia al ataque.** Realiza un modelado de amenazas del diseño completo de la aplicación para analizar la resistencia al ataque. Para ello comprueba una **lista de categorías de riesgos** según el modelo de análisis de amenazas de Microsoft STRIDE.
- ▶ **Análisis de ambigüedad.** Es un subproceso que pretende descubrir nuevos riesgos en base al conocimiento de principios de diseño. Aprovecha múltiples puntos de vista de la **arquitectura de software**, de varios arquitectos, para crear una **técnica de análisis crítica** con el fin de encontrar nuevos defectos, puntos de conflicto y de inconsistencia.

- ▶ **Análisis de debilidad.** Es un subproceso que ayuda al entendimiento del impacto de dependencias del software externo.

2.8. Patrones de diseño

Según se define en el documento de referencias (McGraw, 2005), los patrones de diseño son:

«Una solución general repetible a un problema de ingeniería de software recurrente, que está expresamente destinado a contribuir al diseño de software menos vulnerable, más resistente y tolerante a los ataques» (McGraw, 2005).

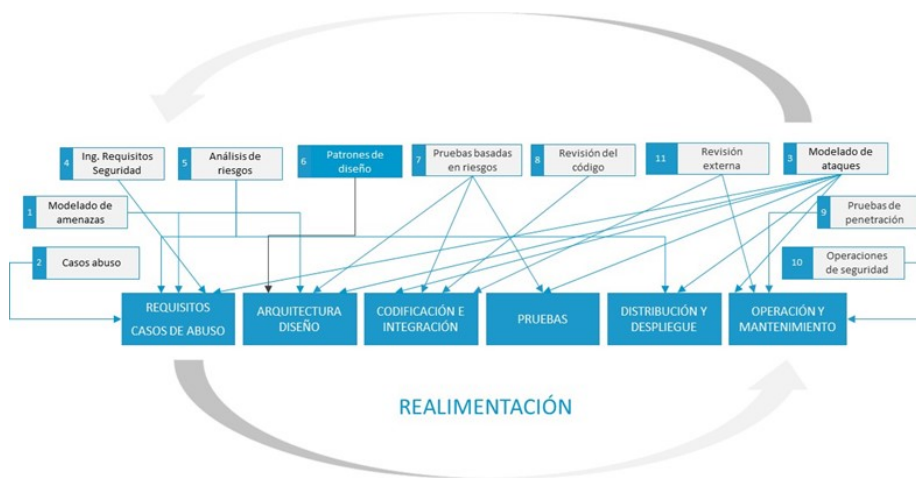


Figura 17. Patrones de diseño. Fuente: elaboración propia.

El concepto de **patrones** es más familiar en **ingeniería de software**, relacionados principalmente con la arquitectura de los sistemas o soluciones para el diseño de estos. Se han desarrollado patrones de diseño de seguridad para casi **todas las fases del SDLC**: requerimientos, análisis, diseño, codificación, etc. El uso adecuado de estos patrones conduce a la **remediación** de los principales **fallos de seguridad**, como, por ejemplo, uno que tuvo un efecto significativo fue un **validador** que construía un modelo de validación de entrada para defenderse contra los ataques de inyección SQL; otros ejemplos son los contruidos para los **lenguajes de programación** en **C** y / o **C++**.

2.9. Referencias bibliográficas

Donald, G. (2003). *Software Engineering Institute, U.S.A. Security Use Cases*. Journal of Object Technology.

Sindre, G., & Opdahl, A. L. (2001). *Capturing security requirements through misuse cases*. NIK 2001, Norsk Informatikkonferanse 2001. <http://www.nik.no/2001>.

Hoglund. (2004). *Exploiting Software: How to Break Code*, By G. Hoglund and G. McGraw. Addison-Wesley Software Security Series.

McGraw, G. (2005). *Software Security: Building Security In*. Addison Wesley Professional.

Redwine Jr., S. T. (Editor). (2006). *Software Assurance: A Guide to the Common Body of Knowledge to Produce, Acquire, and Sustain Secure Software Version 1.1*. US Department of Homeland Security.

Sroka, D. *Presentación HP Fortify User Training*.

Yuan, Xiaohong y Nuakoh, Emmanuel y Williams, Imano y Yu, Huiming. (2015). Developing Abuse Cases Based on Threat Modeling and Attack Patterns. Journal of Software, 10, 491-498. 10.17706/jsw.10.4.491-498.

Implementación simplificada del proceso SDL

Microsoft (2010). *Implementación simplificada del proceso SDL de Microsoft*.

Microsoft. <http://www.microsoft.com/en-us/download/details.aspx?id=12379>

El documento describe los conceptos básicos del Microsoft Security Development Lifecycle (SDL) y las actividades de seguridad individuales en cada una de sus fases.

Secure Development Lifecycles (SDLC): Introduction and Process Models - Bart De Win

secappdev.org. (2017, marzo 10). *Secure Development Lifecycles (SDLC): Introduction and Process Models - Bart De Win* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=L-gL1YQUrwg>

En este vídeo se expone una visión general de los modelos SDLC de última generación con el fin de discutir los fundamentos y las piedras angulares de estos modelos. Esto ayudará a comprender su alcance y los diferentes conceptos. También se desarrolla la perspectiva de los modelos de cascada y de desarrollo ágil.

Threat modeling tool demo

Blackhawk-Technical-College-IT-Web-Software-Developer (s. f.). *Secure Code - Day05 - Presentation02 Threat Modeling Demo* [Vídeo]. Youtube. <https://www.youtube.com/watch?v=7HUOk8BhYW0>

Presentación de las nuevas características de la herramienta de modelado de amenazas Microsoft Modeling Tool de Microsoft.

Threat modeling

secappdev.org (2014, agosto 6). *Threat Modeling - Jim DelGrosso* [Vídeo]. Youtube.
<https://www.youtube.com/watch?v=We2cy8JwVqc>

Vídeo que proporciona una descripción del proceso de modelo de amenazas que utiliza Cigital mediante un ejemplo de aplicación del proceso para identificar fallas potenciales en un sistema. Objetivos:

- ▶ Describir terminología utilizada en el modelado de amenazas.
- ▶ Describir un proceso de modelado de amenazas de un sistema.
- ▶ Realizar un modelo de amenazas de algún sistema ficticio.

Introduction, threat models

MIT OpenCourseWare (2015, julio 14). *1. Introduction, Threat Models* [Video]. Youtube. <https://www.youtube.com/watch?v=GqmQg-cszw4&feature=youtu.be>

Clase del MIT Open Course Ware del Profesor Nickolai Zeldovich acerca del modelado de amenazas.

1. Señala la respuesta correcta. ¿Qué incluye la seguridad del *software*?
 - A. Patrones de codificación.
 - B. Principios de diseño seguro.
 - C. Casos de uso.
 - D. Buenas prácticas de seguridad.

2. Señala la respuesta correcta. Los patrones de ataque en la fase de diseño y arquitectura:
 - A. Ayudan a definir el comportamiento del sistema para prevenir o reaccionar ante otros ataques.
 - B. Específicas debilidades no aprovechadas por los ataques, orientando que técnicas o prácticas de seguridad de desarrollo que evitan estas deficiencias.
 - C. Proporcionan el contexto de las amenazas al que el *software* se va a enfrentar.
 - D. Proporcionan ejemplos de ataques que no aprovechan las vulnerabilidades de la arquitectura elegida.

3. Señala la respuesta correcta. Respecto a los casos de uso de seguridad:
 - A. Analizan y especifican los requisitos de seguridad funcionales.
 - B. Analizan y especifican las amenazas a la seguridad.
 - C. Análisis de vulnerabilidades de activos y amenazas.
 - D. Análisis forense de las actividades maliciosas.

4. Respecto a la ingeniería de requisitos de seguridad, esta NO incluye:
- A. Requisitos de *software* seguro. Requisitos que afectan directamente a la probabilidad de que el *software* sea seguro.
 - B. Requisitos servicios de seguridad.
 - C. Los requerimientos no funcionales de seguridad deben especificar: propiedades que el *software* debe exhibir.
 - D. Los requerimientos no funcionales de seguridad deben especificar: los controles y normas que rigen los procesos de desarrollo, implementación y operación del *software*.
5. ¿Cuál de estas opciones NO forma parte de los tres pasos básicos del análisis de riesgo arquitectónico?
- A. Análisis de ambigüedad.
 - B. Análisis de resistencia al ataque.
 - C. Análisis de robustez.
 - D. Análisis de debilidad.
6. ¿Cuál de las siguientes respuestas es una de las perspectivas que el equipo de desarrollo tiene que adoptar a la hora de diseñar unas pruebas de seguridad basadas en el riesgo?:
- A. Perspectiva gerencia.
 - B. Perspectiva atacante.
 - C. Perspectiva usuario.
 - D. Perspectiva del analista.

7. ¿Cuál de estas opciones NO es una razón principal para añadir prácticas de seguridad en el SDLC?

- A. Mayor probabilidad de capturar adecuadamente los requisitos.
- B. Mayor probabilidad de tomar decisiones de diseño correctas y no cometer errores involuntarios de codificación.
- C. Dificultad de los agentes maliciosos para explotar vulnerabilidades y debilidades del *software*.
- D. Mayor probabilidad de que el *software* funcione correctamente.

8. Señala la respuesta correcta. Un método sistemático para caracterizar la seguridad de un sistema, basado en la combinación y dependencias de las vulnerabilidades de este, que un atacante puede aprovechar para comprometerlo.

- A. Método de ataque.
- B. Patrones de ataque.
- C. Árboles de ataque.
- D. Modelo de ataque.

9. Señalar la respuesta incorrecta sobre los casos de abuso.
- A. Los casos de abuso constituyen unas buenas prácticas para la obtención de los requisitos funcionales de una aplicación, referidos a actividades que debería realizar el sistema.
 - B. Un caso de abuso es la inversa de un caso de uso, es decir, una función que el sistema no debe permitir o una secuencia completa de acciones que resulta en una pérdida para la organización.
 - C. Los casos de abuso, o casos de mal uso, son un instrumento que puede ayudar a pensar de la misma forma que lo hacen los atacantes
 - D. Establecen la base para otros casos de uso de seguridad que proporcionan los medios para contrarrestar o mitigar las amenazas capturadas en los mismos y una manera altamente reutilizable de organizar, analizar y especificar los requisitos de seguridad
10. Indica en qué fase del ciclo de vida de desarrollo del software NO es aplicable el análisis de riesgo arquitectónico.
- A. Especificación de requisitos.
 - B. Diseño del sistema.
 - C. Codificación.
 - D. Operación.

Desarrollo Seguro de Software y Auditoría de la
Ciberseguridad

Tema 3. Seguridad en el ciclo de vida del software en las fases de codificación, pruebas y operación

Índice

Esquema

Ideas clave

- 3.1. Introducción y objetivos
- 3.2. Pruebas de seguridad basadas en riesgo
- 3.3. Revisión de código
- 3.4. Pruebas de penetración
- 3.5. Operaciones de seguridad
- 3.6. Revisión externa
- 3.7. Referencias bibliográficas

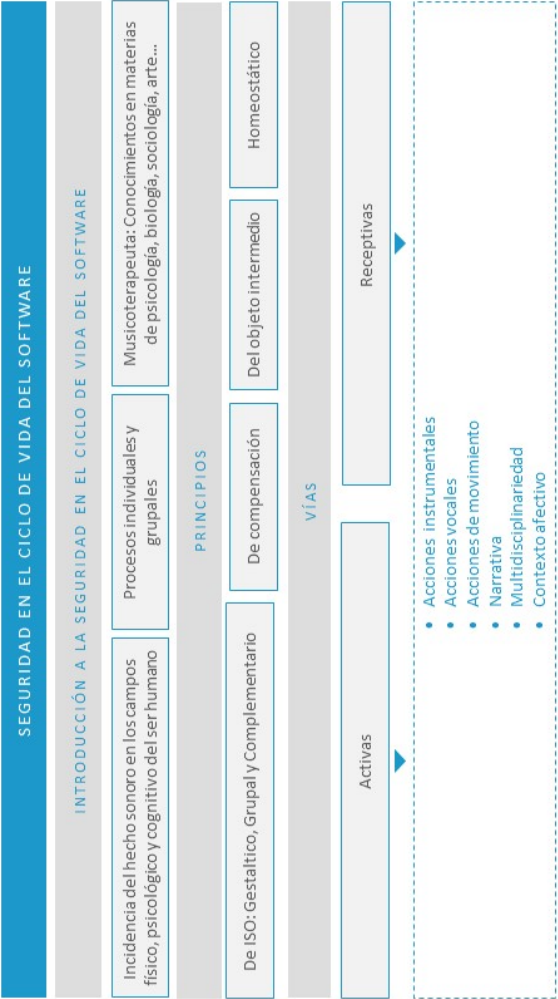
A fondo

A systemic approach for assessing software supply-chain risk

Hybrid Security Assessment Methodology for Web Applications

A Few Billion Lines of Code Later

Test



3.1. Introducción y objetivos

Para conseguir **software confiable** y minimizar al máximo los ataques en la capa de aplicación, es necesario seguir un proceso sistemático o disciplina que aborde la seguridad en todas las etapas del ciclo de vida de desarrollo del software que incluya una serie de buenas prácticas de seguridad (S-SDLC) como especificación, requisitos de seguridad, casos de abuso, análisis de riesgo, análisis de código, pruebas de penetración dinámicas, modelado de amenazas, operaciones de seguridad y revisiones externas, necesarias para asegurar la confianza y robustez del mismo.

Normalmente, el modelo más propicio para el desarrollo de software en las organizaciones suele ser una combinación de dos o más modelos de los ciclos de vida (cascada, espiral, etc.). Sin embargo, hay que tener en cuenta que lo importante no es el modelo seguido, sino la **incorporación de buenas prácticas de seguridad** en las diferentes fases de este y unos hitos o puntos de control donde se verifiquen los entregables de cada fase. Entre las razones principales para añadir prácticas de seguridad en el SDLC tenemos:

- ▶ Mayor probabilidad de capturar adecuadamente los requisitos y tomar decisiones de diseño correctas y no cometer errores involuntarios de codificación.
- ▶ Dificultad de los agentes maliciosos para explotar vulnerabilidades y debilidades del software, pues el resultante de un ciclo de vida S-SDLC será más robusto en su entorno de ejecución y en su interacción con entidades externas.

Los **objetivos** del presente tema son los siguientes:

- ▶ Conocimiento y comprensión de las buenas prácticas de seguridad a incluir en un S-SDLC en las fases de **codificación, pruebas y operación**.

- ▶ Profundizar en el estudio de la principal **práctica de seguridad: revisión de código**.
- ▶ Estudiar las prácticas de seguridad aplicables en estas **fases del SDLC**, como pruebas de penetración, operaciones de seguridad, revisión externa y pruebas de seguridad basadas en riesgo.

3.2. Pruebas de seguridad basadas en riesgo

Hasta hace unos pocos años, las pruebas de software se desarrollaban en base a la demostración de sus requisitos como forma de comprobar el correcto funcionamiento de sus funcionalidades y servicios de seguridad; sin embargo, no sirven para determinar cómo se comportará en condiciones anómalas y hostiles, ni si está libre de vulnerabilidades.

Identificando los riesgos del sistema y diseñando las pruebas en base a ellos bajo la perspectiva de un atacante, un probador de seguridad de software puede enfocar correctamente las áreas de código donde un ataque probablemente pudiera tener éxito.

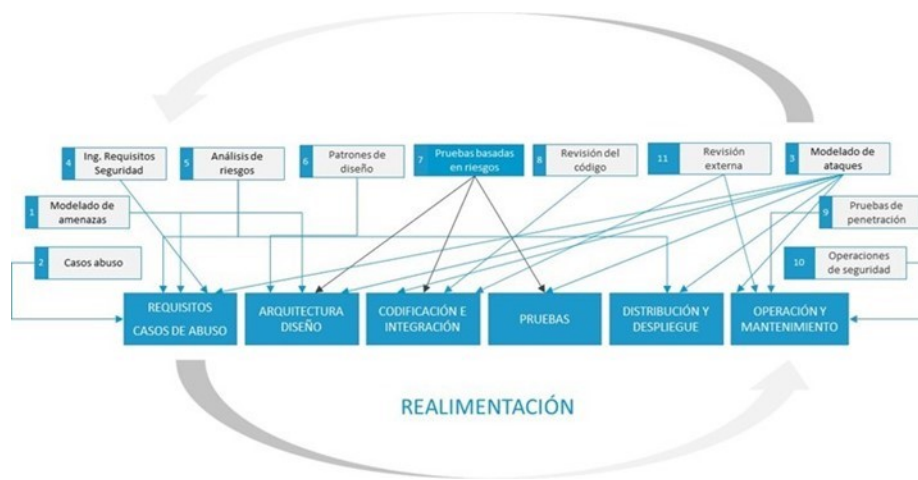


Figura 1. Pruebas seguridad basadas en riesgo. Fuente: elaboración propia.

Aunque los componentes de seguridad, como la criptografía, la autenticación y el control de acceso, jueguen un papel crítico en la seguridad del software, la seguridad en sí misma es una característica de la totalidad del sistema; por lo tanto, no se refiere solamente a los mecanismos y elementos de seguridad. Un **desbordamiento de buffer** es un problema de seguridad independientemente de si existe un

componente de seguridad, o en un interfaz hombre máquina no crítico. Por esta razón, es necesario que las pruebas de seguridad impliquen dos tipos de aproximaciones:

TIPOS DE PRUEBAS SEGURIDAD DEL SOFTWARE

PRUEBAS DE SEGURIDAD FUNCIONALES	¿Cuál es el daño que puede originar la vulnerabilidad si llega a ser explotadas?
PRUEBAS DE SEGURIDAD PERSPECTIVA ATACANTE	¿Es fácil reproducir las condiciones que propicien el ataque?

Figura 2. Tipos de pruebas seguridad del software. Fuente: elaboración propia.

Los **objetivos de las pruebas de seguridad basadas en el riesgo** son los siguientes:

- ▶ Verificar la operación confiable del software bajo condiciones hostiles de ataque.
- ▶ Verificar la fiabilidad del software, en términos de comportamiento seguro y cambios de estado confiables.
- ▶ Verificar la falta de defectos y debilidades explotables.
- ▶ Verificar la capacidad de supervivencia del software ante la aparición de anomalías, errores, y el manejo de estas mediante excepciones, que minimicen el alcance e impacto de los daños que puedan resultar de los ataques.

Las **pruebas de seguridad** deberían comenzar a nivel de componente antes de la integración del sistema. Se deben de utilizar los modelos de ataque, casos de abuso y análisis de riesgos desarrollados al comienzo del ciclo de vida, para mejorar el plan de pruebas con casos diseñados desde el punto de vista de los atacantes basados en los escenarios de abuso. Esto implica tanto pruebas de caja negra como de caja blanca.

Técnicas de pruebas que son particularmente útiles para las pruebas basadas en riesgo incluyen las mostradas en el siguiente diagrama:

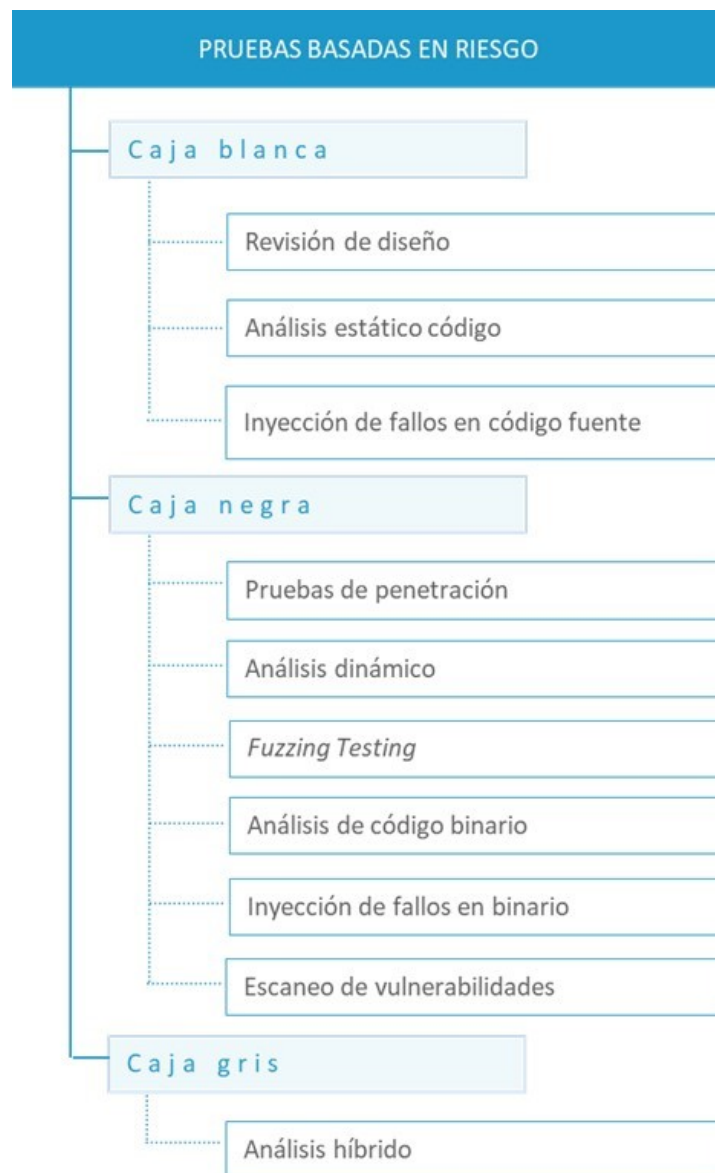


Figura 3. Tipos de pruebas seguridad basadas en riesgo. Fuente: elaboración propia.

El **análisis de caja blanca** implica el análisis y el entendimiento tanto de código fuente como del diseño. Esta clase de pruebas es muy eficaz en el hallazgo de

errores de programación, bugs, y debilidades de diseño, *flaws*. Las exploraciones de código pueden automatizarse con un **analizador estático**. Una desventaja de esta clase de pruebas consiste en que se podría encontrar una vulnerabilidad potencial donde en realidad existe un **falso positivo**. Hay que revisar el resultado de las herramientas de análisis estático para identificarlos.

PUNTOS FUERTES	PUNTOS DÉBILES
La vulnerabilidad se reporta en la línea de código fuente.	Falsos positivos si no se configura bien la herramienta.
Los fallos se pueden encontrar aunque no estuvieran accesibles en ejecución.	Acceder al código fuente no es siempre posible.
Cobertura total si se tiene acceso a todo el código de la aplicación.	Algunas debilidades son solo expuestas al desplegar la aplicación.
Imprescindible para corregir, lo más temprano posible, la gran mayoría de los defectos de seguridad de las aplicaciones.	El <i>software</i> no es fácil de probar si se trata de pruebas remotas.

Figura 4. Puntos fuertes y débiles pruebas caja blanca. Fuente: adaptado de López, S. (2012).

- Revisión de diseño.** Las vulnerabilidades de nivel de diseño son la categoría de defectos más **difícil** de manejar, además son tanto frecuentes como críticos. Lamentablemente, averiguar si un programa tiene vulnerabilidades de nivel de diseño requiere conocimiento y experiencia. Ejemplos de problemas de nivel de diseño incluyen manejo de errores en sistemas orientados a objetos, objetos compartidos, relaciones de confianza, canales de datos sin protección, mecanismos de control de acceso incorrectos, carencia de auditorías, registro (logs) y errores en el ordenamiento de eventos.
- Revisión de código o análisis estático de código.** Se considera una de las prácticas de seguridad más importantes, consiste básicamente en el análisis del código fuente para detectar errores, construcciones inseguras de codificación e indicadores de vulnerabilidades o debilidades.

- **Inyección de fallos en código fuente.** Una forma de análisis dinámico en el que el código fuente se ha «instrumentalizado» mediante la inyección de errores. A continuación, se deberá compilar y ejecutar el código instrumentado para observar los cambios en el estado y el comportamiento que surgen cuando las partes instrumentadas del código se ejecutan. De esta manera, se puede determinar y cuantificar cómo, incluso, el software reacciona cuando es forzado en estados anómalos, tales como los provocados por fallos intencionales. Esta técnica ha demostrado ser particularmente útil para detectar el uso incorrecto de punteros y matrices, y condiciones de carrera.

La **Inyección de fallos** es un complejo proceso de prueba y, por lo tanto, tiende a ser limitada al código que requiere garantía muy alta de seguridad. Para la realización de estas pruebas se utilizan **motores inyección**. Este instrumento usa el navegador para interceptar transacciones y permitir a un analista estudiarlas.

El **análisis de caja negra** se refiere al análisis de un programa en ejecución al que se le sondea con diferentes tipos de entradas. Por tanto, se centra en estructuras de datos, componentes, APIs, estado de programa, etcétera. No usa análisis de código fuente de ninguna clase. La visión obtenida durante el análisis de riesgo arquitectónico es muy útil en la planificación de este tipo de pruebas de seguridad.

PUNTOS FUERTES	PUNTOS DÉBILES
▣ Pruebas «reales» de <i>software</i>: alta confianza en los resultados.	▣ No es posible determinar la cobertura de todo el código.
▣ No es necesario facilitar el código fuente de la aplicación.	▣ Fallos que no se ejecuten en las pruebas no saldrán a la luz
▣ El <i>software</i> es fácilmente probado a través de la red: pruebas de infraestructura.	▣ Requiere desplegar y ejecutar la aplicación.

Figura 5. Puntos fuertes y débiles pruebas caja negra. Fuente: adaptado de López, S., 2012.

- ▶ **Pruebas de penetración.** Su propósito se centra en la determinación de vulnerabilidades internas a un componente o entre ellos, que estén expuestas al acceso externo y si pueden ser explotadas para comprometer el software.

La explotación de las vulnerabilidades encontradas se puede realizar de forma automática con las siguientes herramientas:

Herramientas de explotación de vulnerabilidades:

- ▶ **Metasploit Framework**

- Licencia: comercial / libre
- S.O.: Windows, Linux

- ▶ **Core Impact**

- Licencia: comercial
- S.O.: Windows, Linux

- ▶ **Canvas**

- Licencia: comercial
- S.O.: Windows, Linux

- ▶ **Análisis dinámico (Dynamic Application Security Testing-DAST).** Detectan debilidades y vulnerabilidades de seguridad en aplicaciones web mientras se están **ejecutando**. Emplea técnicas de inyección de fallos en una aplicación, por ejemplo, entrada de datos maliciosos, tal y como se representa en la figura siguiente, para identificar vulnerabilidades comunes, como inyección SQL, *cross-site scripting*, *open redirect*, *command injection*, *path traversal*, etc. También identifica problemas de configuración del servidor, autenticación, inicio de sesiones, etc. Lo más eficiente es

la **realización de pruebas conjuntas SAST y DAST**, pues ayudan a minimizar la cantidad de falsos positivos.



Figura 6. Funcionamiento pruebas Análisis Dinámico (DSAT). Fuente: López, S. (2012).

Herramientas DAST:

► [Acunetix WVS](#)

- Licencia: comercial / libre
- S.O.: Windows

► [AppScan](#)

- Licencia: comercial
- S.O.: Windows

► [Burp Suite](#)

- Licencia: comercial / libre
- S.O.: Windows/lun

► [GamaScan](#)

- Licencia: comercial

- S.O.: Windows

► [Grabber](#)

- Licencia: *open source*
- S.O.: Python

► [Grendel-Scan](#)

- Licencia: *open source*
- S.O.: Windows, Linux y Macintosh

► [N-Stealth](#)

- Licencia: comercial
- S.O.: Windows

► [Inviciti](#)

- Licencia: comercial
- S.O.: Windows

► [Nikto](#)

- Licencia: *open source*
- S.O.: Unix/Linux

► [NTOSpider](#)

- Licencia: comercial
- S.O.: Windows

► [ParosPro](#)

- Licencia: comercial
- S.O.: Windows

► [QualysGuard](#)

- Licencia: comercial
- S.O.: N/A

► [Retina](#)

- Licencia: comercial
- S.O.: Windows

► [SecPoint Penetrator](#)

- Licencia: comercial
- S.O.: Windows, Linux y Macintosh

► [Trustkeeper Scanner](#)

- Licencia: comercial
- S.O.: SaaS

► [Vega](#)

- Licencia: *open source*
- S.O.: Windows, Linux, Macintosh

► [Wapiti](#)

- Licencia: *open source*
- S.O.: Windows, Linux y Macintosh

► [WebApp360](#)

- Licencia: comercial
- S.O.: Windows

► [WebInspect](#)

- Licencia: comercial
- S.O.: Windows

► [WebKing](#)

- Licencia: comercial
- S.O.: Windows, Linux y Solaris

► [Websecurify](#)

- Licencia: comercial / libre
- S.O.: Windows, Linux y Macintosh

► [Wikto](#)

- Licencia: *open source*
- S.O.: Windows

► [Zap](#)

- Licencia: *open source*

- S.O.: Windows, Linux, Macintosh

► **Análisis código binario.** Es comparable a la exploración del código fuente, pero se dirige al **código ensamblador** o ejecutable compilado binario del software antes de ser instalado y ejecutado. No hay escáneres de código binario específico, se utilizan herramientas de ingeniería inversa y análisis de ejecutables binario; por ejemplo, descompiladores, desensambladores y escáneres de código binario como los utilizados por Veracode para analizar el **código máquina** y modelar una **representación independiente** del idioma de los comportamientos del programa, el control y los flujos de datos, árboles y las llamadas externas de función. Ejemplos de este tipo de herramientas son IDAPRO, WinDbg, etc.

► **Inyección de fallos en binarios.** La inyección de fallos de seguridad induce tensiones en el software, crea problemas de **interoperabilidad** entre los componentes y simula fallos en el entorno de ejecución. Simula tipos de fallos y anomalías que pudieran resultar de patrones de ataque o la ejecución de **lógica maliciosa** que hacen el software vulnerable. En tiempo de ejecución, el probador modifica los datos que se pasan por el entorno del software. Sin embargo, los fallos inyectados no deben limitarse solo a aquellos que simulan ataques. Para obtener una comprensión más completa de todos los posibles comportamientos del software y sus estados, el probador también debe inyectar fallos que simulan condiciones muy poco probables, incluso los «imposibles».

► **Fuzz testing.** Al igual que en la inyección de fallos binario, consiste en la **introducción de datos no válidos** (por lo general producido por la modificación de una entrada válida) al software a través de su entorno o a través de otro componente de este. Las pruebas se llevan a cabo mediante un **fuzzer**, programa o *script* que realiza, modifica o combina entradas del software, para revelar cómo se comporta. Suelen ser específicos de un tipo particular de entrada, como HTTP, y se escriben

para ser utilizados para probar un programa específico, por lo que no son fácilmente reutilizables. Sin embargo, su valor radica en su **especificidad**, ya que a menudo puede revelar vulnerabilidades de seguridad que las herramientas genéricas de evaluación no detectan. Para que sean eficaces se requiere que el probador tenga una **comprensión completa** del software que está probando y la forma en que interactúa con entidades externas, cuyos datos simulará el *fuzzer*. En la siguiente tabla se presenta herramientas de este tipo.

Herramientas para la auditoría *fuzzing*:

- ▶ **Mangle:** *fuzzer* que genera etiquetas HTML y se autolanzará dentro de un navegador.
- ▶ **Spike:** colección de *fuzzers*.
- ▶ **Wfuzz:** *fuzzer* para web que aplica fuerza bruta sobre el protocolo HTTP. Realiza pruebas de *path discovery* (recursivas) o de fuerza bruta sobre variables (pasadas por GET o por POST). Emplea diccionarios (en el caso de las inyecciones SQL el diccionario está preparado con los patrones SQL más empleados).
- ▶ **Netcat:** Utilidad Unix que lee y escribe datos a través de conexiones de red usando los protocolos TCP o UDP. Está diseñada para ser una utilidad de tipo *back-end* que pueda ser usada directa o fácilmente manejada por otros programas y scripts.
- ▶ **Radamsa:** *fuzzer* de caja negra basado en mutaciones.
- ▶ **Blab:** *fuzzer* basado en gramática.
- ▶ **American Fuzzy Lop:** *fuzzer* basado en mutaciones.
- ▶ **Zzuf:** CERT *basic fuzzing framework*. Búsqueda de *bugs*.
- ▶ **Sulley:** extras para gestionar *fuzzing*, como parte de las pruebas de penetración.

- ▶ **Hping2:** herramienta de red capaz de enviar paquetes ICM/UDP/TCP hechos a medida, y de monitorizar las respuestas del host destino. Maneja fragmentación y tamaños arbitrarios de paquetes.
- ▶ **Sniffit:** herramienta de monitorización y *packet* de *sniffer* para paquetes de TCP/UDP/ICMP capaz de dar información técnica muy detallada acerca de estos paquetes.
- ▶ **Firewalk:** emplea técnicas del estilo *traceroute* para determinar las reglas de filtrado que se están usando en un dispositivo de transporte de paquetes.
- ▶ **Escaneo de vulnerabilidades.** Este tipo de pruebas analizan los sistemas en busca de vulnerabilidades conocidas. Disponen de información sobre vulnerabilidades existentes en los sistemas operativos y aplicaciones mediante bases de datos actualizadas, que utiliza para la detección de estas.

La herramienta más utilizada es **Nessus**, inicialmente de código abierto y versión gratuita, y actualmente en dos versiones, una **profesional comercial** y otra de **prueba gratuita** (www.nessus.org). Otras herramientas de extendido uso son OpenVas, Nexpose de Rapid 7, ISS Real Secure, GFI LANguard, QualysGuard, Nmap y Retina.

A continuación, se enumeran y se suministra información de varias herramientas con diversas funcionalidades que existen actualmente:

- ▶ **Nessus**
 - Licencia: libre
 - S.O.: Linux/Unix, Windows

▶ [Nexpose](#)

- Licencia: comercial
- S.O.: Linux/Unix, Windows

▶ [AppDetective](#)

- Licencia: comercial
- S.O.: Windows

▶ [Open Vas](#)

- Licencia: Libre
- S.O.: linux

▶ [GFI LANguard](#)

- Licencia: comercial
- S.O.: Windows

▶ [MBSA](#)

- Licencia: freeware
- S.O.: Windows

▶ [Nmap](#)

- Licencia: GPL
- S.O.: Linux/Unix, Windows, Mac OS X

► [Retina Network Security Scanner](#)

- Licencia: comercial
- S.O.: Windows

► [SATAN](#)

- Licencia: comercial
- S.O.: Linux/Unix

► [SDS \(Shadow Database Scanner\)](#)

- Licencia: comercial
- S.O.: Windows

► [SolarWinds Engineer's Toolset](#)

- Licencia: comercial
- S.O.: Windows

► [Symantec Enterprise Security Manager](#)

- Licencia: comercial
- S.O.: Windows

El **análisis de caja gris** es una combinación de las dos anteriores. Se trata de un tipo de análisis de seguridad, que combina pruebas de caja blanca y negra, que tiene en cuenta la interfaz del aplicativo y su **código fuente**.

- **Análisis híbrido o Interactive Application Security Testing (IAST).** Se trata de otra de las pruebas enfocadas al análisis de aplicaciones, tanto web como de otro tipo, que implica el uso del código fuente y del binario ejecutable, compilado a partir

de dicho código. Para la realización del análisis, se ejecuta el programa compilado con un agente dentro del mismo, al tiempo que se «alimentan» las interfaces externas de la muestra. El revisor sigue y analiza los datos que el programa produce como resultado de su ejecución, de forma que cualquier vulnerabilidad o anomalía que surja en dicha interfaz se traza simultáneamente con el código fuente que la genera con mayor eficacia. En realidad, en este tipo de revisión se realizan **tres tipos de análisis**:

- ▶ **Análisis de cobertura.** Pone de manifiesto las interacciones entre las diferentes partes del programa.
- ▶ **Análisis de frecuencia de espectro.** Revela las dependencias entre los componentes del programa.
- ▶ **Análisis de patrones.** Permite buscar patrones específicos en la ejecución del programa, tales como excepciones no capturadas, omisiones, errores de memoria dinámica y problemas de seguridad.

Dado que el agente IAST está trabajando dentro de la aplicación, su análisis se aplica a toda la aplicación incluido su código. Se comprueba el control de los tiempos de ejecución, flujos de datos, configuración, peticiones y respuestas HTTP, bibliotecas, *frameworks* y otros componentes. El acceso a toda esa información permite al motor IAST producir resultados **más precisos** y verificar una gama más amplia de problemas de seguridad que SAST o DAST.

Herramientas de análisis híbrido:

- ▶ [Valgrind](#): para todo tipo de aplicaciones.
- ▶ [INSURE++](#): herramienta para análisis de errores para todo tipo de aplicaciones escrita en C, C++.
- ▶ [SCA Fortify](#) más Web Inspect de HP: disponen de un agente para integrar el resultado de las dos herramientas.
- ▶ [IBM Appscan](#), con el agente GLASS BOX.
- ▶ [Acunetix](#) más Acusensor.
- ▶ [SEEKER](#).

Las **pruebas de seguridad basadas en el riesgo** (pruebas de seguridad desde el punto de vista del atacante), según lo anteriormente expuesto, se deberían estructurar en las siguientes fases:

- ▶ **Descomponer** el sistema en sus componentes fundamentales.
- ▶ **Identificar** las interfaces de los componentes.
- ▶ **Clasificar** las interfaces de los componentes por su riesgo potencial.
- ▶ **Averiguar** las estructuras de datos usadas por cada interfaz.
- ▶ **Encontrar** problemas de seguridad inyectando datos maliciosos, apoyándose en el estado del riesgo ante las posibles amenazas.

En la siguiente figura se muestran las diferentes pruebas a realizar en cada una de las fases del S-SDLC:

DISEÑO Y ARQUITECTURA	• Revisión de diseño
CODIFICACIÓN	• Revisión del código • Inyección de fallos en código fuente • <i>Fuzzing Testing</i>
PRUEBAS	• Análisis dinámico • Inyección de fallos en código fuente • Análisis híbrido • Pruebas de penetración • Análisis código binario • <i>Fuzzing Testing</i> • Inyección de fallos en binarios • Escaneo de vulnerabilidades
OPERACIÓN - PRODUCCIÓN	• Pruebas de penetración • Análisis dinámico • Escaneo de vulnerabilidades

Figura 7. Distribución pruebas de seguridad a lo largo de las fases del S-SDLC. Fuente: elaboración propia.

3.3. Revisión de código

El análisis estático de código fuente, tal y como comenta McGraw (2005), se considera la **actividad más importante de entre las mejores prácticas de seguridad** que se han de realizar en el curso del desarrollo de una aplicación. En los siguientes apartados se van a analizar los tipos y categorías de herramientas disponibles, tanto comerciales como de *open source*, para qué lenguajes están disponibles, cómo y cuándo se tienen que utilizar y cómo se enmarca el uso de estas herramientas en el **proceso de revisión del código**.

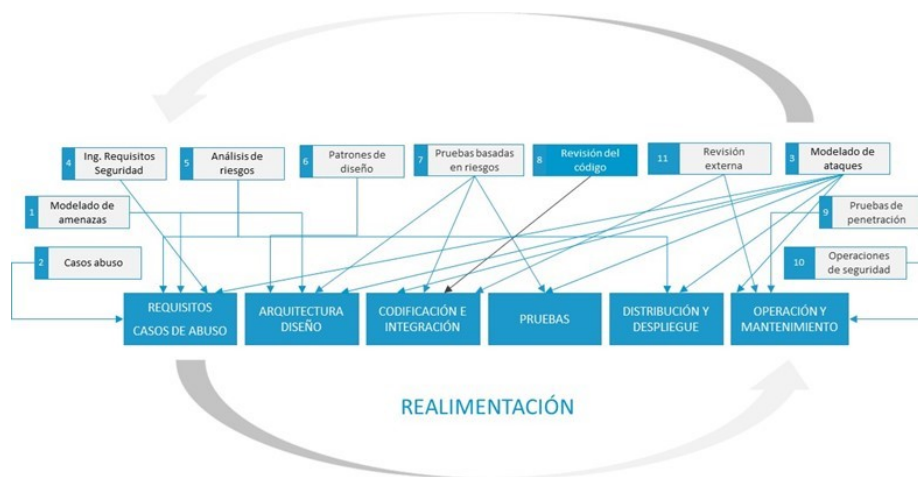


Figura 8. Revisión de código. Fuente: elaboración propia.

Los problemas de seguridad de una aplicación pueden ser resultado de **dos tipos de errores principales**:

- ▶ **Errores simples** que comete el programador por confusión, un *lapsus* momentáneo, etc.
- ▶ **Carencias de conocimientos** del programador.

Con esto en mente, el **análisis estático de código fuente** es adecuado para identificar problemas de seguridad por ciertas razones:

- ▶ Las herramientas de análisis estático comprueban el código a fondo y coherentemente, sin ninguna tendencia. Un análisis valioso debe ser lo más imparcial posible.
- ▶ Examinando el **código** en sí mismo, las herramientas de análisis estático a menudo pueden indicar la causa de origen de un problema de seguridad, no solamente uno de sus síntomas.
- ▶ El **análisis estático** puede encontrar errores tempranamente en el desarrollo, aún antes de que el programa sea ejecutado por primera vez.
- ▶ Cuando un investigador de seguridad descubre una nueva variedad de ataque, las herramientas de análisis estático ayudan a **comprobar de nuevo** una gran cantidad de código.

La queja más común y contrastada contra las herramientas de análisis estático de seguridad es que producen demasiado ruido, es decir, **falsos positivos**: un problema descubierto en un programa cuando ningún problema existe en realidad. Un gran número de falsos positivos puede causar **verdaderas dificultades**.

Los falsos positivos son, seguramente, indeseables, pero desde una perspectiva de seguridad, los **falsos negativos** son mucho peores. Con un falso negativo, un problema existe en el programa, **pero la herramienta no lo detecta**. La **penitencia por un falso positivo** es la cantidad de **tiempo gastado** repasando el resultado. La **penitencia por un falso negativo** es mucho mayor, pues no solo se paga el precio asociado por tener una vulnerabilidad en el código, se vive con un **sentido falso de seguridad** que se deriva del hecho de que la herramienta hizo parecer que todo era correcto.

Todas las herramientas de **análisis estático de código** producen algunos falsos positivos y falsos negativos, su balance es normalmente indicativo del propósito de la herramienta de esta.

- ▶ **Objetivo descubrir *bugs*.** El coste de omitir un *bug* dentro de la primera categoría es relativamente pequeño hablando en términos de seguridad.
- ▶ **Objetivo descubrir defectos relevantes de seguridad.** La penitencia por *bugs* de seguridad pasados por alto es elevada, por lo que este tipo de herramientas, en general, producen más falsos positivos para reducir al mínimo falsos negativos.

Herramientas de revisión de código

Las **herramientas de análisis de seguridad estáticas** usan muchas de las mismas técnicas encontradas en los compiladores, pero su objetivo está más enfocado a identificar problemas de seguridad, por lo que aplican estas técnicas de manera diferente.

Para una herramienta de comprobación de propiedades, al buscar potenciales vulnerabilidades de desbordamiento de *buffer* habría que comprobar la propiedad «**el programa no tiene acceso a una dirección fuera de los límites de memoria asignada**».

Fortify Software e IBM fabrican herramientas de análisis estático de código que caen dentro de esta categoría.

Consulta más información sobre estas herramientas

en: <https://www.microfocus.com/es-es/products/static-code-analysis-sast/overview>

En la práctica, lo importante es que estas herramientas **suministran importantes resultados**. El hecho de que sean imperfectas no les impide tener un valor significativo. Los principales factores prácticos que determinan la utilidad de una herramienta de análisis estático son:

- ▶ La capacidad de la herramienta para comprender el programa que se analiza.
- ▶ El equilibrio que la herramienta hace entre la precisión, la profundidad y la escalabilidad.
- ▶ Porcentaje de falsos positivos y falsos negativos de la herramienta.
- ▶ El conjunto de errores que la herramienta comprueba.
- ▶ Las características de la herramienta para hacerla fácil de usar.

Arquitectura de las herramientas de análisis estático de código

Independientemente de las técnicas de análisis usadas, todas las herramientas de **análisis estático** funcionan, aproximadamente, del mismo modo, tal y como se muestra en la siguiente figura. Todas **aceptan el código**, construyen un modelo (abstracción del código, modelo de autómata de estado finito, etc.) que representa el programa, analizan dicho modelo en combinación con una **base de conocimiento de seguridad** y terminan presentando sus resultados al usuario.

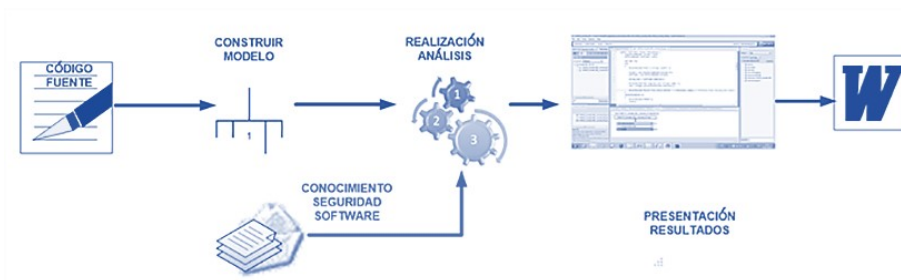


Figura 11. Diagrama de bloques para una herramienta genérica de análisis de código. Fuente: elaboración propia.

Hay algunas técnicas importantes y estructuras de datos que comparten compiladores y las herramientas de análisis estático, como son los componentes que tienen la mayoría de los compiladores:

- ▶ **Analizador léxico.** Constituye la primera fase de la herramienta en recibir como entrada el código fuente de un programa (**secuencia de caracteres**) y produce una salida compuesta de símbolos (**componentes léxicos**). Estos símbolos sirven para una posterior etapa del proceso de traducción y son la entrada para el analizador sintáctico. Realiza, además, funciones como eliminar espacios en blanco, saltos de línea, tabuladores, ignorar comentarios, detección y recuperación de errores.
- ▶ **Analizador sintáctico.** Convierte la entrada de componentes léxico de la etapa anterior en una estructura del árbol, más útil para el posterior análisis. Capturan la jerarquía implícita de la entrada.
- ▶ **Analizador semántico.** Utiliza la entrada el árbol sintáctico para comprobar restricciones de tipo y otras limitaciones semánticas.
- ▶ **Estructuras de datos como las tablas de símbolos y de tipos.** Es una estructura de datos donde a cada símbolo en el código fuente de un programa se le asocia información como la ubicación, tipo de datos y ámbito de cada variable, constante o procedimiento.

Las herramientas de análisis estático realizan varios tipos de análisis:

- ▶ **Análisis estructural.** Realiza comprobaciones de los detalles de la gramática y la sintaxis del texto de programa creando una estructura de datos denominada **árbol de sintaxis abstracto (AST)**. Con la tabla de símbolos realiza la comprobación de tipos *typechecking*.
- ▶ **Análisis de flujo de control.** Exploración de los diferentes caminos de ejecución que se pueden seguir cuando una función se ejecuta.

- ▶ **Análisis de flujo de datos.** Examinan los caminos de movimiento de datos a través de un programa; por lo general, atraviesan el gráfico de flujo de control de una función y anotan dónde se generan los valores de datos y dónde se usan.
- ▶ **Taint Propagation.** Análisis de flujo de datos para determinar qué es lo que un atacante puede controlar desde las diversas entradas a la aplicación. Se requiere saber por dónde entra la información en el programa y cómo se mueve a través de él. Mediante esta técnica se encuentran muchos defectos de validación de entrada.
- ▶ **Pointer Aliasing.** Los alias de los punteros son otra clase de problema del flujo de datos. La finalidad del análisis de alias es determinar qué punteros apuntan a la misma posición de memoria.
- ▶ **Análisis local.** Analiza una función individualmente en cuanto a posibles caminos de ejecución, lo que elimina los **camino falsos**, que son caminos por los cuales el código nunca puede ejecutarse porque son lógicamente incoherentes.
- ▶ **Análisis global.** Este análisis requiere hacer comprobaciones de las interacciones entre las funciones.
- ▶ **Interpretación abstracta.** Es una técnica general formalizada para descartar los aspectos del programa que no son relevantes. Modela el programa como una máquina abstracta consistente en una caracterización matemática que recopila información sobre el flujo de control y de los datos de este, simulando su comportamiento.
- ▶ **Model checking.** Para propiedades temporales de seguridad como «la memoria solo debería ser liberada una vez» y «solo los punteros no nulos deberían ser referenciados» es fácil representarlas como pequeños autómatas de estado finito.
- ▶ **SAT Solvers.** El denominado problema **boolean satisfiability** consiste en averiguar si, dada una expresión, hay alguna combinación en los valores de las variables de la expresión que haga la expresión TRUE.

A continuación, se van a enumerar algunas de estas herramientas, tanto comerciales como libres (*open source*), para lenguajes como **C/C++** y **Java**, aunque las herramientas comerciales soportan varios lenguajes más. Las herramientas libres son, en la mayoría de los casos, proyectos de investigación de universidades. Este apartado se limita a enumerar algunas de ellas e indicar su tipo de licencia y los lenguajes que es capaz de revisar.

Herramientas de análisis estático de código fuente

► [SCA](#) de Fortify Software de HP

- Licencia: comercial.
- Lenguaje: C, C++, Java y otros

► [AppScam](#) de IBM.

- Licencia: comercial
- Lenguaje: C, C++, Java y otros

► [Prevent](#) de Coverity

- Licencia: comercial
- Lenguaje: C, Java y otros

► [K8 Inshight](#) de Klocwork,

- Licencia: comercial
- Lenguaje: C, Java y otros

► [VERACODE](#)

- Licencia: SaaS

- Lenguaje: C, Java y otros
- ▶ [CXSUITE \(CHECKMARX\)](#)
 - Licencia: comercial
 - Lenguaje: C, Java y otros
- ▶ [CodeSonar](#) de Grammatech,
 - Licencia: comercial
 - Lenguaje: C, C++
- ▶ [SATURN](#) de Stanford University.
 - Licencia: libre
 - Lenguaje: C
- ▶ [BOOP \(C\)](#) de Graz University of technology.
 - Licencia: libre
 - Lenguaje: C
- ▶ [Magic](#) de Carnegie Mellon University.
 - Licencia: libre
 - Lenguaje: C
- ▶ [Jtest](#) de Parasoft
 - Licencia: comercial
 - Lenguaje: Java

► [FindBugs](#) de Maryland k,

- Licencia: libre
- Lenguaje: Java

► [Java PathFinder](#)

- Licencia: libre
- Lenguaje: Java

► [Cppcheck](#)

- Licencia: libre
- Lenguaje: C,C++

► [Polyspace](#) de Mathworks.

- Licencia: comercial
- Lenguaje: C, ada

► [Pc-lint](#) de Gimpel Software

- Licencia: comercial
- Lenguaje: C, C++

3.4. Pruebas de penetración

Una vez terminada la fase de desarrollo se despliega el sistema. Se deben llevar a cabo muchas de las operaciones o actividades de seguridad relacionadas con la puesta en marcha de la aplicación para su posterior paso a producción y explotación por parte de los usuarios. Las **actividades de seguridad** a realizar en esta fase comprenden la implementación y comprobación de la eficacia de las salvaguardas, tanto del tipo de software (por ejemplo, autenticación) como de los dispositivos hardware (por ejemplo, *firewall*) que se derivaron de la actividad de análisis de riesgos realizada en la fase de diseño.

La comprobación de la eficacia de las salvaguardas implementadas se realiza principalmente mediante las pruebas de penetración, que tiene como principal misión verificar cómo el software se comporta y resiste ante diferentes tipos de ataque.

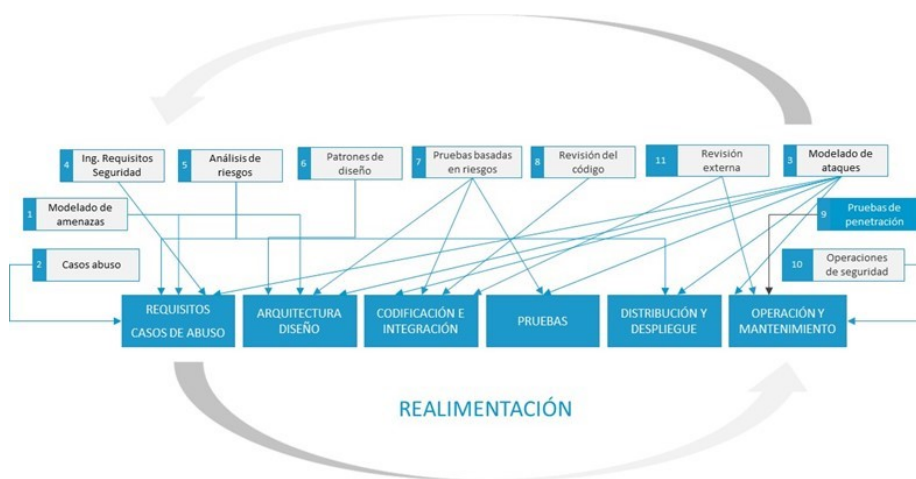


Figura 10. Test de penetración en el SDLC. Fuente: elaboración propia.

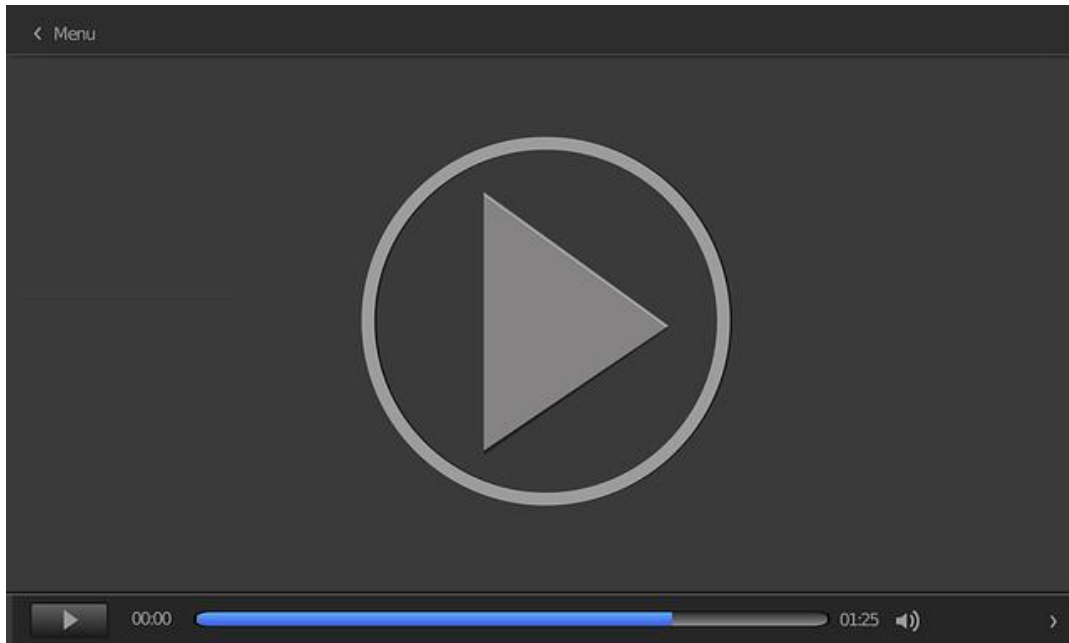
Las pruebas de penetración deben centrarse en aspectos de comportamiento del software, sus interacciones y vulnerabilidades que no puedan ser detectadas mediante otras pruebas realizadas fuera del entorno de producción. Deben tratar de encontrar problemas de seguridad que puedan originarse en su **arquitectura y diseño** (frente a errores de codificación que se manifiestan como vulnerabilidades), ya que este tipo de problema tiende a ser pasado por alto por otras técnicas de prueba.

El **plan de pruebas de penetración** debe incluir los «peores» escenarios en los que se puedan reproducir vectores de ataques e intrusiones que se consideran altamente perjudiciales, como los escenarios de amenazas internas. El plan de pruebas debe capturar:

- ▶ La política de seguridad del sistema se supone que debe respetar o hacer respetar.
- ▶ Amenazas previstas.
- ▶ Riesgos de seguridad (conducido por casos de abuso, riesgos arquitectónicos y modelos de ataque).
- ▶ Secuencias de ataques probables que se puedan producir.

Las pruebas de penetración de una aplicación deben centrarse en aspectos de comportamiento del software, sus interacciones y vulnerabilidades que no puedan ser detectadas mediante otras pruebas realizadas.

En este vídeo, *Pruebas de Penetración en desarrollo de aplicaciones*, se estudia la buena práctica de seguridad «Pruebas de Penetración» en el contexto del desarrollo y pruebas de seguridad de una aplicación.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=c42d5ade-161a-4fa2-b7a9-adf700cff2b6>

Consideraciones sobre las pruebas de penetración

Una buena parte de los defectos y vulnerabilidades del software no está relacionada directamente con la funcionalidad de seguridad. Muchas de las cuestiones asociadas con la seguridad implican un mal uso de una aplicación, inesperado y descubierto por un atacante, normalmente. Las pruebas de penetración tienen que verificar los **aspectos negativos** del sistema; es decir, se debe probar la seguridad de la aplicación en base a los **riesgos** (conducido por casos de abuso, riesgos arquitectónicos y modelos de ataque) para determinar cómo se comporta ante los ataques.

Las pruebas de penetración son **pruebas de caja negra**. En cualquier caso, la prueba de aspectos negativos es un desafío mucho mayor que la verificación de un aspecto positivo. Es realmente **fácil** probar si un componente funciona adecuadamente o no, pero es muy **difícil** demostrar si realmente un sistema es seguro a un ataque malévolo.

Si la realización de pruebas para detección de aspectos negativos no revela ningún defecto, estas únicamente demuestran que ningún defecto ocurre en las condiciones particulares de esas pruebas, en ningún caso demuestran que ningún defecto existe.

Las **pruebas de penetración** son, de forma general, las más aplicadas de todas las mejores prácticas de seguridad del software, y suelen ser parte del proceso de aceptación de final. A causa de restricciones de tiempo, la mayor parte de este tipo de evaluaciones son realizadas de forma apresurada como un punto de la lista de comprobación de seguridad al final del ciclo de vida.

Una limitación principal de este acercamiento es que casi siempre representa una tentativa **demasiado corta y muy tardía** para abordar el problema de la seguridad al final del ciclo de desarrollo.

El verdadero valor de las pruebas de penetración viene de sondear un sistema en su ambiente final de operación. El **entendimiento del ambiente de ejecución y de los problemas de configuración** es el mejor resultado de cualquier prueba de penetración. Esto es así, sobre todo, porque estos problemas pueden ser solucionados en realidad tarde en el ciclo de vida. Las **pruebas de penetración** llegan al corazón del entorno de ejecución y de los aspectos de configuración rápidamente.

El éxito de una prueba de penetración depende de muchos factores, pocos de los cuales se prestan a la métrica y la estandarización. La primera variable y más obvia es la **habilidad**, el **conocimiento**, y la **experiencia del probador**. El resultado de estas depende de ello.

Todos estos problemas palidecen en comparación con el problema de que las pruebas de penetración a menudo son usadas como una excusa para declarar la victoria de seguridad «y ya está todo hecho». Lamentablemente, las pruebas de penetración realizadas sin basarse en el análisis de riesgos de seguridad conducen a una **falsa sensación de seguridad**.

Herramientas para test de penetración

Uso de herramientas para realizar las pruebas de penetración. Normalmente, la realización de este tipo de pruebas de **caja negra** engloba la realización de otros tipos de pruebas:

- ▶ Escaneo de vulnerabilidades.
- ▶ Explotación de vulnerabilidades.
- ▶ Fuzz testing.
- ▶ Análisis dinámico (DAST) para aplicaciones web.

Con las herramientas de **escaneo de vulnerabilidades** se encuentran vulnerabilidades conocidas con poco esfuerzo, que aprovechan las herramientas de **explotación de vulnerabilidades**. Las herramientas de **análisis dinámico y de fuzzing** pueden observar cómo se ejecuta un sistema, pueden **someter** los datos mal formados, malévolos y arbitrarios en los puntos de entrada del sistema en una tentativa de destapar fallos. Los defectos se reportan al personal de prueba para su análisis. Cuando es posible, el empleo de estas herramientas se debe dirigir por los resultados de análisis de riesgo y los modelos de ataque.

3.5. Operaciones de seguridad

El proceso final para llevar a cabo, previo al paso a producción de la aplicación segura, se compone de las actividades centrales de:

- ▶ Distribución.
- ▶ Despliegue.
- ▶ Operaciones.

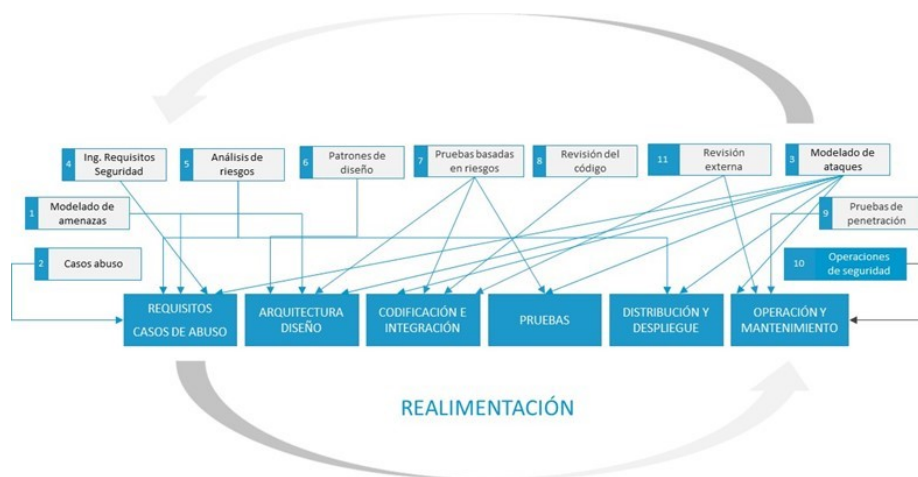


Figura 14. Operaciones seguridad. Fuente: elaboración propia.

Distribución

El objetivo de la distribución segura es reducir al mínimo las posibilidades de acceso y manipulación del software durante la transmisión de un proveedor a su consumidor (envío a través de medios físicos o descarga de red) por los agentes maliciosos.

A la hora de realizar la **distribución y despliegue** del software desarrollado, es recomendable el realizar las siguientes buenas prácticas:

- ▶ **Cambio de los valores de configuración** predeterminados durante el desarrollo.
- ▶ Utilizar **mecanismos de distribución estándar**, como los utilizados en los derechos de propiedad intelectual.
- ▶ Distribuir el software con una **configuración por defecto segura** y lo más restrictiva posible.
- ▶ Realizar una **guía de configuración de seguridad**.
- ▶ Proporcionar una **herramienta de instalación automática**.
- ▶ Establecer un **medio de autenticación** para la persona que va a ejecutar la instalación y configuración.
- ▶ Los interfaces de configuración proporcionados por la herramienta o el *script* de instalación deben ser **seguros**.
- ▶ **Revisión y limpieza** de todo el código fuente por el visible usuario (por ejemplo, código del cliente de aplicaciones web).

Despliegue

Una configuración cuidadosa y la personalización del entorno de despliegue de cualquier aplicación de software pueden mejorar enormemente su seguridad. El **diseño de un entorno de despliegue** adaptado para una aplicación requiere de un proceso:

- ▶ Comienza en el nivel de **componente de red**.
- ▶ Continúa por el **sistema operativo y otro software de base** como puede ser un gesto de base de datos, etc.

- Termina con la propia configuración de seguridad de la aplicación y el sistema.

El software puede haber sido diseñado y desarrollado para ser extremadamente seguro, pero no lo será si sus parámetros de configuración no se establecen como el diseñador lo diseñó. De manera similar, los parámetros de configuración de su entorno de ejecución deben ajustarse de modo que el software no sea innecesariamente expuesto a amenazas potenciales, a esta tarea se le denomina «bastionado».



Figura 13. Capas del sistema a proteger. Fuente: elaboración propia.

El **bastionado del software** comprende los procesos y herramientas necesarias para realizar el control y corrección de los defectos y debilidades de las configuraciones del software sobre la base de un proceso de control de configuración. En España, el **Centro Criptológico Nacional (CCN)** y, en Estados Unidos, el departamento de Defensa, y otros departamentos, elaboran directrices de configuración seguras y listas de control para productos de **software comercial**. Podemos encontrar algunas en:

- [Centro Criptológico Nacional \(CCN\)](#)

► NIST Listas de comprobación de configuración de productos TIC

Operaciones

Muchos **desarrolladores de software** argumentan que la distribución, el despliegue y las operaciones no son parte del proceso de desarrollo de software. Hay que ajustar los controles de acceso a la red y niveles del sistema operativo, así como diseñar un sistema de registro de eventos, de monitorización, de *backup* y recuperación de los sistemas de ficheros, que será lo más eficaz durante operaciones de respuesta ante incidentes. Los ataques llegarán, y se debe estar **preparado para defenderse** de ellos y para deshacer el daño ocasionado después de que un ataque ha tenido lugar.

3.6. Revisión externa

Un **análisis externo** por personal ajeno al equipo de diseño es bastante eficaz y fundamental aportando otra visión de la seguridad del sistema y del riesgo y contribuyendo a una mejora de la seguridad, porque seguramente este análisis va a descubrir alguna amenaza y riesgo residual existente. Después, terminado el análisis externo habrá que revisar el análisis de riesgos y, si hay nuevas amenazas y riesgos, gestionarlos para que sean mitigados; si es necesario, también se deberán realizar cambios en la arquitectura hardware y software, realizar cambios en el código y, por lo tanto, repetir la revisión de este, los *tests* de seguridad basados en el riesgo que ha cambiado y volver, después del despliegue de la aplicación, a pasar los *tests* de penetración.

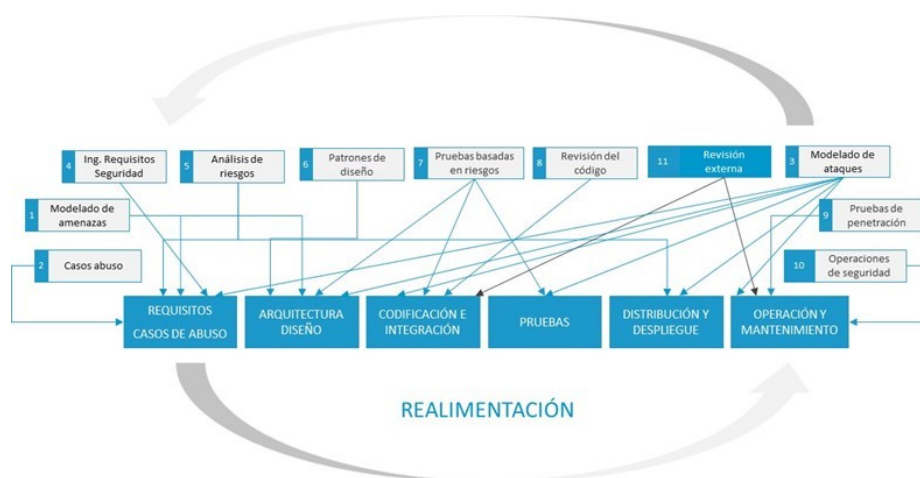
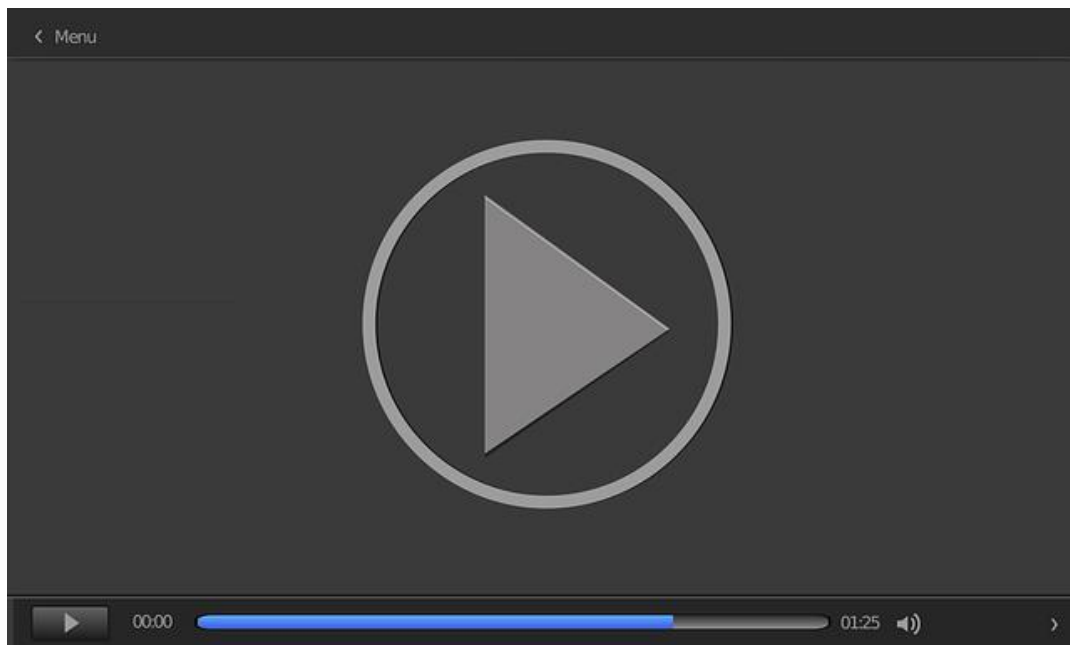


Figura 16. Revisión externa. Fuente: elaboración propia.

Esto conforma un esquema de seguridad en el **ciclo de vida de carácter cíclico** en el que es posible que se tengan que realizar varias veces el mismo tipo de actividades, consecuencia de la naturaleza cambiante y de continua evolución de las aplicaciones y también del carácter cíclico de los distintos ciclos de vida de desarrollo de aplicaciones donde, en muchos casos, se van refinando prototipos que evolucionan hasta conseguir el sistema final.

Una práctica común en la adquisición es aceptar un software que satisface la funcionalidad con poca o ninguna consideración con respecto a la especificación y aseguramiento de las propiedades de seguridad. Esto aumenta la exposición y riesgos de seguridad de la organización. Es este vídeo (*Seguridad en las adquisiciones de código*) se estudian las mejores prácticas que permiten verificar la seguridad del software adquirido.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=7e666bb0-4150-4be9-8932-adf700cff21e>

3.7. Referencias bibliográficas

López, S. (2012). *Descubriendo el valor de la Seguridad en las Aplicaciones Web con IBM AppScan*. IBM Corporation.

McGraw, G. (2005). *Software Security: Building Security In*. Addison Wesley Professional.

A systemic approach for assessing software supply-chain risk

Dorofee, A., Woody, C., Alberts, C., Creel, R. y Ellison, R. J. (2013). *A systemic approach for assessing software supply-chain risk*. En Cybersecurity & Infrastructure Security Agency. <https://us-cert.cisa.gov/bsi/articles/best-practices/acquisition/a-systemic-approach-assessing-software-supply-chain-risk>

Esta guía proporciona información sobre cómo incorporar prácticas de aseguramiento del software durante todo el proceso de adquisición, durante las fases de planificación de la contratación de la adquisición, supervisión y aceptación y seguimiento.

Hybrid Security Assessment Methodology for Web Applications

Correa, R., Bermejo Higuera, J. R., Higuera, J. B., Sicilia Montalvo, J. A., Rubio, M. S., y Magreñán, Á. A. (2021). *Hybrid Security Assessment Methodology for Web Applications*. *Computer Modeling in Engineering & Sciences*, 126(1), 89-124. <https://www.ingentaconnect.com/content/tsp/cmesc/2021/00000126/00000001/art00005>

Este artículo presenta una metodología para evaluar y prevenir problemas de vulnerabilidades de seguridad en aplicaciones web. El proceso de análisis se basa en el uso de técnicas y herramientas que permiten realizar evaluaciones de seguridad de caja blanca y caja negra, para llevar a cabo la validación de seguridad de una aplicación web de forma ágil y precisa.

El objetivo de la metodología es aprovechar las sinergias de las herramientas semiautomáticas de análisis de seguridad estático y dinámico y las comprobaciones manuales. Cada una de las fases contempladas en la se apoya en herramientas de análisis de seguridad de distinto grado de cobertura, de forma que los resultados generados en una fase se utilizan como alimentación de las siguientes para obtener un resultado global de análisis de seguridad optimizado.

La metodología puede utilizarse como parte de otras metodologías más generales que no cubren el uso de herramientas de análisis estático y dinámico en las fases de implementación y prueba de un ciclo de vida de desarrollo de software seguro (S-SDLC).

A Few Billion Lines of Code Later

Bessey, A. et al. (2010). *A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World*. Communications of the ACM, Vol. 53 No. 2, 66-75.
<http://cacm.acm.org/magazines/2010/2/69354-a-few-billion-lines-of-code-later/fulltext>

Artículo realizado por el equipo de Coverity que describe la experiencia e implementación de una herramienta de análisis estático comercial.

1. Señala la respuesta correcta. Las perspectivas de las pruebas de seguridad basadas en el riesgo son las siguientes:
 - A. Perspectiva gerencia.
 - B. Perspectiva atacante.
 - C. Perspectiva usuario.
 - D. Perspectiva del analista.

2. Señala la respuesta correcta. El principal problema de las herramientas de análisis estático es:
 - A. Falsos negativos que produce.
 - B. Gran cantidad de defectos que encuentra.
 - C. Reglas de la herramienta.
 - D. Falsos positivos que produce.

3. Señala la respuesta incorrecta. Las herramientas de análisis estático realizan varios tipos de análisis:
 - A. *Taint Propagation*.
 - B. Análisis puntual.
 - C. *Model checking*.
 - D. Análisis de flujo de datos.

4. Señala la respuesta correcta. Los *tests* de penetración:
- A. Demuestran que ningún defecto existe.
 - B. Revisan el código.
 - C. El entendimiento del ambiente de ejecución y de los problemas de configuración es el mejor resultado de cualquier prueba de penetración.
 - D. Las conclusiones de seguridad son repetibles a través de equipos diferentes y varían extensamente dependiendo de la habilidad y la experiencia de los probadores.
5. Señalar la respuesta correcta. A la hora de realizar la distribución y despliegue de l *software* desarrollado es recomendable el realizar las siguientes buenas prácticas:
- A. Distribuir el *software* con una configuración por defecto segura y lo más restrictiva posible.
 - B. Proporcionar una herramienta de instalación automática.
 - C. Cambio los valores de configuración predeterminados durante el desarrollo.
 - D. Todas las anteriores.

6. Señala la respuesta incorrecta. Los objetivos de las pruebas de seguridad basadas en el riesgo son:

- A. Verificar la operación del software bajo en su entorno de producción.
- B. Verificar la capacidad de supervivencia del software ante la aparición de anomalías, errores y su manejo de estas mediante excepciones que minimicen el alcance e impacto de los daños que puedan resultar de los ataques.
- C. Verificar la falta de defectos y debilidades explotables.
- D. Verificar la fiabilidad del software, en términos de comportamiento seguro y cambios de estado confiables.

7. Señalar la respuesta incorrecta. El análisis estático de código fuente es adecuado para identificar problemas de seguridad por ciertas razones:

- A. Las herramientas de análisis estático comprueban el código a fondo y coherentemente, sin ninguna tendencia, que a veces los programadores según su criterio podrían tener sobre algunas partes del código que pudieran ser más «interesantes» desde una perspectiva de seguridad o partes del código que pudieran ser más fáciles para realizar las pruebas dinámicas
- B. Examinando el código en sí mismo, las herramientas de análisis estático a menudo pueden indicar la causa de origen de un problema de seguridad, no solamente uno de sus síntomas.
- C. Cuando un investigador de seguridad descubre una nueva variedad de ataque, las herramientas de análisis estático no ayudan a comprobar de nuevo una gran cantidad de código para ver dónde el nuevo ataque podría tener éxito.
- D. El análisis estático puede encontrar errores tempranamente en el desarrollo, aún antes de que el programa sea ejecutado por primera vez.

8. Señalar la respuesta correcta. La principal misión de los *tests* de penetración es:
- A. Revisar estáticamente el código del sistema.
 - B. Comprobar las vulnerabilidades del software.
 - C. Verificar cómo el software se comporta y resiste ante diferentes tipos de ataque.
 - D. Probar la seguridad de la arquitectura del software.
9. Indicar la respuesta incorrecta. Los factores principales prácticos que determinan la utilidad de una herramienta de análisis estático son:
- A. El equilibrio que la herramienta hace entre la precisión, la profundidad y la escalabilidad.
 - B. Porcentaje de falsos positivos de la herramienta.
 - C. Las características de la herramienta para hacerla fácil de usar.
 - D. La capacidad de la herramienta para comprender el programa que se analiza.
10. Señalar la respuesta correcta. Una herramienta de análisis de código reporta que existe una vulnerabilidad de inyección SQL. Sin embargo, después de la correspondiente verificación, se comprueba que en realidad no existe tal vulnerabilidad. ¿Qué tipo de limitación de las herramientas de análisis de código se ha expuesto?
- A. Un falso positivo.
 - B. Un falso negativo.
 - C. Una vulnerabilidad específica de un lenguaje de programación.
 - D. Ninguna de las anteriores.

Desarrollo Seguro de Software y Auditoría de la
Ciberseguridad

Tema 4. Codificación segura de aplicaciones I

Índice

Esquema

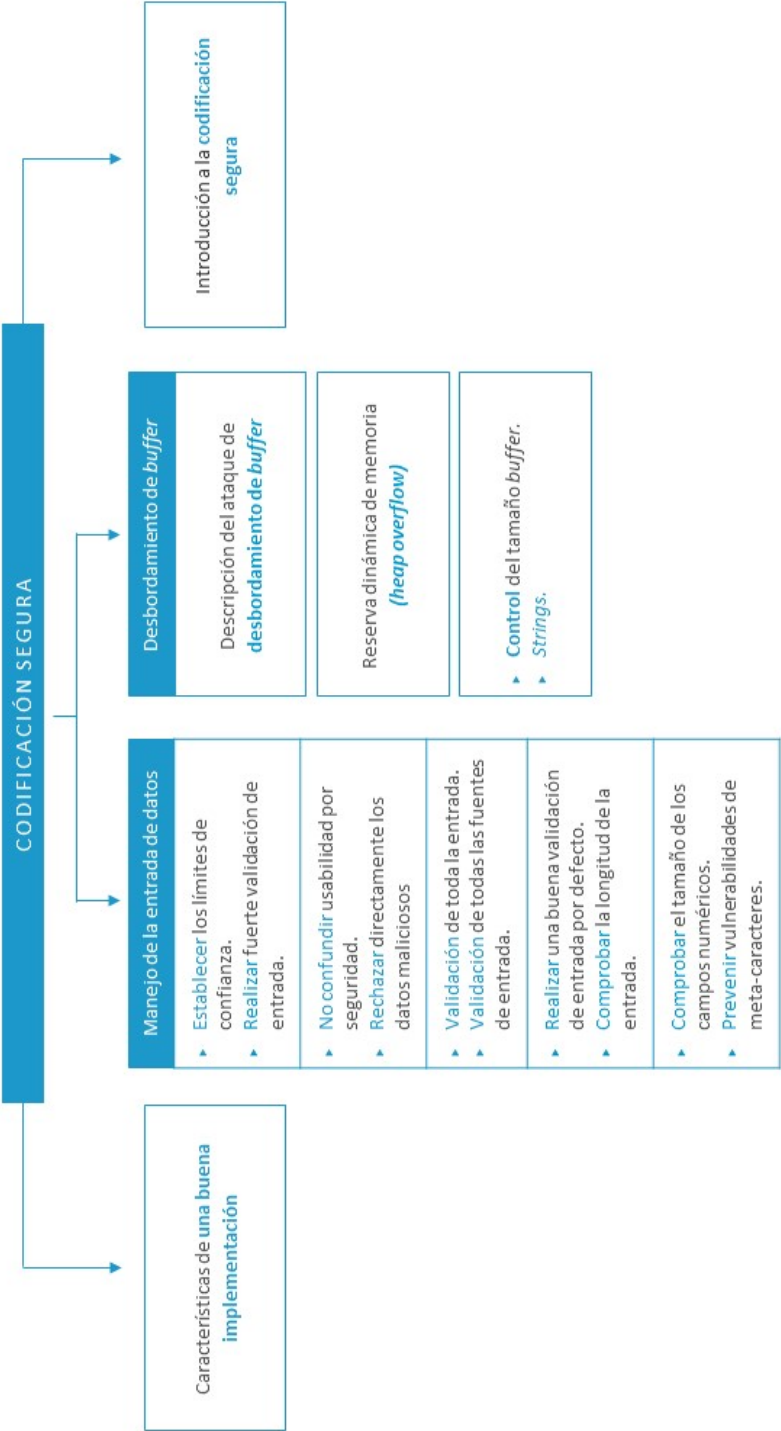
Ideas clave

- 4.1. Introducción y objetivos
- 4.2. Prácticas de codificación segura
- 4.3. Manejo de la entrada de datos
- 4.4. Desbordamiento de buffer
- 4.5. Referencias bibliográficas

A fondo

- Secure coding in C and C++
- Secure Coding Best Practices
- Closing web application security vulnerabilities with Fortify
- OWASP Code Review Guide
- SEI CERT Coding Standards

Test



4.1. Introducción y objetivos

Las principales características que diferencian a un desarrollador que programa teniendo en cuenta las reglas de codificación segura, de uno que no, son el **conocimiento, intención y precaución**.

El profesional con conocimiento de seguridad de software es consciente de que una de las maneras de evitar problemas de seguridad en las aplicaciones es, aparte de requisitos adecuados y uso de principios de diseño seguro, el **seguimiento** por parte de los programadores de prácticas de codificación seguras, que eviten la codificación de errores y debilidades explotables que deriven en **vulnerabilidades explotables** por los diferentes tipos de agentes maliciosos.

La posibilidad de desarrollar un software no confiable es atribuible a todas las partes involucradas en el ciclo de vida de su desarrollo y, en concreto, a los desarrolladores que escriben el código, desempeñando un papel vital en el desarrollo de software seguro libre de vulnerabilidades.

Los principales **objetivos** de este tema son los siguientes:

- ▶ Estudiar una serie de recomendaciones de **buenas y malas prácticas** de implementación.
- ▶ Conocer los **defectos más comunes** que se pueden cometer al codificar en lenguajes como C y Java.
- ▶ Prepararse para poder **analizar el código** en base al conocimiento de los defectos de programación que se pueden cometer.

4.2. Prácticas de codificación segura

Para iniciarse en la **implementación segura de código** es conveniente empezar con una serie de recomendaciones generales para tener en cuenta, como la línea a seguir en la profundización y conocimiento de esta disciplina. A continuación, se presentan una serie de **recomendaciones** que caen en las siguientes categorías:

BUENAS PRÁCTICAS

- ▶ Formarse no mismo.
- ▶ Formación continua.
- ▶ Manejo de los datos con precaución.
- ▶ Desechar código que no ha sido testeado y probado.
- ▶ Insistir en el proceso de revisión del código.
- ▶ Usar listas de comprobación.
- ▶ Revisar el proceso de mantenimiento.

MALAS PRÁCTICAS

- ▶ Usar en el código nombres relativos de ficheros.
- ▶ Referirse a un fichero con el mismo nombre dos veces en el mismo programa.
- ▶ Invocar programas en los que no se confía desde otros en los que se confía.
- ▶ Asumir que los usuarios no son maliciosos.
- ▶ No utilizar librerías seguras que generan números aleatorios como *random*.
- ▶ Invocar *Shell* desde líneas de comandos.
- ▶ Utilizar direcciones IP, MAC o direcciones de correo para identificar usuarios.
- ▶ Utilizar zonas de memoria accesibles por todos los usuarios.
- ▶ Almacenar información sensible en una base de datos sin protección.
- ▶ Visualizar *passwords* en las pantallas de los usuarios.
- ▶ Codificar *passwords* en la aplicación.
- ▶ Almacenar *passwords* en disco sin cifrar.
- ▶ Transmitir *passwords* sin cifrar.

Figura 1. Buenas y malas prácticas de codificación. Fuente: elaboración propia.

Estos pueden ser solo algunos de los aspectos más generales para tener en cuenta a la hora de desarrollar y cómo se verá más adelante. A la hora de hablar de **defectos de implementación de código** hay que tener en cuenta los defectos

particulares que se pueden cometer con el **lenguaje de programación** que se está utilizando, el compilador que se utiliza y otro software de terceros que seguramente no se tendrá posibilidad de verificar, pero que formará parte del sistema y, por lo tanto, podría introducir errores en el sistema que se está implementando.

Los **defectos de implementación** que se pueden cometer caen dentro de las siguientes categorías:

▣ Manejo de la entrada de datos
▣ Desbordamiento de <i>buffer</i>
▣ Errores y excepciones
▣ Privacidad y confidencialidad
▣ Programas privilegiados
▣ Errores aplicaciones web

Figura 2. Defectos de implementación. Fuente: elaboración propia.

4.3. Manejo de la entrada de datos

Una de las medidas de defensa más importantes que un programador puede llevar a cabo es **validar todas las entradas** que el sistema recibe, ya que son la fuente de algunas de las peores vulnerabilidades de las aplicaciones, como el desbordamiento de *buffer*, inyección de SQL y otras.

Los atacantes pueden **manipular** los diferentes tipos de datos de entrada con el fin de entregar cargas maliciosas a las aplicaciones. Los datos de entrada siempre deben ser validados por una **rutina de validación de entradas**, para evitar este tipo de ataques.

Validación de la entrada: conjunto de actividades que aceptan la entrada sin confiar en ella y comprueban su validez, limitándola solo a los valores que se sabe que son aceptables.

Validación de toda la entrada

Estas palabras serán repetidas durante todo el apartado: **validar todas las entradas**. Hay que definir la entrada **detalladamente** y pensar más allá de los datos que un usuario puede enviar. Si una aplicación consiste en **más de un proceso**, se deberá validar la entrada de cada proceso, incluso si aquella entrada llega de otra parte de la aplicación. Hay que validar la entrada incluso si llega por una **conexión segura**, de una fuente confiable o está protegida según permisos de archivo estrictos. Las rutinas de validación de entrada se pueden dividir en dos grupos principales:

- **Comprobaciones de sintaxis** que prueban el formato de la entrada y a menudo puede ser desacoplada de la lógica de aplicación y aplicada cerca del punto donde los datos entran en el programa.

- ▶ **Comprobaciones semánticas** que determinan si la entrada es apropiada, considerando la lógica de la aplicación y la función. Por lo general, tienen que aparecer junto a la lógica de aplicación, porque las dos están estrechamente relacionadas.

Se tendrá que realizar la **validación de entrada** no solo sobre la entrada de usuario, sino también sobre datos de cualquier fuente externa al código del sistema que está siendo desarrollado. Esta lista debería incluir, pero sin limitarse a ella, lo siguiente:

- ▶ Parámetros de línea de comando-
- ▶ Archivos de configuración.
- ▶ Datos recuperados de una base de datos.
- ▶ Subida de archivos.
- ▶ Variables de entorno.
- ▶ Importaciones de archivos planos.
- ▶ Servicios de red.
- ▶ Valores de registro.
- ▶ Propiedades de sistema.
- ▶ Archivos temporales.
- ▶ Variables de entorno.
- ▶ Direcciones URL e identificadores URI.
- ▶ Referencias de otros nombres de archivo.
- ▶ Encabezados Hyper Text Transfer Protocol (HTTP).

- ▶ Parámetros HTTP GET.
- ▶ Campos de formulario (especialmente los ocultos).
- ▶ Listas de selección.
- ▶ Listas desplegables.
- ▶ *Cookies* y comunicaciones Java *applets*.

A continuación, se verá un ejemplo de estos tipos de errores por ausencia de validación en la entrada que conduce una vulnerabilidad conocida como **inyección de SQL**. La Versión 2.1.9, Hiberne (Chess y West, 2007), un paquete popular *open source*, contiene un ejemplo excelente de qué **no hacer** con la entrada de línea de comando. La versión Java de la herramienta SchemaExport de Hibernate acepta un parámetro de línea de comando llamado `-- delimiter`, que se utilizaba para separar los comandos SQL en los *scripts* que generaba. El código se muestra ahora de una forma simplificada.

```
void*.doAlloc(int·sz)·{  
    return·malloc(sz);·/*·Implicit·conversion·here:·malloc()·  
    accepts·an·unsigned·argument.·*/  
}
```



Figura 3. Ejemplo de no validación de la entrada que puede ocasionar una vulnerabilidad de inyección SQL. Fuente: Chess y West, 2007.

La opción `-- delimiter` existe para que un usuario pueda especificar el separador que debería aparecer entre sentencias SQL. Valores típicos podrían ser un punto y coma o un retorno de carro. Pero el programa no coloca ninguna restricción contra el valor del argumento; por lo tanto, a través de un parámetro de línea de comando se puede escribir cualquier cadena que se quiera en la sentencia SQL generada, incluyendo órdenes de SQL adicionales.

Por ejemplo, si una simple **SELECT** fue generada con -- delimiter ';', esto generaría un *script* para ejecutar la sentencia siguiente:

```
SELECT * FROM items WHERE owner = "admin"
```

Pero si la misma consulta fue emitida con la opción malévola -- delimiter ';' DELETE FROM items;' generaría una sentencia que limpia a fondo la tabla *items* con la sentencia siguiente:

```
SELECT * FROM items WHERE owner = "admin"; DELETE FROM items
```

Establecer los límites de confianza

Un **límite de confianza** se puede pensar como una línea dibujada a través de un programa, fuera de la cual no se confía en los datos. Al otro lado de la línea, los datos se asumen seguros para alguna operación particular. El objetivo de la lógica de validación es el de permitir a los datos cruzar el límite de confianza, moverse de la zona no segura a la zona en la que se confía. El modo más fácil de cometer este error es el de permitir **datos confiables y no confiables** mezclados en la misma estructura de datos.

El ejemplo de la figura siguiente demuestra un problema común de frontera de confianza en Java. Los datos no confiables llegan a una **petición HTTP** y el programa almacena los datos en un **objeto de sesión HTTP** sin realizar primero una validación. Como el usuario directamente no puede tener acceso al objeto de sesión, los programadores típicamente creen que pueden **confiar** en la información almacenada en el objeto de sesión. Combinando datos validados y no validados en la sesión, este código viola un límite implícito de confianza.

```
status = request.getParameter("status");  
if (status != null && status.length() > 0) {  
    session.setAttribute("USER_STATUS", status).  
}
```



Figura 4. Ejemplo de violación de los límites de confianza. Fuente: Chess y West, 2007.

Realizar fuerte validación de entrada

Una correcta aproximación a la validación de entrada es **comprobar la entrada** frente a una lista de valores correctos conocidos. Una buena validación de entrada no intenta comprobar valores específicos no válidos. Se considera mejor la comprobación contra:

Whitelisting: lista de valores conocidos válidos.

Cuando el juego de valores de entrada posibles es pequeño, se puede usar la **selección indirecta** para conseguir que el *whitelist* sea imposible de evitar, es la primera opción cuando se realiza la validación de entrada. Al final, se pone:

Lista negra (*Blacklinting*): intento de enumerar todas las entradas posibles inaceptables.

Realizar una buena validación de entrada por defecto

En vez de codificar una nueva solución al problema de validación de entrada cada vez, se deberá diseñar el programa de modo que haya un lugar **claro, constante, y obvio** para la validación de entrada. La aplicación debería hacer pasar toda la entrada por esta lógica de validación, y dicha lógica debería **rechazar** cualquier entrada que no pueda ser validada.

Esto requiere la creación de una **capa de abstracción** sobre las bibliotecas del sistema en la que el programa suele ser introducido. Hacer una buena validación de entrada por defecto, creando una capa de funciones o métodos, se denomina **seguridad API**. La siguiente figura muestra cómo se interpone el API de seguridad entre el programa y las librerías de sistema.

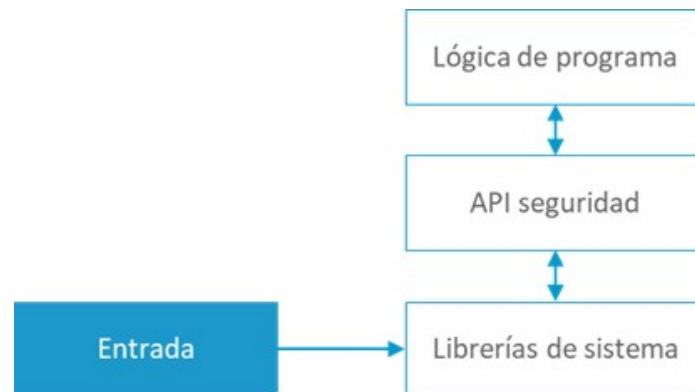


Figura 5. Las API de seguridad implican un amplio contexto para hacer la validación de entrada. Fuente: elaboración propia.

Comprobar la longitud de la entrada

La lógica de validación **front-end** siempre debería comprobar la **longitud de la entrada** contra un mínimo y máximo. Las comprobaciones de longitud son, por lo general, fáciles de añadir, pues no se requiere mucho conocimiento sobre el significado de la entrada. Se deberá tener cuidado cuando el programa transforma la entrada antes del tratamiento, pues esta puede hacerse más larga durante el proceso.

```
if (path != null &&
    path.length() > 0 && path.length() <= MAXPATH) {
    fileOperation(path);
}
```



Figura 6. Comprobación de la longitud de un campo. Fuente: Chess y West, 2007.

Comprobar el tamaño de los campos numéricos

Comprobar la entrada numérica tanto contra un valor máximo como contra un valor mínimo como parte de la validación de la entrada. Tener cuidado con las operaciones que podrían ser capaces de llevar un número más allá de su valor máximo o mínimo.

Cuando un atacante aprovecha la capacidad limitada de una variable de tipo entero, el problema es el desbordamiento de este tipo de datos. En **C** y **C++** el desbordamiento de número entero típicamente supone parte de la preparación para un ataque de desbordamiento de *buffer*. El desbordamiento de número entero en Java no conduce vulnerabilidades de desbordamiento, pero puede causar un **comportamiento indeseable**.

El mejor modo de evitar problemas de desbordamiento de un número entero es comprobar toda la entrada de este tipo de datos, tanto contra una cota superior como contra una cota inferior. Por ejemplo, `malloc()` acepta un entero y realiza una conversión implícita a entero si signo, si `doAlloc()` recibe un número negativo como argumento puede resultar en un intento de reservar una gran cantidad de memoria.

```
void* doAlloc(int sz) {  
    return malloc(sz); /* Implicit conversion here: malloc()  
    accepts an unsigned argument. */  
}
```



Figura 7. Comprobación de tipos. Fuente: Chess y West, 2007.

Prevenir vulnerabilidades de meta-caracteres

Si se permite a los atacantes **controlar** los comandos que se envían a la base de datos, sistemas de ficheros, navegador u otros subsistemas, se podría cambiar la finalidad de este. Se pueden tener **problemas serios** como consecuencia de ello. Como ejemplos, tenemos los siguientes:

Ataque	Implicaciones
SQL Injection	Accede/modifica datos en la base de datos
XPath Injection	Accede/modifica datos en formato XML
SSI Injection	Ejecuta comandos sobre el servidor y accede a datos sensibles
LDAP Injection	<i>Bypass</i> de autenticación
MX Injection	Usa el servidor de correos como una máquina de <i>spam</i>
HTTP Injection	Modifica o intoxica el caché de los sitios web

Tabla 1. Defectos de inyección. Fuente: elaboración propia.

- **Inyección de SQL:** la realización de sentencias SQL donde se combinan palabras clave con datos tiene el problema de que alguien malintencionadamente pueda alterar las estructuras de control y, por lo tanto, el significado de la sentencia, cuando la intención era solamente suministrar datos a la sentencia sin alterar las estructuras de control. Un atacante explota este tipo de vulnerabilidades especificando **meta-caracteres** que tienen un significado especial, como: comillas simples (') o dos puntos (..) —peligroso en sistema de ficheros—. Para comandos de *shell*: punto y coma (;), (&&), carácter de nueva línea (/n) para los ficheros de *log*.

En el ejemplo de la siguiente figura se muestra este tipo de vulnerabilidad en una sentencia SQL que se forma concatenando *strings* de tal forma que se deja abierta a un ataque conocido como **inyección de SQL**:

```
String userName = ctx.getAuthenticatedUserName();
String itemName = request.getParameter("itemName");
String query = "SELECT * FROM items WHERE owner = '"
    + userName + "' AND itemname = '"
    + itemName + "'";
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(query);
```



Figura 8. Ejemplo de *bug* que deja abierto el código abierto a un ataque de inyección de SQL. Fuente:

Chess y West, 2007.

El programador que escribió el código pretendía formar consultas como:

```
SELECT * FROM items WHERE owner = <userName> AND itemname =
    <itemName>;
```

Pero un atacante puede asignar al campo **itemname** la cadena "name' OR 'a'='a", con lo que la sentencia se convierte en:

```
SELECT * FROM items WHERE owner = 'wiley' AND itemname = 'name' OR 'a'='a';
```

La adición de OR 'a'='a' convierte la sentencia pretendida a esta: SELECT * FROM ítems. Esta sentencia extrae todas las filas de la tabla ítems, lo que posibilita que el atacante las pueda visualizar. Este es un pequeño ejemplo de lo que un atacante puede hacer. Muestra que se podrían también borrar todos los datos de la tabla y ejecutar muchas más sentencias distintas de la pretendida, que solo mostrar una fila al usuario.

La forma de ayudar a prevenir este tipo de ataques es utilizar **consultas parametrizadas** que fuerzan a la distinción entre **estructuras de control**, como son las palabras clave de una sentencia SQL de los que son puramente **datos de entrada** por parte del usuario de la aplicación. Los programadores pueden, explícitamente, especificar a la base de datos lo que debe ser tratado como comando y lo que debe ser tratado como datos. Debe evitarse formar consultas construidas con sentencias parametrizadas y cadenas concatenadas:

```
String item = request.getParameter("item");  
String q="SELECT * FROM records WHERE item=" + item;  
PreparedStatement stmt = conn.prepareStatement(q);  
ResultSet results = stmt.executeQuery();
```



Figura 9. Ejemplo de consulta SQL construida con el uso de sentencias parametrizadas y concatenadas.

Fuente: Chess y West, 2007.

- **Manipulación de rutas:** este tipo de error tiene lugar cuando se permite en las entradas de un usuario incluir meta-caracteres de sistemas de ficheros como: *slash* (/), *backslash* (\) y punto (.

Donde se espera una **ruta relativa**, un atacante puede convertirla en una **ruta absoluta** o recorrer el sistema de ficheros a una posición no planeada subiendo en el árbol de directorio. Se denomina a un acceso al sistema de ficheros no autorizado de este tipo manipulación de ruta. El código del ejemplo de la figura siguiente muestra la **entrada a la aplicación** desde una **petición HTTP** para crear un nombre de archivo. El programador no ha considerado la posibilidad de que un atacante podría proporcionar un nombre del archivo como , que hace que la aplicación suprima uno de sus propios archivos de configuración.

```
String rName = request.getParameter("reportName");  
File rFile = new File("/usr/local/apfr/reports/" + rName);  
rFile.delete();
```



Figura 10. Ejemplo de vulnerabilidad de manipulación de ruta. Fuente: Chess y West, 2007.

Las vulnerabilidades de manipulación de ruta son relativamente fáciles de prevenir con *whitelists*.

- **Inyección de comandos:** si se permite al usuario especificar comandos de sistema que su programa ejecuta, los atacantes podrían ser capaces de hacer que el sistema ejecute comandos maliciosos en su nombre. Se denomina inyección de comandos a la ejecución de comandos no autorizada.

El código en el ejemplo de la siguiente figura es para ejecutar un *backup* de una base de datos **ORACLE** en Windows. Se acepta un parámetro que especifica un tipo de *backup* con la utilidad **rman**. Se necesita un acceso privilegiado para poder acceder a la BD; por lo tanto, la aplicación se ejecuta con un **usuario privilegiado**. Un atacante podría pasar como parámetro "&& del c:\\dbms*.*". Cualquier comando que se inyectara se ejecutaría con los privilegios que se tenían para trabajar con la BD.

```
String btype = request.getParameter("backuptype");  
String cmd = new String("cmd.exe /K \"c:\\util\\rmanDB.bat "  
+ btype + "&&c:\\utl\\cleanup.bat\"")  
Runtime.getRuntime().exec(cmd);
```



Figura 11. Ejemplo de inyección de comandos. Fuente: Chess y West, 2007.

- **Falsificación de logs:** los *logs* son un objetivo para atacantes, pues son un recurso valioso para administradores de sistema y desarrolladores. Si los atacantes pueden falsificar el valor que es escrito en los *logs*, podrían ser capaces de fabricar eventos en el sistema mediante la inclusión de entradas corrompidas. Los archivos de *log* falsificados o, de otro modo, archivos de *log* corrompidos, pueden usarse para seguir las pistas de un atacante o implicar a otra parte en la comisión de un acto malicioso.

El ejemplo de la siguiente figura muestra un trozo de código de una aplicación web que intenta leer un número entero desde un objeto *request*. Si el valor, al ser comprobado, no es entero, se escribe el evento de *log* correspondiente indicando lo ocurrido.


```
String val = request.getParameter("val");  
try {  
    int value = Integer.parseInt(val);  
}  
catch (NumberFormatException) {  
    log.info("Failed to parse val = " + val);  
}
```



Figura 12. Ejemplo de falsificación de log. Fuente: Chess y West, 2007.

Si un usuario envía a la aplicación *twenty-one* en el parámetro *val*, se registra la siguiente entrada:

INFO: Failed to parse val=twenty-one

Sin embargo, si un atacante envía:

«twenty-one%0a%0aINFO:+User+logged+out%3dbadguy»

Se registra la siguiente entrada:

INFO: Failed to parse val=twenty-one INFO: User logged out=badguy

4.4. Desbordamiento de buffer

Un desbordamiento de *buffer* ocurre cuando un programa escribe datos fuera de los límites de la memoria asignada.

Las **vulnerabilidades por desbordamiento de *buffer***, por lo general, son explotadas con el objetivo de superponer valores en la memoria en provecho del atacante. Los errores de desbordamiento *buffer* están muy extendidos y normalmente dan a un atacante un control alto del código vulnerable. Este tipo de vulnerabilidades se puede clasificar en los siguientes tipos:



Figura 13. Tipos de vulnerabilidades de desbordamiento de *buffer*. Fuente: elaboración propia.

La mejor forma de prevenir el desbordamiento de *buffer* es utilizar un lenguaje de programación que **force la comprobación de tipos y de memoria** de forma que su gestión sea segura. C#, Java, Python, Ruby o dialectos de C como CCured y Cyclone son lenguajes de este tipo. Se usa el término *safe* para referirse a lenguajes que automáticamente realizan comprobaciones en tiempo de ejecución para impedir a los programas violar los límites de la memoria asignada.

Los **lenguajes seguros** deben proporcionar **dos propiedades** para asegurar que los programas respetan límites de asignación:

- **Seguridad de memoria.** Requiere que el programa no leerá o escribirá datos fuera de los límites de la memoria asignada.

- **Seguridad de tipo.** En los lenguajes que cumplen esa propiedad se asigna un tipo a los objetos, como, por ejemplo, un **tipo int**, puntero a int, puntero a función, etc. Esta propiedad asegura que las operaciones en el objeto son siempre compatibles con su tipo. Sin esta característica, cualquier valor arbitrario podría usarse como una referencia en la memoria. **C y C++** son lenguajes inseguros y ampliamente utilizados y, por lo tanto, el programador es el responsable de prevenir que las operaciones que manipulan la memoria puedan resultar en **desbordamientos** de *buffer*. Un ejemplo de este tipo de error que se puede cometer en este lenguaje es cuando se declara un puntero a un número entero (int) y luego se utiliza como un puntero a una función, tal y como se muestra en ejemplo siguiente:

```
int (*cmp)(char*,char*);
int *p = (int*)malloc(sizeof(int));
*p = 1;
cmp = (int (*)(char*,char*))p;
cmp("hola","adios"); // El programa realiza un crash
```



Figura 14. Error de seguridad de tipos. Fuente: elaboración propia.

La máquina virtual de java (**JVM**), en sí misma, está escrita en C para una plataforma dada. Esto quiere decir que la JVM, en sí misma, puede ser susceptible a **problemas de desbordamiento**.

```
class Echo {
    public native void runEcho();
    static {
        System.loadLibrary("echo");
    }
    public static void main(String[] args) {
        new Echo().runEcho();
    }
}
```



Figura 15. Llamada a un método nativo JNI. Fuente: Chess y West, 2007.

El código de la figura siguiente implementa el método nativo definido en la clase **Echo**. El código es vulnerable a **buffer overflow** causada por una llamada ilimitada a `gets()`.



```
#include <jni.h>
#include "Echo.h" // Echo class compiled with javah
#include <stdio.h>
JNIEXPORT void JNICALL
Java_Echo_runEcho(JNIEnv *env, jobject obj)
{
    char buf[64];
    gets(buf);
    printf(buf);
}
```

Figura 16. Vulnerabilidad de *buffer overflow* en un método nativo. Fuente: Chess y West, 2007.

Ataque de desbordamiento de buffer basado en el stack

En un ataque clásico por desbordamiento de *buffer*, el atacante envía los datos que contienen un segmento de código malévolo a un programa que es vulnerable a este error, basado en el *stack*. Además del código malévolo, el atacante incluye la dirección de memoria del principio del código.

El código malévolo suele ser un **shellcode**: conjunto de instrucciones de programación, generalmente en lenguaje ensamblador y trasladadas a **opcodes** que suelen ser inyectadas en la pila (o *stack*) de ejecución de un programa para conseguir que la máquina en la que reside se ejecute la **operación de tipo malicioso**. Deben ser de longitud corta para poder ser inyectadas dentro de la pila en un espacio reducido y se utilizan para ejecutar código malévolo. En la figura se muestra un ejemplo:

```
#include <unistd.h>
int main() {
    char *scode[2];
    scode[0] = "/bin/sh";
    scode[1] = NULL;
    execve (scode[0], scode, NULL);
}
```



Figura 17. Ejemplo de *shellcode*. Fuente: Wikipedia, s. f.

Esta **shellcode**, que ejecuta la *shell* `/bin/sh`, realiza una llamada al sistema **execve** para realizar la ejecución de la *shell* (ventana de comandos) contenida dentro del *array* *scode*.

Cuando el desbordamiento de *buffer* ocurre, el programa escribe los datos del atacante en el *buffer* y sigue más allá de sus límites hasta que superponga la dirección de vuelta de la función con la dirección del principio del **código malicioso**. Cuando la función devuelve el control, salta al valor almacenado en su dirección de retorno.

Normalmente, esto lo devolvería al contexto de la función llamada, pero debido a que la dirección de retorno ha sido superpuesta, el control salta el *buffer* y comienza a ejecutar el código malicioso del atacante. Para aumentar la probabilidad de adivinar la dirección correcta del código malicioso, los atacantes típicamente rellenan el principio de su entrada con una serie de **instrucciones NOP** (ninguna operación).

El código del ejemplo de la siguiente figura define el problema de función `trouble()`, que asigna un **buffer char** y un `int` en el *stack* (pila) y lee una línea de texto de **stdin** con `gets()` en el *buffer*. Como `gets()` sigue leyendo la entrada hasta que se encuentra un carácter de **final-de-línea**, un atacante puede desbordar el *buffer line* con datos maliciosos. En el ejemplo, esta función declara dos variables locales y usa `gets()` para leer una línea de texto en el *buffer line* del *stack* de 128 octetos.

La pila es un lugar en la memoria de un equipo en el que todas las variables que se declaran e inicializan antes de pasarlas a tiempo de ejecución se almacenan.

Otro ejemplo mostrado en la siguiente Figura 18, expone lo que ocurre en un clásico ataque de **buffer overflow**.

- ▶ El **primer marco de stack** representa el contenido de la memoria después de llamar a `trouble()`, pero antes de que sea ejecutada. La variable local `line` es asignada sobre el `stack` que comienza en la dirección `0xNN`. La variable local `a` está justo encima de `line` en la memoria; la dirección de vuelta (`0x <retorno>`) está encima de `a`. Asumir que `0x <retorno>` apunta a la función que llamó a `trouble()`.
- ▶ El **segundo marco de stack** ilustra un argumento en el cual el `trouble()` se comporta normalmente. ¡Se lee la entrada! ¡EJEMPLO BUENO! y retorna. Se puede ver ahora que `line` está parcialmente llena con un *string* de entrada y que otros valores almacenados sobre el `stack` son inalterados.
- ▶ El **tercer marco de stack** ilustra un escenario en el cual un atacante explota la vulnerabilidad de desbordamiento de *buffer* en `trouble()` y hace que se ejecute el código malévolo en vez de retornar normalmente. En este caso, `line` ha estado llena de una serie de **NOPs**, el código del *exploit*, y la dirección del principio del *buffer*, `0xNN`.

```
void trouble() {  
    int a = 32; /*integer*/  
    char line[128]; /*character array*/  
    gets(line); /*read a line from stdin*/  
}
```



Figura 18. Empleo incorrecto de `gets()` que puede incurrir en desbordamiento de *buffer*. Fuente: Chess y West, 2007.

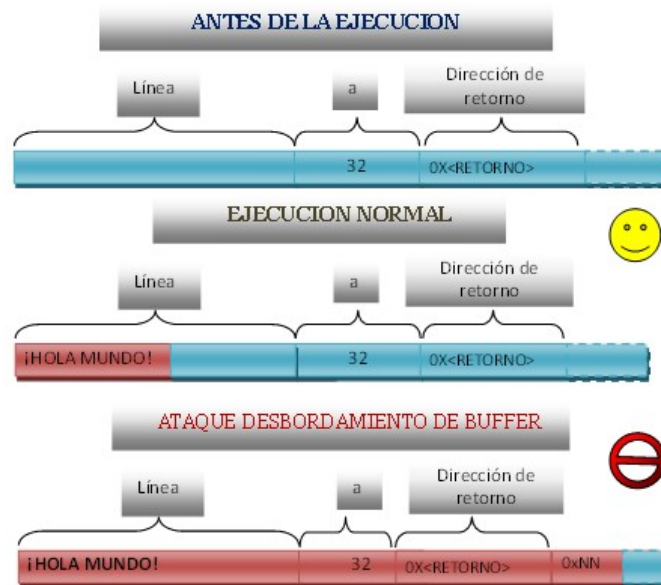


Figura 19. Ejemplo de ataque de *buffer overflow*. Fuente: Chess y West, 2007.

Ataques relacionados con la reserva dinámica de memoria (heap overflow)

Uno de los errores de concepto que se tienen sobre las vulnerabilidades de *buffer overflow*, es que son solo explotables cuando el *buffer* está en el *stack*. Los ataques basados en el *heap* pueden sobrescribir datos importantes almacenados en el mismo y cambiar el flujo del programa. Por ejemplo, se podría sobrescribir el valor de un puntero a una función, de tal forma que cuando el programa invoque la función referenciada por el puntero a la función se ejecutará el *exploit*.

Heap es la sección de la memoria de un equipo donde todas las variables creadas o inicializadas en tiempo de ejecución se almacenan.

Gran parte de las vulnerabilidades relacionadas con la reserva de memoria dinámica se corresponden a fallos de programación que encajan en alguna de las siguientes categorías:

ERRORES HEAP OVERFLOWS

- ▣ Memoria no utilizada, sin desasignar (*memory leaks*)
- ▣ Uso de memoria después de ser liberada (*use after free*)
- ▣ Acceso a una variable después de liberarla (*dereference after free*)
- ▣ Liberar la memoria más de una vez (*double free*)
- ▣ Uso de un puntero con valor NULL (*null dereference*)

Figura 20. Errores de *heap overflows*. Fuente: elaboración propia.

- **Memoria no utilizada, sin desasignar (*memory leaks*).** Favorece de forma potencial ataques de denegación de servicio. Se produce cuando una aplicación **adquiere la memoria y falla al liberarla** de nuevo al sistema operativo. La siguiente función en **C** deliberadamente pierde memoria al no liberar el puntero a la memoria asignada, una vez que el **puntero a** está fuera de ámbito, una vez ejecutada la `function_which_allocates()` finaliza sin liberar **a**.

```
#include <stdlib.h>

void function_which_allocates(void) {
    /* reserve en memoria un array de 45 datos tipo floats */
    float * a = malloc(sizeof(float) * 45);
    /* la zona de memora direccionada ppor el punter a 'a' */
    return
    /* al volver a la función main se olvida de liberar la memoria
    reservada con la función malloc*/
}

int main(void) {
    function_which_allocates();
    /* el puntero "a" ya no existe por lo que no se puede liberar y por
    tanto la memoria seguirá estando asignada*/
}
```

Figura 21. Ejemplo de *memory leaks*. Fuente: Wikipedia, s. f.

- **Uso de memoria después de ser liberada (*use after free*).** Ocurre cuando un programa continúa usando un puntero que ha sido previamente liberado. Si esa memoria se vuelve a reservar, un atacante puede lanzar un ataque de *buffer overflow* catalogado con el CWE-416.


```
char* ptr = (char*)malloc (SIZE);
...
if (tryOperation() == OPERATION_FAILED) {
    free(ptr);
    errors++;
}
...
if (errors > 0) {
    logError("operation aborted before commit", ptr);
}
```



Figura 22. Ejemplo de *use after free*. Fuente: Chess y West, 2007.

- **Acceso a una variable después de liberarla (*dereference after free*).** Caso concreto de *use after free* que ocurre cuando intentamos acceder a la memoria dinámica previamente liberada como consecuencia de un `free()`. En el ejemplo de abajo (Merino, 2011), el puntero a `char` `buf2R1` es liberado, aunque más adelante se vuelve a referenciar desde la función `strncpy`. Como consecuencia, es probable que se corrompan datos asignados posteriormente en dicha dirección de memoria tras haber liberado la misma, o que se produzca un *crash* de la aplicación al intentar sobrescribir «memoria aleatoria» en la que el proceso no tiene permiso.

```
#include <stdio.h>
#include <unistd.h>
#define BUFSIZER1 512
#define BUFSIZER2 ((BUFSIZER1/2) - 8)
int main(int argc, char **argv) {
    char *buf1R1;
    char *buf2R1;
    char *buf2R2;
    char *buf3R2;
    buf1R1 = (char *) malloc(BUFSIZER1);
    buf2R1 = (char *) malloc(BUFSIZER1);
    free(buf2R1); /* libero el puntero*/
    buf2R2 = (char *) malloc(BUFSIZER2);
    buf3R2 = (char *) malloc(BUFSIZER2);
    strncpy(buf2R1, argv[1], BUFSIZER1-1); /* lo vuelvo a utilizar*/
    free(buf1R1);
    free(buf2R2);
    free(buf3R2);
}
```



Figura 23. Ejemplo de *reference free*. Fuente: Merino, 2011.

- **Liberar la memoria más de una vez (*double free*).** Cuando un programa libera un trozo de memoria más de una vez `free()`, las estructuras de datos para gestión de la memoria pueden llegar a corromperse, lo que puede ocasionar que el programa falle, o causar que se llame dos veces a `malloc()`. Esto último puede dar el control a un atacante sobre los datos que se escriben en esa memoria doblemente reservada. Entonces, se tiene un programa potencialmente expuesto a ataque de *buffer overflow*.

```
int * ab = (int*)malloc (SIZE);  
...  
if (c == 'EOF') {  
    free(ab); }  
...  
free(ab);
```



Figura 24. Ejemplo de *double free*. Fuente: Merino, 2011.

- **Uso de un puntero con valor NULL (*null dereference*).** El uso de un puntero con valor *null* puede ocasionar fallos de segmentación en la gestión de la memoria.

```
#include <stdlib.h>  
int main(int argc, char *argv[]){  
    int k=0;  
    int *p=(int *)NULL;  
    switch(k) {  
        case 0:  
            if (*p) k = *p;  
        default:  
            break;  
    }  
    return 0;  
}
```



Figura 25. Ejemplo de *null dereference*. Fuente: elaboración propia.

Manipulación de strings

La estructura de datos de **strings** de C básica serie de caracteres terminada por el carácter nulo, es propensa a **error** y, además, las **funciones de biblioteca** para la manipulación de estos solo empeoran el panorama.

En el presente apartado se introduce al alumno en el estudio de los problemas de seguridades relativas a las funciones de manipulación de *strings* y las de segunda generación; que son más seguras, pues limitan el número de datos copiados por parámetro.

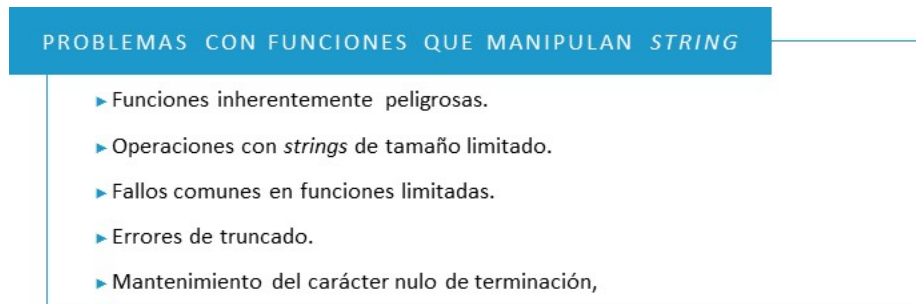


Figura 26. Problemas con funciones que manipulan *string*. Fuente: elaboración propia.

Funciones inherentemente peligrosas

Específicamente, hay que **evitar** el uso de funciones como `gets()`, `scanf()`, `strcpy()` o `sprintf()`.

- ▶ **gets()**. Esta función lee, de la entrada estándar, una corriente de *bytes*, y la almacena en el *array* apuntado por *s* hasta que se encuentra el carácter de nueva línea o de fin de fichero. Puede producir desbordamiento de **buffer**, que se le pasa como argumento, si es más pequeño que la fuente de entrada, que en este caso es la entrada estándar: el teclado. La función `getws()` se comporta de la misma forma.

`char gets(char *s)`

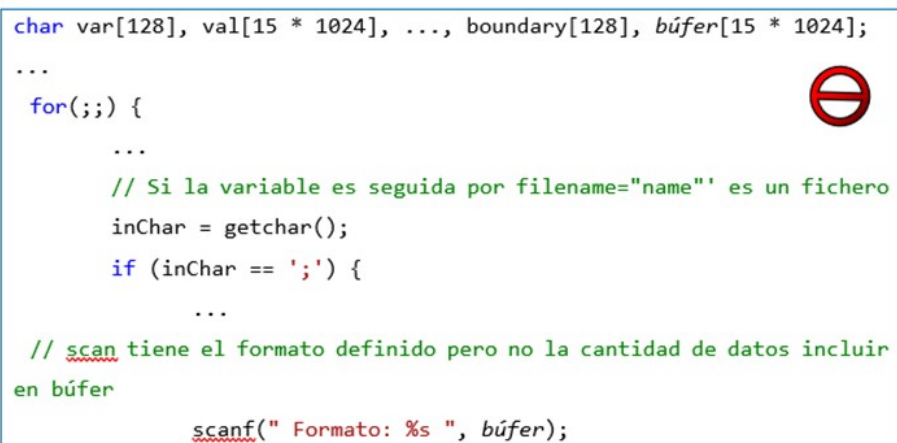
```
char line[512];
gets(line);
```



Figura 27. Llamada insegura a `gets()` similar a la explotada por el gusano Morris. Fuente: Chess y West, 2007.

- **scanf()**. Función que lee de la entrada estándar los caracteres que corresponden al formato que se le especifica. Por ejemplo, si se especifica %s, se leerán los caracteres de la entrada en el *buffer* hasta que llegue un carácter no-ascii, lo que puede ocasionar un **desbordamiento de buffer**. También se puede especificar %255s y limitar a ese número de caracteres los que se pueden leer de la entrada y usarse la función de un modo más seguro. Funciones de similar comportamiento son fscanf(), wscanf()).

scanf(const char *FORMAT [, ARG, ...])



```
char var[128], val[15 * 1024], ..., boundary[128], búfer[15 * 1024];
...
for(;;) {
    ...
    // Si la variable es seguida por filename="name" es un fichero
    inChar = getchar();
    if (inChar == ';') {
        ...
        // scanf tiene el formato definido pero no la cantidad de datos incluir
        // en búfer
        scanf(" Formato: %s ", búfer);
```

Figura 28. Código de w3-msql 2.0.11 vulnerable a un desbordamiento de búfer remoto causada por una llamada insegura a scanf(). Fuente: Chess y West, 2007.

- **strcpy()**. A diferencia de las funciones anteriores, opera en datos ya almacenados en una variable de programa, lo que hace que su manejo suponga obviamente menos riesgo de seguridad. Esta función copia el contenido de un *buffer* en otro hasta que se encuentra un carácter nulo en el *buffer* fuente; por lo tanto, hay que tener cuidado de que el *buffer* fuente sea más pequeño que el destino y que esté terminado por el carácter nulo. Funciones con similar comportamiento son wcsncpy(), lstrcpy ().

char strcpy(char *DST, const char *SRC)

```
char *FixFilename(char *filename, int cd, int *ret) {
...
    char fn[128], user[128], *s;
    ...
    s = strrchr(filename, '/');
    if(s) {
        strcpy(fn, s+1);
    }
}
```



Figura 29. Código del programa php.cgi en PHP / FI 2.0beta10 vulnerable a un desbordamiento de *buffer* remoto causada por una llamada insegura para `strcpy()`. Fuente: Chess y West, 2007.

- **sprintf()**. Para usarla de forma segura, hay que comprobar que el *buffer* de destino pueda acomodarse a la combinación de todos los argumentos de la fuente de entrada, controlando los tamaños **fuentes-destino** y las conversiones de formato que se especifican. Si la longitud del argumento fuente es mayor que la del destino, se producirá un *buffer overflow*. Funciones de comportamiento similar son `fprintf()`, `printf()`, `swprintf()`.

```
int sprintf(char *STR, const char *FORMAT [, ARG, ...])
```

```
char speed[128];
...
sprintf(speed, "%s/%d", (cp = getenv("TERM")) ? cp : "",
        (def_rspeed > 0) ? def_rspeed : 9600);
```



Figura 30. Código de la Versión 1.0 del daemon de Telnet Kerberos 5 que contiene un desbordamiento de *buffer* debido a que la longitud variable de entorno `TERM`, que nunca se valida. Fuente: Chess y West, 2007.

Operaciones con *strings* de tamaño limitado.

Muchas de las primeras vulnerabilidades descubiertas en C, C++ eran causa de operaciones con *strings*. Cuando se revisó el estándar de C se introdujeron nuevas funciones equivalentes a las que se han analizado en el apartado anterior con un

parámetro que limita la longitud del *buffer* de destino de los datos. Conceptualmente, el uso de estas funciones con limitación de *buffer* destino se realiza reemplazando las antiguas funciones con las nuevas:

`strcpy(buf, src) → strncpy(buf, src, sizeof(buf))`

Para asegurarse que ningún desbordamiento de *buffer* ocurre aquí, una herramienta tiene que asegurarse solo de que `dest_size` no es más grande que la cantidad de espacio asignado para `dest`. El tamaño y el contenido de `src` no importan. En la mayoría de los casos, es mucho más fácil comprobar que `dest` y `dest_size` están de acuerdo el uno con el otro.

Función no limitada librería C estándar	Función limitada equivalente librería C estándar	Función limitada equivalente Windows Safe CRT
<code>char * gets(char *dst)</code>	<code>char * fgets(char *dst, int bound, FILE *FP)</code>	<code>char * gets_s(char *s, size_t bound)</code>
<code>int scanf(const char *FMT [, arg, ...])</code>	Ninguna	<code>errno_t scanf_s(const char *FMT [, ARG, size_t bound,...])</code>
<code>int sprintf(char *str, const char *FMT [, arg, ...])</code>	<code>int sprintf(char *str, size_t bound, const char *FMT, [, arg, ...])</code>	<code>errno_t sprintf_s(char *dst, size_t bound, const char *FMT [, arg, ...]) w</code>
<code>char * strcat(char *str, const char *SRC)</code>	<code>char * strncat(char *dst, const char *SRC, size_t bound)</code>	<code>errno_t strcat_s(char *dst, size_t bound, const char *SRC)</code>
<code>char * strcpy(char *dst, const char *SRC)</code>	<code>char * strncpy(char *dst, const char *SRC, size_t bound)</code>	<code>errno_t strcpy_s(char *dst, size_t bound, const char *SRC)</code>

Tabla 2. Funciones ilimitadas con sus correspondientes limitadas. Fuente: adaptado de Chess y West, 2007.

El código de la figura siguiente muestra una vulnerabilidad de desbordamiento de *buffer* que es resultado del empleo incorrecto de funciones de manipulación de **strings ilimitadas** `strcat()` y `strcpy()` en Kerberos 5 Versión 1.0.6, CERT CA-2000-06, [15]. Dependiendo de la longitud de `cp` y `copy`, con cualquiera de las llamadas final a `strcat()` o a `strcpy()` podría desbordarse `cmdbuf`.

La vulnerabilidad ocurre en el código responsable de manipular un *string* de un comando, que inmediatamente sugiere que pudiera ser explotable debido a su proximidad a la entrada de usuario. De hecho, esta vulnerabilidad públicamente ha sido explotada para ejecutar órdenes no autorizadas sobre sistemas comprometidos.

```
if (auth_sys == KRB5_RECVAUTH_V4) {  
    strcat(cmdbuf, "/v4rcp");  
} else {  
    strcat(cmdbuf, "/rcp");  
}  
if (stat((char *)cmdbuf + offst, &s) >= 0)  
    strcat(cmdbuf, cp);  
else  
    strcpy(cmdbuf, copy);
```



Figura 31. Función ilimitada, vulnerabilidad en Kerberos 5. Fuente: Chess y West, 2007.

Errores de truncado

Incluso cuando se usan correctamente, las funciones de **strings limitadas** pueden introducir errores, porque truncan los datos que exceden el límite especificado.

El código de la figura siguiente muestra un **error de truncamiento de string** que se convierte en un problema de terminación. El error está relacionado con el empleo de la función `readlink()` que no termina con carácter nulo su *buffer* destino y puede devolver hasta el número de octetos especificados en su tercer argumento, el código de la figura cae en la trampa demasiado común de terminar a mano con carácter nulo la ruta expandida, *buf*, en este caso, un octeto más allá del final del *buffer*.

Este error por un byte, denominado **off by one**, podría ser inconsecuente dependiendo de lo que se almacena en la memoria justo más allá del *buffer*, pues permanecerá con eficacia terminado por el carácter uno hasta que otra posición de

memoria sea superpuesta. Es decir, `strlen(buf)` devolverá solo el tamaño real del *buffer* más uno, `PATH_MAX + 1`, en este caso.

Sin embargo, cuando **buf** posteriormente sea copiado en otro *buffer* con el valor de vuelta de `readlink()` → `len`, como el límite pasado a `strncpy()`, los datos en **buf** están truncados y el *buffer* de destino se deja sin terminar con el carácter nulo. Este «error de por uno», probablemente causará un serio desbordamiento de *buffer*.

```
char path[PATH_MAX];
char buf[PATH_MAX];
if(S_ISLNK(st.st_mode)) {
    len = readlink(link, buf, sizeof(path));
    buf[len] = '\0';
}
strncpy(path, buf, len);
```



Figura 32. Llamada a `strncpy()` que podría causar error de truncado. Fuente: Chess y West, 2007.

Hay que evitar truncar los datos. Si la entrada proporcionada es demasiado grande para una operación dada, hay que intentar manejar el problema redimensionando dinámicamente los *buffers*, o directamente rehusar realizar la operación e indicar al usuario lo que tiene que ocurrir para que la operación tenga éxito.

Mantenimiento del carácter nulo de terminación

En C, los *strings* dependen de la terminación nula apropiada; sin ello, su tamaño no puede ser determinado. Los **errores de terminación de strings** fácilmente pueden conducir a desbordamientos y errores lógicos. Estos problemas, normalmente se hacen más insidiosos porque ocurren, aparentemente, de manera **no determinista**, dependiendo del estado de la memoria cuando el programa se ejecuta.

Considerar el código de la figura siguiente, como `readlink()`, no termina con carácter nulo su *buffer* de salida, `strlen()` lee la memoria hasta que encuentra un carácter nulo y por lo que a veces produce valores incorrectos de longitud, dependiendo del contenido de memoria después de *buf*.

```
char buf[MAXPATH];  
readlink(path, buf, MAXPATH);  
int length = strlen(buf);
```



Figura 33. *String* no terminado en carácter nulo producido por `readlink()`. Fuente: Chess y West, 2007.

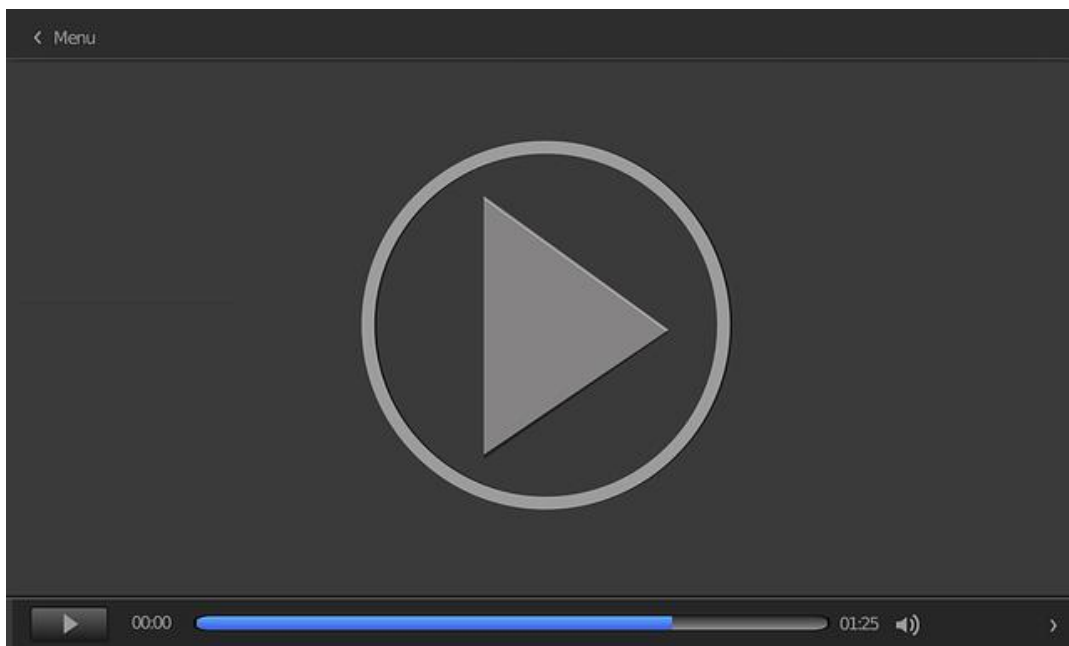
Nunca hay asumir que los datos del mundo exterior están correctamente terminados con carácter nulo. Si el *string* no se termina correctamente, la adición de un octeto nulo en el último octeto del *buffer* impedirá a otras operaciones como calcular mal la longitud del *string* o desbordar el *buffer*.



Figura 34. Ejemplo. Fuente: elaboración propia.

Errores de formato de cadena (format strings)

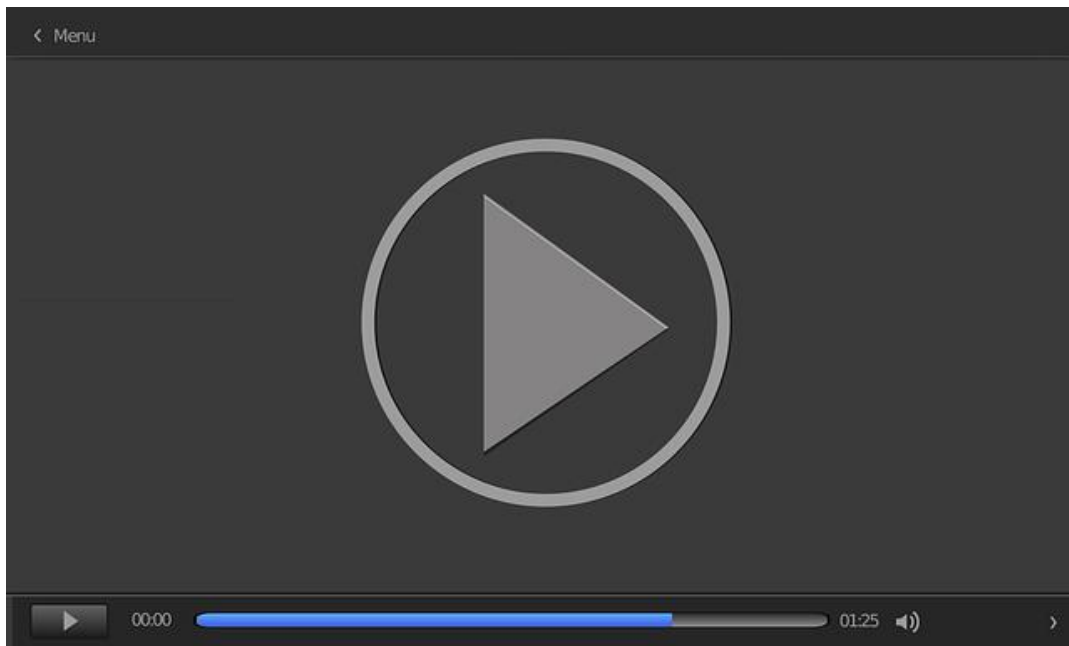
Los errores de formato de *string* se explican en el siguiente video. Una de las vulnerabilidades que se suelen cometer a la hora de programar en lenguaje C y C++ es la de *format string*. En este vídeo (*Vulnerabilidades de format string*) se profundiza en el estudio de este tipo de vulnerabilidad.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=59cadd44-d546-4037-bfae-adf700cff0bf>

El análisis estático de código fuente se considera la actividad más importante de entre las mejores prácticas de seguridad que se han de realizar en el curso del desarrollo de una aplicación. En la siguiente clase magistral (*Herramienta de análisis de código fuente SCA Fortify*) se realiza una introducción a una de las mejores herramientas de análisis estático de código del mercado, Fortify, de la compañía HP.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=efd8066a-ae3a-4108-9638-adf700cff13c>

4.5. Referencias bibliográficas

Chess, B., and West, J. (2007). *Secure Programming with Static Analysis*. Addison-Wesley Software Security Series.

Merino, B. (2011). *Software Exploitation*. INCIBE.

Shellcode. (s. f.). En *Wikipedia*. <https://es.wikipedia.org/wiki/Shellcode>

Secure coding in C and C++

Seacord, R. (2013). *Secure coding in C and C++*. Pearson.
<http://www.microsoft.com/en-us/download/details.aspx?id=12379>

Presentación sobre casos prácticos de errores de programación más comunes que se cometen en el lenguaje de programación C y C++.

Secure Coding Best Practices

Coding Tech (2018, junio 9). *Secure Coding Best Practices* [Vídeo]. Youtube. <https://www.youtube.com/watch?v=zIEdzJccdps>

Vídeo que presenta ejemplos de código que contienen vulnerabilidades y cómo corregirlas. También muestra diferentes formas para hacer frente a esas amenazas desde el momento del diseño. Está enfocada en el lenguaje C++ de aplicaciones o librerías que se ejecutan y enfrentan a ataques del mundo real.

Closing web application security vulnerabilities with Fortify

joseph2535 (s. f.). Closing Web Application Security Vulnerabilities with Fortify [Vídeo]. Youtube. <http://www.youtube.com/watch?v=z9vnunUu6Ac>

En el siguiente vídeo podrás ver cómo utilizar Fortify para detectar vulnerabilidades de seguridad web de tipo *cross site scripting* (XSS) e inyección SQL. Fortify realiza análisis de código fuente y trabaja en varios lenguajes, incluyendo Java, .NET y PHP.

OWASP Code Review Guide

Conklin, L. y Robinson, G. (2017). *CODE REVIEW GUIDE* [Guía].
https://owasp.org/www-pdf-archive/OWASP_Code_Review_Guide_v2.pdf

Guía que presenta prácticas fundamentales de codificación segura y una lista de recursos con más información para profundizar en las prácticas que propone.

SEI CERT Coding Standards

Schiela, R. (2020, mayo 29). *SEI CERT Coding Standards*. Software Engineering Institute.

<https://wiki.sei.cmu.edu/confluence/display/seccode/SEI+CERT+Coding+Standards>

Sitio web que incluye normas de codificación para los lenguajes de programación de uso común como C, C++, Java y Perl, y la plataforma Android™. Estos estándares se han desarrollado a través de un amplio esfuerzo por parte de los miembros de las comunidades de desarrollo y seguridad de software.

1. Señala la respuesta correcta. En los apuntes de la asignatura se presentan una serie de recomendaciones de buenas prácticas:

- A. Manejo de los datos con precaución.
- B. Confiar en software de terceros en operaciones críticas.
- C. Usar listas de errores.
- D. Usar en el código nombres relativos de ficheros.

2. Señala la respuesta correcta. ¿Qué tipo de vulnerabilidad se comete en este código?

- A. *Integer overflows*.
- B. Desbordamiento de *buffer*.
- C. Uso de datos invalidados.
- D. *Use after free*.

3. Señala la respuesta correcta. ¿Qué tipo de vulnerabilidad se comete en este código?

- A. *Integer overflows*.
- B. Desbordamiento de *buffer*.
- C. *Format string*.
- D. *Use after free*.

4. Señala la respuesta correcta. ¿Qué tipo de vulnerabilidad se comete en este código?

- A. *Buffer overflow*.
- B. Validación límites de confianza.
- C. Validación de entrada DNS.
- D. *Memory leaks*.

5. Señala la respuesta correcta. ¿Es el siguiente código correcto?
 - A. Es correcto.
 - B. Es incorrecto.
 - C. No se puede determinar.
 - D. Ninguna de las anteriores.

6. Señala la respuesta correcta. ¿Qué tipo de vulnerabilidad se comete en este código?
 - A. *Integer overflows*.
 - B. Desbordamiento de *buffer*.
 - C. *Format string*.
 - D. *Use after free*.

7. Señala la respuesta correcta. ¿Qué tipo de vulnerabilidad se comete en este código?
 - A. *Integer overflows*.
 - B. Desbordamiento de *buffer*.
 - C. *Format string*.
 - D. *Use after free*.

8. Señala la respuesta correcta. Al realizar una buena validación de entrada por defecto, una mejora con API de seguridad aumenta la capacidad de hacer lo siguiente:
 - A. Entender y mantener la lógica de validación de entrada.
 - B. Actualizar y modificar el intento de introducir la validación coherentemente.
 - C. Aplicar una validación de entrada sensible a contexto coherentemente a toda la entrada.
 - D. Descentralizar la lógica de validación.

9. Señalar la respuesta incorrecta. En el desarrollo de aplicaciones seguras y confiables se requiere el seguimiento de unas buenas prácticas:

- A. Insistir en el proceso de revisión de código.
- B. Formación continua.
- C. Invocar programas en los que no se confía desde otros en los que se confía.
- D. Manejo de los datos con precaución.

10. Señala la respuesta correcta. ¿Cuál es la mejor forma de prevenir ataques de desbordamiento de *buffer*?

- A. Tener precauciones al realizar conversiones de tipo.
- ☐ B. Utilizar un lenguaje de programación que fuerce la comprobación de tipos y de memoria.
- C. Comprobar los límites de memoria.
- D. Comprobar las longitudes del *buffer*.

Desarrollo Seguro de Software y Auditoría de la
Ciberseguridad

Tema 5. Codificación segura de aplicaciones II

Índice

Esquema

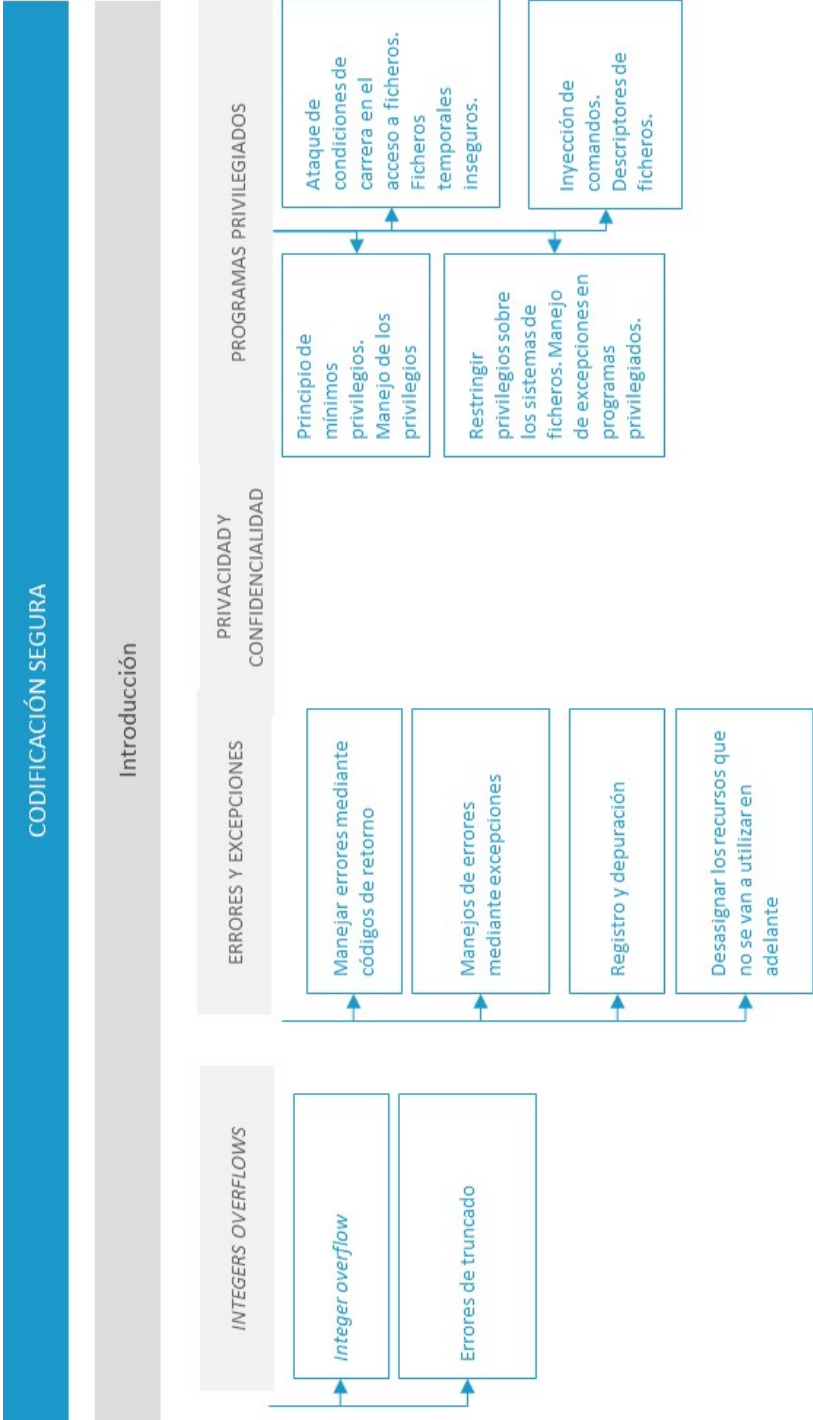
Ideas clave

- 5.1. Introducción y objetivos
- 5.2. Integers overflows
- 5.3. Errores y excepciones
- 5.4. Privacidad y confidencialidad
- 5.5. Programas privilegiados
- 5.6. Referencias bibliográficas

A fondo

- Common vulnerabilities guide for C programmers
- Secure coding techniques
- GOTO 2016. Secure Coding Patterns. Andreas Hallberg
- Secure coding
- SEI CERT Coding Standards

Test



5.1. Introducción y objetivos

La posibilidad de desarrollar **software no confiable** es atribuible a todas las partes involucradas en el ciclo de vida de desarrollo de software y, en concreto, a los desarrolladores de software que escriben el código, quienes desempeñan un papel vital en el desarrollo de **software seguro** libre de vulnerabilidades.

La formación de los programadores en esta materia es muy importante, porque si se conoce qué tipo de **defectos de seguridad** se pueden cometer en un lenguaje de programación, se lograrán evitar muchos de ellos desde el principio. Además, se hacen una serie de recomendaciones para tener en cuenta a la hora de la implementación, tanto de buenas como de malas prácticas.

El encontrar **bugs** de manera temprana conlleva una **reducción** del coste económico, en varios órdenes, con respecto a su descubrimiento en fases posteriores. Este hecho tiene que incitar a las organizaciones a poner especial interés en esta actividad y encuadrarla en los procesos y procedimientos de desarrollo.

De todas formas, inevitablemente, siempre se dejarán **defectos** en el código una vez terminada la fase de desarrollo; es por ello por lo que todavía se puede hacer más por eliminar la mayor parte de defectos posible al realizar un **análisis de código fuente** durante la codificación y al finalizarla.

Los principales **objetivos** de este tema son los siguientes:

- ▶ Estudiar una serie de recomendaciones de buenas y malas **prácticas de implementación**.

- ▶ Conocer los principales errores de programación que derivan en vulnerabilidades explotables, relativos a ***integers overflows***, errores y excepciones, privacidad y confidencialidad y programas privilegiados.
- ▶ Prepararse para poder analizar el código en base al conocimiento de los **defectos de programación** que se pueden cometer.

Si se entienden bien los defectos que se pueden cometer, se estará en disposición de poder analizar el código para encontrarlos y, posteriormente, corregirlos.

5.2. Integers overflows

Todos los tipos de representación de números enteros tienen una **limitada capacidad** debido a que se representan con un limitado número de bits. Esto es un hecho que normalmente se les olvida a los programadores, quienes pueden ocasionar errores de este tipo al intentar almacenar un valor **demasiado grande** en la variable asociada y generar que sobrepase esos límites inferiores y superiores, o un valor grande positivo se convierta en un valor grande negativo y viceversa, lo que provoca un **resultado inesperado** (valores negativos, valores inferiores, etc.).

En el documento de **INCIBE** (2012) se indica que:

«Este tipo de error puede tener consecuencias graves cuando el valor que genera el *integer overflow* es resultado de alguna entrada de usuario (es decir, que puede ser controlado por el mismo) y cuando, de este valor, se toman decisiones de seguridad, se toma como base para hacer asignaciones de memoria, índice de un array, concatenar datos, hacer bucles. etc. Este error ha sido el caso de vulnerabilidades como CVE-2001-014459 en SSH1 y que permitiría a un atacante ejecutar código arbitrario con los privilegios del servicio ssh; o algunos más recientes como CVE-2011-2371 en Mozilla Firefox».

El código de la siguiente figura muestra un ejemplo de ***integer overflow*** que se da con una función que reserva memoria para un *string* con un tamaño que se lee de un fichero. Antes de reservar el *buffer*, varias líneas de código comprueban que el tamaño no es mayor a 1024 y decrementa su valor en uno para no copiar el carácter de la nueva línea del fichero.

Sin embargo, dependiendo del valor de `readamt`, decrementar su valor podría causar resultados erróneos; si se declara sin signo y un atacante ocasiona que `getstringsize()` devuelva 0, `malloc()` será invocada con 4 294 267 295, que es el valor más grande en una representación de **32 bit** y la operación fallará por **insuficiencia de memoria**.

```
unsigned int readamt;
readamt = getstringsize();
if (readamt > 1024)
return -1;
    readamt--; // don't allocate space for '\n'
buf = malloc(readamt);
```



Figura 1. *Integer overflow*. Fuente: Chess y West, 2007.

Otro **error** de tipo aritmético es el siguiente:

```
int ConcatBuffers(char *buf1, char *buf2, size_t len1, size_t len2){
    char buf[0xFF];
    if((len1 + len2) > 0xFF) return -1;
    memcpy(buf, buf1, len1);
    memcpy(buf + len1, buf2, len2);

    // do stuff with buf

    return 0;
}
```



0x103 len1
+ 0xFFFFFFFFC len2

0xFF

Las dos funciones `memcpy`
intentan realizar una copia > de

Figura 2. Error aritmético. Fuente: presentación de Microsoft «Basic of Secure Development Test».

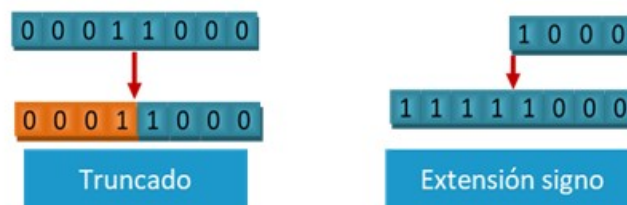
Errores de truncado

Otro tipo de problemas que se pueden presentar en operaciones con números es que se pueden **truncar** cuando un tipo de dato entero, con gran cantidad de bits, se convierte a otro con menor cantidad de bits. El **lenguaje C**, por ejemplo, desecha los bits de mayor orden; si un entero con signo se convierte desde otro más pequeño en número de bits, los bits extra se rellenan de tal forma que el número nuevo conserve el mismo signo.

Esto significa que, si un número negativo es convertido a un tipo de datos más grande, su signo será el mismo, pero su valor aumentará considerablemente, porque sus bits más significativos se modificarán. Si un programador no entiende cuándo y cómo las conversiones tienen lugar, pueden surgir vulnerabilidades.

Conversiones entre enteros con signo y sin signo

Suponiendo que se tienen 4 bits para representar enteros con y sin signo, se tendría:



No pueden representar el mismo número de valores y solo algunos valores pueden ser convertidos desde un dato de tipo **sin signo** a otro **con signo**, y viceversa, sin cambiar el significado, mientras otros no. En el caso de **valores positivos**, el problema es que el 50 % de los valores sin signo requiere poner a uno el bit más significativo.



Figura 4. Error que ocurre cuando un entero de 4 bit es truncado con signo a sin signo y viceversa.

Fuente: Chess y West, 2007.

En el código de la siguiente figura se procesa la entrada de usuario de una serie de *strings* de longitud variable almacenadas en una posición simple `char*`. Los primeros dos octetos de entrada dictan el tamaño de la estructura a ser procesada. El programador ha puesto una cota superior en el tamaño del *string*: la entrada será procesada solo si `len` es menor o igual a 512.

El problema es que `len` está declarado como **tipo *short*** (con signo); entonces, la comprobación contra la longitud de *string* máxima es una comparación con signo. Más tarde, `len`, implícitamente, se convierte a un número entero sin signo en la llamada a `memcpy()`. Si el valor de `len` es negativo, aparecerá que el *string* tiene un tamaño apropiado (se toma la alternativa ***if***), pero la cantidad de memoria copiada por `memcpy()` será muy grande (mucho más grande que 512) y el atacante será capaz de **desbordar *buff*** con los datos en `strm`.

```
char* processNext(char* strm) {  
    char buf[512];  
    short len = *(short*) strm;  
    strm += sizeof(len);  
    if (len <= 512) {  
        memcpy(buf, strm, len);  
        process(buf);  
        return strm + len;  
    }  
    else {  
        return -1;  
    }  
}
```



Figura 5. *Integer overflow* causado por conversión signo-sin signo. Fuente: Chess y West, 2007.

5.3. Errores y excepciones

Los problemas de seguridad, muchas veces, comienzan cuando un atacante busca la forma de violar las **expectativas** del programador, quienes es usual que den menos importancia a las condiciones de error y manejo de excepciones. Por el contrario, es uno de los aspectos que tienen en cuenta los atacantes:

- ▶ **C** usa códigos de error que generan las funciones.
- ▶ **C++** usa una combinación de códigos de error y excepciones sin comprobar.
- ▶ **Java** usa excepciones comprobadas.

Manejar errores mediante códigos de retorno

El **lenguaje C** usa el valor de retorno de una función para comunicar el éxito o el fracaso. Esta aproximación tiene varios **efectos secundarios** no deseables:

- ▶ Hace fácil no tratar los errores: simplemente, no haciendo caso del **valor de retorno** de una función.
- ▶ Conectar la información del error con el código para manejarlo hace los programas más **difíciles de leer**. La lógica de manejo del error se mezcla con la lógica para manejar casos esperados. Esto aumenta la tentación de ignorar errores.
- ▶ No hay ninguna convención universal para comunicar la **información de error**; por lo tanto, los programadores deben investigar el mecanismo que se maneja en cada función.

Vale la pena notar que estos son algunos motivos para que los diseñadores de **C++** y **Java** incluyeran excepciones como un elemento del lenguaje. El efecto que tiene **ignorar** un código de error, como en el ejemplo de la siguiente figura, es que el programador espera que el *buff* termine en nulo; pero, si no es así, debido, por

ejemplo, a un error **I/O**, como no se notifica el error en la siguiente llamada a **strcpy()**, puede resultar en *buffer overflow* (*buff* no tiene terminador nulo).

```
char buf[10], cp_buf[10];
fgets(buf, 10, stdin);
strcpy(cp_buf, buf);
```



Figura 6. No se comprueba el código de retorno. Fuente: Chess y West, 2007.

El código **Java** de la siguiente figura recorre un **conjunto de usuarios** al leer un fichero de datos privado de cada usuario. El programador asume que los archivos tienen siempre exactamente **1 kb** de tamaño y, por lo tanto, no hace caso del valor de retorno de `read()`. Si un atacante puede crear un archivo más pequeño, el programa reutilizará el resto de los datos del archivo anterior, haciendo que el código trate los datos de otro usuario como si los datos pertenecieran al atacante.

```
FileInputStream fis;
byte[] byteArray = new byte[1024];
for (Iterator i=users.iterator(); i.hasNext();) {
    String userName = (String) i.next();
    String pFileName = PFILE_ROOT + "/" + userName;
    FileInputStream fis = new FileInputStream(pFileName);
    try {
        fis.read(byteArray); // El fichero es siempre de 1k bytes
        processPFile(userName, byteArray);
    }
    finally {
        fis.close();
    }
}
```



Figura 7. No se comprueba el código de retorno del método `read()`. Fuente: Chess y West, 2007.

Manejo de errores mediante excepciones

Las excepciones solucionan muchos problemas de manejo de errores. Aunque sea fácil ignorar el valor de retorno de una función simplemente omitiendo la comprobación del código, no hacer caso de una **excepción comprobada** requiere justo lo contrario: un programador tiene que **escribir el código** expresamente para ignorarlo. Las excepciones también permiten la separación entre el código que sigue un **camino esperado** y el código que maneja **circunstancias anormales**.

Las excepciones tienen dos versiones, la diferencia es relativa a si el compilador usa **análisis estático** para asegurar que la excepción es manejada:

- ▶ **Checked.** Si un método declara que lanza una excepción *checked*, todos los objetos que lo utilizan deben manejar la excepción o declarar que también lo lanzan. Esto fuerza al programador a pensar en excepciones *checked* en cualquier parte donde pudieran ocurrir. Los compiladores de Java hacen cumplir las reglas en cuanto a excepciones *checked*, y la biblioteca de clases de Java hace un empleo liberal de excepciones *checked*. La clase Java **java.lang.Exception** es una excepción *checked*.
- ▶ **Unchecked.** Las excepciones *unchecked* no tienen que ser declaradas o manejadas. Todas las excepciones en C++ son *unchecked*, que quiere decir que un programador podría completamente ignorar las excepciones y el compilador no se quejaría. Java ofrece excepciones *unchecked*, también. Sin embargo, el peligro con las excepciones *unchecked* consiste en que los programadores podrían ser inconscientes sobre que una excepción puede ocurrir en un contexto dado y omitir el manejo del error apropiado. Por ejemplo, la función `VirtualAlloc`, de Windows, asigna memoria en el *stack*. Si una petición de asignación es demasiado grande para el espacio de *stack* disponible, `VirtualAlloc` lanza una excepción de desbordamiento de *stack* *unchecked*. Si la excepción no es capturada, el programa fallará, lo que potencialmente posibilite un ataque de denegación de servicio. Siempre que sea posible, se deberá:
 - Capturar las excepciones en el nivel superior, las excepciones se pueden capturar o volver a lanzar.
 - No incluir sentencias *return* en el bloque *finally* de manejo de la excepción, el programa se comporta como si ninguna excepción hubiera ocurrido.
 - Capturar solamente lo que se esté preparando para manejar. La captura de todas las excepciones en el nivel superior es una buena idea, pero la captura de las excepciones dentro de un programa después de una gran cantidad de código puede causar problemas.

- No dejar excepciones *checked* con el bloque *catch* vacío. Si no se sabe qué hacer para tratar una excepción capturada, se puede lanzar una excepción que, por lo menos, notifique que algo raro ha ocurrido:

```
catch (RareException e) {  
    throw RuntimeException("Esto nunca debe ocurrir ", e);  
}
```



Figura 8. Excepción con el bloque *catch* relleno.

Desasignar los recursos que no se van a utilizar en adelante

No liberar recursos como memoria, ficheros, conexiones de red, etc., puede causar problemas de **rendimiento**. Este concepto se conoce como **resource leaks**, que supone un riesgo de seguridad, pues favorecen claramente ataques de denegación de servicio. En cualquier lenguaje que se utilice se tiene que procurar desasignar cualquier recurso que no se vaya a utilizar en adelante en el resto de la aplicación. En el código de la siguiente figura se muestra un ejemplo de **memory leak**, que es, también, un subconjunto de esta categoría.

```
char* getBlock(int fd) {  
    char* buf = (char*) malloc(BLOCK_SIZE);  
    if (!buf) {  
        return NULL;  
    }  
    if (read(fd, buf, BLOCK_SIZE) != BLOCK_SIZE) {  
        return NULL;  
    }  
    return buf;  
}
```



Figura 9. No se libera la memoria de *buf*. Fuente: Chess y West, 2007.

5.4. Privacidad y confidencialidad

En este apartado se va a analizar cómo hay que **manejar las contraseñas**, tanto las que utilizan los usuarios para autenticarse (*inbound passwords*), como las que utiliza un programa que actúa como cliente para autenticarse frente a un servicio final (*outbound passwords*), como una base de datos o un **servidor LDAP**. *Inbound passwords* pueden ser *hashed*; sin embargo, las *outbound passwords* deben ser accesibles al programa, en texto claro.

Para profundizar sobre *inbound passwords*, se recomienda la lectura de este libro: <http://www.cl.cam.ac.uk/~rja14/book.html>

Mantener las passwords fuera del código fuente

Se deberán cifrar las contraseñas con algoritmo seguro y almacenarlas fuera del código. La siguiente figura muestra un trozo de código que muestra un mal uso de *password* codificada dentro de código Java.

```
public class DbUtil {  
    public static Connection getConnection() throws SQLException {  
        return DriverManager.getConnection(  
            "jdbc:mysql://ixne.com/rxsq1", "chan", "0uigoT0");  
    }  
}
```



Figura 10. *Password hardcoded*. Fuente: Chess y West, 2007.

Una buena **estrategia** consiste en:

- **Cifrar las passwords** con una implementación pública de algoritmos criptográficos robustos y almacenarlas cifradas en un fichero de configuración: almacenar la clave necesaria para descifrar la *password* en un fichero separado donde la aplicación pueda acceder, pero no la mayoría de los administradores de sistemas.

- ▶ **Usar *passwords* robustas:** usar generadores de números aleatorios seguros para las *passwords* de conexiones *outbounds*. Hay un compromiso entre los conceptos de **robustez** y de **facilidad** a la hora de elegir una *password*. Las *outbound passwords* no requieren ser recordadas, deben ser largas y aleatoriamente generadas usando un método seguro de generación de números aleatorios.
- ▶ **Números aleatorios:** si se consigue una buena fuente de generación de números aleatorios, se tiene una buena **f fuente de secreto**. La generación de nuevos secretos es una parte crítica en el establecimiento de muchas clases de relaciones de confianza. Los números aleatorios son importantes en la criptografía.

Los computadores son máquinas intrínsecamente deterministas y no son una fuente de aleatoriedad. Si un programa requiere un volumen grande de valores realmente arbitrarios, hay que pensar en comprar un **generador de números aleatorios hardware**. Los generadores de números pseudo aleatorios caen en dos categorías:

- ▶ **PRNGS estadísticos:** producen valores que son uniformemente distribuidos a través de un rango especificado, pero son calculados usando algoritmos que crean una corriente de valores fáciles de reproducir y triviales de predecir.
- ▶ **PRNGS criptográficos:** usan algoritmos criptográficos para ampliar la entropía de una semilla de valor aleatorio en una corriente de valores imprevisibles que, con alta probabilidad, no pueden ser distinguidos de un valor realmente arbitrario. Los PRNGS criptográficos son convenientes cuando la imprevisibilidad es importante, incluyendo los usos siguientes:
 - Criptografía.
 - Generación de contraseñas.
 - Aleatoria de puertos para seguridad.
 - Identificadores externos únicos (identificadores de sesión).

Los errores más comunes y obvios relacionados con números aleatorios ocurren cuando se usa **PRNG estadístico** en una situación que exige que los valores sean sumamente imprevisibles y, por lo tanto, producidos por **PRNG criptográfico**. Otro error ocurre cuando un PRNG no se crea con entropía insuficiente, causando que la corriente de números sea predecible.

- ▶ **Números aleatorios en Java.** Java tiene métodos de generación de números aleatorios estadísticos y criptográficos:
 - Método estadístico: `Random.nextInt()`.

```
String generateCouponCode(String couponBase) {  
    Random ranGen = new Random();  
    ranGen.setSeed((new Date()).getTime());  
    return(couponBase + Gen.nextInt(400000000));  
}
```



Figura 11. Función aleatoria no segura. Fuente: Chess y West, 2007.

Clase con métodos criptográficos: `Java.security.SecureRandom`. El algoritmo, por defecto, y las fuentes de entropía usadas por `SecureRandom`, son opciones buenas para la mayor parte de aplicaciones de software y los usuarios raras veces deberían tener que proporcionar su propia semilla.

- ▶ **Números aleatorios en C y C++.** C/C++ tienen métodos de generación de números aleatorios estadísticos y aleatorios:
 - **Funciones estadísticas:** evitar utilizar `rand()`, `srand()`, `srand48()`, `drand48()`, `lrand48()`, `random()`, `srandom()`.
 - **Funciones criptográficas:** Sobre plataformas de Microsoft, `rand_s()` (del API de seguridad de Microsoft CRT) y `CryptGenRandom()` (de Microsoft CryptoAPI) proporcionan el acceso a valores criptográficamente fuertes arbitrarios producidos por el subyacente `RtlGenRandom()` PRNG.

5.5. Programas privilegiados

La mayor parte de los programas se ejecutan con un **conjunto de privilegios** heredados del usuario con el que se ejecutan. Por ejemplo, un editor de textos puede mostrar solo los archivos que su usuario tiene el permiso de lectura.

Los programas privilegiados tienen privilegios adicionales que les permiten realizar operaciones que sus usuarios, de otra forma, no realizarían.

- ▶ Cuando está **escrito correctamente**, un programa privilegiado concede a usuarios regulares una cantidad limitada de accesos a algún recurso compartido, como la memoria física, un dispositivo de hardware, o archivos especiales, como el archivo de contraseñas o la base de datos de correo.
- ▶ Cuando está **escrito incorrectamente**, un programa vulnerable privilegiado deja, a los atacantes, campo para actuar. En el peor caso, esto puede permitir un ataque de escalada de privilegios vertical, en el que el agente malicioso gana acceso incontrolado a los privilegios elevados del programa.

Otro tipo común de ataque, conocido como **escalada de privilegios horizontal**, ocurre cuando un atacante engaña a los mecanismos de control de acceso de una aplicación para tener acceso a recursos que pertenecen a otro usuario. La siguiente figura ilustra la diferencia entre la escalada de privilegio vertical y horizontal.

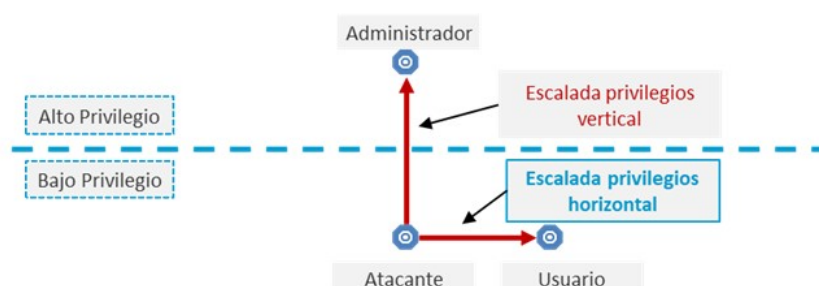


Figura 12. Ataques de elevación de privilegios vertical y horizontal. Fuente: Chess y West, 2007.

Principio de mínimo privilegio

Evidentemente, es uno de los primordiales principios de diseño que, a la hora de implementar el código, hay que ser lo más consecuente posible con lo que se especificó durante la fase de análisis de requisitos. La motivación detrás de este principio está clara: **el privilegio es peligroso**. Cuantos más privilegios tiene un programa, mayor daño potencial puede causar.

Un conjunto reducido de acciones posibles disminuye el riesgo potencial de un programa. La mejor gestión de privilegios es ninguna gestión de privilegio. El modo más fácil de prevenir vulnerabilidades relacionadas con el privilegio es diseñar los sistemas que no requieren componentes privilegiados.

Cuando se aplica al código, el **principio de mínimos privilegios** implica dos cosas:

- ▶ Los programas no deberían requerir que sus usuarios tengan **privilegios extraordinarios** para realizar tareas ordinarias.
- ▶ Los programas privilegiados deberían **reducir** al mínimo la cantidad de daño que pueden causar cuando algo va mal.

Dependiendo de la **funcionalidad del programa**, y de sus necesidades de privilegio, se pueden elevar y bajar sus privilegios en puntos diferentes de su ejecución. Los programas requieren privilegios para una variedad de motivos:

- ▶ Comunicarse directamente al hardware.
- ▶ Modificar el comportamiento del OS.
- ▶ Enviar señales a ciertos procesos.
- ▶ Trabajar con recursos compartidos.

- ▶ Apertura de puertos de red con baja numeración.
- ▶ Cambio de la configuración global (el registro y/o archivos).
- ▶ Protecciones de sistema de archivos principales.
- ▶ Instalación de nuevos archivos en directorios de sistema.
- ▶ Puesta al día de archivos protegidos.
- ▶ Tener acceso a archivos que pertenecen a otros usuarios.

Las transiciones entre estados privilegiados y no privilegiados definen el **perfil de privilegio** de un programa. El perfil de privilegio de un programa puede ser colocado en una de las siguientes **cuatro clases**:

- ▶ **Programas normales** que corren con los mismos privilegios que sus usuarios.
Ejemplo: Emacs.
- ▶ **Programas de sistema** que corren con privilegios de *root* para la duración de su ejecución. Ejemplo: Init.
- ▶ Programas que **necesitan privilegios de *root*** para usar un conjunto fijo de recursos del sistema cuando se ejecutan al principio. Ejemplo: Apache httpd, que necesita el acceso de *root* para usar puertos bajos numerados.
- ▶ Programas que **requieren privilegios de *root*** intermitentemente a lo largo de su ejecución. Ejemplo: un demonio de FTP que usa puertos de baja numeración intermitentemente en todas las partes de su ejecución. En la Figura 13, se representa un esquema de los ejemplos mencionados y cuándo necesitarían privilegios de *root*.

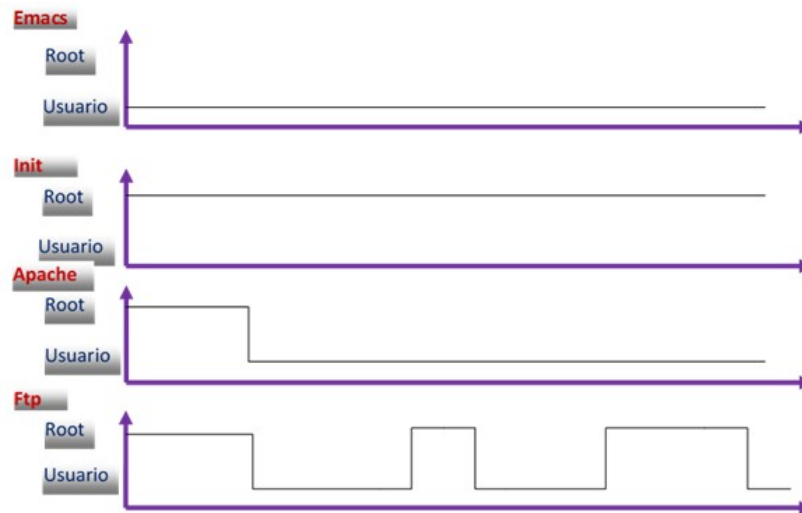


Figura 13. Ejemplo de utilización de privilegios de *root* en la ejecución de varios programas. Fuente: Chess y West, 2007.

El concepto de seguridad que la mayoría de la gente debería repetir es «**no confiar en nada**». Aunque esto sea un ideal que es imposible de realizar, no hay nada que debería ser tan perseguido enérgicamente como un **programa privilegiado**.

Los atacantes usarán, para montar, ataques de escalada de privilegios:

- ▶ Los argumentos al programa.
- ▶ El entorno de ejecución del programa.
- ▶ Los recursos de que el programa depende (sistema de ficheros).

Manejo de los privilegios

Los autores advierten que la mayoría de los programas privilegiados realizan **tres operaciones claves** sobre sus privilegios:

- ▶ Eliminar privilegios con la intención de recuperarlos.
- ▶ Eliminar privilegios permanentemente.

- ▶ Recuperar privilegios previamente almacenados.

Otra aproximación de gestión de privilegios, de acuerdo con el principio de **mínimo privilegio**, es dividir un programa en componentes privilegiados y no privilegiados. Si las operaciones que requieren privilegios pueden ser categorizadas en un proceso separado, esta solución puede reducir bastante el trabajo requerido para asegurar que los privilegios se administran adecuadamente.

Además de la gestión explícita de privilegios y el control de acceso basado en el ID del usuario, se puede restringir los privilegios de un programa limitando su **visión del sistema de archivos** al usar, por ejemplo, la función de sistema `chroot()` en sistemas Linux. Después de una correcta invocación a `chroot()`, un proceso no puede tener acceso a ningún archivo fuera del árbol de directorio especificado para esa llamada. Se denomina a tal entorno **jaula *chroot*** y se usa comúnmente para prevenir la posibilidad de que se pueda corromper un proceso y usarlo para tener **acceso a archivos protegidos**. Por ejemplo, muchos servidores de FTP se ejecutan en jaulas *chroot* para prevenir a un atacante que descubre una nueva vulnerabilidad en el servidor. Esta tentativa de crear una jaula *chroot* tiene **cuatro problemas**:

- ▶ Llamar a `chroot()` no afecta a ningún descriptor de archivo que está actualmente abierto; entonces, se puede apuntar a archivos fuera de la jaula *chroot*. Para estar seguro, se deberá cerrar cada archivo abierto antes de la llamada `chroot()`.
- ▶ La llamada a la función `chroot()` no cambia el directorio actual de trabajo del proceso; entonces, rutas relativas como `../../../../../etc/passwd` todavía pueden referirse a recursos del sistema de archivos fuera de la jaula *chroot*, después de que se ha llamado a la función `chroot()`. Siempre, antes de una llamada a `chroot()`, se incluye una llamada a `chdir("/")`. Verificar que cualquier error de manejo de código entre la llamada a `chroot()` y `chdir()` no abre ningún archivo y no devuelve el control a ninguna otra parte del programa que no sean rutinas de parada. Las funciones `chroot()` y `chdir()` deberían estar tan cerca como sea posible, para limitar la exposición a esta vulnerabilidad.

- ▶ Es también posible, y recomendable, crear una jaula *chroot* apropiada que llame a *chdir()* primero. La llamada a *chdir()*, entonces, puede tomar un argumento constante *("/")*, que es más fácil de verificar durante una revisión de código.
- ▶ Una llamada a *chroot()* puede fallar, por lo que se deberá comprobar el valor de retorno de *chroot()* para asegurarse del resultado de la llamada.
- ▶ Para llamar *chroot()*, el programa debe ejecutarse con privilegios de *root*. En cuanto la operación privilegiada se ha completado, el programa debería eliminar los privilegios de *root* y retornar a los privilegios del usuario de invocación. El código en el ejemplo de la Figura 14, sigue ejecutándose como *root*.

```
chroot("/var/ftpboot");

if (fgets(filename, sizeof(filename), network) != NULL) {
    if (filename[strlen(filename) - 1] == '\n') {
        filename[strlen(filename) - 1] = '\0';
    }
    localfile = fopen(filename, "r");
    while ((len = fread(buf, 1, sizeof(buf), localfile)) != EOF) {
        (void)fwrite(buf, 1, len, network);
    }
}
```



Figura 14. Código de un simple FTP *server*. Fuente: Chess y West, 2007.

Manejo de excepciones en programas privilegiados

Hay que tener en cuenta una serie de directrices generales para manejar **eventos inesperados y excepciones** que toman una importancia aún mayor en programas privilegiados, donde los ataques son más probables y el coste de un *exploit* es mayor.

- ▶ Comprobar cada condición de error.
- ▶ Seguridad sobre robustez: terminación ante errores. No intentar reponerse de errores inesperados o mal entendidos.

- ▶ Inhabilitar señales antes de la elevación de privilegios. De esta forma, se evita tener señales que controlan código que se ejecuta con privilegios. Rehabilitar las señales y después volver a los privilegios estándar que tenía el usuario.

Ataques de escalada de privilegios

A veces, un ataque contra un programa privilegiado se realiza por un **usuario legítimo** del sistema. Normalmente, un atacante usa un proceso de dos pasos para obtener privilegios sobre una máquina:

- ▶ Intenta **encontrar una debilidad** en un servicio de red o en una cuenta de usuario mal protegida y obtiene acceso a una cuenta con bajos privilegios sobre la máquina.
- ▶ Usa este acceso de bajos privilegios para **explotar una vulnerabilidad** en un programa privilegiado y tomar el control de la máquina.

Los ataques de escalada de privilegios pueden tener como objetivo cualquier variedad de vulnerabilidades de software. En este apartado, se tratan las clases de vulnerabilidades que son principalmente un riesgo en programas privilegiados:

- ▶ Condiciones de carrera de acceso a archivos.
- ▶ Permisos de archivo débiles.
- ▶ Archivos temporales inseguros.
- ▶ Inyección de comandos.
- ▶ Mal uso de descriptores de archivo estándar.

Ataque de condiciones de carrera en el acceso a ficheros

Se encuentran en este grupo las vulnerabilidades conocidas como **time-of-check, time-of-use (TOCTOU)**. MITRE clasifica este tipo de vulnerabilidades con el **CWE 367**. Los errores generados como consecuencia de una condición de carrera se producen por el cambio que experimenta el **estado de un recurso** (ficheros,

memoria, registros, etc.) cuando un programa intenta verificar una propiedad y, más tarde, toma una decisión, suponiendo que la propiedad no ha cambiado. Los ataques **TOCTOU** sobre vulnerabilidades del sistema de archivos, generalmente, siguen esta secuencia:

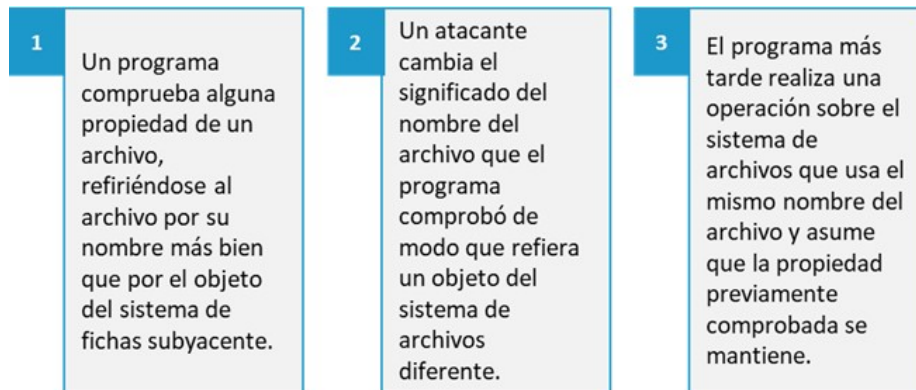


Figura 15. Secuencia de ataque TOCTOU. Fuente: elaboración propia.

La siguiente figura muestra el modo que las operaciones en **lpr** podrían intercalarse con la ejecución del código de un agente malicioso de un **ataque con éxito**. Un atacante invoca **lpr** con el argumento `/tmp/attack` y redirecciona el archivo para que apunte al fichero de sistema `/etc/shadow` durante el tiempo que pasa entre que es comprobado y utilizado. En la figura siguiente, se muestra el ejemplo corregido.

```
for (int i=1; i < argc; i++) {
    /* make sure that the user can read the file, then open it */
    if (!access(argv[i], O_RDONLY)) {
        fd = open(argv[i], O_RDONLY);
    }
    print(fd);
}
```



Figura 16. Ejemplo de código con una vulnerabilidad TOCTOU en el acceso a un fichero. Fuente: Chess y West, 2007.

En la siguiente tabla se muestra un **esquema de tiempos** con las operaciones que pueden darse intercaladas de un **usuario legítimo** y un **atacante**. El

aprovechamiento de las vulnerabilidades de condiciones de carrera de acceso a un archivo depende del hecho de que los sistemas operativos modernos permiten a muchos programas ejecutarse inmediatamente y no se garantiza que la ejecución de cualquier programa dado no se intercale con otro. Aunque dos operaciones pudieran parecer secuenciales en el **código original** de un programa, cualquier número de instrucciones puede ser ejecutado entre ellas.

Tiempo	Lpr	Atacante
Tiempo 1	<code>access("/tmp/attack")</code>	
Tiempo 2		<code>unlink("/tmp/attack")</code>
Tiempo 3		<code>symlink("/etc/shadow", "/tmp/attack")</code>
Tiempo 4	<code>open("/tmp/attack")</code>	

Tabla 1. Esquema de tiempos de una vulnerabilidad TOCTOU en el acceso a un fichero. Fuente: Chess y West, 2007.

La ventana de vulnerabilidad para tal ataque es el período de tiempo entre cuando una propiedad es **comprobada** y cuando se **usa el archivo**. Los atacantes tienen una variedad de técnicas para ampliar la longitud de esta ventana para hacer los *exploits* más fáciles, como el enviar a una señal al proceso víctima, que hace que ceda la CPU a otro proceso. Incluso, con una pequeña ventana, una tentativa de *exploit* puede repetirse hasta que se consiga llevar a cabo.

Hay que tener en cuenta que un ataque de condición de carrera todavía puede darse después de abrir el archivo, si alguna operación posterior depende de una propiedad comprobada anterior a la apertura del archivo.

Por ejemplo, si la estructura pasada a `stat()` se obtiene antes de que un archivo sea abierto, y una decisión posterior sobre si hay que operar sobre el archivo está basada en un valor leído de la estructura de *stat*, entonces una condición de carrera TOCTOU puede darse entre la llamada a `stat()` y la llamada `open()`.

Ficheros temporales inseguros

Algunas funciones intentan crear un único nombre de archivo simbólico que el programador puede usar para crear un archivo temporal. Otras funciones, además, van un poco más lejos e intentan abrir un **descriptor de archivo**.

Ambos tipos de funciones son susceptibles a una variedad de vulnerabilidades que podrían permitir a un atacante **inyectar contenido malévolo** en un programa, hacer que el programa realice operaciones malévolas en nombre del atacante, dar acceso al atacante a información sensible que el programa almacena sobre el sistema de archivos, o permitir un ataque de **denegación** de servicio contra el programa.

Existen dos opciones para crear archivos temporales de **forma segura**:

- ▶ **Almacenar** los archivos temporales bajo un directorio que no es públicamente accesible; de esta forma, se elimina toda la discusión con respecto a ataques.
- ▶ **Generar** los nombres de archivo temporales que sean difíciles de adivinar usando un generador de números criptográficamente seguros pseudoaleatorios (PRNG) para crear un elemento aleatorio en cada nombre del archivo temporal.

Inyección de comandos

Los datos introducidos por un usuario pueden **alterar el propósito** de un comando, consulta, etc., a ejecutar en el entorno de la aplicación, base de datos, etc. La secuencia de actuación de un atacante sería:

- ▶ Un atacante **modifica** el entorno de un programa.
- ▶ El programa ejecuta un comando que usa el **entorno modificado** sin especificar una ruta absoluta.
- ▶ Ejecutando el comando, el programa da **privilegios** a un atacante o la capacidad que, de otra manera, no tendría.

Para entender mejor el **impacto potencial** del entorno sobre la ejecución de comandos en un programa, hay que considerar la vulnerabilidad en la utilidad **ChangePassword**, basada en CGI-Web, que permite a los usuarios **cambiar sus contraseñas** en el sistema (Berkman, 2004). El proceso de actualización de contraseña incluye ejecutar **make** en el directorio `/var/yp`. Es importante notar que debido a que el programa pone al día registros de contraseña, debe ser instalado con **setuid root**. El programa invoca *make* como sigue:

```
system("cd /var/yp && make &> /dev/null")
```

Como el programa no especifica una ruta absoluta para sus variables de entorno antes de la invocación del comando, un atacante puede aprovecharse del programa ejecutarlo localmente para modificar la variable de entorno `$PATH` con el fin de apuntar su código malicioso llamado *make* y luego ejecutar un *script* CGI desde un intérprete de comandos *shell*. El atacante puede ejecutar código arbitrario con privilegios de *root*.

En programas seguros privilegiados, `$PATH` debería contener, únicamente, directorios de *root*, ya que es probable que el programa tenga que ejecutar utilidades de sistema con acceso a directorios controlados por su propietario. No incluir en el directorio actual (.) rutas relativas que un atacante podría ser capaz de manipular. En la mayor parte de los entornos, `$IFS` debería ser puesto a su valor por defecto, `\t\n`.

Descriptores de ficheros

Los **descriptores de archivo estándar** en sistemas Linux `stdin` (FD 0), `stdout` (FD 1), y `stderr` (FD 2) son, por lo general, abiertos por el terminal y usados tanto explícita como implícitamente por funciones como `printf()`. Algunos programas remiten uno o varios de estos descriptores a *streams* de datos diferentes, como un archivo de *log*, para reutilizar su comportamiento implícito, de forma que sea más adecuado al diseño del programa, pero la mayor parte de estos nunca cambia sus valores.

Como con *umask* y con variables de entorno, un **proceso hijo** hereda descriptores de archivo estándar de su padre. Los atacantes pueden aprovechar este hecho para hacer que un programa vulnerable lea o escriba en archivos del sistema, cuando en realidad se espera interactuar con el terminal.

```
fd = open("/etc/passwd", O_RDWR);  
...  
if (!process_ok(argv[1])) {  
    perror(argv[1]);  
}
```



Figura 17. Código vulnerable a ataques a su descriptor de ficheros estándar. Fuente: Chess y West, 2007.

El ejemplo muestra un programa que, como parte de su actividad normal, abre el archivo `/etc/passwd` en modo lectura-escritura y después, condicionalmente, pasa su primer argumento a `stderr()` usado por `perror()`. Se podría creer que estas dos operaciones no están relacionadas y no tienen ningún impacto la uno sobre la otra, pero el código realmente contiene un *bug*.

Un ejemplo de un **ataque sobre la vulnerabilidad del ejemplo anterior** se puede ver en la figura siguiente, el código del primer ejemplo cierra `stderr` (descriptor de archivo 2) y luego ejecuta el programa vulnerable. Cuando el programa abre `/etc/passwd`, `open()` devolverá el primer descriptor de archivo disponible que, debido a que el programa del atacante cerró `stderr`, será el descriptor de archivo 2.

Ahora, en vez de inofensivamente pasar su primer parámetro al terminal, el programa escribe la información a `/etc/passwd`. Aunque esta vulnerabilidad fuera perjudicial, independientemente de la información escrita, este ejemplo es, en particular, fatal, porque el atacante puede escribir una entrada válida a `/etc/passwd`. En los sistemas que almacenan entradas de contraseña en `/etc/passwd` se daría acceso con privilegios de *root* al atacante sobre el sistema.


```
int main(int argc, char* argv[]) {
    (void)close(2);
    execl("victim", "victim", "attacker:<pw>:0:1:Super-User-2:...",
    NULL);
    return 1;
}
```



Figura 18. Ejemplo de ataque a vulnerabilidad del ejemplo anterior. Fuente: Chess y West, 2007.

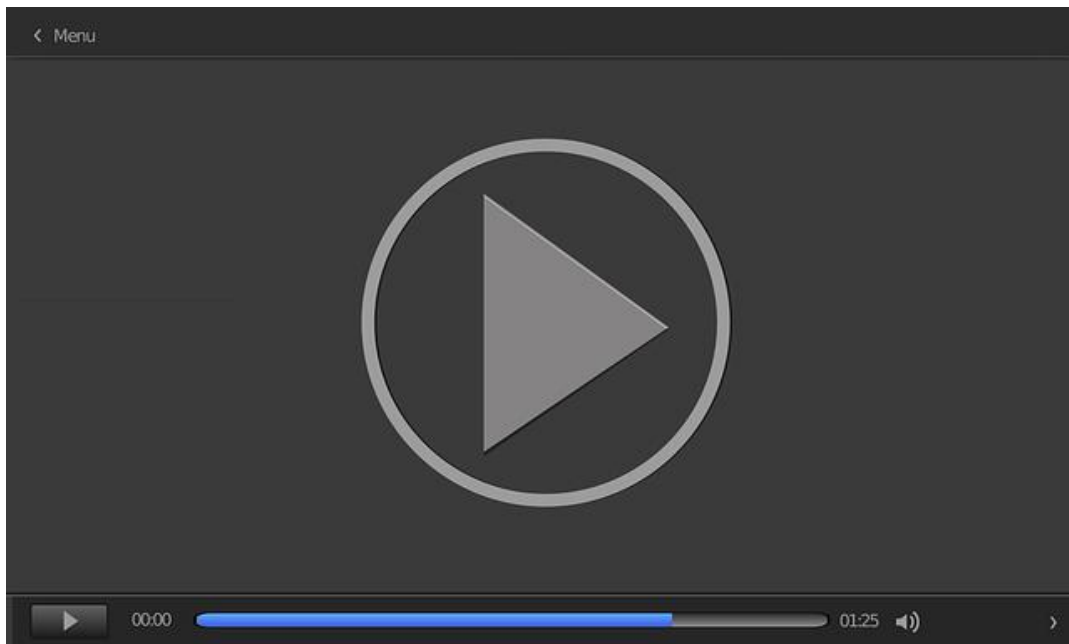
Los programas privilegiados deben asegurar que los tres primeros descriptores de archivo se abran a **archivos seguros**, conocidos antes del comienzo de su ejecución. Un programa puede: abrir los tres primeros descriptores de archivo al terminal deseado, abrir archivos *streams* (si el programa tiene la intención de usar funciones que implícitamente dependen de estos valores), o abrir `/dev/null` tres veces para asegurar que los tres primeros descriptores de archivo se usan. El ejemplo de la figura siguiente ilustra la segunda opción.

```
int main(int argc, char* argv[]) {
    if (open("/dev/null", O_WRONLY) < 0 ||
        open("/dev/null", O_WRONLY) < 0 ||
        open("/dev/null", O_WRONLY) < 0) {
        exit(-1);
    }
    ...
}
```



Figura 19. Código que soluciona vulnerabilidades de descriptor de ficheros asegurando el uso de los tres primeros descriptores de ficheros. Fuente: Chess y West, 2007.

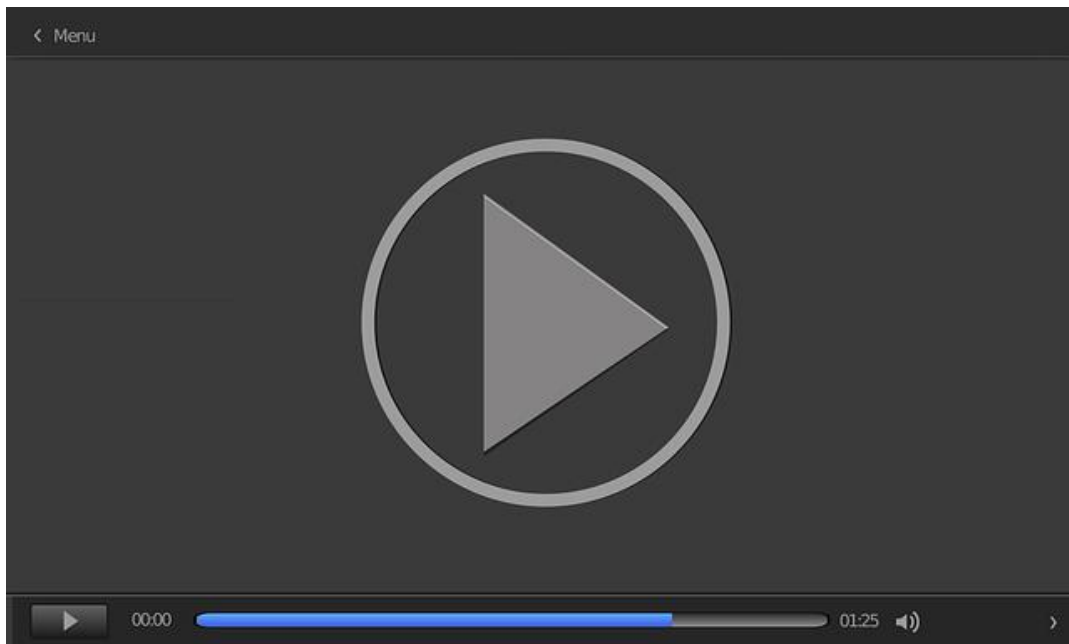
DevOps unifica los dos mundos de las tecnologías de la información, desarrollo y operaciones. Propone ciclos completos y automatizados desde el desarrollo hasta el despliegue del producto. De ahí que lenguajes de implementación de infraestructura como Puppet o Ansible hayan encajado tan bien en este paradigma. La idea práctica es introducir la **capa de seguridad** en DevOps para llegar a evolucionar al término DevSecOps. En este vídeo (*DevSecOps*) se realiza una introducción a DevSecOps.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=ca090dcc-b111-43e9-b6dc-adf700cfeff0>

El **IEEE Center for Secure Design (CSD)** definió una lista de los diez defectos de diseño más importantes del software y las técnicas de diseño para evitarlos. En este vídeo (*Principales 10 debilidades de diseño*) se presentan y estudian los diez citados defectos de diseño.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=556c5759-7155-4ca0-8a1e-adf700cfef88>

5.6. Referencias bibliográficas

Berkman, A. (2004). *ChangePassword 0.8 Runs setuid Shell*.

Chess, B., and West, J. (2007). *Secure Programming with Static Analysis*. Addison-Wesley Software Security Series.

INCIBE. (2012). *Informe de vulnerabilidades, 1er semestre 2012*. INCIBE.

Wagner, D., Foster, J. S., Brewer, E. A. y Aiken, A. (2002). *A First Step Towards Automated Detection of Buffer Overflow Vulnerabilities*. Proceedings of the 7th Network and Distributed System Security Symposium.

Common vulnerabilities guide for C programmers

CERN (s. f.). *Common vulnerabilities guide for C programmers* [Artículo web]. CERN computer security.
<https://security.web.cern.ch/security/recommendations/en/codetools/c.shtml>

Explica las principales librerías inseguras del lenguaje C y cómo pueden provocar ataques de desbordamiento menor.

Secure coding techniques

Professor Messer (2017, diciembre 16). *Secure Coding Techniques - CompTIA Security+ SY0-501 - 3.6*. [Vídeo]. YouTube. <https://www.youtube.com/watch?v=RTifWn01I2g>

Vídeo sobre alguna de las mejores prácticas de desarrollo de aplicaciones que pueden ayudar a hacer que el código sea lo más seguro posible. En concreto, se estudiarán las técnicas de validación de entrada, prevención de XSS y tratamiento de errores.

GOTO 2016. Secure Coding Patterns. Andreas Hallberg

GOTO Conferences (2016, octubre 13). *GOTO 2016 • Secure Coding Patterns • Andreas Hallberg*. [Vídeo]. YouTube. <https://www.youtube.com/watch?v=oqd9bxy5Hvc>

Esta sesión lo introducirá en el desarrollo de una mentalidad segura a la hora de desarrollar aplicaciones.

Secure coding

Software Engineering Institute. *Secure Development*. Carnegie Mellon University.
<http://www.cert.org/secure-coding/>

Sitio web de Software Engineering Institute de la universidad Carnegie Mellon con numerosos recursos de estándares de programación segura, laboratorio de análisis de código fuente, metodología TSP-Secure y herramientas y librerías.

SEI CERT Coding Standards

Schiela, R. (2020, mayo 29). *SEI CERT Coding Standards*. Software Engineering Institute.

<https://wiki.sei.cmu.edu/confluence/display/seccode/SEI+CERT+Coding+Standards>

Sitio web que incluye normas de codificación para los lenguajes de programación de uso común como C, C++, Java y Perl, y la plataforma Android™. Estos estándares se han desarrollado a través de un amplio esfuerzo por parte de los miembros de las comunidades de desarrollo y seguridad de software.

1. Señala la respuesta correcta. Con respecto a los errores y excepciones:
 - A. Si un método declara que lanza una excepción *checked*, todos los objetos que lo utilizan deben manejar la excepción o declarar que lo lanzan también.
 - B. Los compiladores de Java no hacen cumplir las reglas en cuanto a excepciones *checked*.
 - C. Todas las excepciones en C++ son *checked*.
 - D. Las excepciones *unchecked* no tienen que ser declaradas o manejadas.

2. Señala la o las respuestas correctas. ¿Qué tipo de vulnerabilidad se comete en este código?
 - A. *Integer overflows*.
 - B. Desbordamiento de *buffer*.
 - C. *Format string*.
 - D. Manipulación de información privada.

3. Señala la o las respuestas correctas. Se tienen dos opciones para crear archivos temporales de forma segura:

- A. Generar los nombres de archivo temporales que sean difíciles de adivinar, usando un generador de números criptográficamente seguros pseudo-aleatorios (PRNG) para crear un elemento aleatorio en cada nombre del archivo temporal.
- B. Almacenar los archivos temporales bajo un directorio que no es públicamente accesible, eliminando así toda la discusión con respecto a ataques.
- C. Almacenar los archivos temporales bajo un directorio que es públicamente accesible, eliminando así toda la discusión con respecto a ataques.
- D. Generar los nombres de archivo temporales que sean difíciles de adivinar usando un generador de números para crear un elemento en cada nombre del archivo temporal.

4. Señala la respuesta incorrecta. Métodos para detectar y prevenir *intergers overflows*:

- A. Usar tipos sin signo.
- B. Verificar el tipo de los *buffers*.
- C. Restringir la entrada numérica de usuario.
- D. Entender las reglas de conversión entre enteros.

5. Señala la respuesta incorrecta. Los ataques de escalada de privilegios pueden tener como objetivo cualquier variedad de vulnerabilidades de *software*, que son principalmente un riesgo en programas privilegiados:

- A. Archivos de sistema.
- B. Condiciones de carrera de acceso a archivos.
- C. Inyección de comandos.
- D. Mal uso de descriptores de archivo estándar.

6. Señala la respuesta correcta. Consideraciones sobre potenciales vulnerabilidades de Java:

- A. Seguridad de tipos. Los campos que son declarados privados o protegidos, o que tienen protección, por defecto, deberían ser públicamente accesibles.
- B. Campos públicos. Un campo que es declarado público directamente puede no ser accedido por cualquier parte de un programa Java y puede ser modificado por las mismas (a no ser que el campo sea declarado como final).
- C. Serialización. Esta facilidad posibilita que el estado del programa sea capturado y convertido en un byte *stream* que puede ser restaurado por la operación inversa, que es la deserialización.
- D. JVM. La JVM, en sí misma, a menudo está escrita en C para una plataforma dada. Esto quiere decir que, sin la atención cuidadosa a detalles de puesta en práctica, la JVM en sí misma no es susceptible a problemas de desbordamiento.

7. Señala la respuesta correcta. Revisando el programa ¿cuál de las siguientes preguntas describe mejor lo que está haciendo la función?

- A. Asegurar la no violabilidad espacial de la memoria.
- B. Validar la entrada mediante lista blanca.
- C. Asegurar la no violabilidad temporal de la memoria.
- D. Validar la entrada mediante lista negra.

8. Señala la incorrecta. Las funciones de *strings* limitadas son más seguras que las funciones ilimitadas, pero hay todavía mucho margen para el error. Los fallos de programación más comunes que se pueden cometer con las funciones de *string* limitadas:

- A. El *buffer* de destino se desborda porque el límite depende del tamaño de los datos de la fuente, más bien que del tamaño del *buffer* de destino.
- B. El *buffer* de destino se desborda porque su límite se especifica como el tamaño total del *buffer*, más bien que como el espacio restante.
- C. El *buffer* de destino se deja sin un terminador nulo.
- D. El programa escribe a una posición arbitraria en la memoria porque el *buffer* de destino se termina con el carácter nulo.

9. Señalar la respuesta correcta. ¿Qué tipo de vulnerabilidad se comete en este código?

- A. *Integer overflows*.
- B. Desbordamiento de *buffer*.
- C. Condiciones de carrera.
- D. *Use after free*.

10. Señala la respuesta correcta. ¿Cuándo ocurre un ataque de *integer overflow*?
- A. Al realizar una operación de una resta.
 - B. Un entero es usado como si fuera un puntero.
 - C. Un entero es usado para acceder a un *buffer* fuera de sus límites.
 - D. No hay más espacio en el programa para almacenar un entero.

Desarrollo Seguro de Software y Auditoría de la
Ciberseguridad

Tema 6. Introducción al concepto de auditoría de sistemas de información

Índice

Esquema

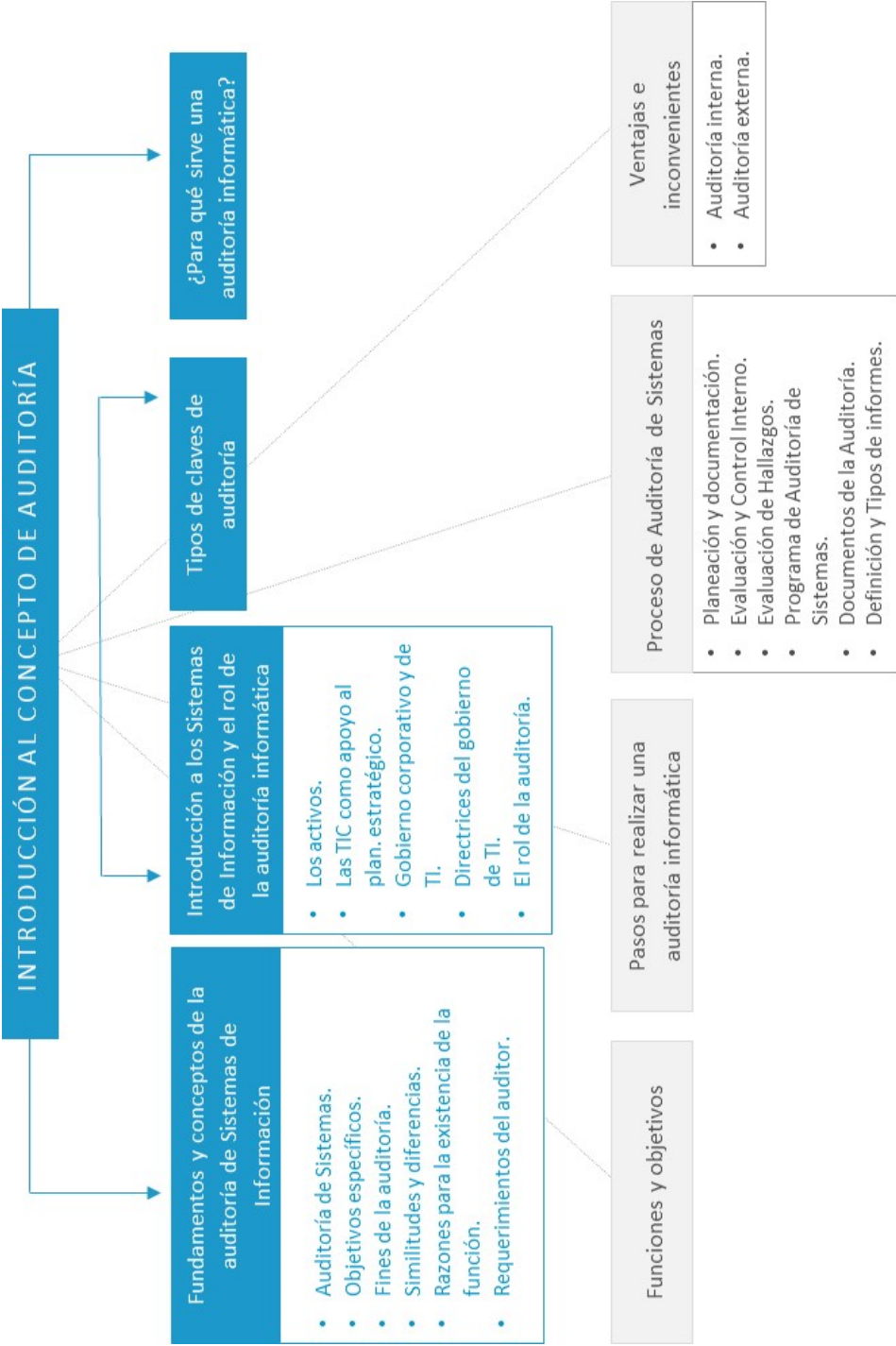
Ideas clave

- 6.1. Introducción y objetivos
- 6.2. Introducción a los sistemas de información y el rol de la auditoría informática
- 6.3. Fundamentos y conceptos de la auditoría de sistemas de información
- 6.4. Tipos y clases de auditoría
- 6.5. ¿Para qué sirve una auditoría informática?
- 6.6. Funciones y objetivos de la auditoría informática
- 6.7. Proceso de auditoría de sistemas
- 6.8. Ventajas e inconvenientes de las auditorías
- 6.9. Referencias bibliográficas

A fondo

- Auditoría de los estándares ISO en las TIC
- ISACA Madrid Chapter
- Auditoría de sistemas. Una visión práctica
- The IS audit process
- Cambios en los controles del Anexo A (ISO 27002:2022)

Test



6.1. Introducción y objetivos

Auditoría

La auditoría se define como un **proceso sistemático** que consiste en obtener y evaluar objetivamente evidencias sobre las afirmaciones relativas a los actos y eventos sobre los sistemas de información, con el fin de determinar el grado de correspondencia entre esas afirmaciones y los criterios establecidos para luego comunicar los resultados a las personas interesadas.

La auditoría es una función de dirección cuya finalidad es analizar y apreciar, con vistas a las eventuales acciones correctivas, el control interno de las organizaciones, para garantizar la integridad de su patrimonio, la veracidad de su información y el mantenimiento de la eficacia de sus sistemas de gestión.

Otras posibles definiciones pueden ser:

- ▶ Es un examen comprensivo de la estructura de una organización en cuanto a los planes y objetivos, métodos y controles, su forma de operación y sus equipos humanos y físicos.
- ▶ Una visión formal y sistemática para determinar hasta qué punto una organización está cumpliendo los objetivos establecidos por la gerencia, así como para identificar los que requieren mejorar.

Criterio de auditoría

El criterio de auditoría responde a **políticas, prácticas, procedimientos o requerimientos** contra los que el auditor compara la información recopilada. Los requerimientos pueden incluir estándares, normas, requerimientos organizacionales específicos y requerimientos legislativos o regulados.

Auditoría informática

En informática, la auditoría es la **revisión y la evaluación** de los controles, sistemas, procedimientos de informática, de equipos de cómputo, su utilización, eficiencia y seguridad, de la organización que participa en el procesamiento de la información, a fin de que, por medio del señalamiento de cursos alternativos, se logre una utilización más eficiente y segura de la información que servirá para una adecuada toma de decisiones.

Auditoría de la información

La auditoría de la información es una rama especializada de la auditoría que promueve y aplica conceptos de auditoría en el área de **sistemas de información**. El objetivo final que tiene el auditor es dar **recomendaciones** a la alta gerencia para mejorar o lograr un adecuado control interno en ambientes de tecnología informática, con el fin de obtener mayor eficiencia operacional y administrativa.

Los **objetivos** del presente tema son los siguientes:

- ▶ Estudiar los fundamentos y principales conceptos relacionados con las auditorías de los sistemas de información.
- ▶ Conocer los diferentes tipos de auditorías.
- ▶ Estudiar las funciones y los objetivos de la auditoría informática.
- ▶ Conocer los pasos para realizar una auditoría informática y el proceso de auditoría de sistemas.
- ▶ Comprender las ventajas e inconvenientes de las auditorías.

6.2. Introducción a los sistemas de información y el rol de la auditoría informática

Este apartado realiza un recorrido por algunos aspectos que se deberían conocer para entender la función de la **auditoría informática** en las organizaciones y su decisiva contribución al gobierno de las TIC.

Los activos de sistemas de información

Según la Organización Internacional de Normalización (ISO) (2004), un activo de sistemas de información es: «algo que tiene valor para la organización».

Un activo de sistemas de información es todo aquello que una entidad considera valioso por contener, procesar o generar información necesaria para el negocio de esta.

Se clasifican en primarios y secundarios, según se muestra en la siguiente figura:

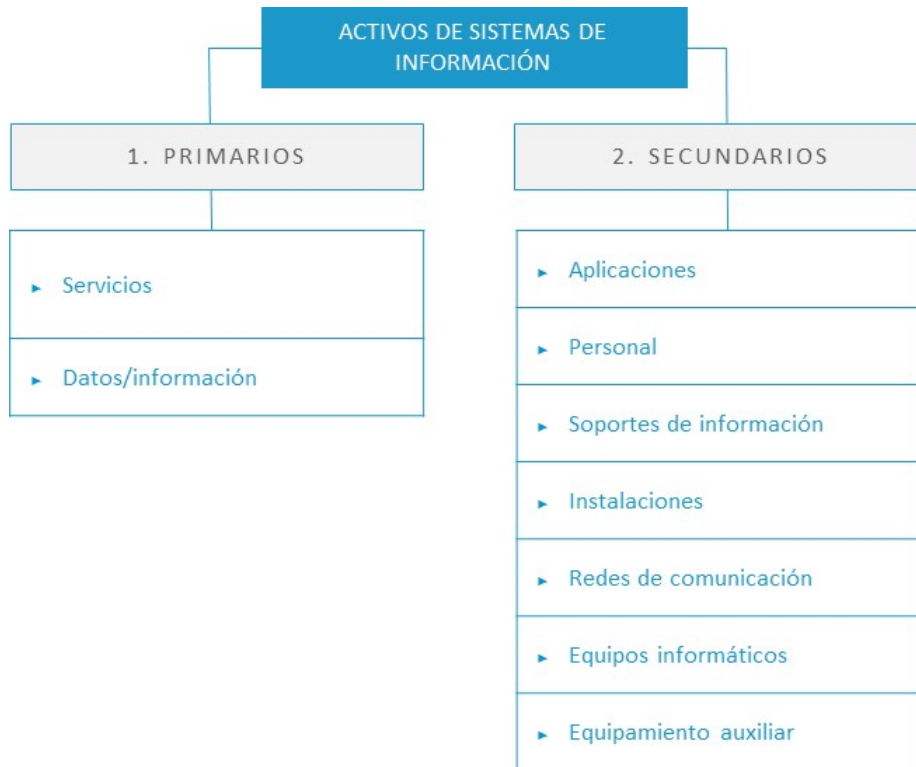


Figura 1. Activos de los sistemas de información. Fuente: elaboración propia.

Todo sistema de información (SI) utilizado por la organización, en régimen de propiedad, subcontratación o pago por uso, debe:

- ▶ **Salvaguardar la propiedad de la información.** Los activos de SI aseguran que la propiedad de los datos sea siempre de la organización.
- ▶ **Mantener la integridad de los datos (información).** Los activos de SI deben garantizar la integridad de la información; es decir, que la información no haya sido alterada por terceros (físicos o humanos).
- ▶ **Asegurar la confidencialidad de la información.** La información solo es accesible y entendible por quien tiene derechos.

- ▶ **Garantizar la disponibilidad de la información.** La organización puede disponer de la información en el momento en que la precisa.
- ▶ **Llevar a cabo los fines de la organización** y utilizar eficientemente los recursos.

El último punto indica que los sistemas de información han de ser **eficaces** (cumplir con los objetivos de la organización) y **eficientes** (el menor uso de recursos posible para lograr su cometido). Están compuestos por los elementos mostrados en la siguiente figura:

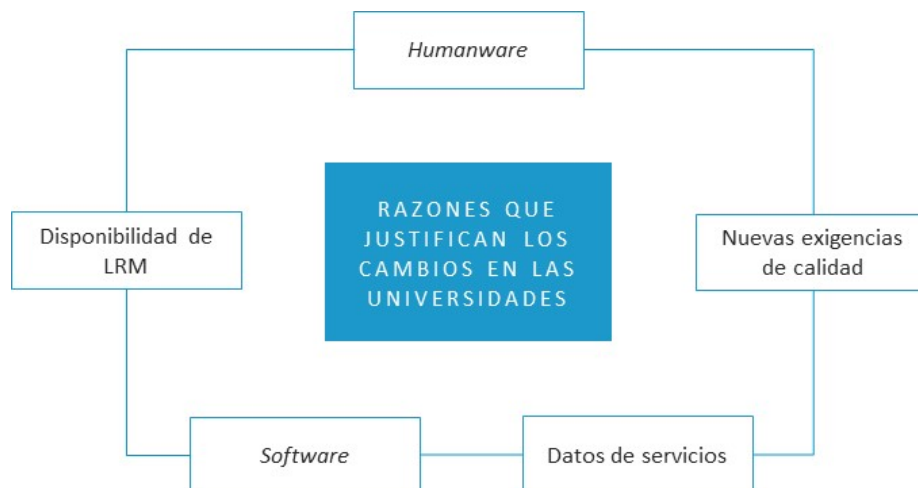


Figura 2. Elementos de los sistemas de información. Fuente: elaboración propia.

Todo sistema de información eficaz y eficiente se basa en:

- ▶ **Planificación.** Un sistema de información no es un elemento aislado, sino que convive de forma colaborativa con otros sistemas de información con los que interactúa en la consecución de los objetivos de negocio. Asumiendo el punto anterior como cierto, el diseño, ejecución o adquisición de un sistema de información no debe ser una decisión aislada, sino que debe formar parte de un **plan**. Es decir, todo sistema de información, si deseamos que sea eficaz y eficiente, debe estar adecuadamente planificado.

- ▶ **Controles.** Más adelante, desarrollaremos ampliamente la definición y tipología de los controles; por el momento, basta decir que un sistema de información eficaz y eficiente debe estar controlado: tienen que existir mecanismos para medir y asegurar que el activo de información lleva a cabo su **cometido** de una forma eficaz y eficiente.
- ▶ **Procedimientos.** El uso de los sistemas de información por parte de la organización debe estar descrito para asegurar su efectividad y eficacia, y basarse en un marco de procesos y procedimientos definido por la propia organización como sustento necesario para la **gestión** posibilitadora del adecuado gobierno.
- ▶ **Estándares.** El marco de gestión indicado en el punto anterior tiene que basarse en los estándares internacionales reconocidos para asegurar la **interoperabilidad** de la organización con otras organizaciones y el libre mercado.
- ▶ **Sistemas de seguridad.** Todo sistema de información debe desenvolverse en el contexto de un marco de gestión de seguridad conducente a asegurar la **integridad, confidencialidad y disponibilidad** de la información que ha sido generada, procesada, tratada o accedida por dicho sistema de información.

La **ausencia** de alguno de los elementos anteriores como base de un sistema de información, especialmente en un entorno informático, puede dar lugar a diversos **riesgos** con impacto en la organización.

Las TIC como apoyo al plan estratégico de las organizaciones

El plan de las **Tecnologías de la Información y Comunicaciones (TIC)** debe dar respuesta a las necesidades del negocio expuestas en el plan estratégico de la organización y ser coherente con este.

El alineamiento del plan estratégico y del plan de las TIC tiene una doble dirección en la medida en que el negocio conforma las necesidades de sistemas de información y la tecnología posibilita medios cada vez más eficaces y eficientes para cubrir los objetivos de negocio, por lo que ambos planes han de retroalimentarse y estar alienados para asegurar el adecuado gobierno de las TIC.

Si bien es necesario el **alineamiento** y la **integración** entre el plan estratégico y el plan de las TIC, así como una adecuada coordinación entre el **CIO** (*chief information officer*) y el **CEO** (*chief executive*), habrá que tener siempre en cuenta que la tecnología está al servicio del negocio, de manera que los avances tecnológicos han de ser entendidos y digeridos por la organización a través de su plan TIC para extraer de ellos todo su **potencial** y contribución al negocio, pero sin condicionarlo o hacerlo **dependiente** de la tecnología (esto que parece obvio evitar, en buena parte de los casos, acontece).

Gobierno corporativo y gobierno de TI

El **gobierno corporativo** se define como un comportamiento empresarial ético, por parte de la dirección y gerencia, para la creación y entrega de los beneficios para todas las partes interesadas (*stakeholders*).

El **ITGI** (**IT Governance Institute**) tiene como misión asistir a los líderes empresariales en su responsabilidad de asegurar que las TIC están alineadas con el negocio y generan valor. Mide si su rendimiento y sus recursos son adecuadamente administrados y sus riesgos gestionados y mitigados.

Para el ITGI, el gobierno de TI es una parte integral del gobierno corporativo que consiste en liderar y definir las estructuras y procesos organizativos que aseguran que las TIC de la empresa soportan y difunden la estrategia y los objetivos de la organización. La responsabilidad es del Comité de Dirección y Gerencia. De ambos conceptos podemos extraer:

- ▶ El gobierno de Tecnología de la Información (TI) es el **alineamiento estratégico** de las TI con la organización, de tal forma que se consigue el máximo valor de negocio por medio del desarrollo y mantenimiento de un control y la responsabilidad efectiva, gestión de la eficiencia y la eficacia (desempeño), así como la gestión de riesgos de TI.
- ▶ El plan informático (plan TIC) tiene que corresponder con el **plan estratégico** de la empresa; el primero es un control general de más alto nivel, tal y como veremos más adelante.

La siguiente ilustración desarrolla una comparativa entre el corporativo y el gobierno TIC:

GOBIERNO CORPORATIVO	PLANIFICACIÓN ESTRATÉGICA SI	GOBIERNO DE TI
División estratégica	Alineación de inversiones de sistemas de información con objetivos de negocio	Alineamiento estratégico
	Explotación de las SI para ventaja competitiva	Entrega de valor de negocio mediante TIC, explotar oportunidades y maximizar beneficios
Gestión de desempeño	Dirigir de manera eficaz y eficiente los recursos de SI	Gestión de desempeño, utilización responsable de los recursos de las TIC
Gestión de riesgos		Gestión de riesgos, gestión apropiada de riesgos de TIC
Política y responsabilidad	Desarrollo de arquitecturas y políticas tecnológicas	
Control y responsabilidad		

Tabla 1. Comparativa entre gobierno corporativo y gobierno TIC. Fuente: elaboración propia.

Es necesario que cada organización administre sus recursos de TI a través de un

conjunto estructurado de controles para asegurar la integridad, exactitud, confidencialidad y disponibilidad que requiere para su negocio.

La **ISACA (Information Systems Audit and Control Association)** y el **ITGI** han desarrollado **COBIT (Control Objectives for Information and Related Technology)**, un marco de objetivos de control para información y tecnologías relacionadas que viene a ser una guía de mejores prácticas dirigida a la gestión de TI. Su última versión es COBIT 5.

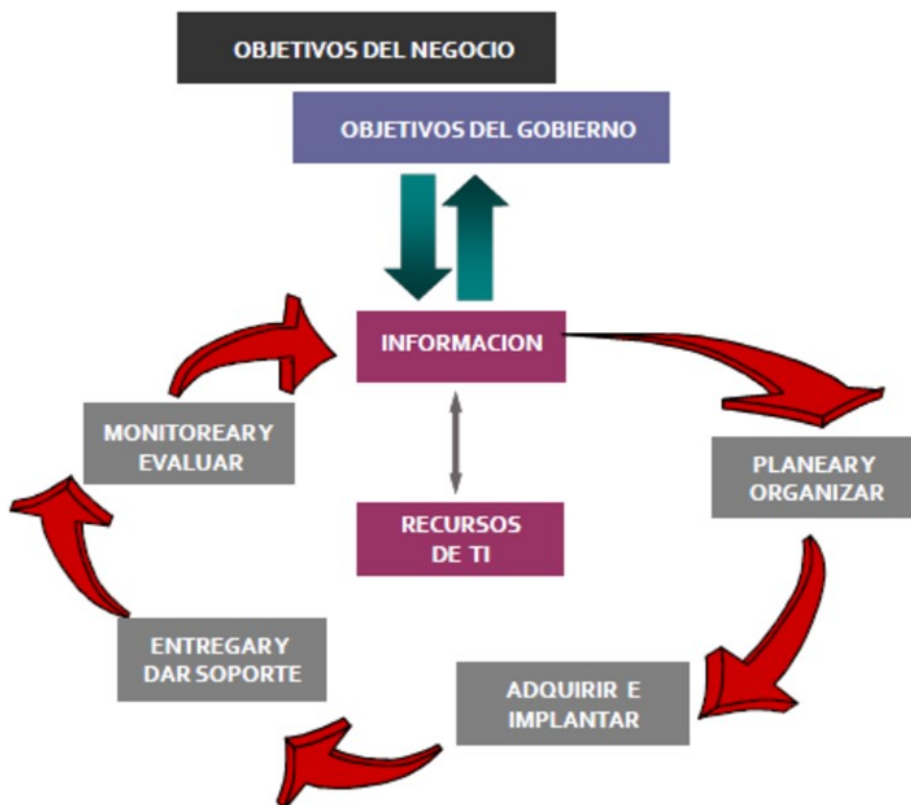


Figura 3. Esquema de los dominios de COBIT. Fuente: interpolados.wordpress.com, 2017.

Directrices del gobierno de TI

El gobierno no solo persigue el logro de los objetivos del negocio (para lo cual es necesaria la gestión), sino que, además, estos objetivos tienen que lograrse asegurando la **sostenibilidad** de la organización o entidad, en equilibrio y

cumplimiento de unos principios de responsabilidad, ética y conducta. Estos principios vertebran las directrices del buen gobierno mencionadas por la ITGI, en concreto:

- ▶ **Responsabilidad.** Todo el mundo debe comprender y asumir sus responsabilidades en la oferta o demanda de TI. La responsabilidad sobre una acción lleva aparejada la autoridad para su realización.
- ▶ **Estrategia.** La estrategia de negocio de la organización tiene en cuenta las capacidades actuales y futuras de las TI. Los planes estratégicos de TI satisfacen las necesidades actuales y previstas derivadas de la estrategia de negocio.
- ▶ **Adquisición.** Las adquisiciones de TI se hacen por razones válidas, basándose en un análisis apropiado y continuo, con decisiones claras y transparentes. Hay un equilibrio adecuado entre beneficios, oportunidades, costes y riesgos, tanto a corto como a largo plazo. Existe una planificación.
- ▶ **Rendimiento.** La TI está dimensionada para dar soporte a la organización al proporcionar los servicios con la calidad adecuada para cumplir con las necesidades actuales y futuras. Es decir, existe una gestión de la capacidad y continuidad de la TI para que sea resistente y asegure la sostenibilidad del negocio.
- ▶ **Conformidad.** La función de TI cumple todas las legislaciones y normas aplicables. Las políticas y prácticas al respecto están claramente definidas, implementadas y exigidas.
- ▶ **Conducta humana.** Las políticas de TI, prácticas y decisiones respecto a la conducta humana, incluyendo las necesidades actuales y emergentes de todas las partes involucradas.



Figura 4. Principio gobierno de TI. Fuente: elaboración propia.

El rol de la auditoría en el gobierno de TI

La auditoría informática es una pieza clave en la medición de la **eficacia de los controles** puestos en marcha por la organización para asegurar el cumplimiento de los objetivos del negocio, ya que es la que está mejor posicionada para:

- ▶ Recomendar prácticas a la alta dirección, con el fin de mejorar la calidad y efectividad de las iniciativas y controles de gobierno de TI implantados.
- ▶ Asegurar el cumplimiento de las iniciativas de gobierno de TI.

La auditoría informática necesitará evaluar los siguientes **aspectos** relacionados con el gobierno de TI:

- ▶ El **alineamiento** de la función de TI con la misión, la visión, los valores, los objetivos y las estrategias de la organización.
- ▶ El **logro** por parte de la función de TI de los objetivos de eficiencia y eficacia establecidos por el negocio.
- ▶ Los **requisitos** legales, de seguridad y los propios de la empresa.
- ▶ El **entorno** de control diseñado y puesto en marcha por la organización.

- ▶ Los **riesgos** intrínsecos dentro de TI, su probabilidad de ocurrencia y su grado de impacto en el negocio

Es importante destacar que **recomendar e informar** a la alta dirección por parte del auditor implica:

- ▶ Definir el **alcance de la auditoría**, incluyendo áreas y aspectos funcionales por cubrir.
- ▶ Establecer el **nivel de dirección** al que se entregará el informe de auditoría.
- ▶ Garantizar el derecho de **acceso a la información** por parte del auditor, poner a su disposición el conjunto de información necesaria y la colaboración por parte de todos los departamentos de la empresa, así como de los terceros relacionados.

Diferencias entre control interno y auditoría informática

La auditoría informática es la revisión independiente, sistemática y objetiva del control interno informático. La siguiente ilustración esquematiza las diferencias entre control interno informático y auditoría informática:

	CONTROL INTERNO INFORMÁTICO	AUDITOR INFORMÁTICO
Semejanzas	▶ Personal interno. Conocimiento especializado en TIC. ▶ Verificación del cumplimiento de controles internos, normativa y procedimientos establecidos por Dirección informática y Dirección general para los SI.	
Diferencias	▶ Análisis de los controles en el día a día. ▶ Informa a la Dirección del Departamento de Informática. ▶ Son personal interno. ▶ El alcance de sus funciones es únicamente sobre el Departamento de Informática.	▶ Análisis de un momento informático determinado. ▶ Informa a la Dirección General de la organización. ▶ Personal interno y/o externo. ▶ Tiene cobertura sobre todos los componentes de los sistemas de información de la organización.

Tabla 2. Control interno informático vs. auditoría informática. Fuente: elaboración propia.

La principal **diferencia** entre ambos radica en que:

- ▶ El **control interno informático** realiza su verificación en el día a día, asegurando la eficacia de los controles; es decir, su función es la de la monitorización y aseguramiento de la eficacia de los controles.
- ▶ La **auditoría informática** lleva a cabo un análisis con una frecuencia puntual, en un momento determinado y con un propósito muy concreto.

Asimismo, su alcance es distinto; ya que, en el caso de **control interno**, es el departamento de informática y, en el caso de la **auditoría**, la revisión abarca aspectos que engloban a toda la organización.

6.3. Fundamentos y conceptos de la auditoría de sistemas de información

Auditoría de sistemas

Se encarga de llevar a cabo la evaluación de normas, controles, técnicas y procedimientos que se tienen establecidos en una organización para lograr **confiabilidad, oportunidad, seguridad y confidencialidad** de la información que se procesa a través de los sistemas de información. La auditoría de sistemas es una rama especializada de la auditoría que promueve y aplica conceptos de auditoría en el área de los sistemas de información.

La auditoría de los sistemas de información se define como cualquier auditoría que abarca la revisión y evaluación de todos los aspectos (o de cualquier porción de ellos) de los sistemas automáticos de procesamiento de la información, incluidos los procedimientos no automáticos relacionados con ellos y las interfaces correspondientes.

El objetivo final que tiene el auditor de sistemas es dar **recomendaciones** a la alta gerencia para mejorar o lograr un adecuado control interno en ambientes de tecnología informática, con el fin de lograr mayor eficiencia operacional y administrativa.

Los objetivos específicos de una auditoría de sistemas son:

OBJETIVOS ESPECÍFICOS DE UNA AUDITORÍA DE SISTEMAS
Evaluación de la seguridad en el área informática
Participación en el desarrollo de nuevos sistemas
▶ Evaluación de controles. ▶ Cumplimiento de la metodología.
Evaluación de suficiencia en los planes de contingencia
▶ Respaldos, prever qué va a pasar si se presentan fallas.
Opinión de la utilización de los recursos
▶ Fraude. ▶ Control a las modificaciones de los programas. Evaluación de controles.
Participación en la negociación de contratos con los proveedores
Revisión de la utilización del sistema operativo y los programas utilitarios
Auditoría de las bases de datos
▶ Estructura sobre la cual se desarrollan las aplicaciones.
Auditoría de la red de teleprocesos
Desarrollo de <i>software</i> de auditoría

Figura 5. Objetivos específicos de una auditoría de sistemas. Fuente: elaboración propia.

El fin principal de la auditoría de sistemas es poder generar una **opinión objetiva** del auditor interno o externo sobre la confiabilidad de los sistemas de información, así como de la eficiencia de las operaciones en el área de TI.

Requerimientos del auditor de sistemas

- ▶ Entendimiento global e integral del negocio, de sus puntos claves, áreas críticas, entorno económico, social y político.
- ▶ Entendimiento del efecto de los sistemas en la organización.
- ▶ Entendimiento de los objetivos de la auditoría.
- ▶ Conocimiento de los recursos de computación de la empresa.
- ▶ Conocimiento de los proyectos de sistemas.

6.4. Tipos y clases de auditoría

Existen infinidad de tipos y clases de auditoría, todo depende de las necesidades de la empresa y los riesgos o requerimientos a los que esté expuesta. Por ello, con carácter enumerativo, pero no limitativo, se expresan diferentes tipos (Argudo, 2017):

TIPOS DE AUDITORÍA	
Auditoría externa o legal	Es la más conocida y consiste en el análisis a través de un profesional auditor externo.
Auditoría interna	Se lleva a cabo por los propios empleados del negocio para investigar la validez de los métodos de operaciones y su coherencia con respecto a la política general de la empresa. Para ello se evalúan ciertos detalles que intervienen en los procesos y mecanismos internos. Es una herramienta clave para el control interno y, una vez finalizado el análisis, emitirá un informe a la dirección, o a órganos superiores del equipo, para evaluar posibles soluciones en referencia a los problemas encontrados.
Auditoría operacional	Este tipo de auditoría se desempeña por un profesional cualificado para ello y tiene como objetivo valorar la empresa y su gestión para aumentar la eficacia y la eficiencia hacia una mejora importante en la productividad. No tiene por qué desarrollarse por alguien interno de la empresa, sino que la propia dirección podrá contratar a un profesional especializado en ello. El auditor analizará el sistema y propondrá ideas con mejoras útiles.
Auditoría de sistemas o especiales	En este grupo encontramos otro tipo de auditorías dirigidas a evaluar otro tipo de factores no económicos, como es el caso de la auditoría de software, entre otros muchos.
Auditoría de seguridad operativa/técnica	Son auditorías o revisiones de seguridad técnica de un sistema o sistemas informáticos o de telecomunicaciones de una organización.
Auditoría de cumplimiento de la seguridad de la información	Son auditorías que verifican el cumplimiento de un determinado estándar o políticas y procedimientos internos de seguridad de la organización de seguridad (por ejemplo, PCI o DSS).
Auditoría pública gubernamental	Se desarrolla por el Tribunal de Cuentas gracias a las competencias adquiridas por diferentes legislaciones.
Auditoría integral	Esta auditoría evalúa por completo toda la información financiera, la estructura de la organización, los sistemas de control interno, el cumplimiento de las leyes y los objetivos empresariales para dar una visión global y certera del cumplimiento de la empresa.
Auditoría forense	Se realizan en las investigaciones criminales con el objetivo de esclarecer los hechos ocurridos.
Auditoría fiscal	Esta auditoría se realiza con el objetivo de velar por el cumplimiento de las leyes tributarias, para que las empresas y organizaciones paguen sus impuestos de forma correcta.
Auditoría financiera	También denominada auditoría contable. Se encarga de examinar y revisar los estados financieros y la preparación de informes de acuerdo con normas contables establecidas.
Auditoría de recursos humanos	Se utiliza para hacer una revisión de la plantilla, las necesidades que posee la empresa y la gestión del talento.
Auditoría ambiental	Se analizan todas las actividades de la empresa para controlar e intentar reducir al máximo el impacto al medioambiente.

Tabla 3. Tipos de auditoría. Fuente: adaptado de Argudo, 2017.

La auditoría de la seguridad operativa/técnica engloba los conceptos de seguridad **física, lógica y operaciones**.

La **seguridad física** se centra en la protección del entorno (ubicación), del hardware y de cualquier elemento físico capaz de tratar información. Para ello, el auditor deberá comprender un amplio abanico de amenazas que pueden estar englobadas en desastres naturales o industriales, entre otros.

Por otro lado, la seguridad lógica se refiere a la seguridad de la información, del software, comunicaciones, arquitectura lógica, accesos... Por ello, el auditor deberá comprender y analizar amenazas que puedan afectar a esta taxonomía.

Otra parte importante que auditar corresponde a las **operaciones de los CPD de seguridad** con el fin de valorar su operación eficaz y eficiente conforme al seguimiento de políticas, procedimientos y planes desarrollados.

6.5. ¿Para qué sirve una auditoría informática?

Lo primero que debemos tener en claro es que la auditoría informática va a ayudar a la empresa a comprobar la **eficiencia** de los sistemas que tiene establecido. Gracias a este análisis sabremos si funciona de forma correcta y, al utilizar los recursos adecuados, veremos si ha surgido algún problema en su interior o las barreras que podemos tener presentes. Todo ello convierte la auditoría informática en una **herramienta clave** para que nuestro sistema siempre esté en marcha de forma correcta.

Además de los aspectos técnicos, estas auditorías también se realizan para conocer si el sistema informático en cuestión cumple con toda la normativa y las leyes que rigen este sector.

Desde la protección de datos hasta, incluso, aspectos medioambientales, la auditoría va a recabar toda esa información. Con ella podremos verificar si nuestro sistema cumple con todos los requisitos legales. Asimismo, es una herramienta perfecta para poder revisar la eficiencia de los recursos que utilizamos. No solo hablamos de elementos materiales, sino que aquí también se pueden incluir todas aquellas personas que forman parte de este sistema informático y hacen uso de él de manera diaria.

6.6. Funciones y objetivos de la auditoría informática

El objetivo principal del auditor es el de **evaluar y comprobar**, en determinados momentos del tiempo, los controles y procedimientos informáticos más complejos, objeto del análisis, desarrollando y aplicando metodologías de auditoría.

La auditoría informática emplea las mismas técnicas descritas de inspección y observación, entrevistas, documentación y procedimientos analíticos propios de cualquier tipo de auditoría, si bien:

- ▶ En la auditoría informática, las técnicas de inspección hacen un mayor uso de software de inspección capaz de automatizar la verificación de los sistemas de información que en el caso del resto de tipos de auditoría.
- ▶ La auditoría informática no se denomina de ese modo por este mayor uso de medios informáticos, sino porque el objeto auditado son los sistemas de información, las TIC.

Es decir, actualmente no es posible verificar, de forma manual, procedimientos informatizados que resuman, calculen y clasifiquen datos, por lo que se debe emplear software de auditoría CAATS (Computer Assisted Audit Techniques). El auditor es responsable de:

- ▶ **Llevar a cabo la auditoría:**
 - Planificar la auditoría.
 - Llevar a cabo las distintas tareas definidas en las fases de la auditoría.
 - Escuchar y observar (recordemos el origen latino del término: «aquel que tiene la capacidad de oír»).

- Gestionar las desviaciones y riesgos que pudieran producirse durante la auditoría.
- ▶ **Informar a la dirección de la organización acerca de:**
 - El diseño y funcionamiento de los controles implantados.
 - La fiabilidad de la información suministrada (su competencia).
 - La suficiencia de las evidencias o la medida en la que sustentan las conclusiones de la auditoría.

El auditor presentará a la dirección de la organización un informe de auditoría que contendrá, entre otros puntos:

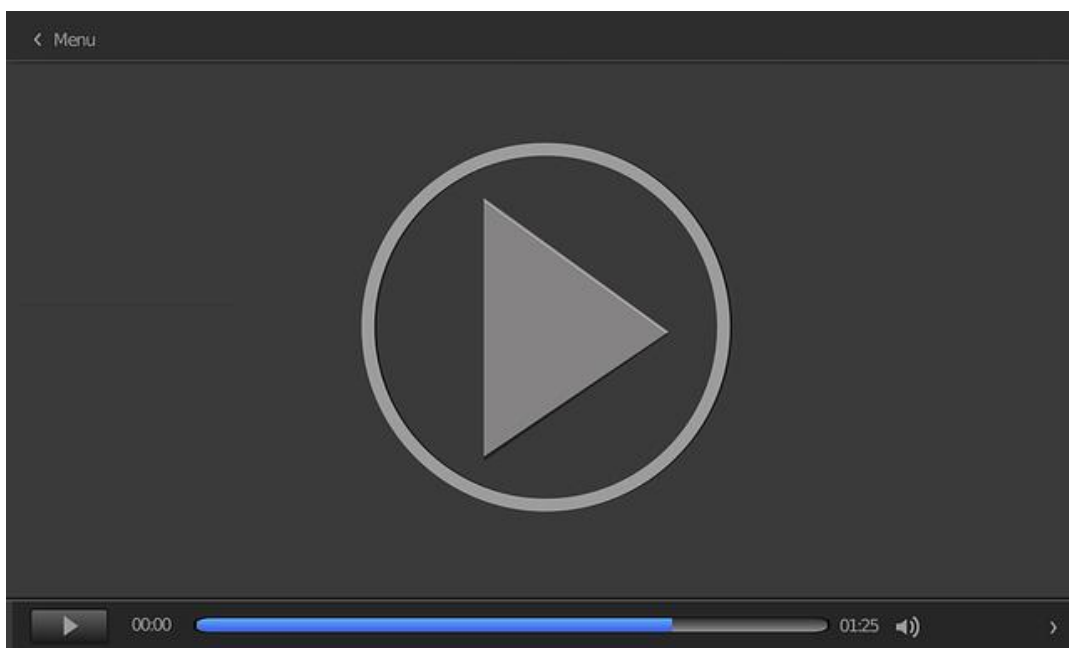
INFORME DE AUDITORÍA

Las conclusiones de la auditoría	
Las no conformidades	▶ Evidencias relacionadas. ▶ Menores y mayores.
Las observaciones	▶ Evidencias relaciones.
Una descripción de la auditoría	▶ Objetivo. ▶ Alcance. ▶ Fases y fechas. ▶ Técnica(s) empleada(s). ▶ Áreas auditadas. ▶ Entrevistados.
La relación de evidencias detectadas y por cada una	▶ Su competencia. ▶ Su suficiencia.

Figura 6. Algunos puntos que debe contener el informe de auditoría. Fuente: elaboración propia.

6.7. Proceso de auditoría de sistemas

En este vídeo (*Proceso de auditoría de sistemas*) se presentan los principales aspectos del proceso de auditoría de sistemas: planeación y documentación, archivo permanente, evaluación y control interno, evaluación de hallazgos, programa de auditoría de sistemas, documentos de la auditoría, definición y tipos de informes.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=a207831c-707d-4684-a915-adf700cfee71>

6.8. Ventajas e inconvenientes de las auditorías

Auditoría interna

► Ventajas

- Facilita una ayuda primordial a la dirección al evaluar de forma relativamente independiente los sistemas de organización y de administración.
- Facilita una evaluación global y objetiva de los problemas de la empresa, que generalmente suelen ser interpretados de una manera parcial por los departamentos afectados.
- Pone a disposición de la dirección un profundo conocimiento de las operaciones empresariales, proporcionado por el trabajo de verificación de los datos y financieros.
- Contribuye eficazmente a evitar las actividades rutinarias y la inercia burocrática que, generalmente, se desarrollan en las grandes empresas.
- Favorece la protección de los intereses y bienes de la empresa frente a terceros.

► Inconvenientes

- El grado de objetividad es menor que en las auditorías externas.
- La responsabilidad del auditor se limita a aquella que emerge del correspondiente contrato de trabajo.
- Puede crear conflictos con personal que sea afectado por el estudio.

Auditoría externa

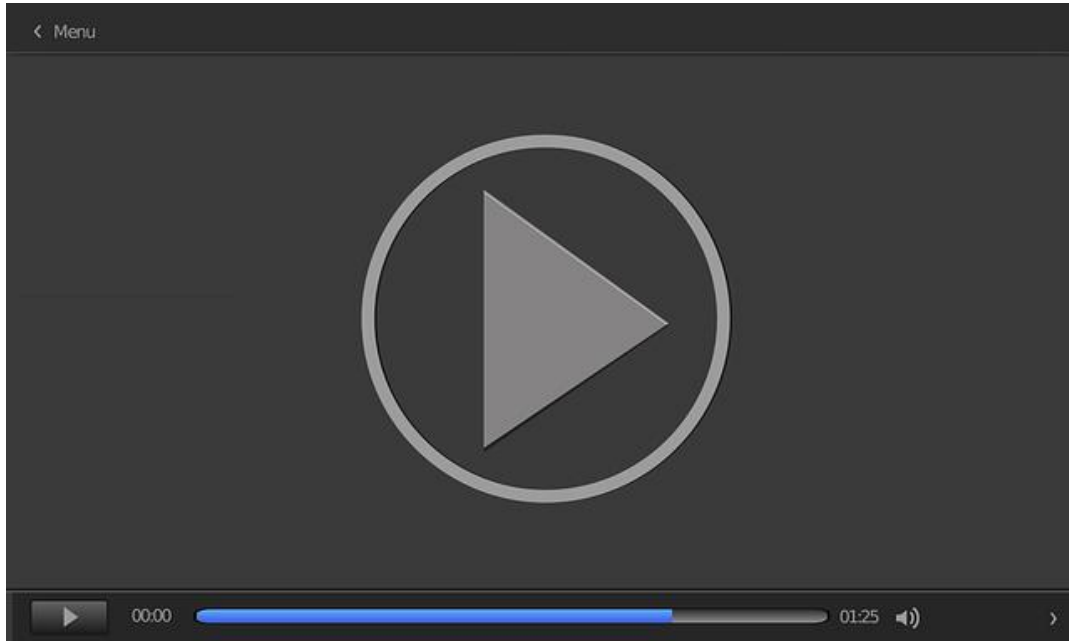
► Ventajas

- Con tiene un alto grado de imparcialidad.
- Se utiliza cuando los recursos de la empresa no son suficientes para mantener un departamento de auditoría interno.
- No necesita la presencia continua del auditor para llevarse a cabo.

► Inconvenientes

- Superan los costos de mantenimiento en comparación con los sistemas de auditoría interna.
- Existe un menor grado de responsabilidad por parte de los auditores.
- No lleva un debido seguimiento en cuanto a las recomendaciones implementadas.
- Se basa más en evaluaciones financieras que en operativas.
- Por lo regular, se limita a un dictamen del auditor.

En este vídeo (*Auditorías técnicas de seguridad en la herramienta Nessus*) se va a realizar una presentación y demostración de la herramienta de escaneo de vulnerabilidades conocida como NISSUS, que es de gran utilidad para la realización de auditorías técnicas de seguridad.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=b3519603-24e3-4a61-b6ce-adf700cfeed6>

6.9. Referencias bibliográficas

Archivos auditoría en sistemas. (s. f.). *Similitudes y diferencias con la auditoría tradicional*. <http://archivosauditoria.blogspot.com/2009/11/similitudes-y-diferencias-con-la.html>

Argudo, C. (20 de abril de 2017). *Tipos de auditoría*. EmprendePyme.

<https://www.emprendepyme.net/tipos-de-auditoria.html>

Auditoría de Sistemas. (2013, septiembre 24). *Ventajas y desventajas*. <http://etitcauditoriasistem.blogspot.com>

Auditoría de sistemas. (2016, mayo 4). *Fases de la auditoría de sistemas*. <https://auditoriasistema12.wordpress.com/>

Interpolados (2017, febrero 31). *Objetivos del negocio*. <https://interpolados.files.wordpress.com/2017/02/31.png?w=924>

Organización Internacional de Normalización. (2004). *ISO/IEC 13335-1:2004. Information technology — Security techniques — Management of information and communications technology security — Part 1: Concepts and models for information and communications technology security management*. Organización Internacional de Normalización. <https://www.iso.org/standard/39066.html>

Auditoría de los estándares ISO en las TIC

Fernández Sánchez, C. M. (2010). *Auditoría de los estándares ISO en las TIC: una herramienta de la Dirección*. Red seguridad, 45, 38-39. <https://dialnet.unirioja.es/servlet/articulo?codigo=3174978>

Este artículo de opinión, elaborado por D. Carlos Manuel Fernández, jefe de Servicio TIC de la Dirección de Desarrollo de AENOR, presenta la hoja de ruta de los estándares ISO en las TIC y el papel de la auditoría interna y externa como herramienta para la dirección.

ISACA Madrid Chapter

ISACA Madrid Chapter. (2020). Inicio [Canal de YouTube].
YouTube. <https://www.youtube.com/channel/UCC1ecsnmROaNG713h1cRsZA>

Los vídeos de este canal son muy interesantes y en línea con la auditoría de los sistemas de información.

Auditoría de sistemas. Una visión práctica

Tamayo, A. (2001). *Auditoría de sistemas. Una visión práctica*. Universidad Nacional de Colombia (Sede Manizales). <https://repositorio.unal.edu.co/handle/unal/60273>

Enriquecedor libro donde se dan unas pautas introductorias a la auditoría de sistemas de información.

The IS audit process

Sayana, A. (s. f.). *Chapter I. The IS audit process*. https://isaudit101.wordpress.com/chapter_i/

Este artículo concentra, en su corta extensión, los aspectos clave de la auditoría de sistemas de información con tanta completitud como concreción y brevedad en su exposición.

Cambios en los controles del Anexo A (ISO 27002:2022)

La principal actualización está en el Anexo A, que ahora tiene 93 controles (frente a los 114 de la versión 2013), reorganizados en 4 dominios (antes eran 14). Algunos controles relevantes para auditoría son:

- ▶ A.5.7 (Threat intelligence): nuevo control sobre inteligencia de amenazas.
- ▶ A.5.23 (Information security for use of cloud services): nuevo control específico para la seguridad en la nube.
- ▶ A.5.30 (ICT readiness for business continuity): reemplaza y consolida controles anteriores sobre continuidad del negocio.
- ▶ A.8.9 (Configuration management): refuerza la gestión de configuraciones seguras.
- ▶ A.8.10 (Information deletion): nuevo control sobre borrado seguro de información.

Estructura de los controles

Los controles ahora siguen una estructura de 5 atributos para facilitar su gestión:

- ▶ Tipo de control (preventivo, detective, correctivo).
- ▶ Propiedades de seguridad (confidencialidad, integridad, disponibilidad).
- ▶ Conceptos de ciberseguridad (identificar, proteger, detectar, responder, recuperar - NIST CSF).
- ▶ Dominios operacionales (gobernanza, protección, defensa, resiliencia).
- ▶ Ciclo de vida de la seguridad (planificación, implementación, evaluación, mejora).

Auditoría interna (cláusula 9.2)

No hay cambios significativos en la cláusula 9.2 (Auditoría interna), que sigue requiriendo:

- ▶ Planificación de auditorías periódicas.
- ▶ Independencia del auditor.
- ▶ Documentación de hallazgos y no conformidades.
- ▶ Acciones correctivas.

Revisión por la dirección (cláusula 9.3)

Tampoco hay cambios importantes, pero se alinea mejor con el enfoque de riesgo y oportunidades introducido en la ISO 27001:2013 (basado en la estructura de alto nivel - HLS).

1. ¿Cuál es uno de los pilares fundamentales de la definición de auditoría?
 - A. Proceso objetivo.
 - B. Proceso subjetivo.
 - C. Calculo el riesgo tecnológico.
 - D. Calculo el riesgo operacional.

2. El objetivo final de la auditoría es:
 - A. Generar un proceso subjetivo.
 - B. Dar recomendaciones a la alta dirección.
 - C. Entregar un informe.
 - D. Comunicar los resultados de manera correcta.

3. Cualquier sistema informático utilizado por las compañías debe:
 - A. Salvaguardar la propiedad de la información.
 - B. Mantener la integridad de los datos.
 - C. Asegurar la confidencialidad de la información.
 - D. Todas las anteriores.

4. ¿Cuál de las siguientes afirmaciones es cierta?
 - A. No hace falta un alineamiento entre el plan estratégico y el plan de las TIC al tratarse de taxonomías de planes totalmente diferentes.
 - B. Las organizaciones con consideración a las TI tienen mayor retorno de inversión a medio y largo plazo que aquellas que no lo consideran ante los mismos objetivos estratégicos, de tal forma que con la madurez tecnológica se llegue a un estado de mayor retorno.
 - C. El plan informático de TI se tiene que corresponder con el plan estratégico de la empresa.
 - D. Ninguna afirmación es cierta.

5. La auditoría del CPD, ¿corresponde con una auditoría de sistemas o de seguridad física?
- A. Al estar los sistemas dentro del CPD, se corresponde en exclusiva con una auditoría de sistemas.
 - B. Al ser una ubicación física, estas auditorías están centradas en las auditorías de seguridad física.
 - C. Es una auditoría de sistemas porque no existen las auditorías de seguridad física.
 - D. Pueden ser las dos, dependiendo de los objetivos de la auditoría.
6. Cuáles son las diferencias más relevantes entre auditoría y control internos.
- A. Realizan la misma función, por lo que la diferencias simplemente es en la nomenclatura que se le dé en cada empresa.
 - B. El control interno tiene mucho más peso que la auditoría interna.
 - C. La auditoría interna tiene mucho más peso que control interno.
 - D. La periodicidad y frecuencia de sus actuaciones.
7. Las auditorías de seguridad solo engloban:
- A. Los temas tecnológicos.
 - B. Los temas físicos.
 - C. Los temas operacionales.
 - D. Todas las anteriores.

- 8.Cuál es uno de los objetivos principales del auditor:
- A. Elaborar un informe de situación y entregarlo.
 - B. Evaluar y comprobar, en determinados momentos, los controles y procedimientos informáticos.
 - C. Pedir evidencias y analizarlas.
 - D. Ninguna de las anteriores.
9. Se denomina auditoria informática porque:
- A. Se utilizan muchos sistemas informáticos para encontrar los resultados.
 - B. Solo pueden realizarse por personas con unos estudios relacionados con la Informática.
 - C. El objeto auditado son los sistemas de información.
 - D. Ninguna de las anteriores.
10. Cuando se encuentran hallazgos en la auditoria:
- A. Se notifica de manera inmediata cualquier hallazgo a la Alta Dirección.
 - B. Se notifica de manera inmediata cualquier hallazgo a nuestro interlocutor.
 - C. Se documentan formalmente y ya se notificarán.
 - D. Ninguna de las anteriores.

Desarrollo Seguro de Software y Auditoría de la
Ciberseguridad

Tema 7. Gobierno y gestión de la función de auditoría

Índice

Esquema

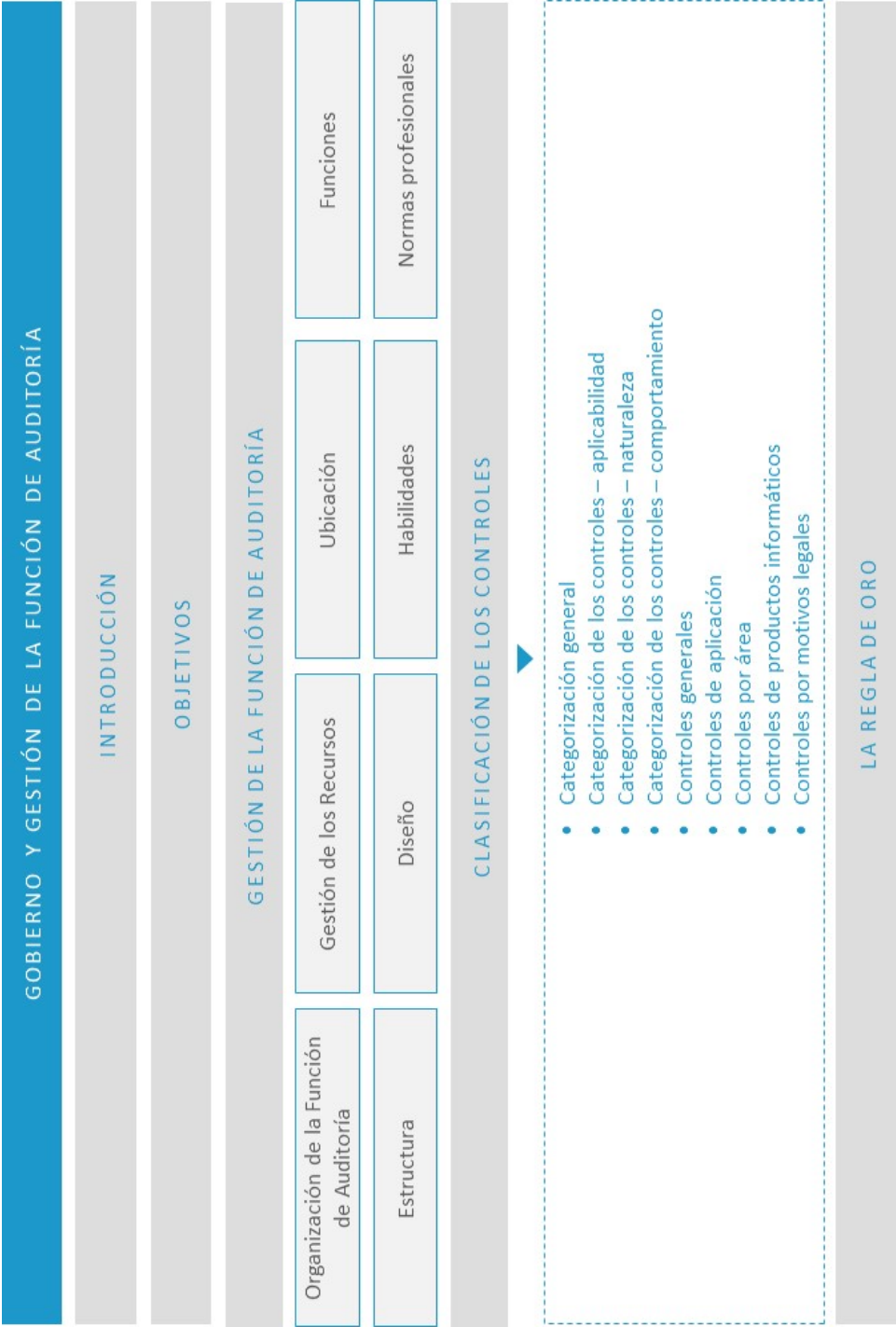
Ideas clave

- 7.1. Introducción y objetivos
- 7.2. Gestión de la función de auditoría
- 7.3. Clasificación de los controles
- 7.4. La regla de oro
- 7.5. Referencias bibliográficas

A fondo

- La revisión de los controles generales en un entorno informatizado
- Auditoría del control interno
- CIS
- Banco de España

Test



7.1. Introducción y objetivos

La información es un recurso clave para todas las empresas y, desde el momento en que se crea hasta que es destruida, la tecnología juega un papel importante. La **tecnología de la información** está avanzando cada vez más y se ha generalizado en las empresas y en entornos sociales, públicos y de negocios. Como resultado, hoy más que nunca, las empresas y sus ejecutivos se esfuerzan en:

- ▶ Mantener información de **alta calidad** para soportar las decisiones del negocio.
- ▶ Generar **valor** al negocio con las inversiones en TI, por ejemplo, alcanzando metas estratégicas y generando beneficios al negocio a través de un uso eficaz e innovador de las TI.
- ▶ Alcanzar la **excelencia operativa** a través de una aplicación de la tecnología fiable y eficiente.
- ▶ Mantener los **riesgos** relacionados con TI en un nivel aceptable.
- ▶ **Optimizar** el coste de los servicios y tecnologías de TI.
- ▶ **Cumplir** con las leyes en constante crecimiento, regulaciones, acuerdos contractuales y políticas aplicables.

Después de comprender esta primera premisa, ¿sería importante realizar auditorías?
¿Qué se entiende por auditoría?

Una auditoría tradicional se define como un proceso sistemático por el que una persona competente e independiente obtiene y evalúa objetivamente evidencias respecto a afirmaciones sobre una entidad económica o un caso con el fin de formarse una opinión e informar sobre el grado en que dicha información se ajusta a un conjunto determinado de estándares.

La asociación ISACA, una fuente confiable de conocimiento y estándares, fundada en 1969, que cuenta con más de 115 000 integrantes en 180 países y cuyo objetivo es ayudar a empresas a construir confianza y maximizar el valor de la información de los sistemas de información, define la **auditoría de los sistemas de información** como:

«Proceso de recolección y evaluación de evidencias para determinar si los sistemas de información y recursos relacionados salvaguardan adecuadamente los activos, mantienen la integridad de los datos y del sistema, proveen información fiable, logran efectivamente las metas de la organización, consumen los recursos de manera eficiente, y tienen en vigor los controles internos que proveen una garantía razonable de que se alcanzarán los objetivos del negocio, operativos y de control» (ISACA, 2019).

¿Por qué son importantes las auditorías de seguridad de sistemas de información?

- ▶ Por la dependencia creciente de la información y de los sistemas que proporcionan dicha información.
- ▶ Por la creciente vulnerabilidad y un amplio espectro de amenazas, incluidas las **ciberamenazas**.
- ▶ Por el coste de las actuales y futuras inversiones en información y en sistemas de información.
- ▶ Por el potencial que tienen las tecnologías para cambiar radicalmente las organizaciones y las prácticas de negocio, crear nuevas oportunidades y reducir costes.

Para asentar los términos, una auditoría de sistemas de información es un **examen** y una **validación** del cumplimiento de los controles y procedimientos utilizados para salvaguardar la confidencialidad, integridad y disponibilidad de los sistemas de información, lo que proporciona al negocio una evaluación **independiente y objetiva**

de los hechos que, en ocasiones, es difícil de obtener cuando se está inmerso en la operación y en la problemática del día a día.

Del mismo modo, se determina si los controles implementados son eficientes y suficientes, se identifican las causas de los problemas existentes en los sistemas de información estableciendo las **acciones preventivas y correctivas** necesarias para mantener los sistemas de información disponibles y confiables e identificando las **causas y soluciones** a los problemas específicos de los sistemas de información que pueden estar afectando a las operaciones y a los objetivos del negocio.

Los **objetivos** del presente tema son los siguientes:

- ▶ Definir y explicar los conceptos relacionados con la gestión de la función de auditoría.
- ▶ Estudiar una clasificación de los diferentes tipos de controles que se pueden auditar en una auditoría.
- ▶ Aprender el concepto de la regla de oro.

7.2. Gestión de la función de auditoría

La función de auditoría debe ser gestionada y conducida en una forma que asegure que las diversas tareas realizadas y logradas por el **equipo de auditoría**, cuyos miembros cumplirán los objetivos de esta mientras se preserva su independencia y competencia. Además, **gestionar** la función de auditoría debería asegurar a la alta dirección las contribuciones al valor agregado respecto a la eficiente gestión de TI y al logro de los objetivos del negocio.

Organización de la función de auditoría

Los servicios de auditoría pueden ser provistos externa o internamente. El rol de la función interna de auditoría debería establecerse en un **estatuto de auditoría** aprobado por la **alta dirección**. La auditoría puede ser parte de la auditoría interna, funcionar como un grupo independiente o estar integrada dentro de una auditoría financiera y operacional para proveer garantía de control; por lo tanto, el estatuto de auditoría puede incluir la **auditoría de SI** como una función de apoyo de auditoría financiera o similar.

Este estatuto debería establecer claramente la responsabilidad y los **objetivos** de la gerencia en lo relativo a la función de auditoría y la delegación de autoridad para la misma. También debería describir la **autoridad, el alcance y las responsabilidades** generales de la función de auditoría. El nivel más alto de gestión y el comité de auditoría, si existe alguno, tendrían que aprobar este estatuto. Una vez establecido, ha de ser modificado solo si dicho cambio puede realizarse y está completamente justificado.

Se debe resaltar que un **estatuto de auditoría** es un documento de **alcance general** que cubre toda la gama de actividades de auditoría en una entidad, mientras que una **carta de compromiso** está más centrada en un ejercicio particular de auditoría, donde se busca que sea iniciado en una organización con un **objetivo específico** en

mente. Si los servicios de auditoría son provistos por una empresa auditora externa, el alcance y los objetivos de estos servicios deben ser documentados en un **contrato formal o declaración de trabajo** entre la organización contratante y el proveedor del servicio.

En cualquier caso, la función de auditoría interna debe ser independiente y reportar a un comité de auditoría, si existe alguno, o al nivel más alto de la gerencia, como, por ejemplo, el consejo de dirección.

Gestión de los recursos de auditoría de sistemas de información

La tecnología está cambiando constantemente. Por lo tanto, es importante que los auditores mantengan su competencia por medio de la actualización de sus habilidades y que obtengan **capacitación** sobre nuevas técnicas de auditoría y áreas de tecnología. El auditor debe mantener su competencia técnica a través de una **educación** profesional continua. Se deben tomar en consideración las habilidades y los conocimientos cuando se planifiquen las auditorías y se asigne personal para tareas específicas de auditoría.

Preferentemente, se debería diseñar un **plan anual** detallado de capacitación del personal basado en la dirección de la organización en términos de tecnología y aspectos relacionados con riesgos que necesiten considerarse. Este plan debería revisarse de manera periódica para asegurar que los esfuerzos y los resultados de capacitación se encuentran alineados con la orientación que esté asumiendo la organización de auditoría. Además, la gerencia de auditoría también debe proporcionar los **recursos de TI** necesarios para realizar apropiadamente auditorías de SI de naturaleza altamente especializada (por ejemplo: herramientas, metodología, programas de trabajo).

Ubicación del departamento de auditoría

Las funciones de una auditoría interna deben quedar enmarcadas dentro de la organización en una unidad que, por su situación jerárquica, le permita la consecución de sus fines. El nivel donde deberá quedar la unidad departamental de auditoría interna reunirá las siguientes características:

- ▶ Que cuente con una **jerarquía** suficiente para poder inmiscuirse en cualquier unidad administrativa de la empresa.
- ▶ Que el tipo de funciones de dicha unidad esté relacionado con **la dirección, el control y la coordinación**.
- ▶ Que tenga suficiente **autoridad** sobre los demás departamentos.

Funciones que desarrollar

Se deberían desarrollar las siguientes funciones:

- ▶ Investigación constante de planes y objetivos.
- ▶ Estudio de las políticas y sus prácticas.
- ▶ Revisión constante de la estructura orgánica.
- ▶ Estudio constante de las operaciones de la empresa.
- ▶ Analizar la eficiencia de la utilización de recursos humanos y materiales.
- ▶ Revisión del equilibrio de las cargas de trabajo.
- ▶ Revisión constante de los métodos de control.

Estructura de la unidad de auditoría

Ubicación jerárquica de una unidad de auditoría y responsabilidades

Con la finalidad de comprender y profundizar en la temática de auditoría, a continuación, se muestran la estructura y los participantes dentro de la auditoría en una compañía:

ESTRUCTURA Y PARTICIPANTES DENTRO DE LA AUDITORÍA EN UNA COMPAÑÍA	
Alta gerencia	▶ Último responsable del sistema de control. ▶ Debe dirigirse a los gerentes que, a su vez, son los responsables en sus respectivas áreas.
Consejo de administración	▶ Fija las pautas y la visión del negocio. ▶ Debe tener un papel activo en el conocimiento de las acciones que se ejecutan. ▶ Debe asegurarse de contar con vías de comunicación efectivas con la alta dirección y las áreas financieras, legales y de auditoría interna.
Auditoría interna	▶ Debe desempeñar un papel de supervisión sobre la eficiencia y permanencia de los sistemas de control. ▶ Debe contar con una ubicación jerárquica adecuada.
Empleados	▶ Tienen la responsabilidad de participar en el esfuerzo de aplicar el control interno. ▶ Deben comunicar al nivel superior las desviaciones que detecten en los códigos de conducta, las políticas establecidas o la legalidad de las acciones realizadas.

Tabla 1. Estructura y participantes dentro de la auditoría en una compañía. Fuente: elaboración propia.

Estructura y atribuciones de las funciones de una unidad de auditoría interna

Una unidad de auditoría interna se estructura de la siguiente manera y posee, entre otras, las siguientes atribuciones.

ESTRUCTURA Y ATRIBUCIONES DE LAS FUNCIONES DE UNA UNIDAD DE AUDITORÍA INTERNA	
Director de auditoría	Dirige a sus subordinados en el desempeño de sus funciones de auditoría interna, que incluye la planeación, coordinación y dirección de sus actividades. Desarrolla políticas y procedimientos para llevar a cabo esta actividad. Planea a corto, mediano y largo plazo las actividades. Reportará única y exclusivamente al superior jerárquico inmediato establecido en el cuadro de la organización.
Gerente de auditoría	Asiste al director en el cumplimiento de sus obligaciones, cubre sus ausencias sobre base rotatoria entre los gerentes de auditoría. Prevé requerimientos de carga de trabajo a corto, mediano y largo plazo en las unidades administrativas y actividades que se le son asignadas. Trabaja directamente bajo la supervisión del director de auditoría, recibiendo de él la orientación y apoyo que sea necesario.
Supervisor de auditoría	Supervisa el cumplimiento de dos o más auditorías en las que concurra. Planea el trabajo que hay que desarrollar y establece prioridades en la auditoría. Selecciona y asigna al personal que sea necesario para ejecutar las auditorías planeadas. Trabaja bajo la dependencia de un gerente de auditoría, provee información tanto a su superior jerárquico inmediato como al director de auditoría.
Encargado de auditoría	Planea y conduce auditorías a las unidades administrativas y actividades de la organización y actúa como líder frente a un equipo de auditores. Ejerce un alto grado de responsabilidad y juicio profesional. Está bajo la supervisión técnica y administrativa de un supervisor de auditoría. Está encargado de elaborar el plan de auditoría y de autorizar los procedimientos y programas de revisión. Es indispensable que el auditor supervisor sea un profesional con suficiente experiencia y capacidad.
Auditor ayudante	Este tiene la responsabilidad de ayudar al auditor supervisor y al de áreas en la recopilación de información, en la planeación del trabajo y en el desarrollo del enfoque aplicado a la auditoría. Desempeña el trabajo asignado bajo la dirección y orientación del auditor encargado. Desarrolla o asiste en la preparación del programa de auditoría. Está bajo la supervisión técnica y administrativa de un encargado de auditoría; cuando se trata de empresas pequeñas puede coordinar actividades.
Auditor principiante (Jr.)	Al encargado de este puesto, generalmente, se le asignan trabajos de rutinas. En la práctica, a esta persona se le guía y supervisa en el desarrollo de su trabajo. El grupo de auditoría debe figurar en el nivel jerárquico de la empresa de donde pueda operar en forma concernida y, además, pueda formar parte de las actividades de la gerencia general, control administrativo y planeación.

Tabla 2. Estructura y atribuciones de las funciones de una unidad de auditoría interna. Fuente: elaboración propia.

Diseño de la estructura de una unidad de auditoría interna

La estructura de la organización es un organismo ideado para ayudar a proyectar las metas; lo fundamental para el diseño de organizaciones es el **conocimiento actual de la empresa**. Sin la comprensión general y específica de la situación actual y la situación futura, la posibilidad de que el sistema propuesto sea bueno se reduce de forma considerable.

Habilidades del equipo auditor

Para los equipos de trabajo, uno de los elementos fundamentales que se tiene que considerar es el relativo a la **experiencia personal de sus integrantes**, ya que de ello depende, en gran medida, el cuidado y la diligencia profesionales que se emplean para determinar la profundidad de las observaciones. Por la naturaleza de la función a desempeñar, existen varios campos que se tienen que dominar:

- ▶ Conocimiento de las áreas sustantivas de la organización.
- ▶ Conocimiento de las áreas adjetivas de la organización.
- ▶ Conocimiento de esfuerzos anteriores.
- ▶ Conocimiento de casos prácticos.
- ▶ Conocimiento derivado de la implementación de estudios organizacionales de otra naturaleza.
- ▶ Conocimiento personal basado en elementos diversos.
- ▶ Responsabilidad profesional.

El equipo auditor debe realizar su trabajo utilizando toda su capacidad, inteligencia y criterio para determinar el **alcance, estrategia y técnicas** que habrá de aplicar en una auditoría, así como evaluar los resultados y presentar los informes correspondientes.

Para este efecto, se debe poner especial cuidado en:

- ▶ Preservar la independencia mental.
- ▶ Realizar su trabajo sobre la base de conocimiento y capacidad profesional adquiridas.

Normas profesionales del equipo auditor

Finalmente, el equipo auditor no debe olvidar que la fortaleza de su función está sujeta a la medida en que afronte su compromiso con respeto y se base en normas profesionales tales como:

- ▶ **Objetividad:** mantener una visión independiente de los hechos, evitando formular juicios o caer en omisiones que alteren de alguna manera los resultados que obtenga.
- ▶ **Responsabilidad:** observar una conducta profesional, cumpliendo con sus encargos de manera oportuna y eficiente.
- ▶ **Integridad:** preservar sus valores por encima de las presiones.
- ▶ **Confidencialidad:** conservar en secreto la información y no utilizarla en beneficio propio o de intereses ajenos.
- ▶ **Compromiso:** tener presente sus obligaciones para consigo mismo y la organización para la que presta sus servicios.
- ▶ **Equilibrio:** no perder la dimensión de la realidad y el significado de los hechos.
- ▶ **Honestidad:** aceptar su condición y tratar de dar su mejor esfuerzo con sus propios recursos, evitando aceptar compromisos o tratos de cualquier tipo.
- ▶ **Institucionalidad:** no olvidar que su ética profesional lo obliga a respetar y obedecer a la organización a la que pertenece.

- ▶ **Criterio:** emplear su capacidad de discernimiento de forma equilibrada.
- ▶ **Iniciativa:** asumir una actitud y capacidad de respuesta ágil y efectiva.
- ▶ **Imparcialidad:** no involucrarse de forma personal en los hechos, conservando su objetividad al margen de preferencias personales.
- ▶ **Creatividad:** ser propositivo e innovador en el desarrollo de su trabajo.

Código de ética profesional

ISACA, organización de referencia mundial en el ámbito de auditoría de sistemas de información, establece un código de **ética profesional** para guiar la conducta de los auditores de sistemas de información. Por ello, cualquier profesional dedicado a esta práctica deberá:

- ▶ Apoyar el cumplimiento de los estándares y procedimientos apropiados para el eficaz gobierno y gestión de la tecnología y los sistemas de información empresariales, entre los que se incluyen: la auditoría, el control, la seguridad y la gestión de riesgos.
- ▶ Llevar a cabo sus labores con objetividad, debida diligencia y rigor profesional, de acuerdo con las normas profesionales.
- ▶ Servir a los intereses de las partes interesadas de manera leal, manteniendo altos estándares de conducta y carácter, y no involucrarse en actos que desacrediten a la profesión o a la asociación.
- ▶ Procurar la privacidad y confidencialidad de la información obtenida en el transcurso de sus funciones, a menos que la autoridad legal requiera su divulgación. Dicha información no será usada para beneficio personal ni será revelada a terceros inapropiados.

- ▶ Mantener la competencia en sus respectivos campos y aceptar encargarse solo de aquellas actividades que razonablemente pueden esperar completar con las habilidades, los conocimientos y la competencia necesarias.
- ▶ Informar a las partes correspondientes los resultados del trabajo realizado, revelando todos los hechos significativos que conozcan que, si no fueran revelados, podrían distorsionar el reporte de los resultados.
- ▶ Apoyar la educación profesional de los interesados para mejorar su comprensión del gobierno y la gestión de la tecnología y los sistemas de información empresariales, incluyendo: la auditoría, el control, la seguridad y la gestión de riesgos.

7.3. Clasificación de los controles

Los controles que deben establecerse en los sistemas de información de una organización son **acciones y mecanismos** definidos para prevenir o reducir el impacto de los eventos no deseados que ponen en riesgo a los activos de una organización.

Se pueden clasificar o categorizar acorde a la siguiente figura:

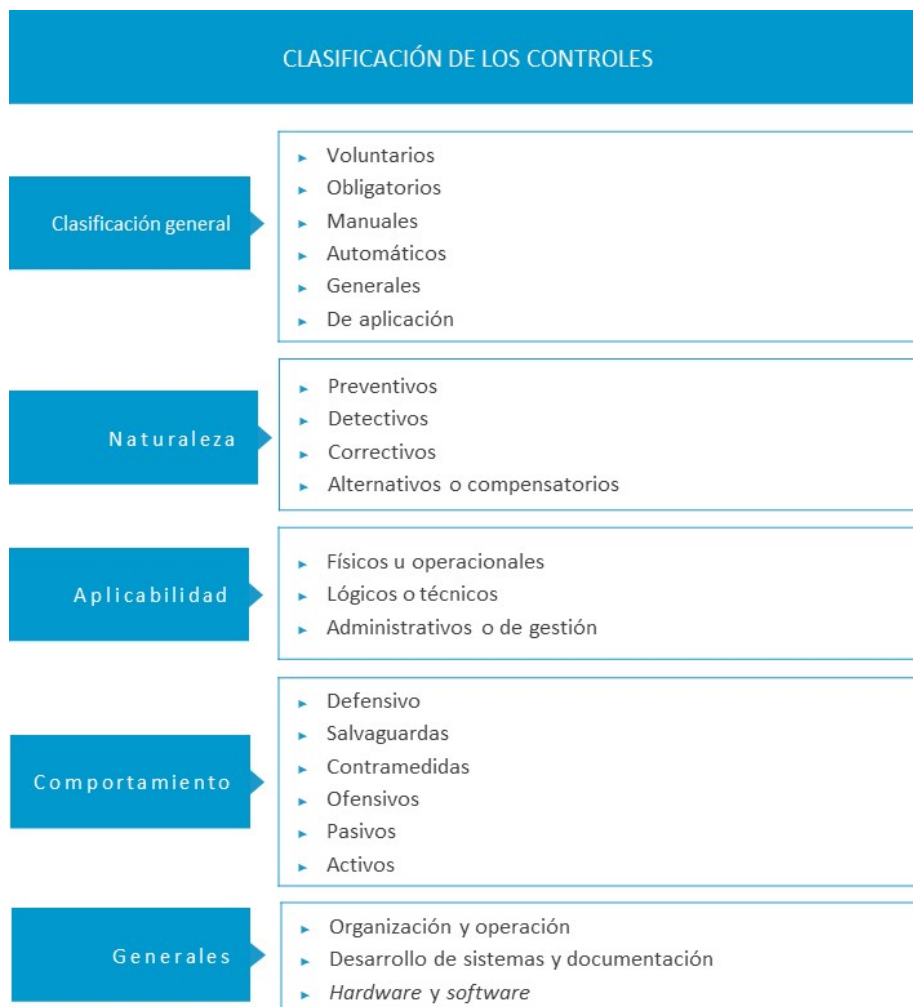


Figura 1. Clasificación de los controles. Elaboración propia.

Categorización general de los controles

En una **clasificación general** de los controles, podemos decir que estos pueden ser:

- ▶ **Voluntarios:** cuando la organización los diseña con el fin de mejorar los procesos.
- ▶ **Obligatorios:** si son impuestos por autoridades externas o reguladoras.
- ▶ **Manuales:** cuando son ejecutados por personas.
- ▶ **Automáticos:** si son llevados a cabo a través de sistemas de información automatizados.
- ▶ **Generales:** cuando van dirigidos al entorno donde operan otros controles.
- ▶ **De aplicación:** cuando operan integrados en el software.

Se debe tener en cuenta que hay tantos controles como riesgos queramos gestionar.

Categorización de los controles con base en su aplicabilidad

En términos generales, los controles de seguridad se pueden dividir en tres grandes grupos dependiendo de su ámbito de aplicabilidad:

- ▶ **Controles de seguridad físicos u operacionales:** son aquellos tipos de controles tangibles orientados a la protección del entorno y de los recursos físicos de la organización. Ejemplos:
 - Puertas.
 - Cerraduras.
 - Ventanas.

- ▶ **Controles de seguridad lógicos o técnicos:** son aquellos controles basados en una combinación de hardware y software. Ejemplos:
 - Criptografía.
 - *Antimalware*.
 - Cortafuegos (*firewalls*).
- ▶ **Controles de seguridad administrativos o de gestión:** son aquellos controles procedimentales, administrativos y/o documentales que establecen las reglas a seguir para la protección del entorno. Ejemplos:
 - Política de seguridad.
 - Gestión de privilegios.
 - Controles de contratación de personal.

Categorización de los controles con base en su naturaleza

Si atendemos a su **naturaleza**, podemos clasificar los controles en preventivos, detectivos y correctivos:

- ▶ **Preventivos.** Actúan sobre la causa de los riesgos con el fin de disminuir su probabilidad de ocurrencia. Es decir, pueden eliminar la vulnerabilidad de un activo (lo que lo hace inseguro) o la amenaza del activo (el elemento de riesgo). También actúan para reducir la acción de los generadores de riesgos. Es decir, pueden actuar de forma preventiva mitigando el impacto que, en caso de que se materializase el riesgo, podría haber sobre el activo.
- ▶ **Detectivos.** Los controles detectivos constituyen la segunda barrera de seguridad y pueden informar y registrar la ocurrencia de los hechos no deseados, accionar alarmas, bloquear la operación de un sistema, monitorizarlo de forma más exhaustiva, dejarlo en cuarentena o alertar a las autoridades.

- ▶ **Correctivos.** Estos controles actúan una vez detectado el evento y permiten el restablecimiento de la actividad normal después de ser detectado el evento no deseable y la modificación de las acciones que propiciaron su ocurrencia, es decir, la eliminación de la causa para evitar futuras ocurrencias de dicho evento.

Categorización de los controles con base en su comportamiento

Si atendemos a su comportamiento, podemos clasificar los controles en **defensivos, ofensivos y alternativos o compensatorios**.

Controles defensivos

Los controles defensivos se pueden organizar en dos subgrupos dependiendo del momento en el que pueden actuar: **salvaguardas y contramedidas**. La línea que hace esta separación es el momento de la ocurrencia de un incidente de seguridad.

Salvaguardas

En este subgrupo se encuentran los diferentes controles implementados para gestionar, prevenir y disuadir cualquier potencial amenaza antes de que se materialice. Posee los siguientes tipos de controles:

- ▶ **Directivos:** controles que permiten la especificación de un modelo de reglas que definirán el comportamiento esperado y aceptado en términos de seguridad en la organización, así como las acciones de defensa relacionadas. Ejemplos:
 - Cuerpo normativo de seguridad.
 - Acuerdos de confidencialidad.
 - Acuerdos de nivel de servicio.

- ▶ **Disuasorios:** controles cuyo objetivo está en inducir a un potencial atacante para que desista de su propósito. Su marco de acción suele ser, fundamentalmente, psicológico. Ejemplos:
 - Avisos de advertencia.
 - Cámaras de videovigilancia inactivas.
- ▶ **Preventivos:** este tipo de controles son la primera línea de acción frente a una posible amenaza. Por lo general, están enfocados en la aplicación de restricciones para evitar un riesgo de forma anticipada. Ejemplos:
 - Vallas de seguridad.
 - Cortafuegos (*firewalls*).
 - Formularios de autenticación.
- ▶ **Compensatorios:** controles alternativos que se usan cuando existe una restricción técnica o del negocio, justificada, que no permite la implementación de un control específico de acuerdo con lo que indican los lineamientos directivos. Ejemplos:
 - Control de software malicioso a través de listas blancas (*whitelists*).
 - Supervisión adicional a las tareas de los empleados.

Contramedidas

En este subgrupo se encuentran aquellos controles cuyo objetivo es identificar, detener, contener y corregir una amenaza cuando ya se ha materializado. Posee los siguientes controles:

- ▶ **De engaño:** controles señuelo que permiten desviar la atención de un potencial atacante hacia un objetivo configurado intencionalmente de forma insegura para que pueda ser atacado, pero que se encuentra aislado y monitorizado por la

organización. Esto permite contener y analizar las acciones del atacante mientras que se optimizan las propias defensas. Ejemplos:

- *Honeypots y honeynets.*
- Cajas fuertes vacías.
- ▶ **Detectivos:** controles que permiten proveer una notificación al personal encargado cuando el incidente ya ha ocurrido o se han detectado comportamientos anormales que pueden ser indicios de la ocurrencia de un incidente. Ejemplos:
 - Registros de eventos (*logs*).
 - Sensores de movimiento.
 - Sistemas de detección de intrusos (*Intrusion Detection Systems, IDS*).
- ▶ **Correctivos:** este tipo de controles están orientados a la contención del daño y la reconfiguración de los activos afectados. Ejemplos:
 - Extintores de fuego.
 - Terminación de conexiones.
 - Finalización del contrato.
- ▶ **De recuperación:** finalmente, este tipo de controles permiten que el activo pueda retornar a la operación normal, corrigiendo cualquier daño causado por el incidente. Ejemplos:
 - Copias de seguridad (*backups*).
 - Uso de centros de procesamiento de datos alternativos.

Es importante resaltar que estas categorías no son necesariamente **excluyentes**; es decir, un control puede operar bajo una o múltiples categorías. Por ejemplo, un **sistema de prevención de intrusiones** (*Intrusion Prevention System, IPS*) es un control defensivo de tipo **detectivo**, ya que genera una notificación del ataque, y **correctivo**, ya que permite la finalización de la conexión maliciosa y el bloqueo de conexiones subsiguientes.

Controles ofensivos

Este tipo de controles permiten la evaluación proactiva de la postura de seguridad corporativa y la mejora continua de los niveles de seguridad ofrecidos por los controles defensivos implementados en la organización. Su naturaleza es **activa** y se basa en la generación de incidentes reales o simulados, emulando, de la forma más verídica posible, las técnicas que podrían usar los atacantes. Su ejecución debe ser explícitamente coordinada por la organización y dentro de unos límites específicos.

Igualmente, no puede ser ejecutada contra organizaciones externas a menos que se tenga una aprobación. Estos controles se pueden subdividir en **pasivos y activos**, dependiendo de la activación o no de una acción que viole explícitamente los controles defensivos (incidente):

Pasivos

Controles que, de forma explícita, no violan ningún control defensivo, por lo que sus acciones no se pueden catalogar como incidentes.

- ▶ **Reconocimiento pasivo:** este tipo de controles se enmarcan en la realización de acciones aceptables dentro del marco normativo de la organización, pero cuyos resultados pueden ofrecer datos al equipo rojo para perfilar sus acciones. Ejemplos:
 - Analizadores del espectro electromagnético.
 - Capturadores de tráfico (*sniffing*).

- Ingeniería social.

Activos

A diferencia de los controles pasivos, estos controles infringen explícitamente una política de seguridad y pueden afectar la integridad, confidencialidad y/o disponibilidad de los activos, por lo que deben ser gestionados de forma muy controlada para minimizar posibles errores.

- ▶ **Reconocimiento activo:** estos controles permiten la detección de posibles vulnerabilidades en los sistemas defensivos mediante la ejecución de pruebas simuladas. Ejemplos:
 - Escáneres de vulnerabilidades.
 - Auditorías técnicas de sistemas.
 - Simulacros de planes de respuesta a incidentes.
- ▶ **Explotación:** estos controles están orientados a la utilización de una vulnerabilidad previamente identificada en un control defensivo para aprovecharse de ella y lograr un objetivo definido. Ejemplos:
 - Herramientas para pruebas de penetración.
- ▶ **Persistencia:** estos controles permiten la continuidad y subsistencia de los privilegios y accesos obtenidos posteriormente a la vulneración del activo. Ejemplos:
 - Controles antiforenses.

Algunos de los **atributos** de un control son:

OBJETIVO DEL CONTROL	Objetivo general del control: qué se pretende medir, conseguir, etc.
DESCRIPCIÓN DEL CONTROL	En qué consiste. Descripción funcional y técnica. Dependencias, etc.
FRECUENCIA DEL CONTROL	Frecuencia de ejecución del control o elementos que lo disparan (<i>trigger</i>).
EJECUCIÓN	Forma de ejecución del control (manual, automática o mixta) e información de ejecución, explotación y operación del control.
MONITORIZACIÓN	Información detallada de la forma de monitorización por parte del control, conteniendo al menos: • Elementos que se monitorizan. • Frecuencia de monitorización. • Plantilla de monitorización (KPI que se analizan). • Informe de monitorización.

Figura 2. Algunos de los atributos de un control. Fuente: elaboración propia.

En la práctica, la función de control interno debe describir con el suficiente nivel de detalle el control; ya que, en buena parte de los casos, es el departamento de explotación y producción quien implanta el control siguiendo las indicaciones descritas por **Control Interno Informático**.

Los **controles correctivos** suelen ser más costosos, porque actúan cuando ya se han presentado los hechos que implican pérdidas para la organización.



Figura 3. Tipos de controles. Fuente: elaboración propia.

Alternativos o compensatorios

Estos controles están basados en la implantación de controles no principales que mitigan el mismo riesgo del control principal. La implantación de este tipo de controles viene dada por la imposibilidad de **implantar**, por temas técnicos u organizativos, el control principal.

Controles generales

Los controles generales contribuyen a asegurar el correcto funcionamiento de los sistemas de información y crean un **entorno adecuado** para el correcto funcionamiento de los controles de aplicación.

La eficacia de los controles generales no es garante por sí sola de la eficacia de los controles de aplicación. Sin embargo, la ineficacia de los controles generales, muy probablemente, implicará la ineficacia de los controles de aplicación.

Controles de organización y operación

La **efectividad** de todos los controles depende de los controles de organización y operación, es decir, de cómo la organización ha definido su organización y operación.

La existencia de políticas y planes estratégicos y de TI, la gestión del presupuesto, la definición de las estructuras organizativas para la adecuada segregación de funciones o un buen marco procedimental son algunos de los controles de organización y operación de cuya eficacia depende buena parte de la efectividad del resto de los controles generales y de los controles de aplicación.

Controles de desarrollo, de sistemas y documentación

Son los controles de los que se dota una organización para regular el **qué, cómo y quién** de las actividades de desarrollo y sistemas del CPD. Estos controles identifican:

- ▶ El conjunto de estándares y buenas prácticas de referencia.
- ▶ Un marco de trabajo para llevar a cabo estas acciones.
- ▶ Procedimientos descriptivos de cada acción y de los responsables de llevarla a cabo.

El objetivo de los controles de desarrollo, de sistemas y documentación es permitir alcanzar la **eficacia** del sistema de información, la **eficiencia** en el uso de recursos, la **integridad** de los datos, la **protección** de los recursos y el **cumplimiento** con las leyes y regulaciones.

Controles de hardware y software de sistemas

A continuación, se describen algunos controles de hardware y software de sistemas entre los que destacan los controles de **seguridad lógica** y de **seguridad física**. Se caracterizan por ser controles que preservan:

- ▶ **Confidencialidad**, al asegurar que solo quien esté autorizado puede acceder a la información.
- ▶ **Integridad**, al asegurar que la información y sus métodos de proceso son exactos y completos.
- ▶ **Disponibilidad**, al asegurar que los usuarios autorizados tienen acceso a la información y a sus activos asociados cuando lo requieren.

Como ejemplo de modelos de control se tienen los incluidos en la norma ISO 27001 y el Modelo de control basado en ITGC.

Controles de aplicación

La seguridad de las aplicaciones, especialmente la seguridad de las aplicaciones web, se ha convertido en el punto de ataque técnico más débil. Los controles comienzan con prácticas seguras de codificación, y debe ser complementado con pruebas de penetración.

Podemos definir un **control de aplicación** como las actividades manuales y automatizadas que aseguran que la información y los sistemas de información, que la generan o procesan, cumplen con ciertos criterios o requerimientos del negocio (COBIT). Estos criterios son:

- ▶ **Efectividad** (requerimiento para los sistemas de información).
- ▶ **Eficiencia** (requerimiento para los sistemas de información).
- ▶ **Confidencialidad** (requerimiento para la información).

- ▶ **Integridad** (requerimiento para la información).
- ▶ **Disponibilidad** (requerimiento para la información).
- ▶ **Cumplimiento** (requerimiento para la información).
- ▶ **Confiabilidad** (requerimiento para la información).

Las aplicaciones deben incorporar controles que garanticen la entrada, actualización, validez y el mantenimiento completos y exactos de la información. Entre los controles de aplicación, podemos distinguir:

- ▶ Controles de **entrada de datos**: procedimientos de conversión y de entrada, validación y corrección de datos.
- ▶ Controles de **tratamiento de datos**: para asegurar que no se dan de alta, modifican o borran datos no autorizados para garantizar su integridad mediante procesos no autorizados.
- ▶ Controles de **salida de datos**: para la gestión de errores en las salidas, etc.

Controles por área

Son los controles que cubren las áreas funcionales ya definidas para una ubicación física, es decir, controles que pudieran ser específicos no de un sistema o aplicación, o generales a la organización, sino **particulares** de una de las áreas funcionales de una ubicación física.

Controles de productos informáticos

Son los controles que debe tener un producto informático comercial, de forma que cumpla con la definición del Control Interno Informático de la organización.

Controles por motivos legales

Requieren de ayuda por parte de un auditor legal. En el sector tecnológico deben existir controles para garantizar el cumplimiento de las Leyes de Propiedad Intelectual (LPI), de Protección de Datos (GDPR), de Servicios de Sociedad de Información y Comercio Electrónico (LSSICE), etc.

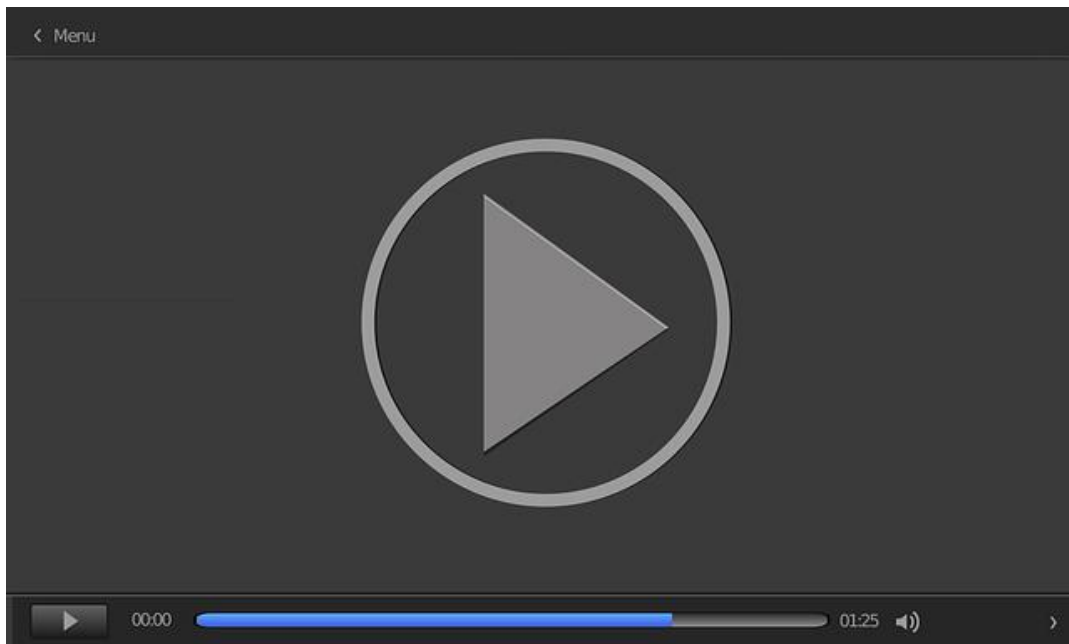
7.4. La regla de oro

La **regla de oro** que el Control Interno Informático de la organización ha de tener siempre en mente para diseñar e implantar los controles es el **principio básico de proporcionalidad**. Es decir, deben cuantificarse los siguientes aspectos:

- ▶ Coste de diseño, implantación, monitorización y mantenimiento del control.
- ▶ El coste puede ser:
 - **Coste del impacto** que tiene el riesgo sobre la organización en el caso de que la amenaza explotara la vulnerabilidad del activo, es decir, en el supuesto que se materializara el riesgo.
 - **Coste potencial** de la no implementación del control.

La organización debe aplicar la regla de oro analizando: riesgo vs. control vs. coste.

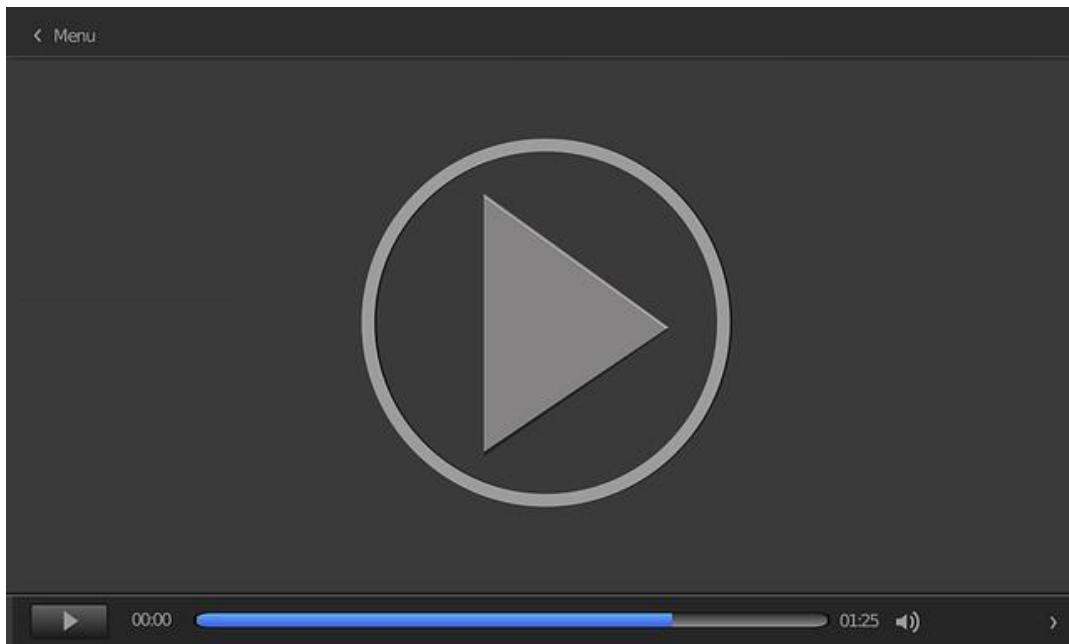
Los controles son acciones y mecanismos definidos para prevenir o reducir el impacto de los eventos no deseados que ponen en riesgo a los activos de una organización. En este vídeo (*Los controles internos de los sistemas como el motor de seguridad en las empresas*) se realiza una introducción a los controles internos de los sistemas como el motor de seguridad en las empresas.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=003d9562-0f36-46b7-883e-adf700cfedb3>

Los 20 controles críticos de seguridad han sido priorizados para ser implementados por las organizaciones que comprenden el riesgo que existe alrededor de los ciberataques. A la hora de adoptar estos controles críticos hay que hacerse la siguiente pregunta: «¿Qué necesito hacer ahora mismo para proteger mi empresa de ataques conocidos?». El adoptar e implementar los controles críticos permite a las organizaciones documentar fácilmente los procesos de seguridad para demostrar su conformidad y mitigar más del 90 % de las amenazas a las que se puede enfrentar una organización. En este vídeo (*Los 20 controles críticos de seguridad [CCS/CIS]*) se presentan los principales controles de esta iniciativa.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=4d86b125-36ef-4f19-b311-adf700cfed36>

7.5. Referencias bibliográficas

ISACA. (2019). *CISA. Review manual*. 27.º ed. ISACA. Organización Internacional de Normalización. (2013). *Norma ISO 27001* (ISO 27001:2013). <https://www.isotools.org/normas/riesgos-y-seguridad/iso-27001/>

La revisión de los controles generales en un entorno informatizado

Minguillón, A. (2010). La revisión de los controles generales en un entorno informatizado. Auditoría pública, 52, 125-136. <https://docplayer.es/2507068-La-revision-de-los-controles-generales-en-un-entorno-informatizado.html>

Este artículo detalla los controles generales y de aplicación, así como algunas normas técnicas de auditoría aplicables según el sector.

Auditoría del control interno

Mantilla, S. A. (2018). *Auditoría del control interno*. Ecoe Ediciones.
<https://www.ecoediciones.com/wp-content/uploads/2018/04/Auditori%CC%81a-del-Control-Interno-4ed.pdf>

Esta publicación analiza el control interno desde una perspectiva de procesos, principalmente. Analiza la cadena de valor que le es inherente y sus componentes centrales: diseño, implementación, evaluación, valoración, auditoría y supervisión. Ciertamente, el centro de atención está puesto en la auditoría.

CIS

CIS. (2020). *CIS controls*. <https://www.cisecurity.org/controls/>

Página de referencia en materia de controles e indicadores tecnológicos de la mano del Center for Internet Security (CIS).

Banco de España

Banco de España. (2020).
Guías. <https://www.bde.es/bde/es/secciones/normativas/Guias/Guias.html>

Guías muy interesantes sobre controles y riesgos tecnológicos, entre otras.

1. Es fundamental que un auditor de sistemas de información:
 - A. Tenga conocimiento de la legislación aplicable en la compañía.
 - B. Tenga conocimiento de los objetivos de la compañía.
 - C. Tenga conocimiento de los sistemas de información.
 - D. Todas las anteriores.

2. ¿Dónde se debe situar un departamento de auditoría interna dentro de una compañía?
 - A. Su situación no es relevante.
 - B. Siempre depende de Tecnología.
 - C. Depende de la empresa, pero lo más recomendable es que dependa de Seguridad.
 - D. En una ubicación jerárquica en la que pueda realizar sus actuaciones de manera independiente.

3. En relación con la clasificación de controles internos, según su naturaleza, podemos afirmar que:
 - A. Son normalmente detectivos, correctivos y preventivos. En ocasiones, se incluyen los controles alternativos.
 - B. Son voluntarios, obligatorios, manuales y automáticos.
 - C. Son voluntarios, obligatorios, manuales, automáticos, preventivos, detectivos y correctivos.
 - D. Son controles generales, de aplicación, de área y legales.

4. En relación con la clasificación de controles internos, según su naturaleza, podemos afirmar que:

- A. Los controles detectivos actúan sobre la causa de los riesgos con el fin de disminuir su probabilidad de ocurrencia.
- B. Los controles preventivos se diseñan para descubrir un evento, irregularidad o resultado no previsto, alertan sobre la presencia de riesgos y permiten tomar medidas inmediatas, pudiendo ser manuales o automáticos.
- C. Las opciones A y B son correctas.
- D. Las opciones A y B son incorrectas.

5. Los controles generales se clasifican en:

- A. Controles de organización y operación, controles de tratamiento de datos, así como controles de hardware y *software* de sistemas.
- B. Controles de entrada de datos, controles de tratamiento de datos y controles de salida de datos.
- C. Controles de organización y operación, controles de desarrollo de sistemas y documentación, así como controles de *hardware* y *software* de sistemas.
- D. Ninguna de las anteriores es cierta.

6. ¿Se puede considerar un control como señuelo?

- A. No, un control siempre debe tener un objetivo claro.
- B. No, los controles tienen que ser reales.
- C. Sí, se utilizan para desviar la atención de un potencial atacante.
- D. Ninguna de las respuestas anteriores es correcta.

7. Dentro de controles generales, podemos afirmar que no son controles de organización y operación:

- A. Seguridad lógica y física, controles de procesamiento y control de presupuestos.
- B. Procedimientos, separación de entornos y estándares y nomenclatura.
- C. Controles de segregación de funciones.
- D. Todos los anteriores son controles de organización y operación.

8. Los controles deben:

- A. Estar implantados.
- B. Cubrir los objetivos por los que fueron implantados.
- C. Ser efectivos.
- D. Todas las anteriores.

9. En relación con los controles de organización y operación, para una segregación de funciones adecuada:

- A. Una misma persona no puede realizar funciones determinadas en un mismo entorno.
- B. Una misma persona no puede realizar funciones determinadas en un entorno y en otro.
- C. Las opciones A y B son correctas.
- D. Las opciones A y B son incorrectas.

10. Los controles de *hardware* y *software* de sistemas:

- A. Preservan las tres dimensiones de la información: confidencialidad, disponibilidad y seguridad.
- B. Preservan las tres dimensiones de la información: fiabilidad, disponibilidad y confidencialidad.
- C. Preservan dos de las tres dimensiones de la información: integridad, disponibilidad y confidencialidad.
- D. Ninguna de las anteriores es correcta.

Los controles de *hardware* y *software* de sistemas, entre los que destacan los controles de seguridad lógica y de seguridad física, se caracterizan por ser controles que preservan tres dimensiones de seguridad (confidencialidad, integridad y disponibilidad).

Desarrollo Seguro de Software y Auditoría de la
Ciberseguridad

Tema 8. El proceso de auditoría de sistemas de la información

Índice

Esquema

Ideas clave

- 8.1. Introducción y objetivos
- 8.2. Estándares de la auditoría
- 8.3. Metodología de la auditoría
- 8.4. Planificación de la auditoría
- 8.5. Herramientas y técnicas de auditoría
- 8.6. Objetivos de la auditoría
- 8.7. Evidencia
- 8.8. Comunicación de los resultados de la auditoría
- 8.9. Referencias bibliográficas

A fondo

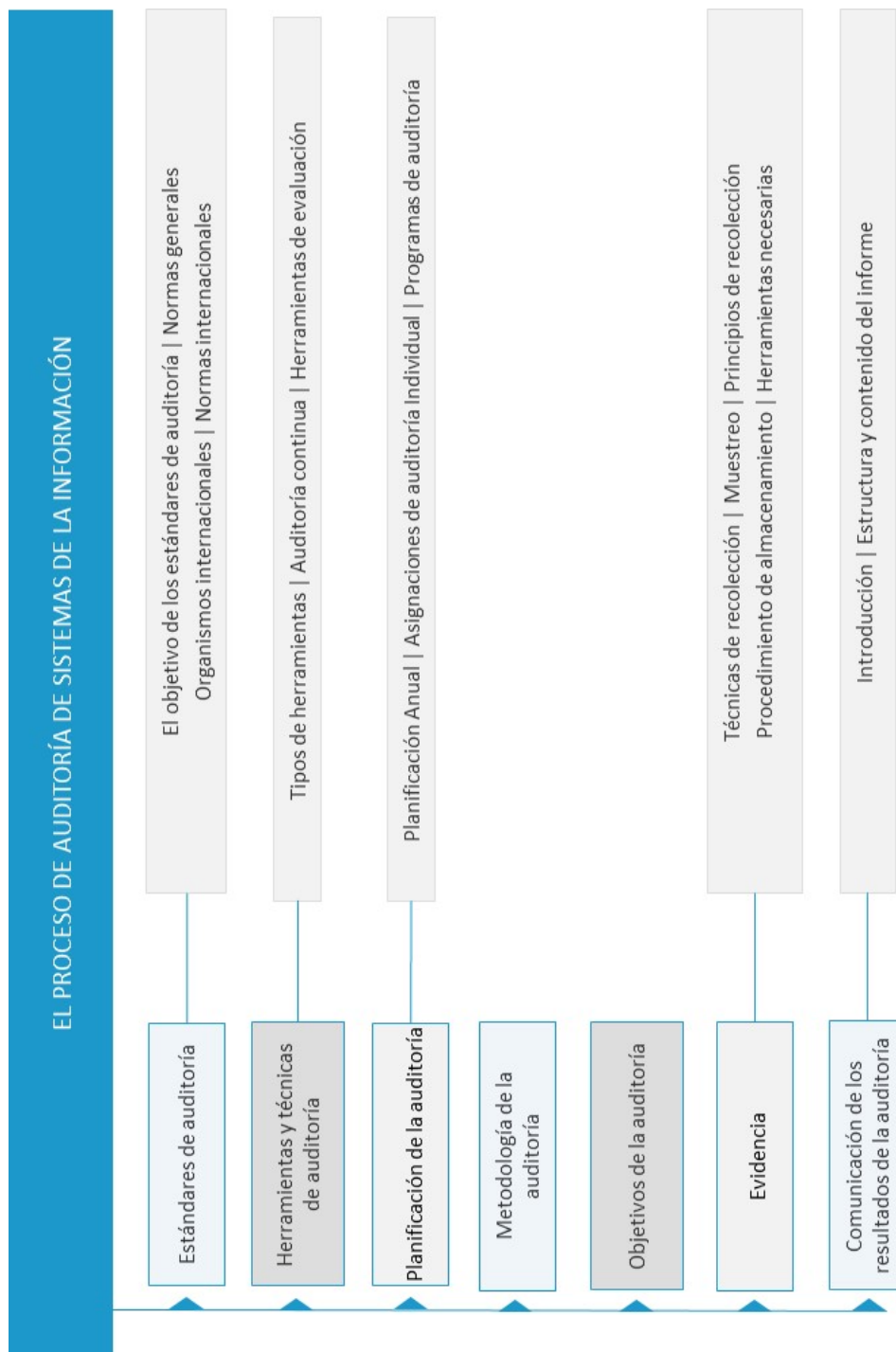
Ciberseguridad

Auditoría de sistemas. Políticas de seguridad para la pyme

Cobertura del riesgo tecnológico: hacia una auditoría interna de TI integrada

Guía para la construcción de un SGCI. Sistema de gestión de la ciberseguridad industrial

Test



8.1. Introducción y objetivos

Se requieren varios pasos para realizar una auditoría, lo que conlleva a que el **auditor de sistemas** deba evaluar los riesgos a los que está sometida la compañía (tecnológicos, seguridad, cumplimiento, etc.) con el fin de realizar una planificación o programa de auditoría. Durante el proceso de auditoría, el auditor debe recabar **evidencias** para evaluar las fortalezas y debilidades de los controles existentes; de esta manera, podrá formarse una imagen **objetiva** para la preparación del informe.

Una correcta **planificación** de la auditoría es lo primero que debe realizar un auditor de sistemas. Para ello, y de manera fundamental, el auditor debe **comprender** la organización, el negocio, la regulación y los estándares aplicables, su problemática, y los riesgos, así como cualquier factor relevante sobre la organización y su contexto, tanto interno como externo. Por este motivo, a continuación, se expresan algunas áreas que deben quedar cubiertas durante la planificación.

- **Comprensión del negocio.** Comprensión del contexto, tanto interno como externo, del universo auditable. Debe incluir la comprensión general de la organización (prácticas comerciales y funciones relacionadas con el tema de la auditoría), así como los tipos de sistemas que se utilizan. De igual manera, el auditor debe comprender la normativa y regulación a la que está expuesto el negocio.

- ▶ **Riesgo y materialidad de auditoría.** Los riesgos de auditoría tienen que ver con una información que pueda tener errores o un auditor de sistemas que no pueda detectar un error que ha ocurrido. Los riesgos en auditoría pueden clasificarse de la siguiente manera.
 - **Riesgo inherente:** cuando un error material no se puede evitar porque no existen controles compensatorios relacionados que se puedan establecer.
 - **Riesgo de control:** cuando un error material no puede ser evitado o detectado de forma oportuna por el sistema de control interno.
 - **Riesgo de detección:** es el riesgo de que el auditor realice pruebas exitosas a partir de un procedimiento inadecuado. El auditor puede llegar a la conclusión de que no existen errores materiales cuando en realidad sí los hay.
- ▶ **Técnicas de evaluación de riesgos.** Al determinar de manera clara el alcance de la auditoría, el auditor debe evaluar los riesgos y determinar cuáles están dentro del alcance y deben ser auditados. Para ello, existen cuatro motivos por los que se utiliza esta evaluación de riesgos:
 - Permitir que la gerencia asigne recursos necesarios para la auditoría.
 - Garantizar que se ha obtenido la información pertinente de todos los niveles.
 - Constituir la base para la organización de la auditoría a fin de administrar eficazmente el departamento.
 - Proveer un resumen que describa cómo el tema individual de auditoría se relaciona con la organización global de la empresa, así como los planes del negocio.

Los **objetivos** del presente tema son los siguientes:

- ▶ Conocer y comprender los principales aspectos que abarcan los estándares internacionales de auditoría.

- ▶ Conocer los aspectos fundamentales de las herramientas y técnicas de auditoría.
- ▶ Aprender y estudiar los principales conceptos relacionados con la planificación de la auditoría.
- ▶ Entender la necesidad del uso de metodologías de auditoría.
- ▶ Definir y explicar los conceptos relativos a los objetivos de la auditoría y las evidencias.
- ▶ Aprender los principales conceptos relacionados con la comunicación de los resultados de la auditoría.

8.2. Estándares de la auditoría

El carácter especializado de la auditoría de sistemas de información, y de las habilidades y conocimientos necesarios para llevar a cabo dichas auditorías, requieren estándares aplicables de forma específica a la auditoría de los sistemas de información. Los estándares definen los **requerimientos obligatorios** para la auditoría de sistemas y la generación del informe.

Una auditoría se realiza con base en un patrón, conjunto de directrices o buenas prácticas sugeridas. Existen estándares orientados a servir como base para auditorías de informática, uno de ellos es **COBIT (Control Objectives for Information and Related Technology)**. Dentro de los objetivos definidos como parámetro, se encuentra el de **garantizar la seguridad de los sistemas**.

Además de este estándar, podemos encontrar el estándar **ISO 27002**, el cual se conforma como un código internacional de buenas prácticas de seguridad de la información y puede constituirse como una **directriz de auditoría** apoyándose de otros estándares de seguridad de la información que definen los requisitos de auditoría y sistemas de gestión de seguridad, como es el estándar **ISO 27001**. Se debe recordar que existen infinidad de estándares y es recomendable contar con un repositorio para poder ver que otros estándares puedan resultar aplicables.

El objetivo de los estándares de auditoría

Informar a los auditores de sistemas del **mínimo nivel aceptado** para resolver las responsabilidades profesionales precisadas en su práctica, de la misma manera, ayuda a proveer información sobre cómo cumplir con los requerimientos que se necesitan para este tipo de prácticas.

Normas generales de la auditoría de sistemas

Desde ISACA (2019) se determina que la naturaleza especializada de la auditoría de los sistemas de información y las habilidades necesarias para llevarlas a cabo requieren el desarrollo y la promulgación de **normas generales** para la auditoría de los sistemas de información.

Organismos internacionales de la auditoría de sistemas

Los organismos internacionales que se ocupan del control y de la auditoría de sistemas de información son las principales fuentes de estándares. A continuación, se muestran dos organismos; pero, al igual que con los estándares, se recomienda que se realice una búsqueda para poder obtener más información sobre otros organismos de auditoría de sistemas.

- ▶ **Information System Audit and Control Association (ISACA):** es la Asociación de Auditoría y Control de Sistemas de Información; comenzó en 1967 y, en 1969, el grupo se formalizó incorporándose bajo el nombre de EDP Auditors Association (Asociación de Auditores de Procesamiento Electrónico de Datos).
- ▶ **Institute of Internal Auditors (IIA):** organización profesional con sede en los Estados Unidos, fundada en 1941, con más de 200 000 miembros en más de 170 países. El IIA es reconocido mundialmente como una autoridad, pues es el principal educador y líder en la certificación, la investigación y la guía tecnológica en la profesión de la auditoría interna.

Normas internacionales de auditoría

A continuación, se enuncian algunas de las normas que el auditor de sistemas de información debe conocer. Ajustarse a estas normas no es obligatorio, pero el auditor de sistemas de información debe estar preparado para **justificar** cualquier incumplimiento a estas. Entre ellas, tenemos las siguientes:

- ▶ Normas 15 y 16 emitidas por **IFAC** (International Federation of Accountants) en la **NIA** (Norma Internacional de Auditoría; en inglés: International Standards on Auditing, ISA), donde se establece la necesidad de utilizar otras técnicas además de las manuales.
- ▶ Norma **ISA 401** sobre sistemas de información por computadora, SAS No. 94 (*the effect of information technology on the auditor's consideration of internal control in a financial statement audit*) dice que una organización que usa tecnologías de información se puede ver afectada en uno de los siguientes cinco componentes del control interno: el ambiente de control, la evaluación de riesgos, las actividades de control, la información, la comunicación y el monitorio, además de la forma en que se inician, registran, procesan y reportan las transacciones.
- ▶ La norma **SAP 1009** (Statement of Auditing Practice), denominada Computer Assisted Audit Techniques (CAAT), o Técnicas de Auditoría Asistidas por Computador, plantea la importancia del uso de CAAT en auditorías en un entorno de sistemas de información por computadora.

8.3. Metodología de la auditoría

Una metodología de auditoría es un conjunto de procedimientos documentados de auditoría, diseñados para alcanzar los objetivos de auditoría planificados. Sus componentes son una declaración del **alcance**, de los **objetivos** y de los **programas** de la auditoría.

La metodología de auditoría debería ser establecida y aprobada por su gerencia para lograr consistencia en el enfoque. Esta metodología tendría que ser formalizada y comunicada a todo el personal de auditoría.

Un producto inicial y crítico del proceso de auditoría debería ser un programa de auditoría que sea una guía para la ejecución y documentación de todos los pasos siguientes de la auditoría y la extensión y el tipo de evidencia revisada.

Como observación, hay que comentar que dos auditores con la misma información y bajo la misma metodología deberán obtener los **mismos resultados**. Si no es así, puede ser porque la información tratada es diferente o porque la metodología no es lo suficientemente concreta y puede malinterpretarse.

Metodología para realizar auditorías informáticas

Existen diversas metodologías de auditorías informáticas y todas dependen de lo que se pretenda revisar o analizar; pero, como estándar, analizaremos las cuatro fases básicas de un proceso de revisión:

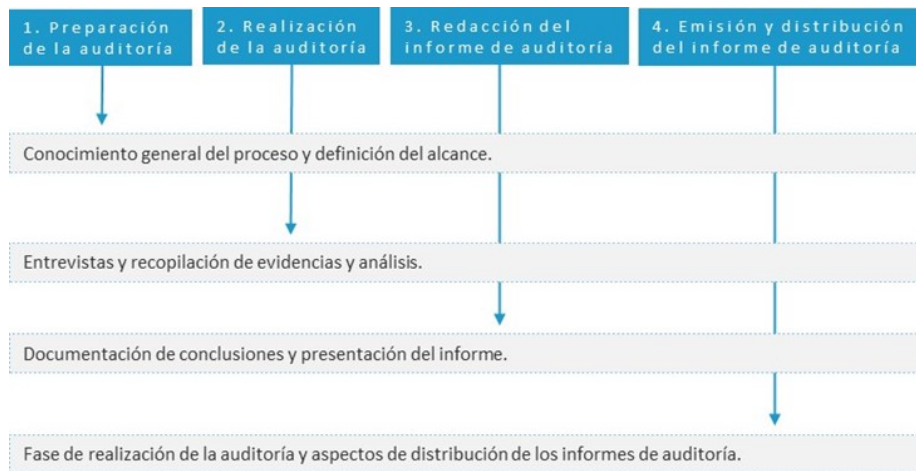


Figura 3. Metodología general para realizar auditorías. Fuente: elaboración propia.

Preparación de la auditoría

Las actividades que el auditor debe realizar con el objetivo de completar adecuadamente la etapa de preparación de la auditoría y asegurar el correcto desarrollo del resto de etapas son, al menos, las siguientes:

- ▶ **Definición del alcance:** es la definición de lo que se va a revisar durante la realización de la auditoría. El auditor debe identificar el alcance tomando en consideración:
 - Las unidades de la organización a revisar. Por ejemplo: área de desarrollo, seguridad lógica, seguridad física, etc.
 - Los sistemas informáticos específicos. Por ejemplo: CRM, ERP, etc.
 - Las ubicaciones físicas consideradas en el alcance de la auditoría. Las diferentes sedes organizativas o delegaciones.
 - Las terceras partes incluidas en el alcance de la auditoría: proveedores, *partners*, etc.

El auditor tiene que evaluar el dimensionamiento del área o sistema a auditar en términos de complejidad, tamaño, a efectos de poder realizar una estimación acertada de los plazos y recursos necesarios para la realización de la auditoría.

- ▶ **Recursos y tiempo:** el auditor tiene que estimar los recursos (auditores, jornadas/hombre, herramientas, etc.) necesarios para la realización de la auditoría, así como el tiempo del que requerirá en función del alcance definido.
- ▶ **Recopilación de información básica:** esta actividad consiste en identificar las fuentes de información más relevantes con el objeto de revisarla antes de la realización de la auditoría. Una vez analizada la información básica, el auditor debe identificar una lista de personas a entrevistar, y especificar, en la medida de lo posible:
 - Nombre y cargo de la persona a entrevistar.
 - Información relacionada con el objeto de la entrevista, como: lista de sistemas a revisar, lista de controles y otra información significativa.
 - Lugar, fecha y hora previstas para la entrevista.
 - Requerimientos, como, por ejemplo: documentación requerida, etc.

Es importante destacar que, si la auditoría es **sorpresiva**, la recopilación de la información básica se realiza por el auditor *in situ*. Si la auditoría no es sorpresiva, previamente se envía una carta de apertura o memorándum donde se indica lo que se necesita.

- ▶ **Programa de trabajo:** el auditor debe definir un plan de trabajo con la asignación de cada auditor al área, sistema o control a revisar, teniendo en cuenta:
 - El alcance definido.
 - Los recursos disponibles.

- El tiempo estimado.
- La información básica analizada.
- ▶ **Plan de comunicación:** uno de los aspectos clave que debe quedar acordado con el auditado, y que conforma un factor crítico de éxito en la realización de la auditoría, es el plan de comunicación. El auditor debe explicar y acordar con el área auditada los aspectos de comunicación.

Realización de la auditoría

Una buena fase de preparación de la auditoría es clave para su adecuada realización, ya que permitirá la apropiada planificación de las tareas y la asignación eficiente de los recursos de auditoría, de manera que se consiga aportar el máximo valor para el negocio.

En esta fase, el equipo auditor realiza la aplicación de la metodología de auditoría informática de acuerdo con el plan de trabajo definido y el programa de entrevistas establecido. En el caso de una auditoría informática basada en la evaluación del riesgo, el auditor aplica EDR y sigue el esquema que recomienda ISACA. Normalmente, el auditor dispone de un EDR genérico que adaptará en función de la organización auditada.

A continuación, se describen las principales etapas que el auditor de sistemas de información tiene que completar para llevar a cabo la auditoría:

- ▶ **Identificación de riesgos potenciales:** consiste en detectar los posibles riesgos en ausencia de controles relacionados. Es decir, el auditor cuantifica el riesgo potencial o inherente a los sistemas de información en ausencia de control.

- ▶ **Identificación de controles fuertes y débiles:** el auditor detecta controles adecuadamente implantados y eficaces (fuertes) y otros parcialmente implantados o que no funcionan (débiles). Por cada control débil, el auditor hace un comentario que puede ser leve, medio o grave según la gravedad y el posible impacto en el negocio.
- ▶ **Selección de las pruebas y técnicas a utilizar:** el auditor decide qué tipo de pruebas, en base a los riesgos, va a diseñar, con la finalidad de objetivar el análisis y obtener evidencias suficientes y competentes que sirvan de base a sus conclusiones.
- **Pruebas sustantivas:** verifican el grado de confiabilidad del SI del organismo. Se suelen obtener mediante observaciones, cálculos, muestreos, entrevistas, técnicas de examen analítico, revisiones y conciliaciones. Verifican, también, la exactitud, integridad y validez de la información.
- **Pruebas de cumplimiento:** verifican el grado de cumplimiento de lo revelado mediante el análisis de la muestra. Proporciona evidencias de que los controles claves existen y que son aplicables efectiva y uniformemente. Una prueba de cumplimiento reúne evidencias de auditoría para indicar: si un control existe, si funciona de forma efectiva, si logra sus objetivos de forma eficiente.
- **Técnicas de *audit trail*, *retrieval*, *tools*.**
- ▶ **Realización de pruebas y obtención de resultados:** el auditor realiza tantas pruebas (de cumplimiento, sustantivas, etc.) como haya estimado. En ocasiones, se utilizan herramientas de apoyo al auditor; por ejemplo, CAAT (Computer Assisted Audit Techniques). El auditor lleva un registro de las pruebas realizadas, así como de los hallazgos obtenidos, la fecha del hallazgo, la evidencia derivada, los comentarios y las observaciones del auditado.

Este registro es muy importante, ya que permite demostrar una conclusión sobre la prueba realizada y forma parte del informe de auditoría y sus anexos. A lo largo de la auditoría, el auditor puede decidir realizar pruebas adicionales sobre aspectos que no hubieran quedado claros o de los que no se disponga de evidencia suficiente y competente. El auditor debe manejar, en todo momento, el plan de trabajo, la asignación de recursos y las posibles desviaciones en tiempo y hacer partícipe al auditado de la gestión de dichas desviaciones.

- ▶ **Conclusiones y comentarios:** el auditor debe analizar y revisar todos los comentarios generados a lo largo de la realización de la auditoría y los resultados de cada prueba deben valorarse, obtener una conclusión, siempre teniendo en cuenta los objetivos y el alcance de la auditoría. También deberá resumir los comentarios acerca de lo analizado (riesgos, incidencias, etc.) y adjuntarlos al informe final. El detalle de todos los comentarios forma parte del informe final, pero en un anexo.
- ▶ **Revisiones y cierre de papeles de trabajo:** el auditor ha de cerrar la auditoría cuando disponga de evidencias pertinentes, suficientes y competentes que sustenten sus conclusiones. Para poder demostrar cualquiera de las conclusiones, deberá guardar correctamente todas las notas recogidas (cuaderno del auditor) y mantener la trazabilidad entre el hallazgo, los comentarios asociados, la evidencia relacionada y la conclusión que se deriva.

Redacción del informe de auditoría

Habitualmente, se elabora un informe preliminar que incluya:

- ▶ Antecedentes.
- ▶ Alcance y objetivo de la auditoría.
- ▶ Metodología utilizada.

- ▶ Posibles limitaciones.
- ▶ Resumen de auditoría.
- ▶ Conclusiones en base a los resultados obtenidos.
- ▶ Discusión con los auditados.

CUERPO DEL INFORME	ANEXO Y COMENTARIOS
▶ Antecedentes.	▶ Hallazgo: número del hallazgo, título del hallazgo, fecha del hallazgo.
▶ Alcance.	▶ Comentarios del auditor: número del comentario, título del comentario, fecha de entrega del comentario.
▶ Resumen.	▶ Evidencia
▶ Conclusiones.	▶ Gravedad.
▶ Anexo y comentarios.	▶ Observaciones del auditorio.
	▶ Fecha de las observaciones del auditorio.

Figura 4. Formato del informe de auditoría y anexos. Fuente: elaboración propia.

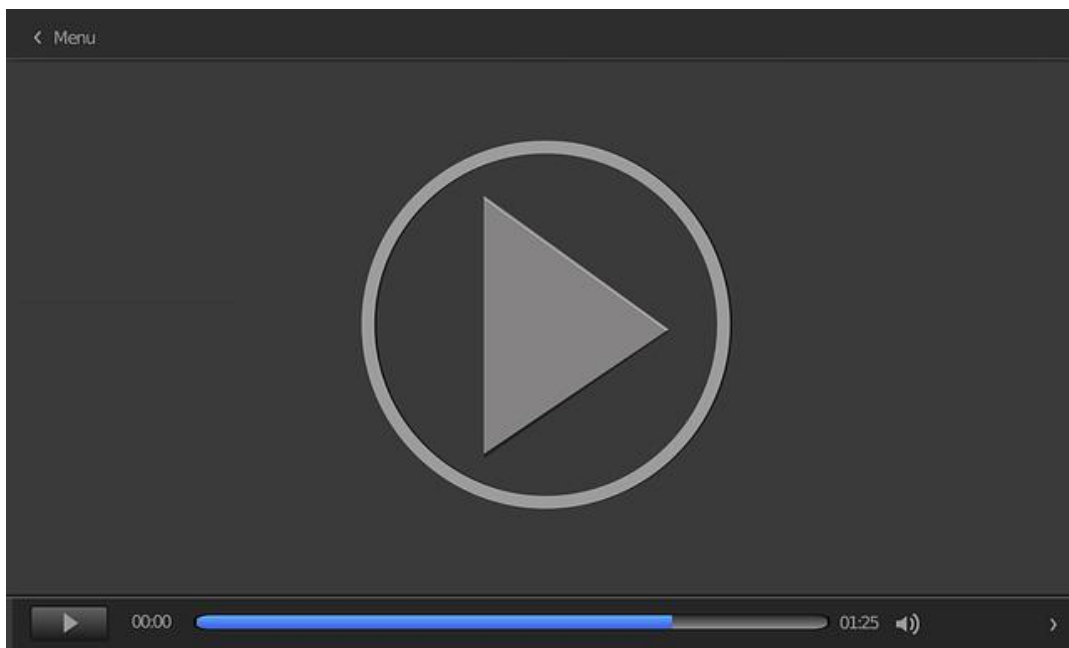
Emisión y distribución del informe de auditoría

La distribución del informe de auditoría debe realizarse de acuerdo con el plan de comunicación acordado entre el auditor y el auditado en la fase de preparación de la auditoría en la que se acuerdan los aspectos de comunicación relacionados con:

- ▶ La fase de realización de la auditoría:
 - Cómo se presentan los comentarios durante la auditoría a cada uno de los roles auditados.
 - Qué tipo de *feedback* se da a la dirección de la organización en caso de comentarios graves, por ejemplo, y con qué frecuencia.
- ▶ Los aspectos de distribución de los informes de auditoría:
 - Audiencia.
 - Formato.
 - Forma de envío.

Los aspectos de comunicación son clave y deben ser escrupulosamente cuidados por el auditor y acordados con la dirección de la organización auditada.

En una auditoría informática basada en una metodología de Evaluación de Riesgos (EDR), el auditor conduce todo el proceso de auditoría a partir de la evaluación inicial del riesgo potencial existente sobre el Sistema de Información de la organización. En este vídeo (*Metodología de auditoría basada en riesgos*) se presentan los principales aspectos para tener en cuenta a la hora de realizar una auditoría basada en riesgos.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=70aefa77-abd6-4dea-b7b3-adf700cfec21>

8.4. Planificación de la auditoría

Planificación anual

La planificación de la auditoría está constituida por la planificación tanto a corto como a largo plazo. La planificación a corto plazo toma en cuenta los aspectos relevantes de auditoría que serán cubiertos durante el año, mientras que la planificación a largo plazo se refiere a los planes de auditoría que tomarán en cuenta aspectos relacionados con **riesgos** debidos a los cambios en la dirección estratégica de TI de la organización que afectarán al ambiente de TI de la organización.

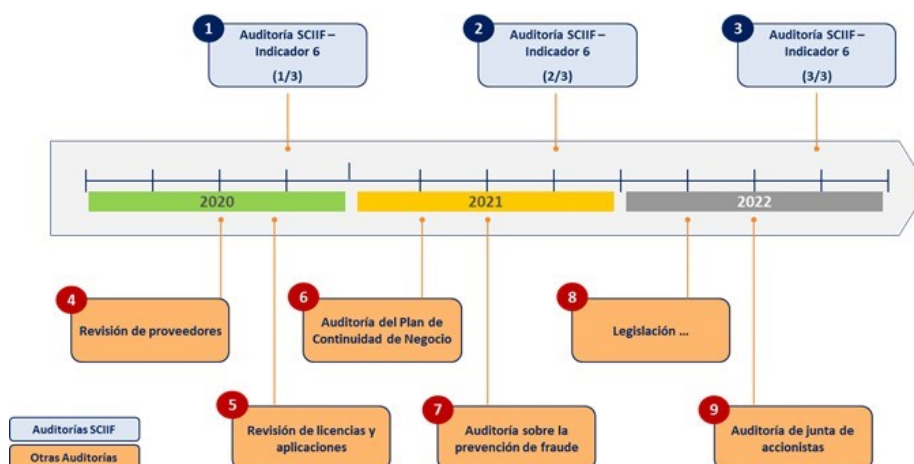


Figura 5. Ejemplo de plan de auditoría trienal. Fuente: elaboración propia.

Todos los procesos relevantes que representan el plan de negocio de la entidad se deben incluir en el universo de la auditoría. La **finalidad de la auditoría** es poder dar soporte al negocio y ayudar a que consiga sus objetivos. El universo de la auditoría, idealmente, enumera **todos los procesos** que se pueden tener en cuenta para las auditorías. Cada uno de estos procesos puede estar sujeto a una evaluación de riesgos cuantitativa o cualitativa mediante la evaluación del riesgo con respecto a factores de riesgo relevantes y definidos.

La evaluación de los factores de riesgo debe basarse en criterios objetivos, aunque la subjetividad no puede eliminarse completamente. Por ejemplo, con respecto al factor de la reputación, los criterios según los que se puede solicitar datos del negocio es posible clasificarlos como:

- ▶ **Alto.** Un problema de proceso puede tener como consecuencia daños a la reputación de la entidad que tarden más de seis meses en remediarse.
- ▶ **Medio.** Un problema de proceso puede tener como consecuencia daños a la reputación de la entidad que tarden entre tres y seis meses en remediarse.
- ▶ **Bajo.** Un problema de proceso puede tener como consecuencia daños a la reputación de la entidad que tarden menos de tres meses en remediarse.

Asignaciones de auditoría individual

Además de la planificación general anual, cada asignación individual de auditoría se debe planificar adecuadamente. El auditor de sistemas de información tendrá que entender que otras **consideraciones**, tales como los resultados de las evaluaciones periódicas de riesgos, los cambios en la aplicación de la tecnología, la evolución de aspectos de privacidad y los requerimientos regulatorios, son capaces de afectar al enfoque general de la auditoría.

El auditor de sistemas de información debería, también, considerar las **fechas límites** establecidas para la implementación o actualización de los sistemas, las tecnologías actuales y futuras, los requerimientos de los dueños del proceso de negocio y las limitaciones de recursos de los sistemas de información.

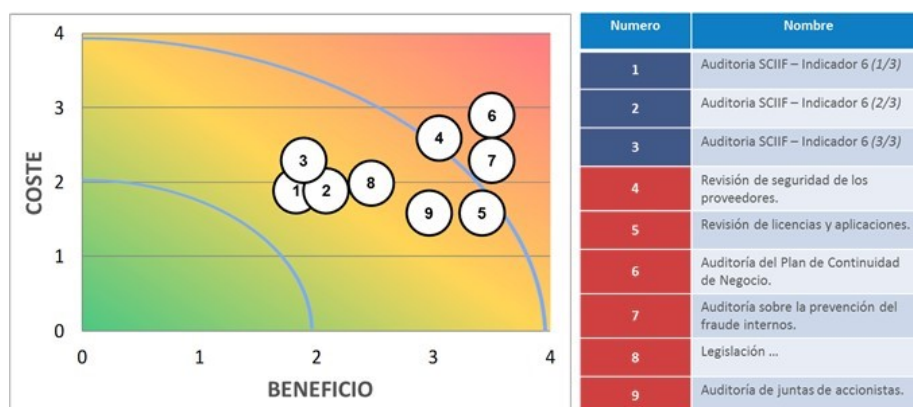


Figura 6. Evaluación coste y beneficio de la auditoría. Fuente: elaboración propia.

Al planificar una auditoría, el auditor de sistemas de información debe poseer una comprensión general del **contexto** bajo revisión. Esto debería incluir una comprensión general de las diversas prácticas y funciones de negocio relativas al sujeto de la auditoría, así como los tipos de sistemas de información y la tecnología que soporta la actividad. Por ejemplo, el auditor de sistemas de información tendría que estar **familiarizado** con el negocio y con el marco regulatorio aplicable.

2020												
Ene	Feb	Mar	Abr	May	Jun	Jul	Ago	Sep	Oct	Nov	Dic	
				5				4				
No.	Nombre	Descripción	Acciones					Aplicaciones	Estimación			
4	Revisión de seguridad de los proveedores	Realización de una revisión de los aspectos de Seguridad de la Información sobre los proveedores de la compañía en España, tanto a nivel de cumplimiento tecnológico, como del cumplimiento contractual y de la legislación en nuevas tecnologías aplicable, desde la vertiente contractual y tecnológica.	1. Planificación e inicio 2. Identificación del universo auditable 3. Definición del marco de controles 4. Revisión de contratos 5. Personalización del marco de controles para cada proveedor 6. Revisión de controles 7. Resultados, entregables y definición de un plan de acción.					N/A	Depende del alcance			
5	Revisión de licencias y aplicaciones	Realización de una revisión de la usabilidad de las licencias de las aplicaciones mas relevantes de la compañía y análisis del coste/rentabilidad de estas.	1. Planificación e inicio. 2. Identificación del alcance. 3. Revisión del perfilado 4. Revisión de los contratos. 5. Revisión de utilización de las aplicaciones por parte de usuario. 6. Resultados, entregables y definición de un plan de acción.					N/A	Depende del alcance			

Figura 7. Ejemplo de planificación. Fuente: elaboración propia.

Los pasos que un auditor de sistemas de información podría dar para lograr la comprensión del negocio son, entre otros:

- ▶ Lectura de antecedentes, incluyendo publicaciones de la industria, reportes anuales y reportes de análisis financieros independientes.
- ▶ Revisión de reportes anteriores o informes relacionados con TI (provenientes de auditorías externas o internas o revisiones específicas tales como revisiones regulatorias).
- ▶ Revisión del negocio y de los planes estratégicos de TI a largo plazo.
- ▶ Revisión de los análisis de riesgos realizados en la compañía.
- ▶ Entrevistas a los gerentes clave para entender pormenores del negocio.
- ▶ Identificar las regulaciones específicas aplicables a TI.

Otro componente básico de la planificación es lograr **correspondencia** entre los recursos de auditoría disponibles y las tareas definidas en el plan de auditoría. El auditor de sistemas de información que prepare el plan debe considerar los requerimientos del proyecto de auditoría, recursos de personal y otras limitaciones.

Programas de auditoría

Un programa de auditoría es un conjunto de acciones, procedimientos e instrucciones paso a paso de auditoría que debe realizarse para completarla y que son capaces, en su conjunto, de alcanzar los objetivos de auditoría. Los programas de auditorías se basan en el alcance y el objetivo de la asignación en particular. Los auditores de sistemas de información evalúan, a menudo, las funciones y los sistemas de TI desde perspectivas diferentes, tales como la **seguridad** (confidencialidad, integridad y disponibilidad), la **calidad** (efectividad, eficiencia), **fiduciaria** (cumplimiento, confiabilidad), el **servicio** y la **capacidad**.

Para cada auditoría específica se deberá elaborar el programa de auditoría que incluya los procedimientos por aplicar, su alcance, la temporalidad y el personal designado para ejecutar cada tarea de la auditoría. El **esquema** típico de un programa de auditoría incluye lo siguiente:

ESQUEMA TÍPICO DE UN PROGRAMA DE AUDITORÍA

1	Objetivo de auditoría	Es donde se indica el propósito del trabajo de auditoría por realizar.
2	Alcances de auditoría	aquí se identifica los sistemas específicos o unidades de organización que se han de incluir en la revisión en un período de tiempo determinado.
3	Planificación previa	donde se identifica los recursos y destrezas que se necesitan para realizar el trabajo, así como las fuentes de información para pruebas o revisión y lugares físicos o instalaciones donde se va a auditar.
4	Procedimiento de auditoría para:	▶ Recopilación de datos. ▶ Identificación de lista de personas por entrevistar. ▶ Identificación y selección del enfoque del trabajo. ▶ Identificación y obtención de políticas, normas y directivas. ▶ Desarrollo de herramientas y metodología para probar y verificar los controles existentes. ▶ Procedimientos para evaluar los resultados de las pruebas y revisiones. ▶ Procedimientos de comunicación con la gerencia. ▶ Procedimientos de seguimiento.

Tabla 1. Aspectos que incluye un esquema típico de un programa de auditoría. Fuente: elaboración propia.

El programa de auditoría se convierte también en una guía para documentar los diversos pasos de auditoría y para señalar la ubicación del material de evidencia. Es la estrategia de auditoría y el plan de auditoría. Este identifica el alcance, los objetivos y los procedimientos de auditoría para lograr evidencia **suficiente, competente y confiable** para obtener y sustentar las conclusiones y opiniones de auditoría. Los procedimientos generales de auditoría son los pasos básicos en la ejecución de una auditoría y habitualmente incluyen lo siguiente:

- ▶ Obtención y documentación del conocimiento sobre el área u objeto de la auditoría.
- ▶ Evaluación de riesgos y planificación general de la auditoría y cronograma.
- ▶ Una planificación de auditoría detallada que incluiría los pasos de auditoría necesarios y un desglose del trabajo planificado en un cronograma anticipado.
- ▶ Revisión preliminar del área u objeto de la auditoría.
- ▶ Evaluación del área u objeto de la auditoría.
- ▶ Verificación y evaluación de la pertinencia de los controles diseñados para cumplir los objetivos de control.
- ▶ Pruebas de cumplimiento.
- ▶ Pruebas sustantivas.
- ▶ Reporte, comunicación de los resultados.
- ▶ Seguimiento en casos en los que hay una función de auditoría interna.

El auditor de sistemas de información debe entender los procedimientos para la prueba y evaluación de los controles de sistemas de información. Estos procedimientos podrían incluir:

- ▶ El uso de software generalizado de auditoría para examinar el contenido de los archivos de datos.
- ▶ El uso de software especializado para evaluar el contenido de los archivos de parámetros de la base de datos y de aplicación del sistema operativo.
- ▶ Técnicas de elaboración de diagramas de flujo para la documentación de aplicaciones automatizadas y del proceso de negocio.
- ▶ El uso de registros (*logs*) o informes de auditoría disponibles en los sistemas operativos o de aplicación.
- ▶ Revisión de la documentación.
- ▶ Observación y consultas.
- ▶ Inspección y verificación.
- ▶ Revisión del rendimiento de controles.

El auditor de sistemas de información debe tener una comprensión suficiente de estos procedimientos que le permita planificar pruebas apropiadas de auditoría. Los programas de auditoría deben ser lo suficientemente flexibles para permitir, en el transcurso de esta, modificaciones, mejoras y ajustes, a juicio del encargado de la auditoría en cada momento.

8.5. Herramientas y técnicas de auditoría

Se definen como herramientas de auditoría a un conjunto de programas y ayudas que dan asistencia a los auditores de sistemas, analistas, ingenieros de software y desarrolladores. Por otra parte, podemos decir que son elementos físicos utilizados para llevar a cabo las acciones y pasos definidos en la técnica.

- ▶ **¿Para qué se utilizan?** Las herramientas se utilizan para valorar registros, planes, presupuestos, programas, controles y otros aspectos que afectan la administración y control de una organización o las áreas que la integran.
- ▶ **¿Para qué se aplican?** Las herramientas se aplican para investigar algún hecho, comprobar alguna cosa, verificar la forma de realizar un proceso, evaluar la aplicación de técnicas, métodos o procedimientos de trabajo, verificar el resultado de una transacción y comprobar la operación correcta de un sistema software, entre otros muchos aspectos.

Tipos de herramientas

Cuestionarios. Elementos para recopilar datos mediante preguntas impresas o fichas en las que el entrevistado responde de acuerdo con su criterio. De esta manera, el auditor obtiene información útil que puede concentrar, clasificar e interpretar por medio de su tabulación y análisis para evaluar lo que está auditando y emitir una opinión sobre el aspecto investigado.

Checklists. El conjunto de preguntas recibe el nombre de *checklist*. Estas deben ser contestadas oralmente, ya que superan en riqueza y generalización a cualquier otra forma.

Entrevistas. Se puede considerar la principal actividad de un auditor, sin importar el tipo de auditoría que realice. Estas entrevistas consisten en la recopilación de información sobre el **aspecto** que va a auditar. Como cualquier tipo de entrevista, es

conveniente señalar que para realizarla de manera eficiente es indispensable entender y seguir un procedimiento bien estructurado, cuya eficacia en las auditorías tradicionales es muy utilizada. Como norma general, existen dos tipos de entrevistas:

- ▶ **Entrevistas libres:** son las entrevistas en las que se sigue un guion básico para obtener la información requerida, pero la participación del entrevistado es libre y sin ninguna atadura.
- ▶ **Entrevistas dirigidas:** este tipo de entrevistas son recomendables para rectificar o ratificar datos con el entrevistado, siempre que se establezca perfectamente el guion que va a seguirse durante la entrevista.

Trazas y/o huellas. Se utilizan para comprobar la ejecución de las validaciones de datos previstas. Las trazas no deben modificar en absoluto el sistema. Si la herramienta de auditoría produce incrementos apreciables de carga, se convendrá de antemano las fechas y horas más adecuadas para su empleo. Son programas encaminados a verificar lo correcto de los cálculos de nóminas, primas, etc.

Herramientas de apoyo a la auditoría. Este tipo de herramientas están orientadas a que se incremente la productividad y efectividad del auditor, efectuar auditorías con valor agregado, que todos los papeles de trabajo sean estandarizados, optimización con la emisión del informe de auditoría.

Herramientas de planificación y registro. Este tipo de herramientas se enfocan en la planificación estratégica basada en evaluación de riesgos, la planificación y asignación de recursos, trabajo distribuido geográficamente, almacenamiento centralizado, seguimiento.

Al existir innumerables herramientas, se recomienda navegar en la red para poder adquirir más conocimientos sobre las herramientas comúnmente utilizadas en la práctica de la auditoría de sistemas. Hay que tener en cuenta que estas herramientas son muy especializadas, por lo que el volumen que existe es muy amplio. Por ello se aconseja ajustar la búsqueda a algún objetivo concreto.

Auditoría continua

La auditoría continua de sistemas de información también se realiza mediante típicos procedimientos automatizados de auditoría. Las herramientas GRC (*governance, risk and compliance*) pasaron de ser planillas donde se registran los riesgos de distintos sectores de la empresa a ser sistemas que se **conectan** a las principales aplicaciones, dando una visión en tiempo real de los riesgos en los sistemas. Estas herramientas se enfocan en los riesgos que puedan surgir del privilegio de usuarios, controles generales, controles de aplicación y patrones de fraude.

8.6. Objetivos de la auditoría

Los objetivos de la auditoría son las **metas específicas** que deben cumplirse por parte de la auditoría. En contraste, un **objetivo de control** hace referencia a cómo debería funcionar un control interno. Una auditoría puede incorporar, como generalmente hace, varios objetivos de auditoría.

Los objetivos de auditoría se centran, a menudo, en validar que existen controles internos para minimizar los riesgos del negocio y que estos funcionan según lo esperado. Incluyen el aseguramiento del cumplimiento con requerimientos legales y regulatorios, así como la confidencialidad, integridad, confiabilidad y disponibilidad de recursos de información y de TI. La gerencia de auditoría puede dar al auditor de sistemas de información un objetivo general de control para revisar y evaluar al llevar a cabo una auditoría.

Un elemento clave en la planificación de una auditoría de sistemas de información es **traducir los objetivos** de auditoría básicos y de amplio alcance en objetivos específicos de auditoría de sistemas de información. Por ejemplo, en una auditoría financiera u operacional, un objetivo de control podría ser **asegurar** que las transacciones sean debidamente registradas en las cuentas del libro mayor del sistema contable. Sin embargo, en la auditoría de sistemas de información, el objetivo podría ampliarse para asegurar que existen funciones de edición para detectar los errores en la codificación de las transacciones que podrían tener un impacto en las actividades de registro de las cuentas.

8.7. Evidencia

La evidencia es cualquier información usada por el auditor de sistemas de información para determinar si la entidad o los datos que están siendo auditados cumplen con los criterios u objetivos establecidos y soporta las conclusiones de auditoría. Es un **requerimiento** que las conclusiones del auditor se basen en evidencia suficiente, relevante y competente.

Al planificar el trabajo de auditoría de sistemas de información, el auditor de sistemas de información debe tomar en cuenta el tipo de evidencia de auditoría a ser recopilada, su uso como evidencia de auditoría para alcanzar los objetivos de auditoría y sus diferentes niveles de confiabilidad.

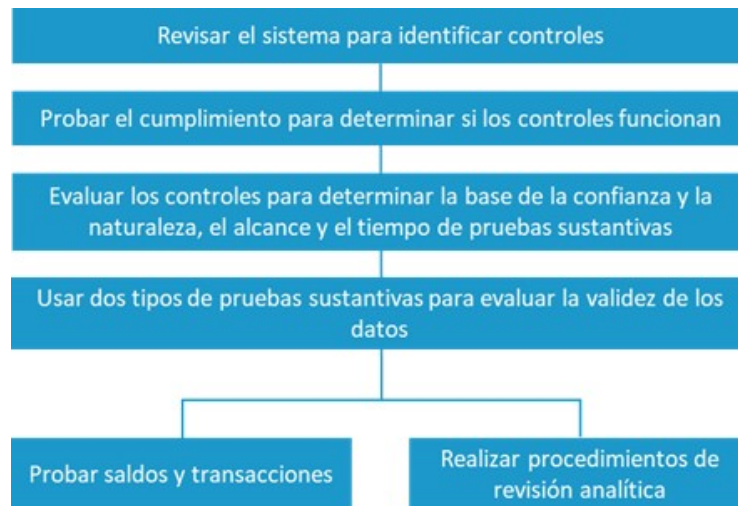


Figura 8. Planificación trabajos auditores sistemas de información. Fuente: elaboración propia.

Los determinantes para evaluar la **confiabilidad** de la evidencia de auditoría incluyen:

- ▶ **Independencia del proveedor de la evidencia.** La evidencia obtenida de fuentes externas es más confiable que la obtenida dentro de la organización.
- ▶ **Credenciales de la persona que suministra la información o evidencia.** El auditor de sistemas de información debería siempre considerar las credenciales y las responsabilidades funcionales de las personas que suministran la información o evidencia independientemente de si estas personas son internas o externas a la organización.
- ▶ **Objetividad de la evidencia.** La evidencia objetiva es más confiable que la evidencia que requiere opinión o interpretación considerable. La revisión de inventario de medios de un auditor de sistemas de información es una evidencia objetiva y directa. El análisis de un auditor de sistemas de información de la eficiencia de una aplicación, basado en discusiones con determinado personal, puede no ser una evidencia de auditoría objetiva.
- ▶ **Tiempo de disponibilidad de la evidencia.** El auditor de sistemas de información debería considerar el tiempo durante el cual la información existe o está disponible al determinar la naturaleza, el tiempo y la extensión de las pruebas de cumplimiento y, si fuera aplicable, las pruebas sustantivas.

Es importante que los auditores de sistemas de información tengan una comprensión de las reglas de la evidencia, ya que es posible que encuentren una variedad de tipos de evidencia.

La **recolección de evidencia** es un paso clave en el proceso de auditoría. El auditor de sistemas de información debe ser consciente de las diversas formas de evidencia de auditoría y de cómo puede ser recopilada y revisada. Las técnicas para la recopilación de evidencia son:

- ▶ **Revisión de las estructuras organizacionales de sistemas de información.** Una estructura organizacional que provee una adecuada separación o segregación de funciones es un control general clave en un ambiente de sistemas de información.
- ▶ **Revisión de políticas y procedimientos de sistemas de información.** Un auditor de sistemas de información debe revisar si existen políticas y procedimientos apropiados, determinar si el personal entiende las políticas y los procedimientos, y garantizar que se sigan las políticas y los procedimientos.
- ▶ **Revisión de los estándares de sistemas de información.** Primero, el auditor de sistemas de información debería entender los estándares vigentes existentes dentro de la organización.
- ▶ **Revisión de la documentación de sistemas de información.** Un primer paso al revisar la documentación para un sistema de información es entender la documentación existente vigente con que cuenta la organización.

Técnicas de recolección de evidencias

- ▶ **Entrevistas al personal apropiado.** Las técnicas de entrevista constituyen una habilidad importante para el auditor de sistemas de información. Las entrevistas deberían ser organizadas de antemano con objetivos claramente comunicados, seguir un patrón fijo y ser documentadas por medio de notas de entrevista. Un buen método es un formulario de entrevista o lista de verificación preparada por un auditor de sistemas de información.
- ▶ **Observación de procesos y desempeño de empleados.** La observación de procesos es una técnica clave de auditoría para muchos tipos de revisiones.

- ▶ **«Redesempeño».** El proceso de repetición de ejecución es una técnica de auditoría clave que, generalmente, proporciona una mejor evidencia que las otras técnicas y es, por consiguiente, utilizada cuando al combinar la investigación, observación y examen de la evidencia no se obtiene una garantía suficiente de que un control funcione de manera efectiva.
- ▶ **Pruebas de recorrido.** La prueba de recorrido es una técnica de auditoría para continuar el entendimiento de los controles.

Muestreo

El muestreo es usado cuando las consideraciones de tiempo y de coste impiden una verificación total de todas las transacciones o eventos en una población definida previamente. La población está constituida por la totalidad de los elementos que son necesarios examinar. Una **muestra** es un subconjunto de miembros de una población utilizada para realizar pruebas. El muestreo se usa para **inferir características** de una población, con base en las características de una muestra.

Los **dos enfoques** generales para muestreo de auditoría son el estadístico y el no estadístico:

- ▶ **Muestreo estadístico.** Es un enfoque objetivo para determinar el tamaño y los criterios de selección de la muestra. El muestreo estadístico usa las leyes matemáticas de la probabilidad para:
 - Calcular el tamaño de la muestra.
 - Seleccionar los objetos de la muestra.
 - Evaluar los resultados de la muestra y hacer la inferencia.
- ▶ **Muestreo no estadístico.** El número de elementos que serán examinados de una población (tamaño de la muestra) y qué elementos seleccionar (selección de la muestra) son determinados con base en el juicio del auditor. Estas decisiones están

basadas en el **criterio subjetivo** respecto a qué ítems o transacciones tienen mayor importancia y mayor riesgo.

Principios durante la recolección de evidencias

El **RFC 3227** es un documento que recoge las directrices para la recopilación de evidencias y su almacenamiento, y puede llegar a servir como estándar de facto para la recopilación de información en incidentes de seguridad o en la realización de periciales informáticas. Si bien es cierto que las medidas en la recogida de evidencias durante la auditoría no tienen que ser tan severas como esas, es muy recomendable conocerlas y, si es conveniente, aplicarlas.

- ▶ Capturar una imagen del sistema tan **precisa** como sea posible.
- ▶ Realizar notas **detalladas**, incluyendo fechas y horas indicando si se utiliza horario local o UTC.
- ▶ **Minimizar** los cambios en la información que se está recolectando y **eliminar** los agentes externos que puedan hacerlo.
- ▶ En el caso de enfrentarse a un dilema entre **recolección y análisis**, elegir primero recolección y después análisis.
- ▶ Recoger la información según el **orden de volatilidad** (de mayor a menor).
- ▶ Tener en cuenta que, por cada dispositivo, la recogida de información puede realizarse de **distinta manera**.

Consideraciones sobre la privacidad

- ▶ Es muy importante tener en consideración las pautas de la empresa en lo que a **privacidad** se refiere. Es habitual solicitar una autorización por escrito de quien corresponda para poder llevar a cabo la recolección de evidencias.

- ▶ No hay que entrometerse en la privacidad de las personas sin una **justificación**. No se deben recopilar datos de lugares a los que normalmente no hay razón para acceder, como ficheros personales, a menos que haya suficientes indicios.

Procedimiento de recolección

El procedimiento de recolección debe ser lo más detallado posible; además, hay que procurar que no sea ambiguo y reducir al mínimo la toma de decisiones.

PASOS del procedimiento de recolección	¿Dónde está la evidencia? Listar qué sistemas están involucrados en el incidente y de cuáles se deben tomar evidencias.
	Establecer qué es relevante. En caso de duda, es mejor recopilar mucha información que poca.
	Fijar el orden de volatilidad para cada sistema.
	Obtener la información de acuerdo al orden establecido.
	Comprobar el grado de sincronización del reloj del sistema.
	Según se vayan realizando los pasos de recolección, preguntarse qué más puede ser una evidencia.
	Documentar cada paso.
	No olvidar a la gente involucrada. Tomar notas sobre qué gente estaba allí, qué estaban haciendo, qué observaron y cómo reaccionaron.

Tabla 2. Pasos del procedimiento de recolección. Fuente: elaboración propia.

El procedimiento de almacenamiento

Cadena de custodia

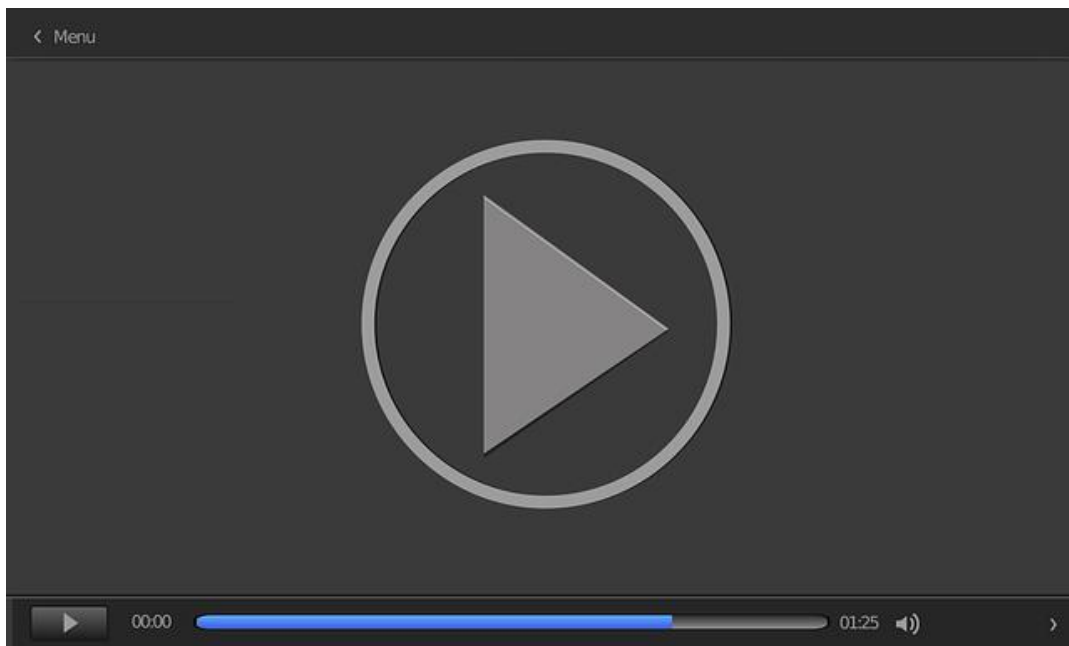
La cadena de custodia debe estar claramente documentada y se deben detallar los siguientes puntos:

- ▶ Dónde, cuándo y quién descubrió y recolectó la evidencia.
- ▶ Dónde, cuándo y quién manejó la evidencia.
- ▶ Quién ha custodiado la evidencia, por cuánto tiempo y cómo la ha almacenado.
- ▶ En el caso de que la evidencia cambie de custodia, cuándo y cómo se realizó el intercambio, incluyendo número de albarán, etc.

8.8. Comunicación de los resultados de la auditoría

Los auditores de sistemas de información deberían ser conscientes de que, al final, ellos son los **responsables** frente a la alta dirección y el comité de auditoría del consejo de dirección. Deberían sentirse en libertad para comunicar los asuntos o preocupaciones a dicha gerencia. Un intento de negar acceso por niveles inferiores a la alta dirección limitaría la independencia de la función de auditoría.

En este vídeo (*Comunicación de los resultados de la auditoría*) se presentan los principales aspectos relacionados con la comunicación de los resultados de la auditoría.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=7bf4466c-09b8-42cf-b2ae-adf700cfec86>

8.9. Referencias bibliográficas

ISACA. (2019). *CISA. Review manual*. 27. ° ed. ISACA.

Ciberseguridad

Corletti, A. (2017). *Ciberseguridad (una estrategia informático/militar)*. DarFe Learning & Consulting S. L. <https://darfe.es/index.php/es/nuevas-descargas/category/3-libros>

Libro recomendable de Alejandro Corletti sobre ciberseguridad; si se comprenden los términos y la filosofía, se podrá auditar.

Auditoría de sistemas. Políticas de seguridad para la pyme

INCIBE. (s. f.). *Auditoría de sistemas. Políticas de seguridad para la pyme*. [Archivo PDF].

<https://www.incibe.es/sites/default/files/contenidos/politicas/documentos/auditoria-sistemas.pdf>

Aunque es plausible visualizar los contenidos de INCIBE, este documento de política de seguridad para pymes es curioso.

Cobertura del riesgo tecnológico: hacia una auditoría interna de TI integrada

Instituto de Auditores Internos de España. (2014). *Cobertura del riesgo tecnológico: hacia una auditoría interna de TI integrada*. [Archivo P D F] . https://auditoresinternos.es/uploads/media_items/f%C3%A1bricati-web.original.pdf

Publicación del IAI muy recomendable para metodologías y *paper*. Un ejemplo muy constructivo.

Guía para la construcción de un SGCI. Sistema de gestión de la ciberseguridad industrial

Centro de Ciberseguridad Industrial. (2014, diciembre 29). *Guía para la construcción de un SGCI. Sistema de gestión de la ciberseguridad industrial (primera parte)*. [Archivo PDF]. <https://www.cci-es.org/activities/guia-para-la-construccion-de-un-sistema-de-gestion-de-la-ciberseguridad-industrial-primera-parte/>

El Centro de Ciberseguridad Industrial (CCI) es muy interesante y dirigido. Para saber auditarlo hay que comprender cómo implantarlo; este documento es muy enriquecedor para comprender la filosofía y los puntos esenciales.

1. ¿Qué acciones son fundamentales en la planificación de una auditoría?
 - A. Comprensión del negocio.
 - B. Posibles riesgos.
 - C. Metodología que se debe utilizar.
 - D. Todas las anteriores.

2. ¿Cuál es la finalidad de un estándar de auditoría?
 - A. Es una metodología utilizada por cada una de las empresas auditoras.
 - B. Los estándares definen los requerimientos obligatorios para la auditoría de sistemas y la generación de informe.
 - C. Es una guía y se tiene que seguir de manera exacta.
 - D. Ninguna de anteriores.

3. ¿Cuántos organismos de auditoría existen?
 - A. Solo uno, ISACA.
 - B. Solo uno, IIA.
 - C. Dos, ISACA e IIA.
 - D. Existen mucho más que dos organismos.

4. ¿Para qué se usan las herramientas de auditoría?
 - A. Para verificar de una manera clara los medios técnicos.
 - B. Para valorar registros, planes, presupuestos, programas, controles y otros aspectos que afectan a la administración y control de una organización o las áreas que la integran.
 - C. Para industrializar el proceso o actuación de auditoría.
 - D. No existen herramientas de auditoría como tal.

5. ¿Un conjunto de preguntas que deben ser contestadas oralmente por el interlocutor constituye una herramienta de auditoría?
 - A. Sí.
 - B. No.
 - C. Depende del conjunto de preguntas.
 - D. Ninguna de las anteriores.

6. ¿Una herramienta de auditoría es lo mismo que una herramienta de evaluación de seguridad?
 - A. Sí.
 - B. No.
 - C. Depende de la herramienta.
 - D. Ninguna de las anteriores.

7. La metodología de auditoría debería ser establecida y aprobada por:
 - A. El departamento de auditoría interna de la compañía.
 - B. El grupo de seguridad de la compañía.
 - C. La gerencia de auditoría.
 - D. Es un documento interno y por modificaciones no es conveniente que esté aprobada.

8. Los objetivos de auditoría se centran a menudo en:
 - A. Entregar un informe de resultados.
 - B. Cumplir con los requerimientos legales o regulatorios.
 - C. Validar que existen controles internos para minimizar los riesgos del negocio.
 - D. Ninguna de las anteriores.

9. Una evidencia puede ser:
- A. Un documento Word.
 - B. Un registro de un sistema.
 - C. Una conversación con un interlocutor.
 - D. Todas las anteriores.
10. El informe final de auditoría:
- A. Contiene el informe final.
 - B. Contiene el informe preliminar.
 - C. Las respuestas A y B son correctas.
 - D. Ninguna de las respuestas anteriores es correcta.

Desarrollo Seguro de Software y Auditoría de la
Ciberseguridad

Tema 9. Auditorías operativas/técnicas

Índice

Esquema

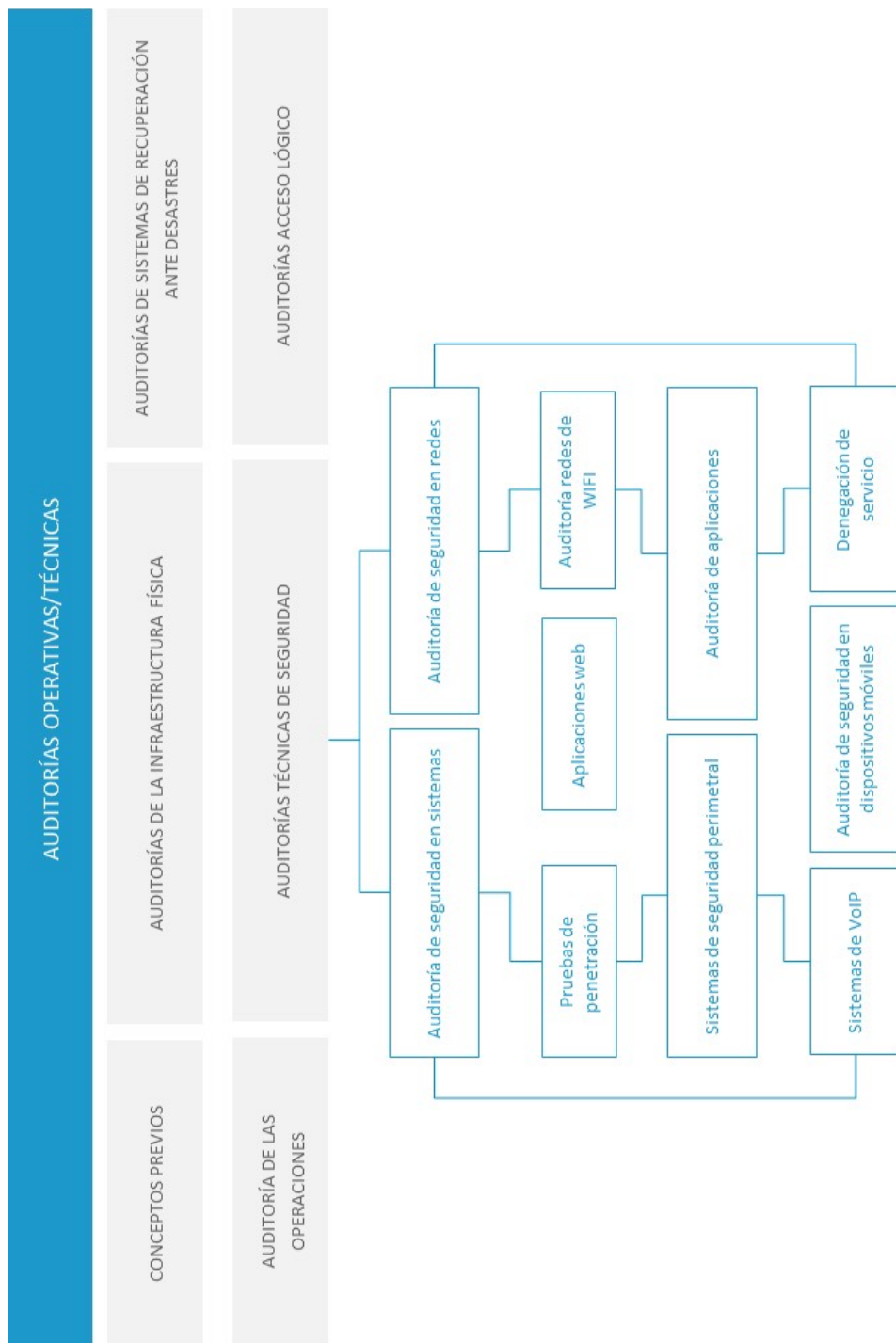
Ideas clave

- 9.1. Introducción y objetivos
- 9.2. Auditorías de la infraestructura física
- 9.3. Auditorías de sistemas de recuperación ante desastres
- 9.4. Auditoría de las operaciones
- 9.5. Auditorías técnicas de seguridad
- 9.6. Auditoría del acceso lógico
- 9.7. Referencias bibliográficas

A fondo

- Writing a penetration testing report
- Auditoría: principios y generalidades
- Auditoría de sistemas. Seguridad física de instalaciones de TI
- Monográfico: Nmap
- Tracking penetration test activities

Test



9.1. Introducción y objetivos

Se han estudiado conceptos generales relativos a la realización y ejecución de auditorías y sus principales tipos. Dentro de los **tipos de auditorías**, en lo que concierne a **seguridad** nos interesan las auditorías de los sistemas de información y telecomunicaciones. Esta última engloba los dos siguientes tipos:

- ▶ **Auditoría de seguridad operativa/técnica.** Son auditorías o revisiones de seguridad técnica de un sistema o sistemas de información o de telecomunicaciones de una organización o de su operación.
- ▶ **Auditoría de cumplimiento de la seguridad de la información.** Son auditorías que verifican el cumplimiento de un determinado estándar o políticas y procedimientos internos de seguridad de la organización de seguridad, por ejemplo: PCI o LOPD.

La auditoría de la seguridad operativa/técnica abarca los conceptos de **seguridad física, lógica y las operaciones**. La seguridad física se refiere a la protección del hardware y los soportes de datos, así como la seguridad de los edificios e instalaciones que los albergan. El auditor debe contemplar situaciones de incendios, inundaciones, sabotajes, robos, catástrofes naturales, etc.

Por su parte, la seguridad lógica se refiere a la seguridad en el uso de software, hardware, comunicaciones, la protección de los datos, procesos y programas, así como la del acceso ordenado y autorizado de los usuarios a la información.

Otra parte importante por auditar son las **operaciones de seguridad** de los centros de procesos de datos (CPD), con el fin de valorar su operación eficaz y eficiente conforme al seguimiento de políticas, procedimientos y planes desarrollados y establecidos en la organización.

En el presente tema nos centraremos en el estudio de las auditorías de seguridad **operativas/técnicas**, entre las que encontramos las siguientes:

- ▶ Auditoría de la infraestructura física.
- ▶ Auditoría de recuperación ante desastres.
- ▶ Auditoría de las operaciones.
- ▶ Auditoría técnica de seguridad informática.
- ▶ Auditoría del acceso lógico.

Los **objetivos** del presente tema son los siguientes:

- ▶ Definir y explicar los conceptos relacionados con las auditorías de seguridad operativas/técnicas.
- ▶ Definir y explicar los conceptos relacionados con las auditorías de cumplimiento de la seguridad de la información.
- ▶ Obtener el conocimiento necesario para poder realizar los diferentes tipos de auditorías que comprenden la seguridad de la información.

9.2. Auditorías de la infraestructura física

Actualmente, los CPD concentran los recursos necesarios para el procesamiento de la información de una organización. Proporcionan una infraestructura física y ambiental en cuanto a temperatura y humedad, conectividad de red, energía eléctrica, protección contra incendios y seguridad física.

Afrontan **riesgos** de distinta naturaleza, algunos están relacionados con:

- ▶ Problemas **industriales**, como subidas de tensión, fuego motivado por condiciones ambientales inadecuadas y otros relacionados con desastres naturales, como pueden ser inundaciones, terremotos, huracanes, etc.
- ▶ Errores **humanos**, como apagado no voluntario o borrados de datos accidentales.
- ▶ Riesgos que vienen motivados por **acciones malintencionadas** como, por ejemplo: robo de datos, actos vandálicos, incidentes terroristas, disturbios, sabotaje o la sustracción de equipos por parte de personas malintencionadas que accedan.

Para mitigar los riesgos anteriores es necesario implantar una serie de controles **físicos u operacionales** orientados a la protección del entorno y de los recursos físicos de la organización.

En esta era de tecnología avanzada, es fácil olvidar la importancia de los controles físicos y enfocarse solo en los controles lógicos. Sin embargo, incluso con excelentes controles de acceso lógico, las amenazas físicas indicadas anteriormente pueden comprometer la seguridad y disponibilidad de los sistemas.

De acuerdo con ISACA (2019), los controles pueden clasificarse de la siguiente forma:

- ▶ **Administrativos.** Incluyen la política, los procedimientos, el equilibrio, el desarrollo de los empleados, la presentación de informes de cumplimiento, etc.
- ▶ **Técnicos.** Se conocen como controles lógicos e incluyen, por ejemplo, la protección perimetral, la red, los sistemas de detección de intrusos (IDS), contraseñas y antivirus, etc.
- ▶ **Físicos.** Incluyen el control de acceso físico: vallas, circuito cerrado de TV (CCTV), sistema de protección contra incendios, etc.

	Administrativos	Técnicos	Físicos
Preventivos	Proceso de registro de usuario	Acceso lógico	Vallas
Detectivos	Auditoría	Sistema de detección de intrusiones	Sensores de movimiento (volumétricos)
Correctivos	Revocación de accesos	Aislamiento de redes	Puertas contra incendios

Tabla 1. Matriz de controles. Fuente: adaptado de ISACA, 2019.

Los diferentes controles que auditar, en lo relativo a la infraestructura física, se encuentran englobados en algunos de los siguientes sistemas:

- ▶ **Sistemas de control de acceso.** Autentifican la entrada física de los trabajadores a las instalaciones. Su objetivo es proteger los sistemas de información y telecomunicaciones que residen en los CPD.
- ▶ **Sistemas de control de intrusiones.** Disponen de una serie de sensores magnéticos en puertas y ventanas, así como sensores de movimiento (volumétricos) que proporcionan alarmas de intrusiones físicas no autorizadas.

- ▶ **Sistemas de protección contra incendios.** Compuestos de centrales de detención, sensores activados por calor o humo divididos en zonas que cubren diferentes partes de la instalación. En los CPD se incluyen, también, centrales automáticas de extinción de incendios que se presentan en dos variedades:

- Sistemas a base de agua.
- Sistemas a base de gas.

Adicionalmente se debe tener un número suficiente de extintores de incendios acorde a la normativa contraincendios en vigor.

- ▶ **Sistemas de circuito cerrado de televisión (CCTV).** Consiste en un conjunto de cámaras conectadas a un sistema que controla diferentes monitores y dispositivos de presentación donde se proyectan las diferentes imágenes captadas por cámaras. Proporciona capacidades de vigilancia y alarma de las zonas e instalaciones cubiertas por el sistema. También disponen de sistemas de grabación.

- ▶ **Sistemas de control de las instalaciones** del CPD que permitan realizar las maniobras en los sistemas de climatización, energía, etc. Y que, sobre todo, proporcione alarmas del tipo:

- Alarmas de agua en el suelo del CPD.
- Alarmas de humedad.
- Alarmas de temperatura en el CPD.
- Alarmas de fluctuación de energía eléctrica.
- Alarmas químicas o de gas (por ejemplo, salas de baterías).

- ▶ **Sistemas resilientes de alimentación eléctrica.** Implementados mediante la inclusión de sistemas de alimentación ininterrumpida (UPS) y grupos electrógenos. La UPS proporciona una energía limpia a los sistemas del CPD y protege las

instalaciones contra fluctuaciones de energía de la red comercial, como picos, sobrecargas, caídas de tensión y apagones que pueden dañar los componentes de los equipos de proceso o causar interrupciones. El grupo electrógeno proporciona energía eléctrica durante cortes de energía prolongados. Para mitigar este riesgo, los centros de datos proporcionan redundancia en sus sistemas de energía:

- Conexión redundante a la red eléctrica con compañías diferentes.
 - Sistemas UPS redundantes.
 - Transformadores de aislamiento.
 - Puesta a tierra de todos los equipos.
 - Grupos electrógenos redundados.
- **Sistemas de climatización de precisión.** Su objetivo es mantener en condiciones óptimas la temperatura y humedad de las instalaciones del CPD. Las condiciones extremas de temperatura y humedad pueden causar daños en los sistemas informáticos. Estos sistemas también deberían estar redundados para mantener la temperatura y la humedad constantes. Por regla general se deben diseñar para proporcionar el doble de capacidad requerida.
- **Protección TEMPEST.** Dado el grado de clasificación de la información que se procesa en un CPD, podría ser necesaria la instalación de protecciones para evitar el robo de información debido a las emanaciones de los diferentes equipos de proceso del CPD y de la organización (Centro Criptológico Nacional, 2020).

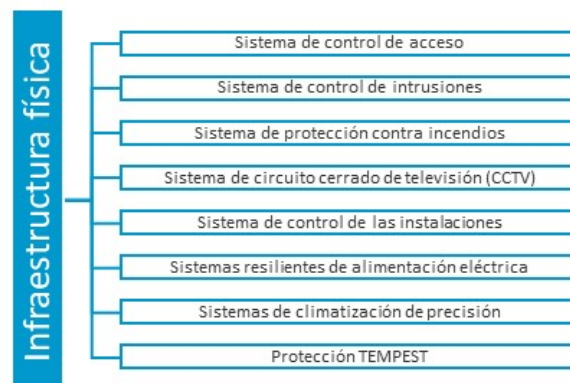


Figura 1. Sistemas de infraestructura física. Fuente: elaboración propia.

A la hora de realizar una **auditoría de los sistemas de infraestructura física**, al menos se deberían abordar las siguientes áreas (Davis et al., 2011):

- ▶ Factores de riesgo externos.
- ▶ Sistemas de seguridad física.
- ▶ Controles ambientales.
- ▶ Alimentación eléctrica.
- ▶ Protección contra el fuego.
- ▶ Protección TEMPEST.

9.3. Auditorías de sistemas de recuperación ante desastres

Todos los CPD de una organización son susceptibles a sufrir cualquier desastre natural o aquellos causados por el hombre, como puede ser el caso común de un **ciberincidente**. En este sentido, toda organización debería disponer de un plan de continuidad de negocio que está formado por un conjunto de planes de actuación. Uno de estos planes es el **Plan de Contingencia de las Tecnologías de la Información y las Comunicaciones (TIC)**. Su objetivo es permitir que la organización siga proporcionando sus servicios críticos en caso de una interrupción desastrosa de sus sistemas informáticos. De acuerdo con INCIBE, este plan consiste en:

«Una estrategia planificada en fases, constituida por un conjunto de recursos de respaldo, una organización de emergencia y unos procedimientos de actuación, encaminados a conseguir una restauración ordenada, progresiva y ágil de los sistemas de información que soportan la información y los procesos de negocio considerados críticos para la empresa» (INCIBE, 2019).



Figura 2. Planes ante desastres. Fuente: INCIBE, 2019.

Este plan incluye, a su vez, un **Plan de Recuperación ante Desastres** (DRP o *Disaster Recovery Plan*, por sus siglas en inglés). Es un **sistema de control** interno que incluye todos los procedimientos necesarios (hardware, software, documentación, personal y soporte logístico) para el restablecimiento de los sistemas que proporcionan los servicios críticos de las TIC que son fundamentales para el funcionamiento de la organización.

Una vez que la criticidad de los procesos de negocio y el apoyo de las TIC se han definido, es necesario examinarlos y revisarlos periódicamente en el marco del DRP, es decir, es un **proceso continuo**. Dentro del plan es importante definir los siguientes conceptos (ISACA, 2019):

- ▶ **Punto de recuperación objetivo (RPO).** Indica el momento más temprano en el que es aceptable recuperar los datos.
- ▶ **Tiempo de recuperación objetivo (RTO).** Indica el momento más temprano en el que las operaciones (y los sistemas informáticos de apoyo) deben reanudarse después del desastre.

La siguiente figura muestra la relación entre el RPO y RTO y las tecnologías de recuperación (ISACA, 2019):

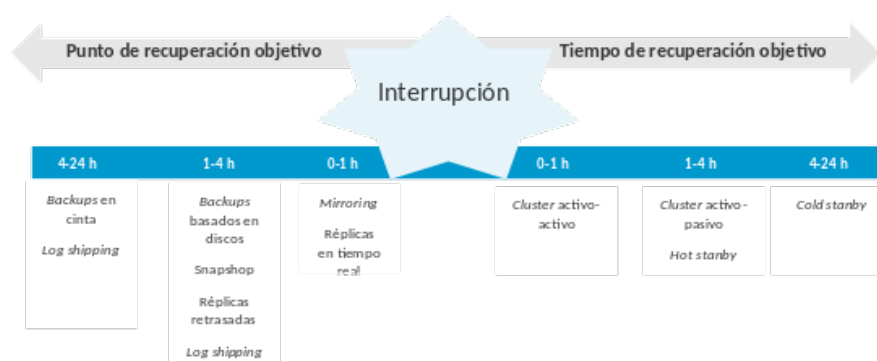


Figura 3. Relación entre el RTO y RPO. Fuente: elaboración propia.

Para realizar un DRP se deberían seguir las siguientes fases (INCIBE, 2019):

- ▶ Alinearlo con el **plan de continuidad de negocio**.
- ▶ **Realizar una evaluación de riesgos**. Se obtiene una visión detallada de las amenazas que pueden causar un desastre que ponga en riesgo la continuidad de negocio.
- ▶ **Llevar a cabo un análisis de impacto de negocio**. Identificar las necesidades de la organización en términos de recuperación, sobre todo en aquellos servicios que consideramos imprescindibles para el funcionamiento de la empresa.
- ▶ Desarrollar las **medidas de recuperación** para los servicios y aplicaciones.
- ▶ **Realizar pruebas**. Se deben programar pruebas periódicas que confirmen la continuidad en caso de desastre.
- ▶ **Actualizar y mejorar el plan**. Dado que las amenazas evolucionan constantemente hay que asegurar que el DRP no se queda desactualizado.
- ▶ **Capacitar a los encargados de ponerlo en marcha, concienciar y difundir el plan**. Todas las partes implicadas en el plan deben conocer al detalle su función en él, así como designar a los encargados de llevarlo a cabo.

El trabajo del auditor es identificar y medir los controles físicos, administrativos y lógicos en las instalaciones, lo que mitiga el riesgo de interrupciones en los sistemas TIC de la organización.

También debe verificar que la ejecución de los planes se traduce en la recuperación exitosa de la infraestructura y servicios críticos. Incluye lo siguiente (ISACA, 2019):

- ▶ La resiliencia del sistema.
- ▶ Copias de seguridad y restauración de datos.

- ▶ Verificar la integridad y precisión del DRP.
- ▶ Evaluar el desempeño del personal involucrado en el DRP.

La activación del DRP podría realizarse como consecuencia de alguno de los incidentes incluidos a continuación:

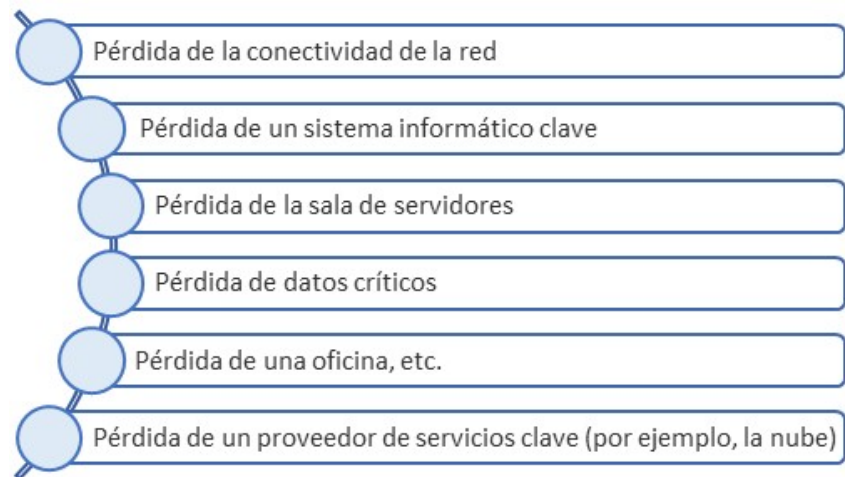


Figura 4. Causas activación del DRP. Fuente: adaptado de ISACA, 2019.

9.4. Auditoría de las operaciones

La operación eficaz y eficiente de un CPD requiere el seguimiento de políticas, procedimientos y planes desarrollados para el mismo. Entre las áreas que deberían cubrirse se encuentran las siguientes (Davis et al., 2011):

- ▶ La vigilancia de las instalaciones.
- ▶ Funciones y responsabilidades del personal del centro de datos.
- ▶ Segregación de las tareas del personal del centro de datos.
- ▶ Respuesta a emergencias y desastres.
- ▶ Mantenimiento de las instalaciones y el equipo.
- ▶ Planificación de la capacidad del centro de datos.
- ▶ Gestión de activos.
- ▶ Contenido y ubicación del almacenamiento fuera de línea.

9.5. Auditorías técnicas de seguridad

Las auditorías técnicas de seguridad tienen como objetivo **analizar la seguridad implantada en los sistemas de información y telecomunicaciones** de una organización y sus servicios (web, correo, wifi, aplicación web, *ftp-file transfer*, etc.) realizando una batería de pruebas planificadas que simulen el comportamiento de un atacante.

Este tipo de auditorías engloban muchos tipos de análisis y pruebas (*test*) de seguridad en función de aquella parte de los sistemas de información y telecomunicaciones que se precise revisar. Con base en lo anterior, los tipos de auditorías técnicas de seguridad que podríamos tener serían:

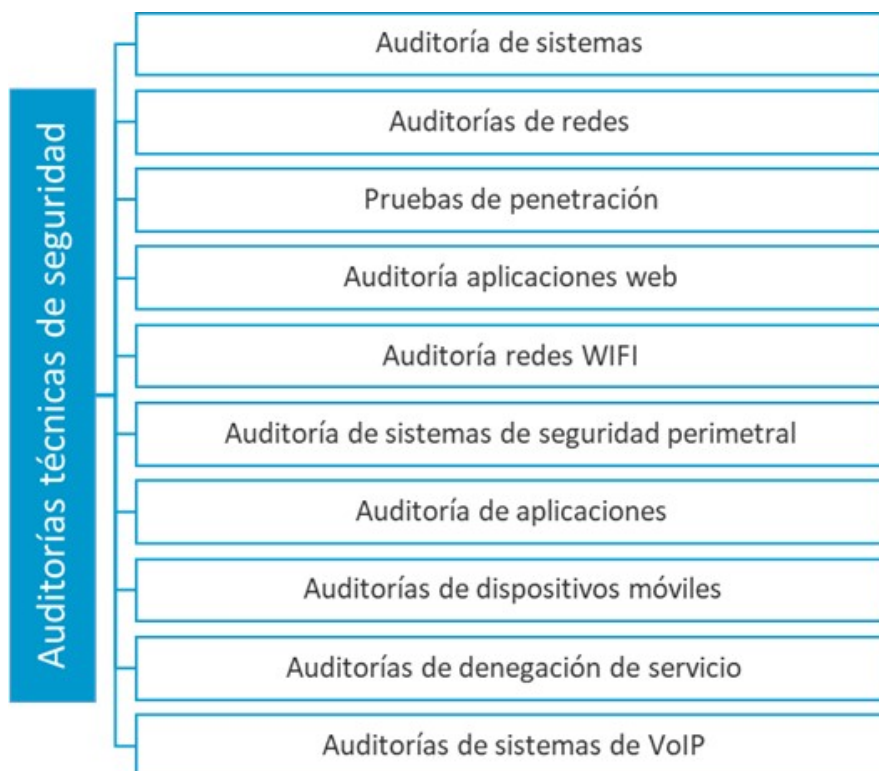


Figura 5. Tipos de auditorías técnicas de seguridad. Fuente: elaboración propia.

Con respecto a la **visibilidad**, esta determina cuál será la información suministrada al evaluador (*pen tester*) para llevar a cabo el análisis, y variará desde que no se le proporcione información alguna hasta que se le suministre, por ejemplo, el código fuente de las aplicaciones para que sean analizadas, información sobre la tipología de la red y sus tipos de sistemas, configuraciones, etc. Las **posibilidades** son:

- ▶ **Caja blanca (*white box*)**. Se facilita **información de antemano** para poder realizar la prueba de seguridad (normalmente, las direcciones IP por testear, arquitectura, etc.). Se analizan en profundidad todas las posibles brechas de seguridad de los sistemas sometidos a estudio al alcance de un atacante. Habitualmente, se permite al auditor la conexión a la red interna y tiene información previa de la organización aportada por el auditado. El resultado es un **informe amplio y detallado de las vulnerabilidades**, así como las recomendaciones para solventar cada una de ellas. A los efectos de definición, estas pruebas se corresponderán, por ejemplo, con una auditoría de código fuente, auditoría de una LAN disponiendo de información de su arquitectura, direccionamiento, etc.
- ▶ **Caja negra (*double blind*)**. Es esencialmente lo mismo que la caja blanca, con la dificultad añadida de que **no se facilita** ningún tipo de información inicial. Dentro del conjunto de esta definición se entenderá que estas pruebas se corresponden con el proceso de prueba de penetración o el análisis de protección perimetral cuando la finalidad es acceder a los sistemas de información de una empresa desde el **exterior** (Internet). El auditor no dispone de información de la empresa, solo la existente de forma pública (Internet, prensa, etc.).

Dentro de las auditorías de seguridad de caja blanca, vamos a englobar los dos primeros apartados indicados en el temario:

- **Caja gris (*double gray box*).** Es una mezcla entre caja negra y blanca. Se simulan ataques reales, pero conociendo de antemano gran parte de la información técnica. Se tiene un **conocimiento limitado de los activos y las defensas** que los protegen, y se simula un ataque que puede ser realizado por un miembro de la organización interno desde la red interna.

En la siguiente figura se muestra una clasificación del tipo de **auditorías técnicas de seguridad** conforme a los criterios anteriores:

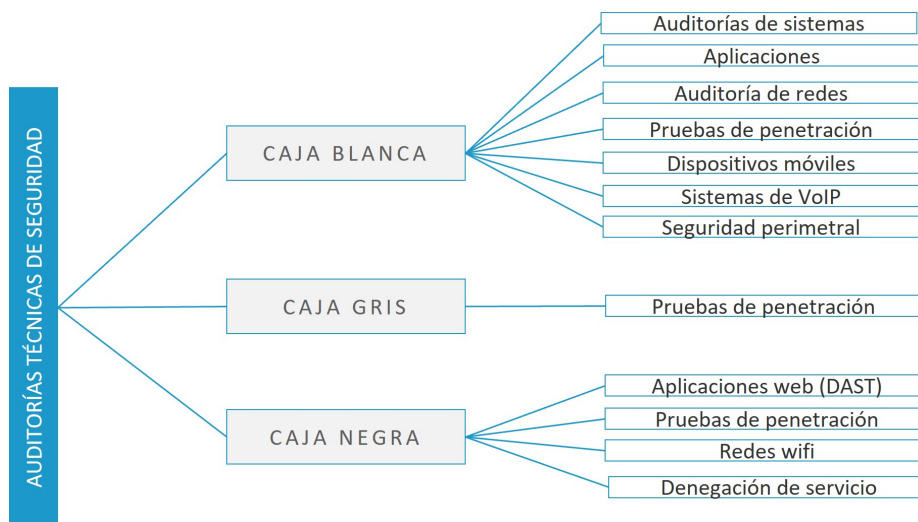


Figura 6. Clasificación de auditorías técnicas de seguridad. Fuente: elaboración propia.

Se entiende por **posicionamiento** el lugar desde donde se llevará a la práctica el análisis de seguridad. Este puede ser desde el interior de la red o desde un terminal externo a la misma, por lo que se clasifica en:

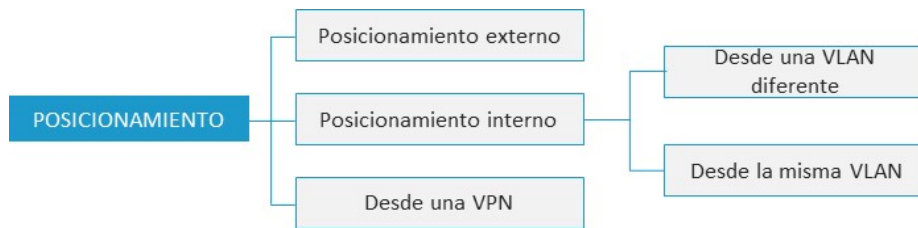


Figura 7. Tipos de posicionamiento. Fuente: elaboración propia.

A continuación, se desarrolla el alcance de algunos de los tipos de auditoría indicados en la Figura 5.

Auditoría de seguridad de sistemas

En este tipo de auditorías se realiza un **análisis de las vulnerabilidades**, configuraciones y procedimientos de seguridad de los diferentes sistemas (bases de datos, servidores de aplicaciones, DNS, etc.) que componen la red de la organización. Los puntos más importantes para revisar cuando se efectúa una auditoría de este tipo (SI) son los siguientes:

- ▶ Revisar la **configuración de seguridad** de los sistemas con respecto a la guía de configuración en vigor en la organización (CCN, NIST, DISA, NSA, CIS Security, etc.).
- ▶ Revisar **arquitecturas de alta disponibilidad**.
- ▶ **Analizar vulnerabilidades**.
- ▶ Revisar que la **administración de equipos** tiene la posibilidad de poner en contacto, en caso de un fallo de equipo, con el fabricante sin intervención manual.
- ▶ Revisar los **informes de disponibilidad** y utilización del hardware.
- ▶ Revisar **documentación de sistemas** (específicamente en las áreas de declaraciones de control de instalación, tablas de parámetros y bitácoras/reportes de actividades).

- ▶ Revisar la **documentación de seguridad del software** de sistema.
- ▶ Determinar si **los recursos del SI están fácilmente disponibles** para procesar aquellos programas de aplicación que requieren un alto nivel de disponibilidad de recursos.
- ▶ Revisar los **procedimientos y controles de seguridad y documentación** ante desastres.
- ▶ **Analizar la fortaleza de contraseñas.**

Entre las **técnicas de pruebas** que aplicar en esta auditoría tenemos:

- ▶ **Análisis de configuraciones de los diferentes sistemas.** Se pueden utilizar herramientas como Nessus y Clara para comprobar si la configuración de los diferentes sistemas es acorde a la guía de configuración en vigor en la organización (CCN, NIST, DISA, NSA, etc.).
- ▶ **Escáner de puertos.** Herramientas para detectar el estado de los puertos de una máquina (y sus servicios) conectada a una red de comunicaciones. Detecta si un puerto está abierto, cerrado o protegido por un cortafuegos.
- ▶ **Footprint y fingerprint.** Técnicas que permiten, a través de herramientas, obtener información del sistema como la versión, parches, *service pack*, etc. (*fingerprint*); e información de DNS, infraestructura e información pública del sistema informático de la organización por auditar (*footprint*).
- ▶ **Pruebas de contraseñas mediante fuerza bruta y diccionarios.** Para analizar la fortaleza de las contraseñas probando todas las combinaciones posibles hasta encontrar aquella que permite el acceso. Fuerza bruta se basa en todas las combinaciones posibles con un set de caracteres definido y una longitud. Las técnicas de prueba basadas en diccionario utilizan como base del ataque un listado de palabras.

- ▶ **Análisis de vulnerabilidades** con herramientas como Nessus, Openvas, Netexpose, etc.

Entre los diferentes **tipos de sistemas** que podríamos auditar tenemos:

- ▶ Sistemas operativos Windows, Linux, etc.
- ▶ Bases de datos.
- ▶ Sistemas de almacenamiento.
- ▶ Entornos virtualizados.
- ▶ Sistemas en la nube.

Auditoría de seguridad en redes

En este tipo de auditorías se evalúa el **nivel de seguridad de la infraestructura de red corporativa** de una organización. Básicamente, consiste en el análisis del tráfico de red en busca de ataques y vulnerabilidades y configuraciones débiles de los equipos de red. El auditor debe revisar los controles sobre la infraestructura de red para:

- ▶ Procedimientos para diseñar y seleccionar la arquitectura de red.
- ▶ La topología y diseño de la red.
- ▶ Los componentes de red importantes y sus configuraciones.
- ▶ Redes de perímetro interconectadas.
- ▶ Usos de la red.
- ▶ Pasarelas a Internet.
- ▶ Operaciones del administrador y los operadores de red.

- ▶ Procedimientos y estándares relativos al diseño, mantenimiento, configuración y uso de red.

Cuando se realiza una **auditoría de infraestructura de red** deben tenerse en cuenta los siguientes aspectos:

- ▶ Revisión de dispositivos de hardware de red.
- ▶ Revisión de servidores de ficheros de los dispositivos de la red (p. ej.: servidor TFTP para la telefonía IP).
- ▶ Revisión de documentación de red.
- ▶ Revisión de armarios y cableado.
- ▶ Revisión de la instalación de los servidores.
- ▶ Revisión de accesos de usuario a la red.
- ▶ Revisión de contraseñas.
- ▶ Revisión de planes de seguridad.
- ▶ Revisión de informes de seguridad.
- ▶ Revisión de mecanismos de seguridad.
- ▶ Revisión de procedimientos de operación y seguridad de la red.
- ▶ Entrevistas al administrador y operadores de red.
- ▶ Entrevistas a los usuarios.

Entre las diferentes **pruebas** que se podrían realizar, tenemos:

- ▶ *Sniffing* de paquetes de red.
- ▶ *Man in the middle*.
- ▶ *ARP Poisoning*.

Pruebas de penetración

Antes de abordar cómo es la anatomía de un ataque, hay que diferenciar entre ataque **ético y malicioso**. La primera y más simple para diferenciar entre sombreros blancos y sombreros negros es la **autorización** (Engebretson, 2011). La autorización es el proceso de **obtener la aprobación** antes de conducir cualquier prueba o ataque. Una vez que se obtenga la autorización, tanto el probador como la empresa auditada deben ponerse de acuerdo sobre el alcance de la prueba.

Las pruebas de penetración o **test de intrusión** son parte de las actividades de lo que se denomina **auditoría de seguridad**, cuyo objetivo es la comprobación del nivel de seguridad para comprobar la capacidad de explotación de un sistema TIC. En la siguiente figura se muestra de forma completa las actividades que podrían comprender unas pruebas de este tipo.

RECOPILACIÓN DE LA INFORMACIÓN		EXPLOTACIÓN DE VULNERABILIDADES			POST-EXPLOTACIÓN		
Recopilación de la información	Mapa de red	Identificación de vulnerabilidades	Penetración	Obtener acceso y elevar privilegios	Listas adicionales	Comprometer usuarios o sitios remotos	Mantener el acceso
Pasiva	Identificar nodos	Identificar servicios vulnerables (vulnerabilidades conocidas)	Determinar exploit o herramienta	Observar acceso	Desarrollar ataques contra contraseñas	Comprometer comunicaciones	Ocultar pistas
Presencia en Internet	Determinar servicios activos	Análisis de vulnerabilidades	Probar exploit o herramienta de la prueba de concepto		Obtener contraseñas	Canales encubiertos	Ocultación de archivos
Análisis DNS	Identificar el perímetro de la red	Identificar vulnerabilidades no conocidas	Disenar exploit personalizado o herramienta	Elevar privilegios	Escudriñar el tráfico y analizarlo	Comprometer usuarios remotos	Borrado de registros
BBDD Trabajo	Investigar los Sistemas Operativos	Identificar vulnerabilidades de configuración	Utilizar exploit o herramienta contra el objetivo		Obtener cookies		Vencer la comprobación de integridad
Motores de búsqueda	Realizar war-dialing	Verificación de falsos positivos y falsos negativos	Comprobar o refutar la existencia de vulnerabilidades		Obtener direcciones de correo electrónico		Vencer antivirus
Sitios de encuesta	Enumeración de los nodos	Lista definitiva y recomendar las medidas inmediatas	Documentar los hallazgos		Identificar rutas y redes		Implementar Rookits
Estadística tiempo actividad	Analizar toda la información recopilada	Estimación probable del impacto			Mapear redes internas		
Redes P2P							
Redes Sociales							
Sitios clandestinos							
Grupos de noticias y e-mails							
Sitios de índices							
Páginas personales							
Activa							
Cuentas de correo. SMTP correos recibidos. SMTP correos rebotados. SMTP confirmaciones de lecturas. BGP. DNS - Transf. Prim. Secund. Servidor. DNS - Transf. Anular dirección. DNS - Direcciones. IPv6. Espejo del sitio web.							

Figura 8. Alcance de las pruebas de penetración. Fuente: elaboración propia.

Auditoría de redes wifi

Uno de los tipos de auditorías más demandados en la actualidad son los de las redes inalámbricas tipo wifi. El ataque a este tipo de redes es uno de los **principales objetivos** de los agentes maliciosos, dado que se puede lograr acceso a la red de las organizaciones desde el exterior al edificio sin requerir conexiones físicas directas.

Esto evita la seguridad de las protecciones de seguridad perimetral más difíciles de explotar, por lo que, para los atacantes, los dispositivos inalámbricos son un vector para mantener un acceso a largo plazo en un entorno objetivo.

Una auditoría de una red wifi consiste, básicamente, en analizar, verificar y comprobar los posibles problemas de seguridad con base en la auditoría de los diferentes controles de seguridad que una red inalámbrica debería tener implementados.

De forma genérica, estos controles de seguridad serían:

CONTROLES POR AUDITOR EN LAS REDES WIFI

Descubrimiento de dispositivos	Identificar las estaciones de comunicaciones y las relaciones entre ellas como base para empezar a evaluar los riesgos existentes.
<i>Fingerprinting</i>	Tras la identificación de los dispositivos con capacidades wifi dentro del área geográfica en donde se realiza el análisis, se debe extraer información relevante de cada dispositivo.
Pruebas sobre la autenticación	La elección de mecanismos seguros de autenticación es una de las bases principales para prevenir accesos no autorizados a redes wifi.
Cifrado de las comunicaciones	El cifrado usado en una red inalámbrica ayuda a proteger la confidencialidad y la integridad de la información transmitida en una red inalámbrica.
Configuración de la plataforma	Verificación de la correcta configuración de la plataforma. La presencia de dispositivos inalámbricos es suficiente por sí misma para crear potenciales agujeros de seguridad.
Pruebas de infraestructura	Verificar la correcta configuración de los puntos de acceso; se debe verificar periódicamente el funcionamiento y la protección de otros elementos que intervienen en su funcionamiento.
Pruebas de denegación de servicio	El objetivo de estas pruebas es identificar qué aspectos pueden ser explotados desde fuera del perímetro de la organización para denegar el servicio.
Gestión sobre directivas y normativas	Las desvinculaciones en las buenas prácticas o en la normativa establecida pueden provocar problemas de seguridad.
Pruebas sobre clientes inalámbricos	Estos controles engloban las verificaciones realizadas contra dispositivos de comunicaciones que actúan como cliente.
Pruebas sobre <i>hotspots</i> y portales cautivos	Pruebas orientadas a evaluar la protección de las redes de comunicaciones abiertas y portales que facilitan la conexión a Internet a los usuarios.

Figura 9. Controles por auditar en las redes wifi. Fuente: elaboración propia.

También existe la metodología OWISAM (Open Wireless Security Assessment Methodology). Unas pruebas importantes que realizar en este tipo de auditorías son:

- ▶ Escaneo de puntos de acceso (AP) falsos (*rogue Aps*).
- ▶ Prueba de conexión con un cliente *wireless* que haga uso de un servicio para el que no está autorizado.
- ▶ Prueba de conexión con un cliente *wireless* que utilice un método de encriptación no adecuado.
- ▶ Prueba de conexión con un cliente *wireless* que utilice un método de autenticación no adecuado.
- ▶ Prueba de un AP con una configuración de autenticación no adecuada.
- ▶ Prueba de un AP con una configuración sin autorización.

Auditoría de seguridad perimetral

El objetivo de este tipo de auditoría es verificar la correcta implantación de controles de acceso a nivel de red para evaluar su seguridad frente a intentos de acceso procedentes del exterior que tiene como propósito el robo de información por parte de agentes maliciosos como *hackers*, *malware*, etc.

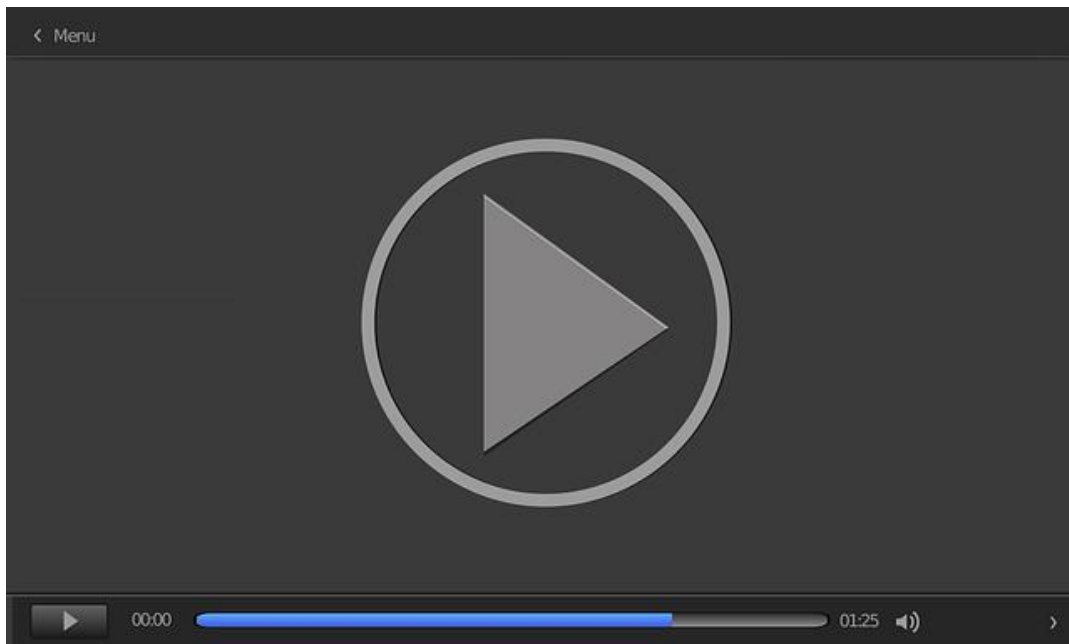
En este tipo de auditorías se busca comprometer las máquinas y la red corporativa mediante la simulación de diferentes tipos de ataque externo que evada las restricciones impuestas por los controles de seguridad del sistema de protección perimetral (DMZ).

El auditor analizará las posibles vías de entrada desde el exterior de la red corporativa hacia la DMZ y zonas internas como si de un atacante se tratase, detectando posibles vectores de acceso a la red de la organización y vulnerabilidades existentes. El alcance de la auditoría, de forma resumida, sería el siguiente:

- ▶ Auditorías de aplicaciones o servicios web y servicios que la organización exponga en el perímetro.
- ▶ Auditoría de sistemas de los servidores y equipos de la DMZ (por ejemplo, servidores DNS externos).
- ▶ Auditoría de sistemas de protección perimetral (*firewalls*, WAF, proxys reversos, etc.).
- ▶ Pruebas de penetración desde el exterior.

Auditoría de aplicaciones

Actualmente, dado el incremento que se está dando en los ataques en la capa de aplicación, es importante realizar auditorías de seguridad de los programas software o aplicaciones. En esta grabación (*Auditorías de seguridad de aplicaciones*) se explican los principales aspectos metodológicos de una auditoría de seguridad de una aplicación.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=3a70f0f9-fea1-429f-96c9-adf700cfeb4a>

Auditorías de dispositivos móviles

El objetivo de este tipo de auditorías es comprobar e identificar el riesgo asociado al uso corporativo de este tipo de dispositivos. En el tema siguiente se presenta una metodología de auditoría de dispositivos móviles, con sistema operativo Android, denominada: Open Android Security Assessment Methodology (OASAM).

Auditorías de denegación de servicio

Una auditoría de denegación de servicio (DOS o DDOS) tiene por objetivo **verificar** cómo se comportan las defensas y medidas reactivas de la red de una organización frente a un ataque de esta naturaleza.

El objetivo de este tipo de ataques no es conseguir acceso no autorizado para leer o modificar información, sino provocar la **inutilización o destrucción** de un activo. Esta particularidad genera que este tipo de ataques informáticos sea distinto al resto, por lo que la forma de afrontarlos también es diferente. La auditoría abarca una serie de pruebas que se incluyen en las siguientes cuatro categorías donde se engloban los diferentes vectores de ataque DOS/DDOS.

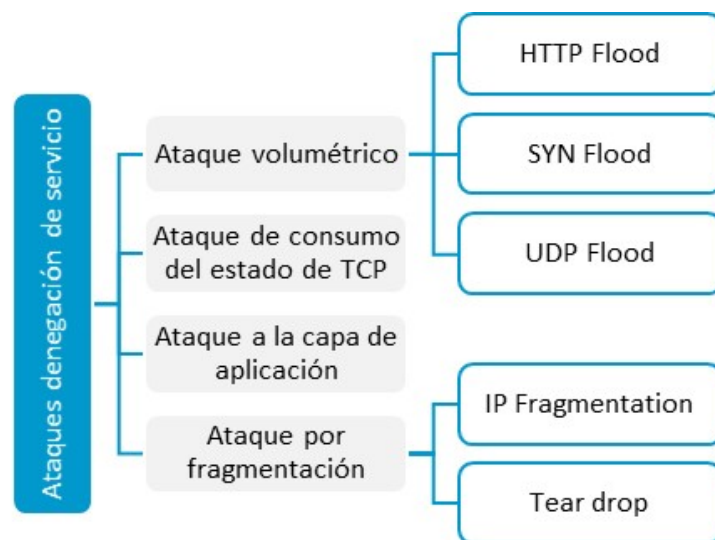


Figura 15. Ataques de denegación de servicio. Fuente: elaboración propia.

- **Ataque volumétrico.** Se basa en el consumo del ancho de banda de la red o el servicio. Esta categoría alberga los vectores de ataque comúnmente utilizados por usuarios malintencionados. Generalmente, estos ataques son conocidos como *flood* (inundación). Los protocolos más explotados son:
 - HTTP Flood.
 - SYN Flood.
 - UDP Flood.

- ▶ **Ataque por fragmentación.** Se basa en desbordar la capacidad de reensamblar los paquetes fragmentados aprovechándose del funcionamiento del protocolo IP. Esta categoría, junto con la desarrollada anteriormente, es la más explotada por usuarios malintencionados. Los vectores de ataque más utilizados son:
 - **IP Fragmentation.** Este vector de ataque se basa en el envío de una gran cantidad de paquetes TCP/IP correctamente fragmentados hacia el sistema objetivo de ralentizar su gestión.
 - **Tear drop.** Este vector de ataque se basa en el envío de una gran cantidad de paquetes TCP/IP fragmentados de forma errónea hacia el sistema con el objetivo de bloquear el sistema destino.
- ▶ **Ataque a la capa de aplicación.** Se basa en el consumo de los recursos de la aplicación para hacerlos inaccesibles a legítimos usuarios.
- ▶ **Ataque de consumo del estado de TCP.** Se basa en el consumo de tablas de estado presentes en los dispositivos de red de la infraestructura (balanceadores, firewalls, servidores de aplicación, etc.).

Las dos últimas categorías se basan en la explotación de vulnerabilidades disponibles en la aplicación y en los elementos de red. Entre las herramientas más utilizadas para realizar este tipo de ataque, tenemos:

- ▶ **HOIC (High Orbit Ion Cannon).** Esta herramienta de software libre permite realizar ataques volumétricos sobre el protocolo HTTP, conocido como HTTP Flood.
- ▶ **LOIC (Low Orbit Ion Cannon).** Esta herramienta de software libre permite realizar ataques volumétricos sobre el servicio de los protocolos HTTP, TCP y UDP; estos vectores de ataque son conocidos como HTTP Flood, TCP Flood y UDP Flood.
- ▶ **BanglaDOS.** Esta herramienta permite realizar ataques volumétricos sobre el servicio de los protocolos HTTP (HTTP Flood).

- **hping3.** A continuación, se indican los comandos a ejecutar para las diferentes pruebas:

- `hping3 -V-1-flood Sp 443 193.33.2.77 -> ICMP Flood.`
- `hping3 -V-2-flood -5 -p 443 193.33.2.77 -> UDP Flood.`
- `hping3 -V -frag-flood Sp 443 193.33.2.77 -> IP Fragmentation.`
- **Metasploit Pro.** Exploit disponible en `auxiliary/dos/tcp/synflood`.
- **Thor Hammer.** Esta herramienta permite realizar ataques volumétricos sobre el servicio los protocolos HTTP (HTTP Flood).

Figura 16. Comandos a ejecutar para las diferentes pruebas. Fuente: elaboración propia.

Auditorías de sistemas de VoIP

Actualmente, los **sistemas de VoIP** están sustituyendo de forma paulatina a los sistemas de telefonía tradicional. La mayoría de las amenazas que afectan hoy en día a este tipo de sistemas son heredadas de las capas de protocolos y software de base y hardware que soportan este tipo de sistemas. Por lo que le son de aplicación los tipos de auditorías de sistemas, pruebas de penetración, aplicaciones web, aplicaciones y denegación de servicio ya estudiados.

Sin embargo, este tipo de sistemas añade otro tipo de **amenazas nuevas** asociadas a los **protocolos de señalización y multimedia** de los sistemas de gestión de llamadas y a los dispositivos de usuarios de los sistemas VoIP. Las amenazas de las redes de telefonía IP se pueden clasificar en las categorías mostradas en la siguiente tabla:

CAPA	ATAQUES Y VULNERABILIDADES
Aplicaciones y protocolos de VoIP	► Fraudes. ► Autenticación. ► SPIT (SPAM). ► <i>Vishing (Phishing).</i> ► <i>Fuzzing.</i> ► <i>Floods (INVITE, REGISTER, etc.).</i> ► Secuestro de sesiones (<i>Hijacking</i>). ► Interceptación (<i>Eavesdropping</i>). ► Redirección de llamadas (<i>CALL redirection</i>). ► Reproducción de llamadas (<i>CALL replay</i>).

Tabla 9. Ataques sistemas VoIP. Fuente: elaboración propia.

Los **objetivos principales** de los ataques a los sistemas de VoIP pueden ser:

- ▶ Realizar denegaciones de servicio al sistema (DoS).
- ▶ Degradar la calidad de servicio o anularla.
- ▶ Robo de información clasificada o confidencial de las conversaciones.
- ▶ Robo de la información y los datos de llamadas.
- ▶ Ataques a la infraestructura que soporta el sistema telefónico VoIP como servidores, sistemas operativos, base de datos, servidores DNS, etc.

A continuación, se presentan algunas diferencias con respecto a las fases clásicas de una prueba de penetración:

Recolección de información. Una de las herramientas más utilizadas para recolección de información de VoIP es la *suit* de utilidades SIPVicious, que está formada por un conjunto de herramientas:

- ▶ **svmap:** escáner que enumera los dispositivos SIP que se encuentran en un rango de IP.
- ▶ **svwar:** identifica extensiones activas en una PBX.
- ▶ **svcrack:** craquea contraseñas en línea para PBX SIP.
- ▶ **svreport:** gestiona las sesiones y exportaciones de informes a diversos formatos.
- ▶ **svcrash:** utilidad para detener exploraciones no autorizadas con svwar y svcrack.

Para visitar la *suit* de utilidades SIPVicious ingresar a:

<https://github.com/EnableSecurity/sipvicious> y <https://www.enablesecurity.com/sipvicious/oss/>

Mapeo de red. NMAP incluye una base de datos que permite identificar dispositivos de VoIP (terminales protocolo SIP): `nmap -sU -p 5060 -sV X.X.X.X`

En este sentido es conveniente saber qué tipo de puertos y sus servicios asociados son los relativos a la VoIP:

Protocolo o servicios de base	Puertos
SIP	TCP/UDP 5060, TCP/UDP 5061.
IAX	TCP/UDP 4569.
RTP	UDP 1024-65535 (audio/vídeo), UDP 1024-65535 (control).
H.323	TCP/UDP 1718 (Discovery), TCP/UDP 1719 (RAS). TCP/UDP 1720 (H.323 setup), TCP/UDP 1731 (Audio Control), TCP/UDP 1024-65536 (H.245).
SCCP	TCP 2000-2001.
MySQL	TCP 3306 (archivos de configuración).
PostgreSQL	TCP 5432 (archivos de configuración).
HTTP (Administración)	TCP 80.
HTTPS	TCP 443.
Manager (Asterisk)	TCP 5038.
LDAP	TCP 389.
TFTP	UDP 69.
SMTP	TCP 25 (envíos correos de voz).
DNS	UDP 53.
SNMP	UDP 162.

Tabla 10. Puertos asociados a los protocolos y servicios asociados a VoIP. Fuente: elaboración propia.

Enumeración. Para poder realizar la enumeración, el atacante tendrá que realizar conexiones activas hacia los objetivos de una forma más específica. Esto le va a ser útil a un administrador para detectar este tipo de actividades. Una fase muy importante y necesaria para poder realizar la mayoría de los ataques es obtener los nombres de usuario y extensiones telefónicas de la organización objetivo del ataque.

Existen diversos métodos para obtener este tipo de datos:

- ▶ **Método REGISTER.** Se envía una petición del protocolo SIP de tipo REGISTER de un usuario que no existe o no se tiene las credenciales de autenticación al servidor de registro. Este responderá con un mensaje «**200 OK**» si es correcto o con un mensaje «**401**» si el usuario existe, o puede que conteste con un mensaje «**403**» por falta autenticación o, si el usuario no existe, responderá directamente con un mensaje «**Forbidden**». Con base en la diferencia de respuestas, este método se utiliza para enumerar y obtener un listado de usuarios válidos de la red VoIP.
- ▶ **Método INVITE.** Se envía una petición del protocolo SIP de tipo INVITE al servidor SIP respondiendo este con un mensaje «**401**» cuando se intenta llamar a un usuario inexistente. Si es correcto, hay que tener la precaución de que el terminal se puede registrar y realizar una llamada. Se puede utilizar **metaexploit** para escanear la red utilizando el módulo `auxiliary/scanner/sip/options`.
- ▶ **Método OPTION.** Es el método que menos ruido provoca en la red del objetivo. Se utiliza también para determinar mediante peticiones **OPTION** qué **codecs** soporta un determinado dispositivo SIP. El servidor contestará con un mensaje «**200 OK**» si el usuario existe o con un mensaje «**404 Not Found**» si no reconoce el usuario.
- ▶ **TFTP.** Gran parte de los dispositivos telefónicos utilizan este protocolo para obtener sus ficheros de configuración de un servidor **TFTP**. Este servicio es altamente inseguro, pues en la configuración de los dispositivos se encuentra información como: contraseñas, extensiones, usuarios, datos de la red, servidores, etc. Pasos:
 - Localizar el servidor TFTP en la red con la herramienta NMAP buscando direcciones con el puerto 69 UDP abierto.
 - Obtener los ficheros de configuración con herramientas como tftpbrute módulo de Metaexploit o TFTP-Bruteforce.

- ▶ **SNMP.** Este protocolo está activo en muchos de los dispositivos VoIP y permite obtener gran cantidad de información. Los pasos serían los siguientes:
 - Localizar dispositivos con soporte **SNMP** en el puerto 162 UDP. Usar las herramientas como **NMAP**, **SolarWind** o **SNMPSweep**.
 - Si se desconoce el **OID** del dispositivo, utilizar la herramienta **SolarWind MIB** para encontrarlo.
 - Con la herramienta **snmpwalk** y el **OID** del dispositivo es posible listar la mayoría de los aspectos de su configuración.
- ▶ **DHCP.** El protocolo DHCP se comunica con todos los dispositivos de una red enviando los paquetes a la dirección MAC de difusión **FF:FF:FF:FF:FF:FF**. Un agente malicioso puede monitorizar todos los pasos del protocolo y en un determinado punto de este, en concreto cuando el servidor acepta un paquete **DHCP ACK** con la configuración del cliente, previo a un paquete **REQUEST** solicitado por el mismo. En este punto es posible, con la herramienta **DHCP ACK Injector**, responder falseando la respuesta del servidor y estableciendo la configuración del cliente conforme a las necesidades del atacante.
- ▶ **DNS.** En sistemas VoIP interesa obtener información de los registros **SRV**, pues definen el número de host y el puerto de los servidores de servicios del protocolo SIP. Se define en el **RFC 2782** y su código de tipo es 33. Una herramienta que nos ayuda a obtener los datos anteriores es el módulo «**enum_dns**» de Metasploit, que pregunta en un dominio por información específica de este registro, necesario para el funcionamiento del protocolo SIP.

Herramientas que automatizan este proceso de enumeración, enfocados a sistemas VoIP son:

- ▶ **Netcat:** permite obtener información sobre el tipo de dispositivo.

- ▶ **Smap:** herramienta que permite identificar dispositivos SIP.
- ▶ **Sivus:** escáner de vulnerabilidades del protocolo IP. Genera peticiones SIP.
- ▶ **Nessus:** escáner de vulnerabilidades que identifica servicios y sistemas.
- ▶ **VolPAudit:** escáner de vulnerabilidades de VoIP.
- ▶ **Sipscan:** herramienta que automatiza el proceso de enumeración mediante el método REGISTER, INVITE u OPTION.

Al igual que en otras disciplinas de la seguridad como el **hacking ético** o el **análisis forense**, para VoIP se tiene una distribución en Linux, denominada [VIPER VAST](#), que contiene gran parte de las herramientas indicadas anteriormente.

9.6. Auditoría del acceso lógico

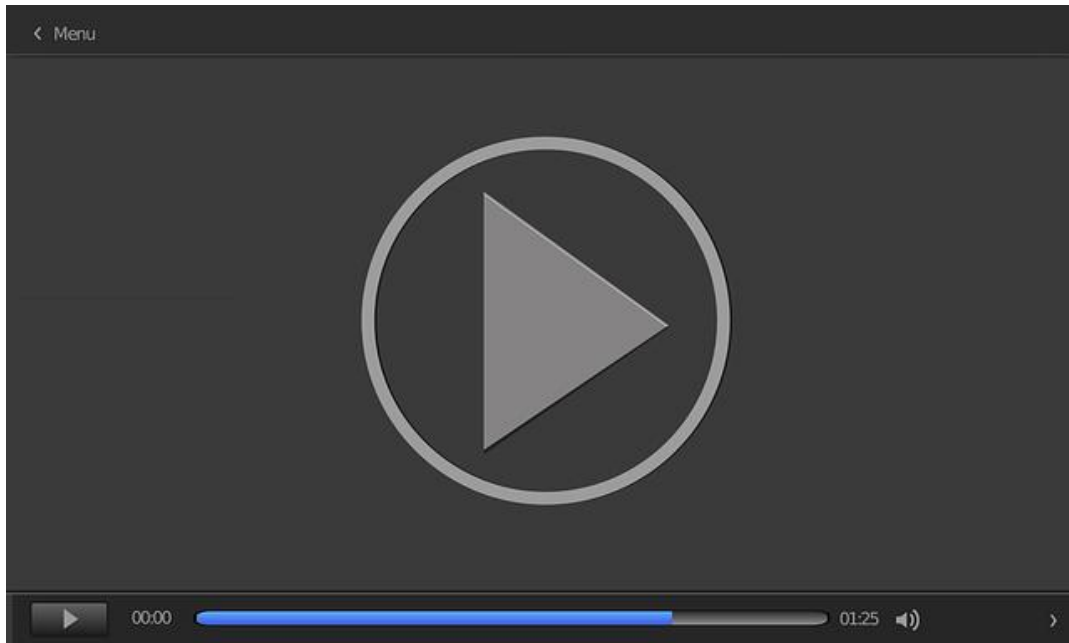
El acceso lógico constituye uno de los controles primarios de protección y gestión de los activos de información. Es importante, por lo tanto, evaluar la efectividad de estos controles para evitar futuras pérdidas debidas a las amenazas a las que están expuestos estos controles.

A la hora de auditar el acceso lógico a las redes y sistemas de una organización, el auditor debe (ISACA, 2019):

- ▶ Obtener una comprensión general de los **riesgos de seguridad** a los que se enfrentan los sistemas TIC de la organización.
- ▶ **Documentar y evaluar los controles sobre las rutas de acceso potencial** para ver si son adecuados, eficientes y efectivos.
- ▶ **Probar los controles sobre las rutas de acceso** para verificar si están funcionando y son efectivos.
- ▶ Evaluar el **ambiente de control de acceso** para determinar si se logran los objetivos de control analizando los resultados de las pruebas y otras evidencias de auditoría.
- ▶ Evaluar el **ambiente de seguridad** para determinar si es adecuado mediante la revisión de las políticas escritas, la observación de las prácticas y los procedimientos y la comparación con los estándares o las prácticas y los procedimientos de seguridad de otras organizaciones.

En la actualidad, una gran cantidad de organizaciones y usuarios sufren ataques debidos a que sus equipos no disponen de certificaciones de seguridad. Actualmente, la norma de referencia para certificar productos de seguridad es la ISO 15408, o Common Criteria (CC), que es una metodología muy costosa y pesada.

Ante tal situación, en orden de simplificar las auditorías de seguridad de los productos, varios países han desarrollado metodologías ligeras de certificación. En este vídeo (*Metodologías ligeras de auditorías de equipos y aplicaciones de seguridad*) se presentan las principales metodologías ligeras desarrolladas.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=6652b5cc-c573-43b4-9acf-adf700cfead2>

9.7. Referencias bibliográficas

Centro Criptológico Nacional. (2020). *Certificación TEMPEST*. <https://oc.ccn.cni.es/tipos-de-certificacion/certificacion-tempest>

Cooke, I. (2017). *IS audit basics: auditing mobile devices*. ISACA Journal, 6. https://www.isaca.org/Journal/archives/2017/Volume-6/Pages/auditing-mobile-devices-spanish.aspx?utm_referrer=

Davis, C., Schiller, M. y Wheeler, K. (2011). *IT auditing using controls to protect information assets*. McGraw-Hill.

Engelbreton, P. (2011). *The basics of hacking and penetration testing*. Elsevier.

INCIBE. (2019, agosto 1). *¿Ya tienes tu Plan de Recuperación ante Desastres* <https://www.incibe.es/protege-tu-empresa/blog/tienes-tu-plan-recuperacion-desastres>

ISACA. (2019). *CISA. Review manual*. 27.º ed. ISACA.

Writing a penetration testing report

Alharbi, M. A. (2010). *Writing a penetration testing report*. SANS Institute [Archivo PDF]. <https://www.sans.org/reading-room/whitepapers/bestprac/paper/33343>

Este artículo presenta una metodología para la redacción y confección de informes, resultados de unas pruebas de penetración, una estructura y un ejemplo.

Auditoría: principios y generalidades

Virtual Training Lteam. (2017, enero 30). *Auditoría Principios y Generalidades* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=i6pwCBrmN8Q>

Presentación que describe los principios y generalidades de una auditoría para un sistema de gestión según la estructura de Alto Nivel de las Normas ISO Anexo SL.

Auditoría de sistemas. Seguridad física de instalaciones de TI

Cristian Duarte Tubio. (2013, octubre 27). *Auditoría de Sistemas* [Vídeo]. YouTube.
<https://www.youtube.com/watch?v=f6s-TnG67Kg>

Presentación del alcance de las diferentes tareas que puede englobar una auditoría de seguridad física en una organización.

Monográfico: Nmap

Mifsud, E. (2012, junio 14). *MONOGRÁFICO: Zenmap - Nmap*. Observatorio tecnológico. <http://recursostic.educacion.es/observatorio/web/es/software/software-general/1050-zenmap?start=1>

En este artículo encontrarás una descripción de Nmap que ha sido diseñada para permitir a administradores de sistemas, y a usuarios curiosos en general, explorar y realizar auditorías de seguridad de redes para determinar qué servidores se encuentran activos y qué servicios ofrecen.

Tracking penetration test activities

Arey, J. (2020, febrero 24). *Tracking penetration test activities*. SANS Institute [Archivo PDF]. <https://www.sans.org/reading-room/whitepapers/monitoring/tracking-penetration-test-activities-39495>

La mayoría de los *pen testers* están obligados a seguir sus acciones durante la realización de una prueba de penetración, pero rara vez lo hacen con suficiente detalle para recrear todas sus actividades con precisión. El seguimiento de las actividades de las pruebas es un reto que a menudo se pasa por alto. Afortunadamente, hay mecanismos de registro automático en la mayoría de los sistemas de *pen test*. La personalización de las configuraciones de registro y la incorporación de herramientas de vigilancia, como *auditd*, pueden ayudar a registrar automáticamente las actividades de prueba en los *tests* de penetración basados en Linux. Este registro adicional permite un seguimiento preciso con suficiente detalle para que un auditor pueda determinar con exactitud qué acciones realizó en unas pruebas de penetración.

1. Una prueba de intrusión es:
 - A. Solo un análisis de vulnerabilidades.
 - B. Un tipo de *hacking* ético.
 - C. Habitualmente de caja blanca, gris y negra.
 - D. No es un pen test.

2. ¿Qué fases habitualmente se utilizan en una auditoría técnica de seguridad (*test* de intrusión)?
 - A. La recopilación de información del sistema por auditar, análisis de vulnerabilidades, explotación/ataque de las vulnerabilidades detectadas y realización de informe técnico y ejecutivo.
 - B. La recopilación de información del sistema por auditar, explotación/ataque de las vulnerabilidades detectadas. Nunca se realiza informe.
 - C. Explotación/ataque de las vulnerabilidades detectadas y realización de informe técnico.
 - D. La recopilación de información del sistema por auditar y realización de informe técnico y ejecutivo.

3. Las pruebas de fuerza bruta se basan en:
 - A. Utilización de ficheros con palabras en diferentes idiomas para averiguar por ensayo/error los usuarios y/o contraseña en las pantallas de *log in*.
 - B. Utilización de todas las combinaciones posibles con un *set* de caracteres definido y una longitud.
 - C. Lo que se denominan *rainbow tables*.
 - D. Ninguna de las respuestas anteriores es correcta.

4. ¿Qué es el *sniffing* de paquetes de red?
- A. Es la técnica para inyectar paquetes en redes inalámbricas.
 - B. Es la técnica pasiva que permite la captura de datos (paquetes de datos) que se distribuyen en una red.
 - C. Es la técnica por la que se intercepta y modifica la información en una red.
 - D. Es la técnica por la que consigues el control de administración en las máquinas servidoras.
5. ¿A qué tipo de pruebas de penetración pertenece la siguiente afirmación? «Se simulan ataques reales, pero conociendo de antemano gran parte de la información técnica. Se tiene un conocimiento limitado de los activos y las defensas que los protegen».
- A. Caja blanca.
 - B. Caja negra.
 - C. *Reversal*.
 - D. Caja gris.
- 6.
- A. Analizar la funcionalidad de los sistemas de información de una organización y sus servicios realizando una batería de pruebas planificadas.
 - B. Analizar la seguridad implantada en los sistemas de información y telecomunicaciones de una organización y sus servicios realizando una batería de pruebas planificadas que simulen el comportamiento de un atacante.
 - C. Verificar los sistemas de información de una organización y sus servicios realizando una batería de pruebas planificadas.
 - D. Todas las anteriores.

7.

- A. Auditoría de seguridad de sistemas.
- B. Auditoría de redes *wifi*.
- C. Auditoría de aplicaciones web.
- D. Todas las anteriores son auditorías técnicas de seguridad.

8.

- A. Del entorno, defectos de las aplicaciones, causadas por las personas de forma accidental y causadas por las personas de forma deliberada.
- B. De origen natural, del entorno, defectos de las aplicaciones, causadas por las personas de forma accidental y causadas por las personas de forma deliberada.
- C. De origen natural, del entorno, defectos de las aplicaciones y causadas por las personas de forma deliberada.
- D. De origen natural, del entorno, defectos de las aplicaciones y causadas por las personas de forma accidental.

9.

- A. Se conoce toda información, como direcciones IP, pero no de la infraestructura.
- B. No se conoce nada de información.
- C. Se conoce alguna información, como direcciones IP, pero no de la infraestructura.
- D. Todas las anteriores.

10.

- A. Incluyen como anexo el informe ejecutivo.
- B. Describen con detalle técnico las fases de la auditoría de seguridad que realizan.
- C. No necesitan definir el objetivo y el alcance de la auditoría.
- D. Ninguna de las anteriores.

Desarrollo Seguro de Software y Auditoría de la
Ciberseguridad

Tema 10. Auditorías de cumplimiento y metodologías de auditoría

Índice

Esquema

Ideas clave

- 10.1. Introducción y objetivos
- 10.2. Auditorías de cumplimiento
- 10.3. Metodologías de auditorías de seguridad
- 10.4. Referencias bibliográficas

A fondo

PTES

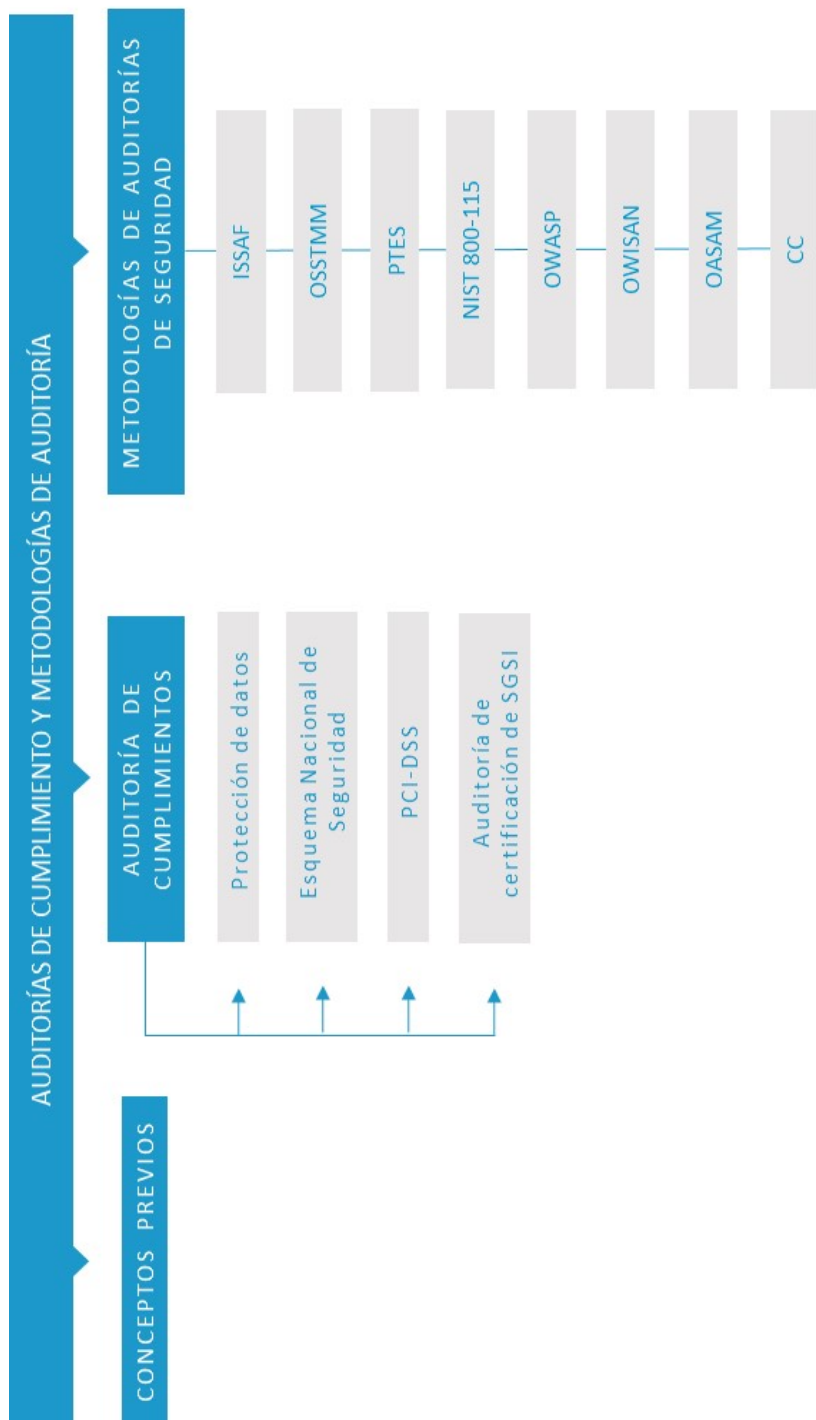
OWISAM: Open Wireless Security Assessment
Methodology

OSSTMM 3.0. Auditoría

Common Criteria en 5 minutos

PCI (industria de tarjetas de pago)

Test



10.1. Introducción y objetivos

La auditoría como actividad genérica se define como la **revisión de la gestión** de una organización en un ámbito determinado, con el propósito de verificar y evaluar si esta se ajusta a la normativa o a las políticas establecidas.

Siguiendo esta línea, la auditoría de seguridad de la información se define como la revisión e inspección **independiente** de las medidas de seguridad implantadas para valorar su idoneidad y eficacia al verificar y asegurar su conformidad con las políticas y los procedimientos establecidos y recomendando los cambios necesarios.

La seguridad de la información no es una actividad estática, sino dinámica, y la auditoría de dicha seguridad es una actividad que se repite periódicamente para poder ofrecer un estado fiable del marco de seguridad establecido en la organización.

Las auditorías de seguridad en función del objetivo se pueden clasificar de la siguiente manera:

- ▶ **De cumplimiento:** tiene como objetivo evaluar la conformidad de los entornos auditados con respecto a una normativa de seguridad de la información establecida. Por ejemplo, el estándar de seguridad ISO 27001, RGPD o de las propias políticas y procedimientos internos de seguridad de la organización.
- ▶ **Técnicas:** auditorías o revisiones de seguridad técnica cuyo alcance está acotado a los sistemas informáticos, o de telecomunicaciones, y son objeto de la revisión.

En el presente tema se estudian las auditorías de cumplimientos y las diferentes metodologías que existen en la actualidad para la realización de auditorías técnicas.

Los **objetivos** son los siguientes:

- ▶ Definir y explicar los conceptos relacionados con las auditorías de seguridad de cumplimiento de seguridad de la información.
- ▶ Estudiar las diferentes metodologías de realización de auditorías de seguridad técnicas existentes en la actualidad.
- ▶ Aprender a seleccionar qué metodología de auditoría de seguridad es la más conveniente para la realización de un tipo concreto de auditoría.
- ▶ Obtener el conocimiento necesario para poder realizar los diferentes tipos de auditorías que comprende la seguridad de la información.

10.2. Auditorías de cumplimiento

Las auditorías en su dimensión de cumplimiento normativo tienen por objetivo identificar riesgos de incumplimiento en la normativa interna, legislación, requisitos sectoriales/regulatorios y estándares, entre otros, con la finalidad de asegurar su **cumplimiento mandatorio**, así como la confidencialidad, integridad y disponibilidad de la información corporativa. En este apartado se estudiarán las siguientes auditorías de cumplimiento:

- ▶ Protección de datos.
- ▶ Esquema Nacional de Seguridad.
- ▶ PCI-DSS.
- ▶ Auditoría de certificación del SGSI.

Protección de datos

Una auditoría de protección de datos tiene por objeto **comprobar y adecuar** el nivel de seguridad de los datos personales que se gestionan en una organización con respecto a lo dispuesto en la Ley de Protección de Datos (LOPD) y el reglamento que la desarrolla. Están obligadas a realizar este tipo de auditoría todas aquellas organizaciones que, al tratar datos de carácter personal, traten, también, datos de alguno de los siguientes tipos:

- ▶ Infracciones, o sanciones, administrativas o penales (por ejemplo, multas de tráfico, sanciones de Hacienda, etc.).
- ▶ Datos que permitan obtener una evaluación de la personalidad del individuo (por ejemplo, las encuestas que realizan las empresas, según los casos).

- ▶ Los referentes a la ideología, religión, creencias, origen racial, salud o vida sexual (por ejemplo, los partes de baja o de alta del trabajador por motivos de salud, los datos médicos, etc.).

En España está en vigor el **Reglamento 2016/679** del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (RGPD). El cumplimiento en materia de protección de datos de una organización transmite a terceros la confianza necesaria de las relaciones de la organización con los **clientes**. En el marco de este reglamento, la auditoría estaría compuesta de las dos siguientes partes:

- ▶ **Auditoría de seguridad.** Consiste, básicamente, en la verificación del análisis de riesgos que hubiera llevado a cabo la entidad, y la valoración actualizada de las amenazas existentes y controles implantados para mitigarlos. Incluye, también, un análisis de vulnerabilidades. Esta fase estaría más enfocada en auditar sobre todo lo estipulado en los siguientes artículos del RGPD (Ramos, 2016):
 - **Seguridad y violaciones de seguridad(Artículos 32 a 34).**
 - **Evaluación de impacto relativa a la protección de datos(Artículo 35).**
 - **Protección de datos desde el diseño(Artículo 25).**
 - **Seguridad del procesamiento(Artículo 32).** Amplía los requisitos de seguridad de las aplicaciones, centrándose en la confidencialidad, la integridad y la disponibilidad.

- ▶ **Auditoría de cumplimiento.** Abarca muchos aspectos jurídicos, por lo que será necesario un equipo de auditores con un perfil mixto (técnicos y especialistas en derecho) con experiencia en protección de datos. Esta fase estaría más enfocada en auditar sobre todo lo estipulado en los siguientes puntos (Ramos, 2016):
 - **Principios (Artículos 5 a 11).** En el artículo 9 se tiene que verificar, sobre todo, la existencia de categorías especiales de datos personales.
 - **Derechos del interesado (Artículos 12 a 23, y 26).** Verificar si se han tomado las medidas oportunas, los casos que se han producido y tratamiento realizado.
 - **Perfiles (Artículos 21 y 22).** Verificar si se están elaborando perfiles y si no procediera o no se cumplen las condiciones.
 - **Responsable del tratamiento (Artículo 24, 28 y 29).** Verificar si están cumpliendo las obligaciones incluidas en el articulado.
 - **Representantes de responsables** o encargados del tratamiento no establecidos en la Unión (**Artículo 27**).
 - **Delegado de protección de datos (DPD) (Artículos 37 a 39).** Designación, la posición dentro de la organización si es interno, y el cumplimiento de sus funciones.
 - **Transferencias de datos a terceros**, países u organizaciones internacionales (**Artículos 44 y 49**).

En la siguiente figura se muestran las fases de este tipo de auditoría:

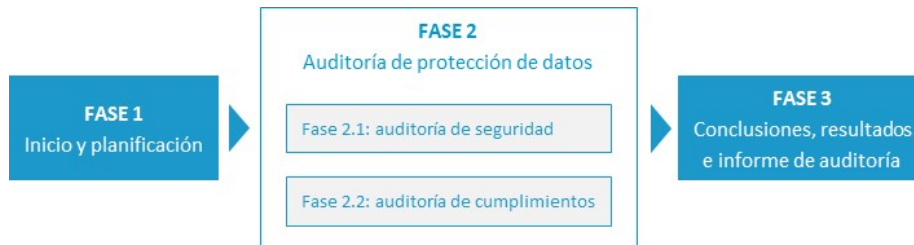


Figura 1. Auditoría protección de datos. Fuente: elaboración propia.

Como métodos de obtención de evidencias tendremos las **revisiones** de la documentación, entrevistas, inspecciones y proveimientos analíticos para la parte técnica. En algún caso se tendrán que realizar muestreos, y las muestras deberán ser representativas de forma que las conclusiones obtenidas sean las mismas que si se hubiera auditado el global.

Por último, **el informe de auditoría** recogerá la adecuación de las medidas y controles a lo dispuesto en el RGPD. Se incluirán las deficiencias, sus medidas correctoras y un plan de acción de resolución.

Esquema Nacional de Seguridad

El **Real Decreto 311/2022**, de 3 de mayo, por el que se desarrolla el Esquema Nacional de Seguridad (ENS) establece los principios básicos y requisitos mínimos, así como las medidas de protección que implantar en los sistemas de información de la Administración de España.

Su objeto es establecer la **política de seguridad** de la utilización de medios electrónicos. Está constituido por unos principios básicos y requisitos mínimos que permiten una protección adecuada de la información. En la Guía de Seguridad de las TIC CCN-STIC 801. Esquema Nacional de Seguridad Responsabilidades y Funciones (Centro Criptológico Nacional, 2019) se establece un proceso de gestión con respecto a la siguiente figura:

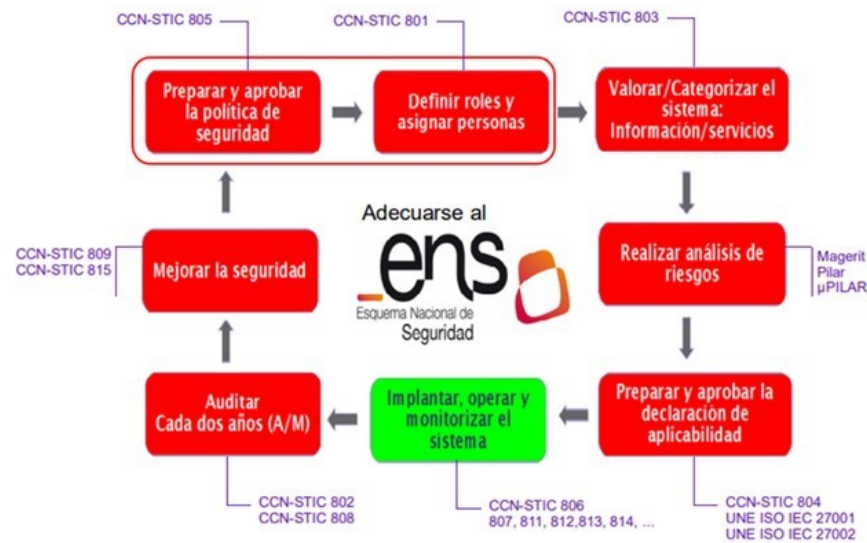


Figura 2. Proceso de gestión. Fuente: Ministerio de Asuntos Económicos y Transformación Digital, S. f.

El ENS establece 73 medidas de seguridad conforme a la siguiente clasificación:

- ▶ **Marco organizativo.** Está constituido por un conjunto de medidas relacionadas con la organización global de la seguridad.
- ▶ **Marco operacional.** Está constituido por las medidas a tomar para proteger la operación del sistema como conjunto integral de componentes para un fin.
- ▶ **Medidas de protección.** Se centrarán en activos concretos, según su naturaleza, con el nivel requerido en cada dimensión de seguridad.



Figura 3. Medidas de protección del ENS. Fuente: Ramiro, 2022.

Los elementos principales del ENS son:

- ▶ **Principios básicos** para tener en cuenta en las decisiones en materia de seguridad.
- ▶ **Requisitos mínimos** que permitan una protección adecuada de la información.

- ▶ **Categorización de los sistemas.** Se clasifican en bajo, medio o alto en función de la naturaleza de la información, de los servicios que se protegen y de los riesgos a los que están expuestos.
- ▶ **Auditoría de la seguridad** que verifique el cumplimiento de las medidas de seguridad aplicables por el ENS.
- ▶ **Respuesta a los incidentes de seguridad.**
- ▶ **Certificación**, como aspecto que considerar al adquirir los productos de seguridad.

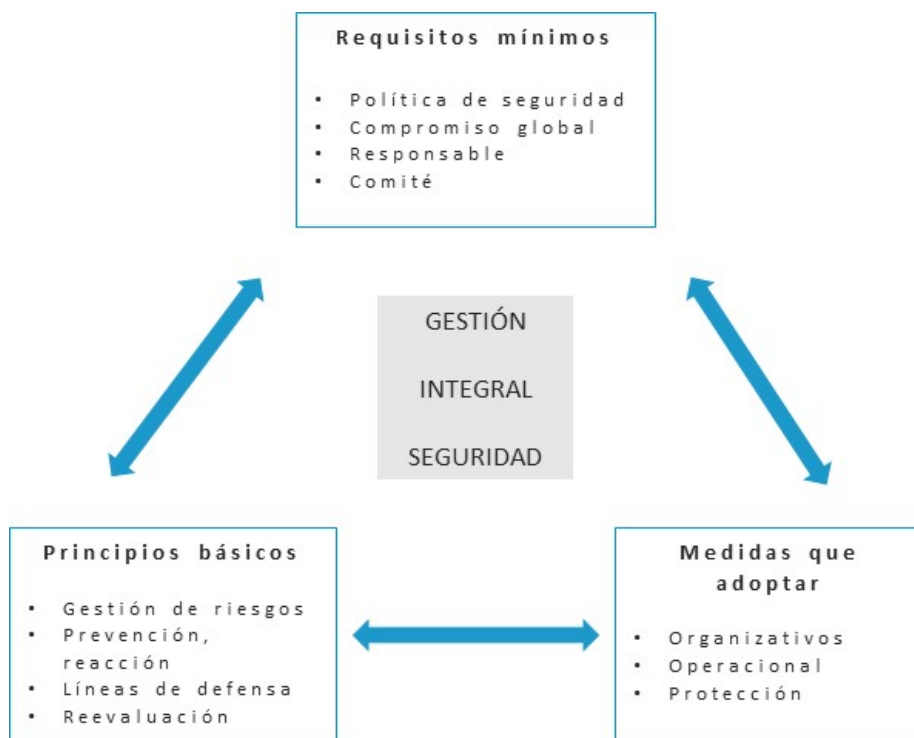


Figura 4. Elementos principales del ENS. Fuente: elaboración propia.

La auditoría del ENS se encuentra regulada en el artículo 34 y el anexo III del RD 3/2010 del ENS. Asimismo, se desarrolla en las siguientes guías de la serie 800:

- ▶ **CCN-STIC 802. Guía de auditoría** (Centro Criptológico Nacional, 2017a): esta guía de auditoría se encuadra dentro de lo previsto en el artículo 42 de la Ley 11/2007, de

22 de junio, de acceso electrónico de los ciudadanos a los Servicios Públicos, y de los requisitos del artículo 34 (Auditoría de la seguridad), y del Anexo III (Auditoría de la Seguridad) del Real Decreto 311/2022 de 3 de mayo.

- ▶ **CCN-STIC 808. Guía de verificación del cumplimiento de las medidas en el Esquema Nacional de Seguridad** (Centro Criptológico Nacional, 2022): guía para auditar los requisitos del Esquema Nacional de Seguridad para un sistema.

Los objetivos de este tipo de auditoría, de acuerdo con el Centro Criptológico Nacional (2017a), son:

- ▶ Dar **cumplimiento** a lo establecido en el RD 3/2010 y verificar el cumplimiento de los requisitos establecidos en los capítulos II y III y en los Anexos I y II del ENS.
- ▶ Emitir una **opinión independiente y objetiva** del cumplimiento del ENS de tal forma que permita a los responsables correspondientes tomar las medidas oportunas para subsanar las deficiencias identificadas.
- ▶ Posibilitar la **obtención** de la correspondiente Certificación de Conformidad.
- ▶ Sustentar la **confianza** que merece el sistema auditado sobre el nivel de seguridad implantado.
- ▶ Calibrar la capacidad del sistema para garantizar la integridad, disponibilidad, autenticidad, confidencialidad y trazabilidad de los servicios prestados y la información tratada, almacenada o transmitida.

Hay que realizar la auditoría de la seguridad periódicamente cada dos años para sistemas de categoría media o alta. La guía CCN-STIC 808 proporciona una completa lista de comprobación para la realización de la auditoría. En la siguiente figura se incluye un ejemplo de verificación de la idoneidad de los controles del sistema.

Aptdo.	Categoría - Dimensiones	Requisito	Aplicabilidad - Auditado	Comentarios
op	MARCO OPERACIONAL			
op.pl	PLANIFICACIÓN			
op.pl.1	Análisis de riesgos			
	Básica - D, A, I, C, T	1.- ¿Dispone de un análisis de riesgos, al menos, informal? Evidencia: Dispone de un documento aprobado por la Dirección en el que se ha realizado una exposición textual en lenguaje natural del análisis de riesgos. Dicho documento no tiene más de un año desde su aprobación. Existe un procedimiento para la revisión y aprobación regular, al menos anualmente, del análisis de riesgos. Respecto a dicho análisis de riesgos: 1.1.- ¿Identifica los activos más valiosos del sistema? Evidencia: En el documento se identifican los servicios que presta la organización y la información que maneja en referencia al cumplimiento de la Ley 11/2007 (p.ej: servicio telemático de tramitación de expedientes, etc.), así como los elementos en los que se sustentan (p.ej: servidores, línea de comunicaciones, aire acondicionado del CPD, oficinas, etc.). 1.2.- ¿Identifica las amenazas más probables? Evidencia: En el documento se identifican las amenazas más probables (p.ej: incendio, robo, virus informático, ataque informático, etc.). 1.3.- ¿Identifica las salvaguardas que protegen de dichas amenazas? Evidencia: En el documento se identifican las salvaguardas de que se disponen para mitigar las amenazas identificadas (p.ej: extintor, puerta con cerradura, antivirus, cortafuegos, etc.). 1.4.- ¿Identifica los principales riesgos residuales? Evidencia: En el documento se identifican las amenazas para las que no existen salvaguardas, o aquellas para las que el grado de protección actual no es el suficiente (p.ej: fuga de información en un soporte USB, etc.). Consultar guías: Criterios de seguridad Capítulo 5	Aplica: ☐ SI Lo audito: ☐ SI	Registros: ☐ Documento: ☐ Registro: Observaciones auditoría:

Figura 5. Guía de verificación de los controles. Fuente: Centro Criptológico Nacional, 2022.

En la guía CCN-STIC 802 se establece una metodología para la realización de la auditoría conforme a los siguientes pasos:

FASES DE AUDITORÍA DEL ENS

PLANIFICACIÓN PRELIMINAR	- Definir el objetivo y alcance de la auditoría. - Identificación de las áreas y personas clave. - Solicitud de información. - Establecer equipo de auditoría.
PROGRAMA DE AUDITORÍA	- Tipos de pruebas que realizar. - Planificación de la auditoría. - Entregable: programa de auditoría.
REVISIÓN Y PRUEBAS	- Revisión de la documentación. - Realización de entrevistas (actas). - Realización de pruebas. - Observaciones presenciales.
CONCLUSIONES DE LA AUDITORÍA	- Elaboración de la presentación preliminar de resultados. - Informe de auditoría: grado cumplimiento con el ENS y acciones correctoras.

Tabla 1. Fases auditoría del ENS. Fuente: elaboración propia.

El informe de auditoría deberá dictaminar sobre el grado de cumplimiento con respecto al ENS, identificar sus deficiencias y sugerir las posibles medidas correctoras o complementarias necesarias, así como las recomendaciones que se consideren oportunas. Se presentarán al responsable del sistema y al responsable de seguridad competentes. Estos informes serán analizados por este último, que expondrá sus conclusiones al responsable del sistema para que adopte las medidas correctoras adecuadas.

PCI DSS

Las **normas de seguridad de los datos de la industria de las tarjetas de pago** (PCI DSS) incluye un conjunto de requisitos de referencia que tiene por objetivo proteger los datos del titular de la tarjeta y de las cuentas. Conforman un conjunto de medidas de protección desarrolladas por las principales empresas de tarjetas de pago.

Además, acorde a la legislación de protección de datos en vigor, se puede requerir la protección específica de la información de identificación personal u otros elementos de datos (por ejemplo, el nombre del titular de tarjeta).

Como parte de sus acuerdos con las empresas de tarjetas, los comerciantes y otros tipos de empresas que gestionan datos de tarjetas pueden ser multados si no cumplen los requisitos de cumplimiento normativo de PCI DDS.

Se aplica **obligatoriamente** a las empresas que almacenan, procesan o transmiten datos de tarjetas de pago, entre las que se incluyen:

- ▶ Comerciantes.
- ▶ Procesadores.

- ▶ Adquirentes.
- ▶ Entidades emisoras.
- ▶ Proveedores de servicios de pagos como pasarelas de pago, centros autorizadores, servicios gestionados, infraestructura tecnológica, desarrollo de software, etc.

Los datos concretos que se deben proteger son los siguientes (PCI Security Standards Council, 2013):

- ▶ Datos del titular de la tarjeta (CHD).
- ▶ Datos confidenciales de autenticación (SAD).

En la siguiente tabla se incluye una definición más detallada de los datos anteriores:

DATOS DE CUENTAS	
Los datos de los titulares de las tarjetas incluyen:	Los datos confidenciales de autenticación incluyen:
- Número de cuenta principal (PAN). - Nombre del titular de la tarjeta. - Fecha de vencimiento. - Código de servicio.	- Contenido completo de la pista (datos de la banda magnética o datos equivalentes que están en un chip). - CAV2/CVC2/CCV2/CID. - PIN/bloqueos de PIN.

Tabla 2. Datos de las cuentas a proteger. Fuente: adaptado de PCI Security Standards Council, 2022.



Figura 6. Tipos de datos en una tarjeta de pago. Fuente: Acosta (2019).

No se deben almacenar los datos confidenciales de autenticación después de la autorización, incluso si están cifrados. El número primario de cuenta (*primary account number*, PAN) siempre debe ser protegido durante su **procesamiento**, transmisión o almacenamiento. Si se almacena con otros elementos de los datos del titular de la tarjeta, únicamente el PAN debe ser ilegible.

Los requisitos que auditar con esta norma son los presentados en la siguiente tabla (PCI Security Standards Council, 2022):

Objetivos de control	Requisitos (controles)
Desarrollar y mantener redes y sistemas seguros	- Instale y mantenga una configuración de <i>firewall</i> para proteger los datos del titular de la tarjeta. - No usar los valores predeterminados suministrados por el proveedor para las contraseñas del sistema y otros parámetros de seguridad.
Proteger los datos del titular de la tarjeta	- Proteja los datos del titular de la tarjeta que fueron almacenados. - Cifrar la transmisión de los datos del titular de la tarjeta en las redes públicas abiertas.
Mantener un programa de administración de vulnerabilidad	- Proteger todos los sistemas contra <i>malware</i> y actualizar los programas o los <i>softwares</i> antivirus regularmente. - Desarrollar y mantener los sistemas y las aplicaciones seguros.
Implementar medidas sólidas de control de acceso	- Restringir el acceso a los datos del titular de la tarjeta según la necesidad de saber que tenga la empresa. - Identificar y autenticar el acceso a los componentes del sistema. - Restringir el acceso físico a los datos del titular de la tarjeta.
Supervisar y evaluar las redes con regularidad	- Rastree y supervise todos los accesos a los recursos de red y a los datos del titular de la tarjeta. - Probar periódicamente los sistemas y procesos de seguridad.
Mantener una política de seguridad de información	Mantener una política que aborde la seguridad de la información para todo el personal.

Tabla 3. Ejemplo de lista de comprobación de requisitos. Fuente: adaptado de PCI Security Standards Council, 2022.

Dada la complejidad de estos requisitos, la norma proporciona una lista de comprobación de cumplimiento de alto nivel que puede ser de utilidad a la hora de la realización de una auditoría. En la siguiente tabla se muestra un ejemplo.

Requisitos A3	Procedimientos de prueba	Guía
A3. 1. Implementar un programa de cumplimiento de la PCI DSS		
A3.1.1 La gerencia ejecutiva deberá y establecerá la responsabilidad de la protección de los datos del titular de la tarjeta y un programa de cumplimiento de la PCI DSS para incluir: ▪ Responsabilidad general de mantener el cumplimiento de la PCI DSS. ▪ La definición de un estatuto para el programa de cumplimiento de la PCI DSS. ▪ Proporcionar actualizaciones a la gerencia y a la junta directiva sobre las iniciativas y los problemas de cumplimiento de la PCI DSS, incluidas las actividades de remediación, al menos anualmente.	A3.1.1.a Revise la documentación para verificar que la gerencia ha asignado la responsabilidad general de mantener el cumplimiento de la PCI DSS de la entidad. A3.1.1.b Revise el estatuto de la PCI DSS de la empresa para verificar que se describen las condiciones en las que se organiza el programa de cumplimiento de la PCI DSS. A3.1.1.c Revise las actas de reuniones y/o presentaciones de la gerencia ejecutiva y la junta directiva para garantizar que las iniciativas de cumplimiento de la PCI DSS y las actividades de remediación se comunican al menos anualmente.	La asignación de la gerencia ejecutiva de las responsabilidades de cumplimiento de la PCI DSS garantiza la visibilidad a nivel ejecutivo en el programa de cumplimiento de la PCI DSS y permite la oportunidad de hacer preguntas adecuadas para determinar la eficacia del programa e influir en las prioridades estratégicas. La responsabilidad general del programa de cumplimiento de la PCI DSS puede ser asignada a los roles individuales y/o a las unidades de negocio dentro de la organización.
Referencia de la PCI DSS: Requisito 12.		

Tabla 4. Ejemplo de lista de comprobación de requisitos. Fuente: adaptado de PCI Security Standards Council, 2022.

El **alcance del entorno** (*cardholder data environment, CDE*), que auditar con respecto a esta norma, está compuesto por personas, procesos y tecnologías que procesan, almacenan y transmiten datos de titulares de tarjetas o datos confidenciales de autenticación.

Entre los beneficios que obtiene una organización al certificarse con esta norma tenemos:

- ▶ Disminución del riesgo tanto tecnológico como reputacional.
- ▶ Obtención de una certificación con respecto a la norma PCI DSS.
- ▶ Aumento de la confianza de los clientes.
- ▶ Mejora de la operatividad de la organización debido al cumplimiento normativo.
- ▶ Facilitar a los clientes la forma de pago.

El proceso de la realización de una auditoría conforme a la norma PCI DSS incluye la realización de los siguientes pasos:



Figura 10. Pasos auditoría PCI DSS. Fuente: elaboración propia.

- ▶ **Confirmar el alcance de la evaluación.**
- ▶ **Auditoría del entorno** según los procedimientos de pruebas de cada requisito.
 - Análisis de documentación relativa a la implantación de la norma.
 - Análisis del entorno de cumplimiento para asegurar que el alcance es el determinado en la validación posterior del cumplimiento de la norma.

- Revisión de los flujos de datos relativos a las tarjetas de crédito.
- Revisión de los activos de información (primarios y secundarios) que intervienen en los procesos de transmisión, procesamiento y almacenamiento de datos de tarjetas de créditos y las medidas de seguridad implementadas.
- Revisión de las relaciones con terceras partes (proveedores de servicios) y cómo influyen en el cumplimiento de la norma.
- Pruebas de cumplimiento basadas en muestreo.
- Entrevistas con el personal de empresa cliente.
- ▶ Completar el **informe correspondiente de la evaluación**. El cumplimiento con PCI DSS se puede demostrar mediante dos formas:
 - Cuestionario de autoevaluación (SAQ).
 - Informe sobre cumplimiento (ROC).

Reporte de cumplimiento de PCI DSS	Cuestionario de autoevaluación <i>Self-Assessment Questionnaire (SAQ)</i>	Evaluación formal de cumplimiento
¿Quién debe usarlo?	Comercios y proveedores de servicio con base en las categorizaciones de cada marca y en función de la cantidad de transacciones anuales procesadas (nivel 2, nivel 3 y nivel 4)	Comercios, proveedores de servicio, adquirientes, emisores y otras entidades financieras catalogadas como nivel 1 por cada programa de las marcas.
Tipos de plantillas	Existen 9 tipos de plantilla de SAQ dependiendo del canal de pago empleado: SAQ A, SAQ A-EP, SAQ B, SAQ B-IP, SAQ C, SAQ C-VT, SAQ P2PE, SAQ D para comerciantes y SAQ D para proveedores de servicios	Se debe emplear la plantilla del documento de Reporte de Cumplimiento (<i>Report on Compliance – RoC</i>) por parte del asesor QSA
Documentación a presentar	• Cuestionario de autoevaluación (SAQ) aplicable • Declaración de Cumplimiento (<i>Attestation on Compliance – AoC</i>)	• Reporte de Cumplimiento (<i>Report on Compliance – RoC</i>) • Declaración de Cumplimiento (<i>Attestation on Compliance – AoC</i>)
¿Requiere de un Asesor Cualificado de PCI DSS (<i>Qualified Security Assessor - QSA</i>)?	No es obligatorio. No obstante, tanto las marcas de pago como los adquirientes pueden exigirlo de forma discrecional.	Si. Las evaluaciones formales de cumplimiento de PCI DSS deben ser ejecutadas obligatoriamente por un QSA.
¿Requiere de escaneos de vulnerabilidades ejecutados por un Proveedor Aprobado de Escaneo (<i>Approved Scanning Vendor - ASV</i>)?	Depende del tipo de SAQ empleado. No obstante, tanto las marcas de pago como los adquirientes pueden exigirlo de forma discrecional.	Si
Periodo de validez del reporte	Anual	Anual

Figura 7. Tipología de informes para demostrar cumplimiento con el estándar PCI DSS. Fuente: Acosta (2019).

- ▶ Completar la **atestación de cumplimiento (AOC)** para proveedores de servicios o comerciantes según corresponda.
- ▶ Presentación de **resultados**.
 - Auditoría anual en sitio realizada por un *qualified security assessor* (QSA), que son organizaciones independientes que han sido calificadas por el Consejo de Estándares de Seguridad PCI para validar la adhesión de una entidad a la norma PCI DSS.
 - Informe sobre cumplimiento (ROC).
 - Escaneo de vulnerabilidades trimestral (*quarterly network scan*) ejecutado por un proveedor aprobado de escaneo (ASV).
 - Declaración de cumplimiento (AoC).

Una opción utilizada para simplificar los procesos de evaluación en organizaciones que tienen una gran cantidad de instalaciones o de componentes del sistema es el **muestreo**. Para cada instancia donde se hayan utilizado muestras, el asesor debe (PCI Security Standards Council, 2022):

- ▶ Documentar la justificación de la técnica de muestreo y el tamaño de la muestra.
- ▶ Documentar y validar los procesos y controles de los PCI DSS estandarizados que se utilizan para determinar el tamaño de la muestra.
- ▶ Explicar cómo la muestra es apropiada y representativa de toda la población.

Cuando una entidad no puede cumplir con un requisito explícitamente establecido, puede implantar **controles de compensación** que mitiguen el riesgo asociado al mismo. Los controles de compensación deben cumplir con los siguientes criterios (PCI Security Standards Council, 2022):

- ▶ Cumplir con el propósito y el rigor del requisito original de las PCI DSS.

- ▶ Proporcionar un nivel similar de defensa al proporcionado por el requisito original, de manera que compense el riesgo que mitiga el requisito original.
- ▶ El simple cumplimiento con otros requisitos de las PCI DSS no constituye un control de compensación.

Auditoría de certificación del SGSI

Un Sistema de Gestión de Seguridad de la Información (SGSI) permite a las organizaciones **conocer, gestionar y minimizar** los riesgos relacionados con la seguridad de la información de una forma sistemática y eficiente. La certificación del SGSI con respecto a la norma ISO 27001, por parte de la organización, tiene, entre otras, las siguientes ventajas:

VENTAJAS DE LA CERTIFICACIÓN DEL SGSI CON RESPECTO A LA NORMA ISO 27001	
Para los sistemas de información	Para el negocio
▪ Sistematizar las actividades de SI. ▪ Ahorro de recursos en las actividades de SI, lo que mejora la motivación e implicación de los empleados. ▪ Analizar riesgos: identificar amenazas, vulnerabilidades e impactos en la actividad empresarial. ▪ Establecer objetivos y metas que permitan aumentar el nivel de confianza en la seguridad. ▪ Planificar, organizar y estructurar los recursos asignados a seguridad de la información. ▪ Identificar y clasificar los activos de información. ▪ Seleccionar controles y dispositivos físicos y lógicos adecuados a la estructura de la organización. ▪ Asegurar el nivel necesario de disponibilidad, integridad y confidencialidad de la información. ▪ Establecer planes para una adecuada gestión de la continuidad del negocio. ▪ Establecer procesos y actividades de revisión, mejora continua y auditoría de la gestión y tratamiento de la información.	▪ Integrar la gestión de la seguridad de la información con otras modalidades de gestión empresarial. ▪ Mejorar la imagen, confianza y competitividad empresarial. Certificación y reconocimiento por terceros. ▪ Comprobar su compromiso con el cumplimiento de la legislación: protección de datos de carácter personal, servicios sociedad de información, comercio electrónico, propiedad intelectual, etc. ▪ Dar satisfacción a los accionistas y demostrar el valor añadido de las actividades de seguridad de la información en la empresa.

Tabla 5. Ventajas de la certificación del SGSI con respecto a la norma ISO 27001. Fuente: elaboración propia.

Antes de la realización por AENOR de la auditoría oficial, conviene realizar una **auditoría interna**. La ISO 27001 especifica que el objetivo de la auditoría interna es verificar que se cumplen:

- ▶ Los requisitos de la empresa.
- ▶ Los requisitos de la ISO 27001. Básicamente, auditar los controles del Anexo A de la norma.

Su finalidad es la de poder realizar cualquier correctivo necesario previo a las auditorías de certificación externa y, de esta manera, contribuir con la **gestión responsable** de la información y la mejora continua. Por lo tanto, es necesario realizar estas auditorías internas de forma periódica para poder determinar si los objetivos, los controles, los procesos y los procedimientos del Sistema de Gestión de Seguridad de la Información continúan estando conforme a los requisitos que establece el estándar internacional ISO 27001, además de la legislación y los reglamentos correspondientes.

A la hora de realizar este tipo de auditoría, se deberían tener en consideración los siguientes puntos:

- ▶ **Tiempo.** Dado que se tiene que evaluar una cantidad de controles significativos (114), es necesario que las evaluaciones se realicen de forma pausada, con la finalidad de que los resultados sean adecuados y esperados.
- ▶ **Responsabilidades.** Como el número de controles es significativo, es indispensable dividir el trabajo entre varios auditores, la división se debe realizar dependiendo de las habilidades de cada auditor involucrado.

- ▶ **Preparación.** Realizar un plan de auditoría, un cronograma de trabajo que simplifique o indique en qué fecha los departamentos van a hacer auditados, etc.
- ▶ **Departamentos.** Gestionar los accesos correspondientes a todos los departamentos y unidades de la organización.
- ▶ **Hallazgos.** Distribuir el informe de la auditoría con todas las áreas, esto ayudará a tomar las medidas necesarias, así como a despejar todas las consultas o inquietudes que puedan tener las entidades auditadas. Propósito de establecer una mejora continua dentro de los procesos de la organización.
- ▶ **Correcciones.** Una vez realizado el informe, es indispensable programar seguimientos de los hallazgos incluidos en el mismo, a fin de cumplir con las recomendaciones antes presentadas.

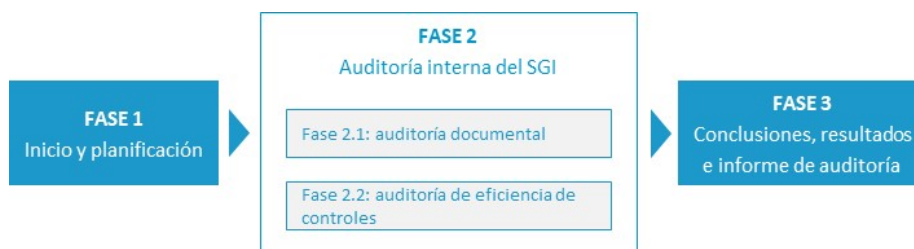


Figura 11. Auditoría Interna SGSI. Fuente: elaboración propia.

Por otra parte, la certificación del SGSI aporta la **confianza** en los sistemas de información, para la que es necesario la realización de una **auditoría externa**. En España, esta auditoría de certificación del SGSI la realiza AENOR. Se lleva a cabo en **dos fases**, como puede apreciarse en la siguiente figura:

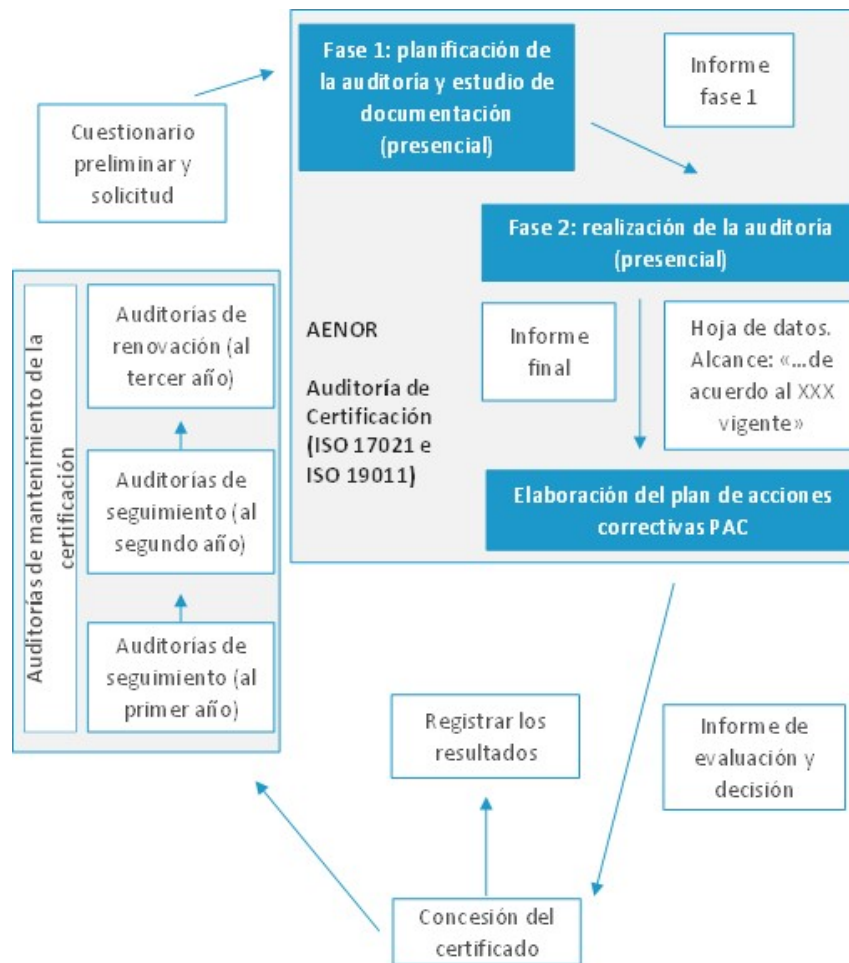


Figura 12. Fases de auditoría del SGSI. Fuente: elaboración propia.

La auditoría del SGSI se rige por los principios de la ISO 17021, de manera que se realiza un cuestionario preliminar y una solicitud de auditoría, tras los que se efectúa la auditoría en dos fases:

- **Fase 1 (no presencial).** Comprende el estudio de la documentación (declaración de intenciones) política, procedimientos/controles, informe de análisis de riesgos, declaración de aplicabilidad. Luego de la fase 1 se emite un informe con las observaciones o no conformidades detectadas que la organización puede subsanar antes de la fase 2, en cuyo caso no formará parte del informe final de auditoría.

- ▶ **Fase 2 (presencial).** Es la fase en la que se busca evidencia de la implantación de los controles y se genera el informe final de auditoría.

La organización debe elaborar y presentar un **plan de acciones correctivas** de cuya evaluación y valoración, por parte de la certificadora, se desprende la concesión del certificado. Luego de la concesión del certificado, la organización debe pasar auditorías de seguimiento al primer y segundo año, una auditoría de renovación al tercer año de forma sucesiva, para asegurar la mejora continua del SGSI.

A continuación, se presentan una serie de softwares que ayudan a realizar este tipo de auditorías:

- ▶ **ISOTools:** herramienta que permite gestionar el ciclo de vida de las no conformidades desde su creación, clasificación y análisis, hasta la posterior gestión de las acciones correctivas necesarias para su resolución. Es una herramienta para emplear durante las auditorías, por lo que permite controlar el grado de cumplimiento de la actividad y evaluar su eficacia.
- ▶ **Conformio:** solución *online* de software que proporciona unos pasos claros para implementar proyectos ISO 27001.
- ▶ **SandaSGRC:** herramienta para gestionar y demostrar el cumplimiento de legislaciones (LOPD, ENS, infraestructuras críticas...), estándares internacionales (ISO 27001, ISO 27002, ISO 22301, PCI DSS...) y políticas corporativas de forma eficiente y centralizada.
- ▶ **Kawak:** proporciona los insumos que necesitan para tomar las medidas preventivas y reactivas sobre la confidencialidad, disponibilidad e integridad de la información. Identifica, valora y prioriza los riesgos asociados a tus activos de información.

- ▶ **ENAXIS:** provee los siguientes módulos para la implementación de sistemas de gestión de seguridad de la información: análisis de riesgos, control de documentos, indicadores, no conformidades/acciones correctivas y preventivas, auditorías y proyectos.
- ▶ **ISOWin:** gestiona el RGPD, ISO 27001, ISO 9001, ISO 14001 y otros proyectos de cumplimiento ISO.

10.3. Metodologías de auditorías de seguridad

Una metodología es un conjunto de procedimientos documentados diseñados para alcanzar los objetivos planeados declarando el alcance, objetivos y programa de trabajo.

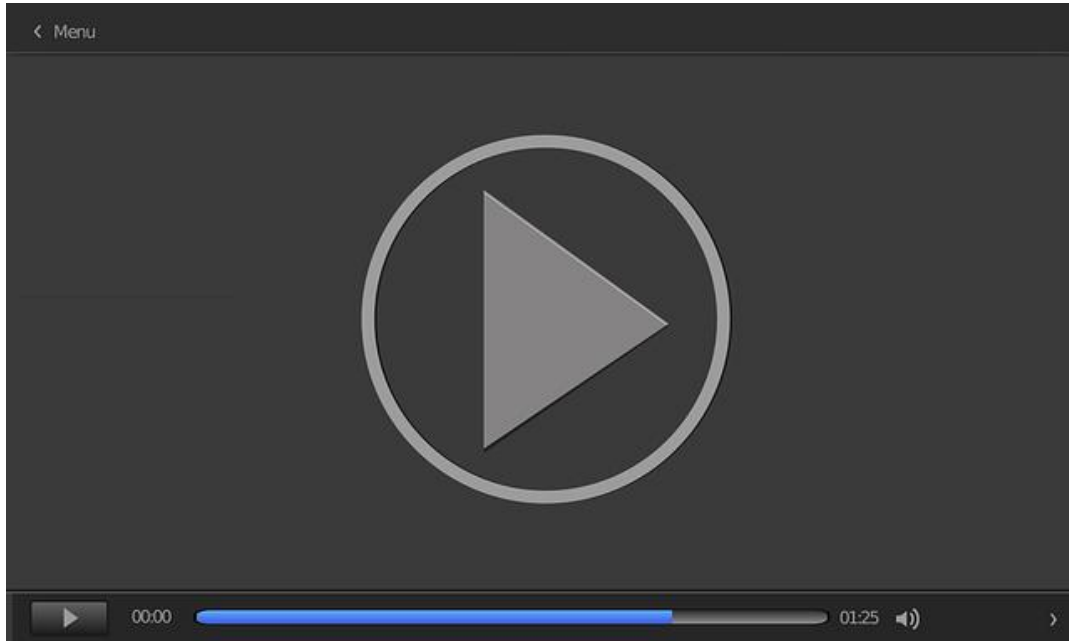
Existe mucha literatura sobre la forma en la que se debe realizar la evaluación de seguridad, por lo que se hace necesario extraer los **principales potenciales** de cada una de estas para constituir una metodología eficiente y aplicable a la evaluación de seguridad de redes de sistemas informáticos. Las diversas metodologías para la realización de auditorías técnicas de seguridad son:

- ▶ Information System Security Assessment Framework (ISSAF).
- ▶ Open Source Security Testing Methodology Manual (OSSTMM).
- ▶ Penetration Testing Execution Standard (PTES).
- ▶ Technical Guide to Information Security Testing and Assessment (NIST 800-115).
- ▶ Open Web Application Security Project (OWASP).
- ▶ Open Wireless Security Assessment Methodology (OWISAM).
- ▶ Open Android Security Assessment Methodology (OASAM).
- ▶ Common Criteria ISO 15408 (CC).

Information System Security Assessment Framework (ISSAF)

Information System Security Assessment Framework (ISSAF) es una metodología estructurada de análisis de seguridad PTF (*penetration testing framework*). Se trata de un marco de trabajo detallado que incluye una serie de prácticas y conceptos en todas y cada una de las tareas en una evaluación de seguridad. En este vídeo

(*Information System Security Assessment Framework [ISSAF]*) se presenta ISSAF, una metodología de auditorías de seguridad realizada por el grupo abierto de la seguridad de la información (OSSIG).

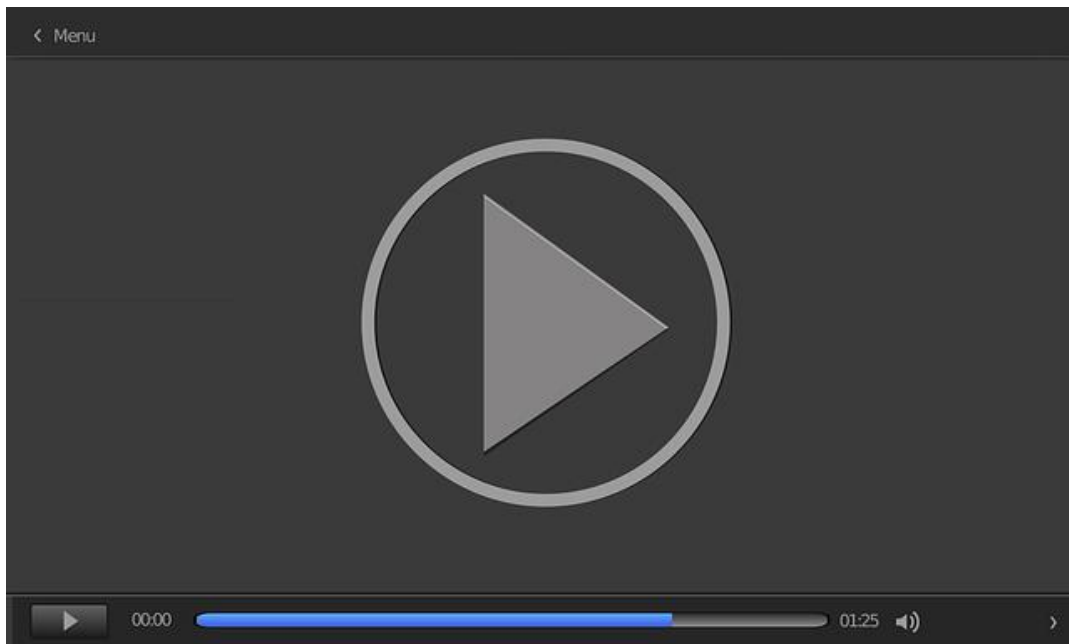


Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=312bda01-5e31-475b-90d9-adf700cfe9ff>

Open Source Security Testing Methodology Manual (OSSTMM)

La metodología del Open Source Security Testing Methodology Manual (OSSTMM) es uno de los estándares más completos y comúnmente utilizados en auditorías de seguridad de sistemas desde Internet. Incluye un marco de trabajo que describe las fases que habría que realizar para la ejecución de la auditoría. En este vídeo (*Open Source Security Testing Methodology Manual [OSSTMM]*) se profundiza en los principales aspectos de la citada metodología.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=8101fae0-a0c2-444d-b378-adf700cfe98e>

Penetration Testing Execution Standard (PTES)

PTES es un estándar que consta de siete apartados que recogen un conjunto de buenas prácticas que tratan de servir de punto de referencia para la realización de pruebas de seguridad mediante el uso de determinadas herramientas y procedimientos.

Accede a la página web a través del siguiente enlace: http://www.pentest-standard.org/index.php/Main_Page

Abarca desde la comunicación inicial (el planeamiento del alcance de las pruebas de penetración, la recopilación de información para producir inteligencia, modelado de las amenazas, estudio y comprensión de la organización, investigación de vulnerabilidades y explotación de estas) hasta la presentación del informe que

captura todo el proceso, de manera que tenga sentido para el cliente y le muestre el valor del proceso realizado. Las principales **secciones definidas por la norma**, como base para la ejecución de pruebas de penetración, son las siguientes:

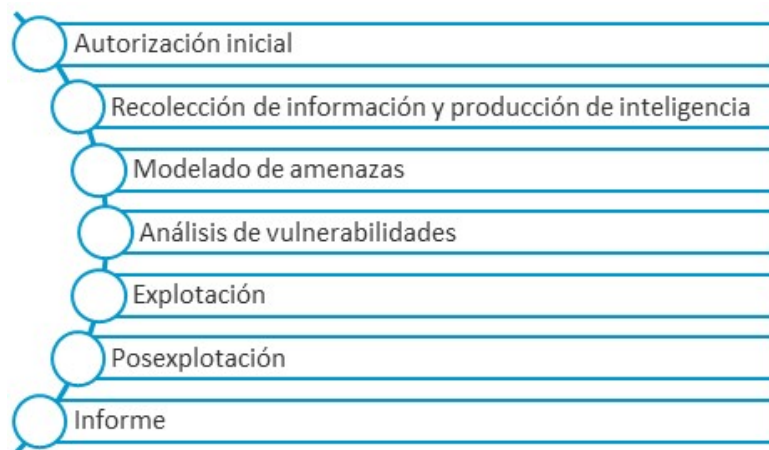


Figura 13. Fases metodología PTES. Fuente: elaboración propia.

Es una excelente fuente de información para cualquier profesional del sector de la seguridad. Como ninguna prueba de penetración es igual a otra, la prueba estará comprendida entre una **prueba de una aplicación web** o de una red hasta el compromiso total de un equipo. Dichos niveles permitirán a una organización definir la cantidad de sofisticación que esperan de su adversario y permitir al probador aumentar la intensidad en aquellas áreas donde la organización más los necesita.

Technical Guide to Information Security Testing and Assessment (NIST 800-115)

Metodología desarrollada por el Instituto Nacional de Normas y Tecnologías (NIST), que tiene por objetivo proporcionar una metodología que sirva para la realización de evaluaciones de seguridad y la propuesta de estrategias para mitigar los fallos. Contiene recomendaciones prácticas para diseñar, implementar y mantener los procesos y procedimientos asociados con la seguridad. Establece **cuatro fases** en la evaluación (Scarfone et al., 2008):

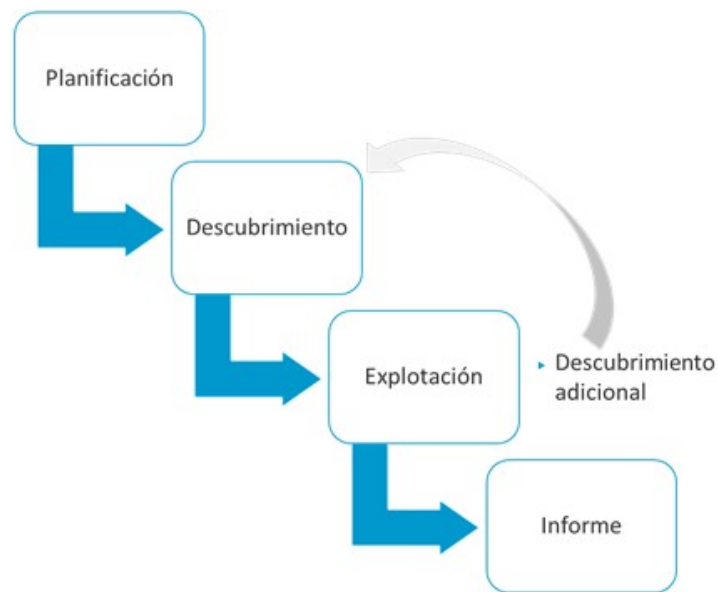


Figura 14. Fases de la metodología. Fuente: elaboración propia.

- ▶ **Fase 1. Planificación.** Se planifica la evaluación de seguridad por realizar, identificando objetivos, amenazas y controles de seguridad que pueden utilizarse para mitigar riesgos.
- ▶ **Fase 2. Descubrimiento.** Se realizan el descubrimiento de vulnerabilidades y su análisis.
- ▶ **Fase 3. Explotación.** En esta fase se ejecutan las pruebas de penetración con base en los objetivos identificados en la sección anterior.
- ▶ **Fase 4. Informe.** Se analizan los resultados identificando las vulnerabilidades encontradas y se recomiendan las formas de mitigar los riesgos a través del informe final.

Las **principales secciones** de la norma son:

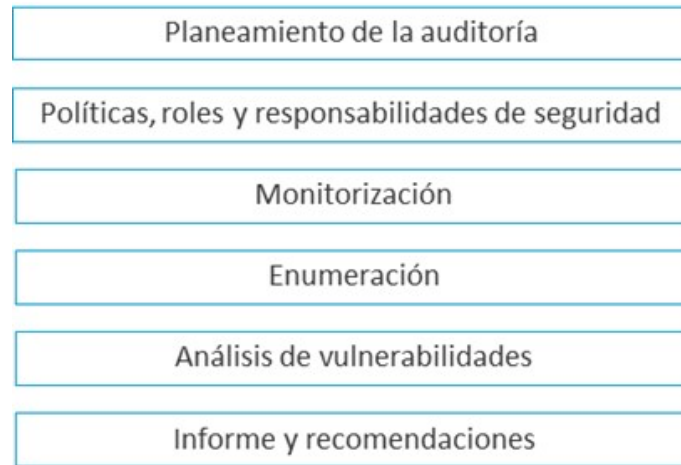


Figura 16. Secciones de la metodología. Fuente: elaboración propia.

Open Web Application Security Project (OWASP)

Una de sus principales metodologías de prueba de la **seguridad de aplicaciones web** se desarrolla en el documento OWASP Testing Guide 4.0 (Meucci y Muller, 2015), que constituye una referencia en materia de auditorías de seguridad de aplicaciones web. Contempla un total de 87 controles, estructurados en torno a 11 categorías, así como las **técnicas y herramientas** requeridas para su verificación:

- 1.OTG-INFO.** Recolección de información.
- 2.OTG-CONFIG.** Gestión de la configuración.
- 3.OTG-IDENT.** Pruebas sobre la administración de identidades.
- 4.OTG-AUTHN.** Pruebas de autenticación.
- 5.OTG-AUTHZ.** Autorización.
- 6.OTG-SESS.** Gestión de sesiones.
- 7.OTG-INPVAL.** Validación de datos.

8.OTG-ERR. Manejo de errores.

9.OTG-CRYPST. Pruebas sobre debilidades en la criptografía.

10.OTG-BUSLOGIC. Lógica del negocio.

11.OTG-CLIENT. Lado del cliente.

De los controles que se van a detallar en los siguientes apartados, no siempre se aplicarán en todas las auditorías, por lo que se pueden encontrar diferentes auditorías sobre las que se haya aplicado un número de controles diferente.

Open Wireless Security Assessment Methodology (OWISAM)

Metodología abierta para el análisis de seguridad *wireless* que busca llenar el vacío actual en cuanto a mejores prácticas a la hora de realizar una auditoría de una red inalámbrica. La **metodología de seguridad OWISAM** define un total de 64 controles técnicos, agrupados en 10 categorías, que especifican un conjunto de pruebas necesarias para garantizar con éxito una auditoría de seguridad sobre una infraestructura inalámbrica. La lista de controles es la siguiente:

OWISAM TOP 10 - 2013			OWISAM
#	CÓDIGO	TIPO DE CONTROL	DESCRIPCIÓN DE LOS CONTROLES
1	OWISAM-DI	Descubrimiento de dispositivos	Recopilación de información sobre las redes inalámbricas
2	OWISAM-FP	Fingerprinting	Análisis de las funcionalidades de los dispositivos de comunicaciones
3	OWISAM-AU	Pruebas sobre la autenticación	Análisis de los mecanismos de autenticación
4	OWISAM-CP	Cifrado de las comunicaciones	Análisis de los mecanismo de cifrado de información
5	OWISAM-CF	Configuración de la plataforma	Verificación de la configuración de las redes
6	OWISAM-IF	Pruebas de infraestructura	Controles de seguridad sobre la infraestructura Wireless
7	OWISAM-DS	Pruebas de denegación de servicio	Controles orientados a verificar la disponibilidad del entorno
8	OWISAM-GD	Pruebas sobre directivas y normativa	Análisis de aspectos normativos que aplican al uso de redes de Wi-Fi
9	OWISAM-CT	Pruebas sobre los clientes inalámbricos	Ataques contra clientes inalámbricos
10	OWISAM-HS	Pruebas sobre hotspots y portales cautivos	Debilidades que afectan al uso de portales cautivos

Figura 23. Tabla de categorías de pruebas. Fuente: Owisam, 2015.

Open Android Security Assessment Methodology

Esta metodología tiene por objetivo el análisis de seguridad de aplicaciones Android. Incluye una taxonomía completa y consistente de vulnerabilidades en aplicaciones Android, que es de utilidad no solo para desarrolladores de aplicaciones, sino también para los encargados de analizar vulnerabilidades en las mismas (Medianero, 2016). Los 59 controles de seguridad de esta metodología se estructuran en las siguientes 9 categorías:

- 1.**OASAM-INFO** *Information Gathering*. Obtención de información y definición de superficie de ataque.
- 2.**OASAM-CONF** *Configuration and Deploy Management*. Análisis de la configuración e implantación.
- 3.**OASAM-AUTH** *Authentication*. Análisis de la autenticación.
- 4.**OASAM-CRYPT** *Cryptography*. Análisis del uso de criptografía.
- 5.**OASAM-LEAK** *Information Leak*. Análisis de fugas de información sensible.
- 6.**OASAM-DV** *Data Validation*. Análisis de gestión de la entrada de usuario.
- 7.**OASAM-IS** *Intent Spoofing*. Análisis de la gestión en la recepción de *intents*.
- 8.**OASAM-UIR** *Unauthorized Intent Receipt*. Análisis de la resolución de *intents*.
- 9.**OASAM-BL** *Business Logic*. Análisis de la lógica de negocio la aplicación.

Common Criteria ISO 15408 (CC)

Estándares ISO 15408 para realizar una evaluación fiable de las características y propiedades de seguridad. Son un conjunto de directrices para la **validación de la seguridad de los productos** de las tecnologías de la información y comunicaciones (TIC).

Es un sistema reconocido a nivel mundial que permite la adquisición y aseguramiento de los **niveles estrictos de integridad** aplicados en la especificación, implementación y evaluación de un producto TIC. Estos productos, normalmente, son evaluados por un laboratorio de pruebas neutral de terceros a través de requisitos de seguridad basados en niveles predefinidos de control de evaluación (EAL). Está orientado principalmente a:

- ▶ Certificación de productos (software o hardware).
- ▶ Especificación de requisitos de seguridad.
- ▶ Soporte para la implementación de características de seguridad.
- ▶ Especificación de requisitos para los procesos de adquisición.

El proceso de evaluación comienza mediante el envío al laboratorio acreditado por parte del proveedor del producto de una serie de documentos:

- ▶ *Protection profile* (PP). Identifica los requisitos y especificaciones de seguridad que debe cumplir una familia de productos TIC a evaluar.
- ▶ *Target of evaluation* (TOE). Es el producto TIC que pretende validar con respecto a las características de seguridad y su funcionamiento.
- ▶ *Security target* (ST). Especifica los requisitos funcionales y no funcionales de seguridad específicos que se solicitan evaluar en el TOE, con respecto a su PP tomado como referencia.

Existen **dos pruebas de evaluación**: requisitos de seguridad funcional (SFRs) y requisitos de aseguramiento de la seguridad (SARs):

- ▶ Requisitos **funcionales de seguridad (SFR)**. La norma posee un catálogo estándar de funciones que definen las características individuales de seguridad que posee un producto:
 - ASE. *Security target evaluation*.
 - ADV. *Development*.
 - AGD. *Guidance documents*.
 - ALC. *Life-cycle support*.
 - ATE. *Tests*.
 - AVA. *Vulnerability assessment*.
- ▶ Requisitos de **aseguramiento de seguridad (SAR)**. Describe las medidas para asegurar el cumplimiento de las especificaciones y funciones de seguridad adoptadas durante el desarrollo, las pruebas y la evaluación del producto.

Finalmente, se realiza el proceso de **evaluación del TOE** para tratar de establecer el nivel de confianza que puede ser asignado al producto:

- ▶ **EAL1 (funcionalidad probada)**. Proporciona evidencia en cuanto a que la evaluación coincide con la descrita en la documentación.
- ▶ **EAL2 (estructuralmente probado)**. Los resultados de las pruebas proporcionan seguridad en la funcionalidad del producto.
- ▶ **EAL3 (probado y verificado metódicamente)**. Apoya las pruebas de caja gris.

- ▶ **EAL4 (diseñado, probado y revisado metódicamente).** Se busca vulnerabilidades independientes.
- ▶ **EAL5 (diseñado y probado semiformalmente).** Se busca la máxima seguridad aplicando técnicas de seguridad.
- ▶ **EAL6 (diseño verificado y probado semiformalmente).** Se realiza un análisis apoyado en un diseño modular y en una exposición ordenada de la implementación del producto.
- ▶ **EAL7 (diseño verificado y probado formalmente).** Confirma de forma independiente y completa los resultados de las pruebas de caja blanca (*white box*) realizadas por el desarrollador.

Los productos que deseen obtener esta certificación deben pasar una serie de pruebas realizadas por un laboratorio certificado.

La metodología que usa la certificación se denomina *Common Methodology for Information Technology Security Evaluation* (CEM) y sus guías pueden ser descargadas desde el siguiente enlace:

<https://www.commoncriteriaportal.org/cc/>

Existen tres guías:

- ▶ *Introduction and general model.*
- ▶ *Security functional requirements.*
- ▶ *Security assurance requirements.*

10.4. Referencias bibliográficas

Acosta, D. (2019, mayo 30). *¿Qué es PCI DSS?* PCI Hispano. <https://www.pcihispano.com/que-es-pci-dss/>

Agencia Española de Protección de Datos. (2019). *Listado de Cumplimiento Normativo* [Archivo PDF]. <https://www.aepd.es/sites/default/files/2019-11/guia-listado-de-cumplimiento-del-rgpd.pdf>

Centro Criptológico Nacional. (2017a). *Guía de Seguridad de las TIC CCN-STIC 802. ENS. Guía de auditoría* [Archivo PDF]. <https://www.ccn-cert.cni.es/series-ccn-stic/800-guia-esquema-nacional-de-seguridad/502-ccn-stic-802-auditoria-del-ens/file.html>

Centro Criptológico Nacional. (2022). *Guía de Seguridad de las TIC CCN-STIC 808. Verificación del cumplimiento del ENS* [Archivo PDF]. 25-26. <https://www.ccn-cert.cni.es/series-ccn-stic/800-guia-esquema-nacional-de-seguridad/518-ccn-stic-808-verificacion-del-cumplimiento-de-las-medidas-en-el-ens/file.html>

Centro Criptológico Nacional. (2019). *Guía de Seguridad de las TIC CCN-STIC 801. Esquema Nacional de Seguridad Responsabilidades y Funciones* [Archivo PDF]. <https://www.ccn-cert.cni.es/series-ccn-stic/800-guia-esquema-nacional-de-seguridad/501-ccn-stic-801-responsabilidades-y-funciones-en-el-ens/file.html>

Medianero, D. (2016, septiembre 29). OASAM. GitHub. <https://github.com/b66l/OASAM>

Meucci, M., y Muller, A. (2015). OWASP Testing Guide 4.0. OWASP.

Página web de Common Criteria (<https://www.commoncriteriaportal.org/>).

Ministerio de Asuntos Económicos y Transformación Digital. (S. f.). *Esquema Nacional de Seguridad - ENS*. Portal Administración Electrónica (PAe). https://administracionelectronica.gob.es/pae_Home/pae_Estrategias/pae_Seguridad_Inicio/pae_Esquema_Nacional_de_Seguridad.html#.ZAYdDJGZPrC

PCI Security Standards Council. (2022). *Payment Card Industry, Data Security Standard. Requirements and Testing Procedures* [Archivo PDF]. https://docs-prv.pcisecuritystandards.org/PCI%20DSS/Standard/PCI-DSS-v4_0.pdf?agreement=true&time=1648815252915

Ramiro, R. (2022, mayo 30). El nuevo ENS 2022 y sus principales cambios. *Ciberseguridad Blog*. <https://ciberseguridad.blog/el-nuevo-ens-2022-y-sus-principales-cambios/>

Ramos, M. A. (2016). La auditoría y el Reglamento Europeo de Protección de Datos (R G P D) . *Informáticos Europeos Expertos*. <http://www.iee.es/pages/bases/articulos/prodaud.html>

Scarfone, K., Souppaya, M., Cody, A., y Orebaugh, A. (2008). *Technical guide to information security testing and assessment*. National Institute of Standards and Technology.

OWISAM (2015). *Owisan top 10 – 2013* [Gráfico]. OWISAM. <https://www.tarlogic.com/wp-content/uploads/2015/02/grafico-auditoria-seguridad-wifi-1.png>

PTES

MediaWiki. (2012, abril 30). *PTES Technical Guidelines*. http://www.pentest-standard.org/index.php/PTES_Technical_Guidelines

Documento que describe las directrices técnicas de la metodología de pruebas de penetración PTES.

OWISAM: Open Wireless Security Assessment Methodology

Rooted CON. (2014, marzo 14). *Andrés y Miguel Tarasco - OWISAM: Open Wireless Security Assessment Methodology [Rooted CON 2013]* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=OsiFu1fOrmY>

Presentación que describe los aspectos más destacados de la metodología OWISAM: Open Wireless Security Assessment Methodology. Está realizada por Andrés y Miguel Tarascó.

OSSTMM 3.0. Auditoría

Adriana Tello. (2013, marzo 1). *OSSTMM 3.0 - Auditoría* - www.ddoshostingprotection.com [Vídeo]. YouTube. <https://www.youtube.com/watch?v=1IUc9zdv9IE>

Presentación acerca de la metodología Open Source Security Testing Methodology Manual (OSSTMM).

Common Criteria en 5 minutos

Allied Telesis. (2016, febrero 16). *Common Criteria in 5 minutes, What is Common Criteria?* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=ceg4hyrcHJc>

El Common Criteria es un conjunto de directrices internacionalmente reconocidas para la seguridad de los productos de las TIC. Este vídeo explica por qué es importante la certificación Common Criteria, quién la utiliza y cuál es la diferencia entre los perfiles de protección y los niveles de garantía de evaluación.

PCI (industria de tarjetas de pago)

PCI Security Standards Council. (2018). *PCI (industria de tarjetas de pago). Normas de seguridad de datos. Requisitos y procedimientos de evaluación de seguridad. Versión*

3.2.1. https://es.pcisecuritystandards.org/_onelink_/pcisecurity/en2es/minisite/en/docs/PCI_DSS_v3-2-1-ES-LA.PDF

Guía de verificación del cumplimiento de las medidas en el Esquema Nacional de Seguridad. El objeto de esta guía es auditar los requisitos del Esquema Nacional de Seguridad para un sistema.

1. La finalidad de una auditoría de cumplimiento es:
 - A. Evaluar el grado de cumplimiento de una norma de seguridad.
 - B. Analizar el código estático del código de aplicaciones para buscar vulnerabilidades, tanto de aplicaciones del tipo web como de cualquier tipo de aplicación.
 - C. Evaluar el nivel de seguridad de las LAN corporativas de carácter interno de una organización.
 - D. Analizar la protección que proporciona una red de seguridad perimetral desde el exterior.

2. Las auditorías de cumplimiento son del tipo:
 - A. Caja blanca.
 - B. Caja negra.
 - C. Caja gris.
 - D. Ninguna de las anteriores.

3. Señala la respuesta incorrecta. Una metodología tiene que proporcionar finalmente un proceso sencillo y estructurado de pruebas que sea:
 - A. Válido para siempre.
 - B. Coherente con las leyes individuales y locales y el derecho a la privacidad.
 - C. Basado en el mérito y esfuerzo de los probadores y analistas.
 - D. Consistente y repetible.

4. ¿Cuál de las siguientes metodologías se considera un marco de trabajo?
- A. Open Source Security Testing Methodology Manual (OSSTMM).
 - B. Penetration Testing Execution Standard (PTES).
 - C. Technical Guide to Information Security Testing and Assessment (NIST 800-115).
 - D. Information System Security Assessment Framework (ISSAF).
5. ¿Qué metodología se enfoca en criterios de evaluación?
- A. The Open Source Security Testing Methodology. Manual (OSSTMM).
 - B. The Information System Security Assessment Framework (ISSAF).
 - C. Technical Guide to Information Security Testing and Assessment (NIST 800-115).
 - D. The Penetration Testing Execution Standard (PTES).
6. ¿Cuál no es una metodología de pruebas de penetración?
- PTES.
 - ISSAF.
 - OSSTMM.
 - PMBOK.
7. Dentro de la metodología OSSTMM se definen una serie de pruebas, ¿cuál de las siguientes no lo es?
- A. Búsqueda de vulnerabilidades.
 - B. Escaneo de la seguridad.
 - C. Test de intrusión.
 - D. Auditoría de informática.

8. ¿En qué metodología se testea la seguridad desde un entorno no privilegiado hacia un entorno privilegiado para evadir los componentes de seguridad, procesos y alarmas, y ganar acceso privilegiado?

- A. ITIL.
- B. PMBOK.
- C. ISSAF.
- D. OSSTMM.

9. ¿Cuál no es un control de la metodología Open Web Application Security Project (OWASP)?

- A. Autorización.
- B. Gestión de sesiones.
- C. Información clasificada.
- D. Manejo de errores.

10. Señala la respuesta incorrecta. De acuerdo con la metodología Common Criteria ISO 15408 (CC), el proceso de evaluación comienza mediante el envío al laboratorio acreditado por parte del proveedor del producto de una serie de documentos:

- A. *Protection profile* (PP). Identifica los requisitos y especificaciones de seguridad que debe cumplir una familia de productos TIC por evaluar.
- B. *Target of evaluation* (TOE). Es el producto TIC que pretende validar con respecto a las características de seguridad y su funcionamiento.
- C. *Security target* (ST). Especifica los requisitos funcionales de seguridad que se solicitan evaluar en el TOE, con respecto a su PP tomado como referencia.
- D. *System design* (SD). Incluye el diseño detallado de la familia de productos TIC por evaluar.